

IC Validator User Guide

中文学习译本

原版版本：X-2025.06，June 2025

Synopsys, Inc.

原文版权归 Synopsys, Inc. 所有（Version X-2025.06, June 2025）。

本文件为非官方个人学习译本，仅供学习参考，不得用于商业传播。

命令名、选项名、文件名、PXL 函数名等技术标识符保持英文原文不译。

目录

关于本用户指南	ii
本版本新增内容	ii
相关产品、文档与商标	ii
排版约定	ii
客户支持	iii
包容性与多样性声明	iii
第一章 IC Validator 基础	1
1.1 在设计流程中使用 IC Validator 工具	1
1.1.1 在 IC Compiler 中运行 IC Validator	2
1.1.2 在 IC Compiler II 中运行 IC Validator	3
1.1.3 与 StarRC 寄生参数提取配合	3
1.1.4 从 Custom Compiler 平台运行	4
1.2 IC Validator 命令行选项	5
1.2.1 常用 (Common)	5
1.2.2 主机配置 (Host Configuration)	5
1.2.3 Runset 配置	6
1.2.4 DRC 相关选项	7
1.2.5 LVS / PERC 相关选项	8
1.2.6 Explorer 相关选项	9
1.2.7 输入数据库选项	9
1.2.8 输出选项	10
1.2.9 调试选项 (Debug)	11
1.2.10 优化控制	12
1.2.11 许可选项	12
1.2.12 通用选项	12
1.3 编译器指令	13
1.3.1 对 64 位坐标的支持	13
1.3.2 仅运行 (Run-Only) 执行	15
1.3.3 环境变量	15
1.3.4 向编译器提供附加信息	16
1.3.5 由环境变量创建预处理器定义	16
1.3.6 裁剪层	18

1.3.7	提升函数调度优先级	18
1.4	Runset 配置	18
1.5	Runset 插入	20
1.6	退出码	20
1.7	内联函数	21
1.8	分布式处理	23
1.8.1	指定主机名与 CPU 数量	23
1.8.2	修改运行中作业的资源	24
1.8.3	在 Platform LSF 中运行	24
1.8.4	在 Univa Grid Engine 中运行	25
1.8.5	许可证要求	26
1.9	Runset 缓存	26
1.9.1	缓存文件	27
1.9.2	配置 Runset 缓存	27
1.9.3	将 Runset 与缓存文件匹配	29
1.10	使用 IC Validator 运行 Knowledge Assistant	29
1.11	将 IC Validator 命令行选项与 Runset 缓存配合使用	30
1.11.1	兼容的命令行选项	30
1.11.2	影响 Runset 缓存的命令行选项	30
1.11.3	不兼容的命令行选项	31
1.12	层映射	31
1.12.1	Runset 层映射	32
1.12.2	数据层映射	33
1.13	使用 PXL Runset 加密	35
1.13.1	操作	35
1.13.2	加密指令	35
第二章	许可与资源需求	37
2.1	许可方案	37
2.2	许可等待	38
2.2.1	用户控制	38
2.3	许可缓存	39
2.4	需要哪些 IC Validator 许可证	39
2.4.1	Apex 许可	40
2.4.2	Elite 许可	40
2.4.3	Base 许可	40
2.4.4	按技术的许可需求	40
2.4.5	按功能的许可需求	42
2.4.6	多核许可	43
2.4.7	令牌许可	44
2.4.8	超线程	44
2.5	资源需求	45
2.5.1	空间需求	45

2.5.2	内存使用	45
2.5.3	运行时间与内存使用报告	46
2.5.4	操作系统	49
2.5.5	系统限制	49
2.5.6	网络环境	50
第三章	输出文件	51
3.1	输出文件说明	51
3.2	通用 DRC 与 LVS 输出文件	51
3.2.1	结果文件	52
3.2.2	错误文件	53
3.2.3	摘要文件	54
3.2.4	规则文件	55
3.2.5	技术文件	55
3.2.6	树结构输出文件	56
3.2.7	虚拟 Cell 文件	56
3.2.8	分布式日志文件	56
3.2.9	分布式统计文件	56
3.2.10	分布式工作目录	56
3.2.11	图形错误结构	57
3.3	LVS 专用输出文件	57
3.3.1	LVS 调试文件	58
3.3.2	提取的版图网表文件	58
3.3.3	提取的版图 SPICE 网表文件	58
3.3.4	Account 文件	58
3.3.5	包围盒文件	58
3.3.6	等价文件	58
3.3.7	器件拉平摘要文件	58
3.3.8	比较日志文件	59
3.3.9	比较树文件	59
3.3.10	比较目录输出文件	59
3.3.11	单个等价点摘要文件	60
3.3.12	分区原理图网表	60
3.3.13	分区版图网表	60
3.3.14	编辑错误	60
第四章	IC Validator Dashboard	61
4.1	简介	61
4.2	从 IC Validator 获取 Dashboard 电子邮件更新	61
4.3	独立启动 IC Validator Dashboard	62
4.4	初始目录设置	62
4.5	IC Validator 图形用户界面	63
4.5.1	IC Validator Dashboard Summary 页面	63

4.5.2	IC Validator Dashboard CPU History 页面	65
4.5.3	IC Validator Memory History 页面	65
4.5.4	IC Validator Dashboard 选项	66
第五章	IC Validator Live DRC	67
5.1	关于 IC Validator Live DRC 工具	67
5.2	在 Custom Compiler 中使用 IC Validator Live DRC	67
5.2.1	设置环境	68
5.2.2	查看作业进度	70
5.2.3	运行 IC Validator Live DRC	71
5.2.4	查看与修复违例	71
5.2.5	附加选项	72
5.3	在 Virtuoso 中使用 IC Validator Live DRC	73
5.3.1	设置环境	74
5.3.2	启动 IC Validator Live DRC 检查	75
5.3.3	在 IC Validator Live DRC 中浏览结果	75
5.3.4	附加选项	76
5.4	在 IC Compiler II 与 Fusion Compiler 中使用 IC Validator Live DRC	84
5.4.1	设置环境	84
5.4.2	为设计规则检查显示对象	86
5.5	设置预编译缓存目录	87
5.6	在 IC Validator 中复现 IC Validator Live DRC 运行	87
第六章	IC Validator Explorer DRC	89
6.1	IC Validator Explorer DRC 简介	89
6.2	Explorer DRC 模式类型	89
6.2.1	执行 Explorer DRC	90
6.3	配置 Explorer DRC	91
6.4	Explorer DRC 输出文件	92
6.4.1	Explorer DRC RESULTS 文件	92
6.4.2	Explorer DRC Errors 文件	93
6.4.3	Explorer DRC VUE 文件	93
6.5	使用 Explorer DRC 调试	93
6.6	Explorer DRC 许可证	94
第七章	IC Validator Explorer LVS	95
7.1	IC Validator Explorer LVS 简介	95
7.2	配置 Explorer LVS	95
7.3	执行 Explorer LVS	97
7.4	Explorer LVS 许可证	98
第八章	PXL	99
8.1	PXL 体系结构	99
8.2	一般定义	100

8.2.1	标识符	100
8.2.2	字符串字面量	101
8.2.3	数值常量	102
8.2.4	关键字	102
8.2.5	运算符	109
8.2.6	注释	110
8.2.7	环境变量引用	110
8.3	变量与运算符	111
8.3.1	变量	111
8.3.2	运算符	113
8.3.3	成员引用	116
8.3.4	优先级与结合性	116
8.4	数据类型与类型系统	117
8.4.1	受支持的数据类型	118
8.4.2	用户定义结构体与类型	120
8.4.3	类型分析	127
8.4.4	嵌套容器	129
8.5	流程控制	131
8.5.1	条件语句	131
8.5.2	循环	131
8.5.3	执行终止	133
8.6	函数	133
8.6.1	函数定义	133
8.6.2	函数原型	134
8.6.3	函数调用	135
8.6.4	匿名函数	138
8.6.5	函数重载	139
8.6.6	函数递归	139
8.7	程序结构	139
8.7.1	复合语句	140
8.7.2	违例	140
8.7.3	执行模型	143
8.7.4	预处理器	143
8.7.5	限定符	147
8.8	模块化代码示例	148
第九章	动态链接库支持	150
9.1	动态链接函数	150
9.2	使用动态链接函数	150
第十章	统一填充	152
10.1	概述	152
10.2	填充图案类型	153

10.2.1 多边形填充图案	154
10.2.2 可调矩形填充图案	157
10.2.3 堆叠填充图案	157
10.2.4 Cell 填充图案	162
10.2.5 可扩展 Fill Cell 图案	162
10.2.6 条带填充图案	165
10.3 填充到填充间距	166
10.3.1 相同图案实例之间的间距	166
10.3.2 不同图案之间的间距	169
10.3.3 间距选项的规则与限制	170
10.4 目标区域与填充到信号间距	171
10.5 范围与宽度相关间距	175
10.6 多边形分组	175
10.7 信号对齐填充	177
10.7.1 为信号对齐填充定义填充图案	178
10.8 改进图案插入	178
10.9 节距对齐填充	179
10.10 判据分析	181
10.11 输出层	184
10.12 层压缩	187
第十一章 处理边	188
11.1 将边层与父层关联	188
11.2 拓扑运算	188
11.3 逻辑运算	189
11.4 边操作	190
第十二章 关键面积分析流程	191
12.1 IC Validator 关键面积分析器 Runset	193
第十三章 DRC 错误分类	197
13.1 使用 DRC 错误分类	197
13.1.1 CELL_LEVEL 模式下的层次考虑	198
13.1.2 层次化 DRC 错误分类	198
13.1.3 错误分类最佳实践	198
13.1.4 数据库命名约定	198
13.2 DRC 错误分类流程	199
13.2.1 DRC 错误分类流程示例	199
13.2.2 pydb_report 实用程序	206
13.2.3 pydb_export 实用程序	210
13.3 使用 PERC 错误分类	214
13.3.1 匹配已分类错误	215
13.3.2 层次化 PERC 错误分类	215

第十四章 DRC Waivers	217
14.1 定义 Waivers	217
14.1.1 使用 VUE 定义 Waivers	218
14.1.2 使用命令行实用程序定义 Waivers	220
14.1.3 自定义 Waiver 属性	220
14.2 创建 Waiver 数据库	221
14.3 使用 Waiver 数据库	222
14.3.1 匹配模式	222
14.3.2 Waiver 报告文件	223
第十五章 错误管理实用程序	226
15.1 错误管理实用程序	226
15.2 DRC 错误分组	231
15.3 拆分错误数据库	231
15.4 合并错误数据库	232
15.5 VUE 文件支持	233
15.6 就地修改数据库	233
15.7 层次考虑	234
15.8 XML 命令文件	234
15.8.1 用 XML 命令文件分类错误	238
15.9 将错误导出到版图	239
15.10 管理所引用的 cPYDB 路径	240
第十六章 图案匹配	241
16.1 概述	241
16.2 使用图案匹配流程	241
16.3 创建图案库	242
16.3.1 准备输入版图	242
16.3.2 创建图案库示例	244
16.3.3 检查图案库创建输出	246
16.4 查看与锁定图案库	248
16.5 检查版图	248
16.5.1 图案匹配 Runset 示例	248
16.5.2 检查图案匹配输出	250
16.5.3 图案匹配后操作	251
16.6 在 IC Compiler 工具中检测并修复热点	252
16.6.1 设置物理签核选项	252
16.6.2 运行图案匹配以检测热点	252
16.6.3 自动修复热点图案	252
16.6.4 使用用户定义修复指导修复热点	253

第十七章 图案库管理器	255
17.1 入门	255
17.1.1 支持的 IC Validator WorkBench 版本	255
17.1.2 启动图案库管理器	255
17.1.3 与 IC Validator WorkBench 通信	256
17.1.4 图案库管理器 GUI	257
17.2 创建新图案库	257
17.2.1 创建新图案库名称	258
17.2.2 添加新图案	258
17.3 编辑图案	263
17.3.1 向图案添加边容差	263
17.3.2 向图案附着图案属性	264
17.3.3 在 IC WorkBench EV Plus 中编辑图案版图	265
17.4 导出与导入图案库	266
17.4.1 导入 Pattern Database 格式	266
17.4.2 导出 Pattern Library Manager 格式	267
17.5 在图案库管理器中运行图案匹配	267
17.6 限制与局限	268
附录 A IC Validator 体系结构	269
A.1 基本体系结构	269
附录 B LVL 实用程序	270
B.1 说明	270
B.2 命令行选项	271
B.3 使用 OpenAccess 库格式	275
B.3.1 OpenAccess 层映射文件	276
B.3.2 单元映射	277
B.3.3 OpenAccess 基线库的错误输出格式	277
B.3.4 设置单元视图名	278
B.4 层文件	278
B.4.1 选择层子集	278
B.4.2 映射层	278
B.5 比较层子集	279
B.5.1 按名称比较	280
B.5.2 按编号比较	280
B.6 文本处理	281
B.7 LVL 输出	281
B.7.1 退出状态	282
B.7.2 错误文件	282
B.7.3 输出目录	283
B.8 使用模型	283
B.9 QuickLVL 实用程序	283

B.9.1	单元级 Quick LVL	284
B.9.2	LVL 与 QuickLVL 的差异	284
B.9.3	聚焦差异层与区域的自动增量 DRC	285
附录 C	版图完整性管理	286
C.1	使用版图完整性管理	286
C.2	版图完整性管理示例	287
C.2.1	基本示例	287
C.2.2	在不同版图格式上运行	288
C.2.3	合并 LIDB 的不匹配报告	289
C.2.4	层次模式	290
C.2.5	层次单元检查示例	290
C.2.6	层次标记层检查示例	292
C.2.7	标记层检查	294
C.2.8	混合 Milkyway 与 GDSII 流程	295
C.3	icv_lidb 实用程序	296
C.3.1	命令行选项	297
C.3.2	退出状态	298
C.3.3	输出文件	298
C.4	icv_lidb_report 实用程序	298
C.4.1	命令行选项	299
C.4.2	输出文件	299
附录 D	IC Validator PXL Debugger	300
D.1	运行 PXL Debugger	300
D.2	PXL Debugger 命令	301
D.2.1	break	302
D.2.2	cont	303
D.2.3	delete	304
D.2.4	disable	304
D.2.5	enable	305
D.2.6	finish	305
D.2.7	info break	306
D.2.8	list	306
D.2.9	next	307
D.2.10	print	308
D.2.11	quit	308
D.2.12	record	308
D.2.13	source	309
D.2.14	step	309
D.2.15	where	310

附录 E	PYDB Perl API	311
E.1	PYDB Perl API 概述	311
E.2	访问 PYDB	312
E.3	常见任务示例	312
E.3.1	列出数据库中的违例	313
E.3.2	列出数据库中的单元	313
E.3.3	访问特定违例中的错误	313
E.3.4	执行直接 SQLite 查询	314
E.4	PYDB 模式	315
附录 F	第三方许可	316
F.1	许可概述	317
F.2	OSS 软件包声明	318
F.2.1	ANTLR	318
F.2.2	Boost	318
F.2.3	cx_Freeze	319
F.2.4	Flatbuffers	320
F.2.5	libdp	320
F.2.6	libxml2	321
F.2.7	MariaDB Connector	322
F.2.8	MariaDB	326
F.2.9	MCPP Public Domain Code	330
F.2.10	NumPy*	330
F.2.11	patchELF	335
F.2.12	pyparsing*	336
F.2.13	Python 3.6 License	336
F.2.14	scikit-learn	337
F.2.15	SciPy*	338
F.2.16	Shroud-1.0	347
F.2.17	SQLite	348
F.2.18	TclLib	350
F.2.19	Tcl/Tk	350
F.2.20	zlib	351
F.2.21	Zstandard	351
F.3	标准 OSS 许可文本	352
F.3.1	Apache-2.0	352
F.3.2	Artistic-1.0-Perl	356
F.3.3	GPL-1.0	358
F.3.4	GPL-3.0	363

关于本用户指南

About This User Guide

本用户指南介绍 PXL、IC Validator 体系结构、命令行选项以及其他基础信息。本指南旨在帮助初级与高级用户加深对 PXL、IC Validator 体系结构以及命令行选项的理解。

本前言包括以下小节：

- 本版本新增内容
- 相关产品、文档与商标
- 排版约定
- 客户支持
- 包容性与多样性声明

本版本新增内容

关于新功能、增强与变更、已知限制，以及已解决的 Synopsys Technical Action Requests (STAR) 信息，请参阅 SolvNetPlus 站点上的 *IC Validator Release Notes*。

相关产品、文档与商标

有关 IC Validator 工具的更多信息，请参阅 Synopsys SolvNetPlus 支持站点上的文档：<https://solvnetplus.synopsys.com>

您可能还希望参阅以下相关 Synopsys 产品文档：

- Synopsys® Custom Compiler™
- Synopsys IC Compiler™
- Synopsys IC Compiler™ II
- Synopsys Fusion Compiler™
- Synopsys StarRC™

排版约定

Synopsys 文档使用下列约定。

表 1: 文档排版约定

约定	说明
Courier	表示语法，例如 <code>write_file</code>
Courier italic	表示语法中的用户自定义值，例如 <code>write_file design_list</code>
Courier bold	表示示例中用户原样输入的内容，例如 <code>prompt> write_file top</code>
紫色高亮	- 在示例中表示特别关注的信息； - 在命令语法中表示默认值，例如 <code>include_enclosing = true false</code>
[]	表示语法中的可选参数，例如 <code>write_file [-format fmt]</code>
...	表示参数可按需重复，例如 <code>pin1 pin2 ... pinN</code>
 	表示在多个备选中选择其一，例如 <code>low medium high</code>
\	表示命令行续行
/	表示目录层次
粗体	表示带有关联操作的图形用户界面（GUI）元素
Edit > Copy	表示菜单命令路径，例如打开 Edit 菜单并选择 Copy
Ctrl+C	表示组合键，例如按住 Ctrl 再按 C

客户支持

客户支持通过 SolvNetPlus 提供。

访问 SolvNetPlus

SolvNetPlus 站点包含技术文章知识库以及对 Synopsys 工具常见问题的解答，并可访问软件下载、文档与技术支持等在线服务。

访问地址：<https://solvnetplus.synopsys.com>

如提示，请输入用户名与密码。若尚无 Synopsys 账号，请按页面说明注册。若使用 SolvNetPlus 时需要帮助，请点击右上角菜单栏中的 **REGISTRATION HELP**。

联系客户支持

请访问 <https://solvnetplus.synopsys.com>。

包容性与多样性声明

Synopsys 致力于营造包容的环境，使每位员工、客户与合作伙伴都感到受欢迎。公司正在审查并从产品及面向客户的材料中移除排他性用语；内部亦推进从工程与工作环境上去除带有偏见的语言，包括嵌入软件与 IP 中的术语。同时，Synopsys 努力确保 Web 内容与软件应用可供不同能力的用户使用。由于部分 IP 实现仍遵循尚在审查中的业界标准规范，软件或文档中可能仍可见非包容性用语。

Chapter 1

IC Validator 基础

IC Validator Basics

本章简要概述 IC Validator 物理验证工具，并介绍 IC Validator 命令行选项与分布式处理。

IC Validator 工具基础包括以下各节：

- 在设计流程中使用 IC Validator 工具
- IC Validator 命令行选项
- 编译器指令（Compiler Directives）
- 退出码（Exit Codes）
- 内联函数（Inlined Functions）
- 分布式处理（Distributed Processing）
- Runset 缓存（Runset Caching）
- 在 Runset 缓存下使用命令行选项
- 层映射（Layer Mapping）
- 使用 PXL Runset 加密

1.1 在设计流程中使用 IC Validator 工具

Using the IC Validator Tool in Your Design Flow

所有 IC Validator 运行均基于您的 IC Validator runset 执行。可通过命令行执行、借助 VUE 的交互式执行，或从其他 Synopsys 工具（如 StarRC、IC Compiler）中启动 IC Validator。加载 IC Validator 前须设置两个环境变量：

- 将 ICV_HOME_DIR 设为 IC Validator 安装目录
- 将 PATH 设为包含 \$ICV_HOME_DIR/bin

要在设计流程中成功运行 IC Validator：

- 创建并调试 `runset` 文件。`runset` 通常包含：
 - 版图数据库相关信息
 - 通用选项
 - 层赋值（`layer assignments`）
 - 数据库检查（如 `snap`、`grid` 检查）
 - 设计规则检查（DRC）、电气规则检查（ERC）、`fill` 图案，或版图与原理图对照（LVS）
- 对版图数据库运行 IC Validator DRC 与 ERC，对照设计规则文档验证设计。
- 使用 IC Validator 错误文件或 VUE 审查并调试 DRC/ERC 结果。
- 运行 `fill-pattern` 生成（含对所填层的基本 DRC 检查）。
- 运行 IC Validator LVS。
- 使用错误文件或 VUE 审查并调试 LVS 结果。

参见

参见附录 A「IC Validator 体系结构」。

1.1.1 在 IC Compiler 中运行 IC Validator

Running the IC Validator Tool Within the IC Compiler Tool

可在 IC Compiler 内部运行 IC Validator，该流程称为 In-Design 物理验证。相关 IC Compiler 命令包括：

- `signoff_drc`：使用晶圆厂认证的 IC Validator DRC `runset` 执行 DRC。
- `signoff_autofix_drc`：自动 DRC 修复。流程中 IC Validator 会：
 - 执行签核级 DRC；
 - 生成 `route guidance`，协助路由器做针对性修复；
 - 验证路由器完成的修复。
- `signoff_metal_fill`：以基于图案（`foundry-qualified metal-fill runset`）或基于 `track`（技术文件信息）的模式插入金属与 `via fill`；`fill` 图形写入 Milkyway 设计的 FILL view。
- `signoff_fix_isolated_via`：自动修复孤立 `via`。
- `report_metal_density`：计算金属密度与密度梯度并标注到设计上。

注意

在 IC Compiler 中使用前，须确认 IC Validator 能正确解析并运行该 `runset`。

参见

详见 SolvNetPlus 上的 *IC Compiler Implementation User Guide*，或 IC Compiler man 页。

1.1.2 在 IC Compiler II 中运行 IC Validator

Running the IC Validator Tool Within the IC Compiler II Tool

可在 IC Compiler II 内部运行 IC Validator (In-Design 物理验证)。相关命令包括:

- `signoff_check_drc`: 使用晶圆厂认证的 IC Validator DRC runset 执行 DRC。
- `signoff_fix_drc`: 自动 DRC 修复 (签核 DRC、生成 route guidance、验证修复)。
- `signoff_create_metal_fill`: 图案模式或 track 模式插入金属/via fill; fill 写入 ICC2 输入设计的设计 view。
- `signoff_fix_isolated_via`: 自动修复孤立 via。

注意

在 IC Compiler II 中使用前, 须确认 IC Validator 能正确解析并运行该 runset。

参见

详见 SolvNetPlus 上的 *IC Compiler II Implementation User Guide*, 或 ICC2 man 页。

1.1.3 与 StarRC 寄生参数提取配合

Running the IC Validator Tool With StarRC Parasitic Extraction

可在 IC Validator runset 中用下列函数生成供 StarRC 使用的数据:

- 寄生提取 (PEX) 层映射函数:
 - `pex_conducting_layer_map()`
 - `pex_via_layer_map()`
 - `pex_marker_layer_map()`
 - `pex_remove_layer_map()`
 - `pex_viewonly_layer_map()`
 - `pex_ignore_cap_layer_map()`

这些函数将 IC Validator 的 `polygon_layer` 对象映射到 StarRC TCAD_GRD_FILE 层, 以生成层映射文件。

- 生成连通版图数据库及映射文件: `pex_generate_results()`

表 1.1 列出了 IC Validator-StarRC 流程中不支持的 LVS 设置 (应设为 `false`)。

表 1.1: IC Validator-StarRC 流程中不支持的 LVS 设置

不支持的设置	参数默认值	建议设置
<code>compare(memory_array_compare=true)</code>	true	false
<code>merge_series(multiple_paths=true)</code>	false	false

参见

PEX 函数的详细语法与用法见 *IC Validator Reference Manual*。

1.1.4 从 Custom Compiler 平台运行

Running the IC Validator Tool From the Custom Compiler Platform

可从 Custom Compiler 的 Verification 菜单或命令行运行 IC Validator 的 DRC 与 LVS。更多信息见 SolvNetPlus 上的 Custom Compiler 文档。

设置并运行 DRC

Setting Up and Running DRC 在 Custom Compiler 中：

1. 在 Layout 窗口选择 **Verification > DRC > Setup and Run**。
2. 选择 IC Validator 并指定所需运行参数。
3. 点击 **Apply** 或 **OK** 运行。

注意

勾选 **View Output** 可打开 Custom Compiler Text Viewer，以标签页查看 runset、结果、错误与运行日志。

若参数已在 Setup and Run 中设好，可选 **Verification > DRC > Run**。

启动 VUE 调试：

- 运行后：**Verification > DRC > Debug**
- 若 Setup 未设为 ICV：先执行 `db::setPrefValue xtDRCtool -value icv`，再选 Debug

打开 Text Viewer（访问 runset、cell.RESULTS、cell.LAYOUT_ERRORS、stdout.drc.log）：**Verification > DRC > View Output**。

设置并运行 LVS

Setting Up and Running LVS

1. 在 Layout 或 Schematic 窗口选择 **Verification > LVS > Setup and Run**。
2. 选择 IC Validator 并指定参数。
3. 点击 **Apply** 或 **OK**。

注意

勾选 **View Output** 可在 Text Viewer 中查看 runset、结果、错误与日志。

参数已设好时选 **Verification > LVS > Run**。

VUE: **Verification > LVS > Debug**；若 Setup 未设为 ICV，先 `db::setPrefValue xtLVSTool -value icv`。

Text Viewer (runset、cell.RESULTS、cell.LAYOUT_ERRORS、cell.LVS_ERRORS、stdout.lvs.log):
Verification > LVS > View Output。

1.2 IC Validator 命令行选项

IC Validator Command-Line Options

命令行语法为:

```
icv [command-line-options] [runsetfile]
```

其中 runsetfile 是用 PXL 编写的指令集，告知 IC Validator 如何执行检查。多数情况下需要 runset；例外包括比较两个网表，或向运行中的作业添加/移除主机等。

选项按用途分为: Common、Host Configuration、Runset Configuration、DRC、LVS/PERC、Explorer、Input Database、Output、Debug、Optimization Controls、Licensing、General。

1.2.1 常用（Common）

Common

表 1.2: IC Validator 常用命令行选项

选项	说明
runsetfile	IC Validator runset。
-c cell	覆盖 library() 的 cell 参数，以及 compare() 的 layout_top_cell。用 -c "*" 时可在未指定顶层 cell 的情况下对 OASIS/GDSII 运行：工具在库中查找顶层 cell；若有多个候选则取第一个并在 summary 中告警。勿在下列情形使用 -c "*"：含 compare() 的 LVS runset；在 ASSIGN 段结束前调用 get_top_cell()；与 -ndg/-C 合用；设置了 incremental_options() 的 select_window/exclude_window；非 GDSII/OASIS 数据库。
-f format	覆盖 library() 中的库格式（GDSII、Milkyway、NDM、OASIS 或 OpenAccess）。
-vue	生成 VUE 输出；也可在 error_options() 中将 create_vue_output 设为 true。

1.2.2 主机配置（Host Configuration）

Host Configuration

用于指定主机、CPU、内存与磁盘用法。

表 1.3: IC Validator 主机配置命令行选项

选项	说明
<code>-host_init ...</code>	在单机或多机上启动作业。形式: <code>hostname</code> (用满 CPU)、 <code>number_of_cpus</code> (本机)、 <code>hostname:n</code> 、LSF/SGE/NB (从对应环境自动取主机与 CPU)。可多次指定,如 <code>-host_init hosta:8 hostb:8</code> 。详见「指定主机名与 CPU 数」。
<code>-host_disk</code>	为 <code>-host_init/-host_add</code> 指定的主机指定本地 <code>group</code> 目录。本地盘写满后改用 <code>run_details</code> 的网络路径。默认 <code>write_vue_info</code> 不使用该目录; 可设环境变量 <code>_ICV_NI_USE_HOST_DISK</code> 覆盖。
<code>-host_login</code>	登录远程主机以启动分布式进程; 默认 <code>rsh</code> 。例: <code>icv -host_login ssh</code> 。 <code>-host_init/-host_add LSF SGE NB</code> 会覆盖默认, 但显式 <code>-host_login</code> 优先。
<code>-host_memory M G T</code>	通过重调度与线程化尽量维持目标内存; 不能用于降低单条命令内存。上限为机器物理内存。未指定时不设内存目标; <code>-host_add</code> 不继承主作业内存设置。
<code>-host_add ...</code>	向已运行作业添加主机; 用法同 <code>-host_init</code> 。默认单次仅支持单主机 <code>grid</code> 请求。
<code>-host_remove list</code>	从作业中移除所列主机并关闭相关进程 (经预留系统获取的主机可释放)。
<code>-host_thread_mode</code>	SMT: 使用 SMT CPU; 或 <code><value></code> (0–100): 专用于线程化的 CPU 百分比。两者同时给出时先评估 SMT。
<code>-host_elastic_cpu_shutoff</code>	运行中作业的最大核数, 默认 2000。
<code>-host_elastic cmd</code>	自动增删主机; <code>cmd</code> 须能调用 <code>icv -host_add</code> 。可在启动时或对运行中作业指定。
<code>-i name</code>	覆盖 <code>library()</code> 的 <code>library_name/merge_libraries</code> 。多个 <code>-i</code> 时第一个替换主库名, 后续构成 <code>merge</code> 库; 格式同主库, 无层映射。
<code>-container</code>	在容器中运行 IC Validator。见 Synopsys Containers / Installation Guide。
<code>-filecache</code>	将部分文件存于内存以减少磁盘占用。

1.2.3 Runset 配置

Runset Configuration

表 1.4: IC Validator Runset 配置命令行选项

选项	说明
<code>-D name[=value]</code>	定义符号名, 可选值, 默认 1。传字符串时注意 shell 勿剥掉引号, 或写入 <code>clf</code> 。

选项	说明
-U name	取消先前由 -D 定义的符号名。
-I directory	预处理器 include 目录；每次一个路径，可多次指定，顺序有关。先搜当前目录再搜 include。非安装目录下须用绝对路径。
-clf file	从文本文件加入命令行选项（自左向右替换）。文件内容不经 shell 做通配/续行处理；以 \$ 开头的变量会按环境变量展开。可多次指定。
-append_runset	将额外 runset 文件追加到主 runset 末尾。
-runset_config file	指定配置文件以更改 options/compare 相关函数：options 类函数视作插在 options 末与 assigns 起之间；compare 相关视作插在 compare() 之前（compare() 本身由配置内容修改）。一般不支持变量。
-cache-only	仅为 runset 在本地缓存目录生成缓存文件，不经引擎运行。
-ece	向 shell 报告额外错误码。见「退出码」。
-norscache	禁用 runset 缓存。

1.2.4 DRC 相关选项

DRC

表 1.5: IC Validator DRC 命令行选项

选项	说明
-svc / -uvc comment	按 violation comment 选择/抑制。
-svn / -uvn name	按 violation name 选择/抑制。
-sfm / -ufn function	按函数名选择/抑制。参数可含 * 与 [] 范围；含通配须加引号。各类别最终取「选中且未抑制」的交集；缺省选择但有抑制则选其余；两者皆无则全选。最终为空则报错退出。
-smu	将 comment/name/function 三类匹配列表改为并集（默认交集）。
-bct option	阻断指定命令输出的前向数据流：extent/connect/both/none（默认）。须与 -il 或 -iln 合用；所选命令本身不被阻断。
-il "layers"	按层名定义按层增量运行的增量层；引号内空格分隔。匹配 ASSIGN 中 assign 的 name（或左侧赋值名）。未找到任一指定层则报错退出。
-iln "..."	按层号（及 datatype）定义增量层，支持范围与逗号语法。
-dv_trace_error_nets	自动启用 delta voltage net tracing（在 runset 末插入调用；若已有则忽略）。
-idrc_f / -idrc_i / -idrc_c	自动增量 DRC：基线版图格式、路径、顶层 cell。

1.2.5 LVS / PERC 相关选项

LVS / PERC

表 1.6: IC Validator LVS/PERC 命令行选项

选项	说明
-s schematic	覆盖 schematic() 的 schematic_file (不覆盖 library file); 须同时用 -sf。
-sdn device_names	仅运行设备名匹配的 device 命令。
-sf SPICE VERILOG ICV	覆盖原理图网表格式。
-stc cell	覆盖原理图顶层 cell (compare()/schematic())。
-C	仅在已有原理图网表与先前生成的版图网表间做比较 (不做 DRC/提取)。用于数据未变仅改 compare 设置时重跑。
-ex	仅做器件提取, 不调用 compare()。
-ndc	仅做 EERC 网表域检查: 复用已有网表, 禁用 polygon/device/compare。
-nvn	无 runset 的网表对网表 (NVN) 流程; 命令行上的 runset 不被读取。
-check_all_properties	Compare 自动检查所有已提取属性 (忽略 check_property[_off]())。
-disable_compare_device_setting	忽略部分或全部 compare device 设置 (FILTER MERGE SHORT ALL)。
-nvnetlist_mode	NVN 模式: LVL (均按 layout 名) 或 SVS (均按 schematic 名)。
-create_lvs_short_output	创建 VUE 并生成 LVS Short Finder 输出; 可限条数或 ALL, 默认 200。输出在 ./run_details/short_out_db。
-define_empty_cell_as_device	空 cell (X-card) 忽略 (默认) 或识别为 GENDEV (NONE BOTH)。
-dhe true false	启用/禁用双层次器件提取, 覆盖 runset 中 init_device_matrix(dual_hierarchy_extraction=)。
-dvp LVS ALL	提取全部属性 (默认) 或仅 LVS compare 所用属性。
-use_alter_starrrc	在用户指定与默认内嵌 StarRC 版本间切换。
-e file	将文件中的 equiv_options()/lvs_black_box_options() 并入 runset (可累积)。
-gse true false	启用/禁用系统生成的 equivalence cell 配对。
-inter_die	对 2.5D/3D 设计启用 inter-die LVS。
-ln / -lnf	覆盖 read_layout_netlist() 的文件名/格式 (须成对使用)。
-lvs_property_epsilon	浮点属性比较的 epsilon (如 $10^{-5}/10^{-6}$)。
-nwl wire_log_file	提供 wireLog 以指导短路解析, 覆盖 NetTran 自动生成文件。
-starrrc_cores n	自动启动 StarRC 时使用的核数。

选项	说明
-udn device_names	不运行设备名匹配的 device 命令。
-vueshort [n ALL]	提取指定数量或全部 short，默认 200。
-zn	压缩由 netlist()/write_spice() 等及 compare() 生成的网表。

1.2.6 Explorer 相关选项

Explorer

表 1.7: IC Validator Explorer 命令行选项

选项	说明
-explorer mode	Explorer: standalone、auto[:tier] 或 tiers[:tier]。见第 6 章。
-explorer_lvs	运行 Explorer LVS。见第 7 章。
-explorer_lvs_layers	指定检查层: METAL ALL user_custom_cell_list。
-explorer_lvs_cells	指定检查 cell: AUTO ALL。
-explorer_lvs_checks	SHORTS 或 ALL (含 LVS compare)。
-explorer_lvs_preserve_cells	保留不被删除/黑盒的 cell。
-explorer_lvs_preserve_cell_file	同上，文件输入 (每行一个 cell)。
-explorer_lvs_preserve_hierarchy_cells	保留 equiv_cells 及其子 equiv_cells。
-explorer_lvs_preserve_hierarchy_cells_file	同上，文件输入。
-explorer_script	tier 0/1 完成后用于添加资源的脚本。
-explorer_tiers_file	自定义 Explorer DRC tiers 文件。

1.2.7 输入数据库选项

Input Database

表 1.8: IC Validator 输入数据库命令行选项

选项	说明
-64	启用 64 位坐标支持 (Elite 许可)。见「对 64 位坐标的支持」。
-add_shape_file	替换/追加 library() 的 add_shape_files 多边形文件列表 (可多次)。
-lf file	指定层映射文件 (runset 层号 $\mapsto$ 库层号)。见「层映射」。
-m factor	覆盖 library() 中的 magnification。
-ml filename	插入 format_merge_library_options() (覆盖 runset 中同类调用)。
-p library_path	覆盖 library() 的库路径。

选项	说明
-rsi "info,..."	报告输入库信息（仅 GDSII/OASIS）；覆盖 run_options() 中对应设置。
-small_cell	强制 small cell 模式以尽快运行小设计；过大则报错退出。
-sub_cell	在大型 OASIS 中仅跑小子单元，避免读全库。
-milkyway_version	打印当前版本支持的 Milkyway schema。
-mw_default_layers	定义若已着色仍按未着色映射的 MW 层号。
-ndm_version	打印支持的 NDM schema。
-ndm_default_layers	类似 ICC2 write_gds -write_default_layers: 着色层亦作 default 层再读一次。
-ndm_design_label	输入 NDM 库的 design label。
-ndm_exclude_layers	排除的 NDM 技术层名（逗号分隔，支持通配）。
-ndm_layer_map_format	icc2 icc_default icc_extended: Milkyway 格式层映射文件时须指定。
-ndm_multiple_rules_per_layer	NDM 层映射映射到文件中所有目标。
-ndm_shape_use_for_pg_mesh	映射为 pg_mesh 的 shape use 列表；默认 "ring stripe"。
-oa_version	打印 OpenAccess 动态库版本。
-oa_cell_map	覆盖 openaccess_options() 的 cell 映射文件。
-oa_color_mapping	OA 颜色层映射策略:ignore strict full compatibility lockOnly。
-oa_dm6	使用兼容 DM6 插件的 OA 共享库（默认）。
-oa_layer_map	覆盖 OA 层映射文件。
-oa_object_map	覆盖 object 映射文件。
-oa_lib_defs	覆盖 OA library definition 文件。
-oa_view	覆盖 view 名。

1.2.8 输出选项

Output

表 1.9: IC Validator 输出命令行选项

选项	说明
-g group_path	覆盖 run_options 的 group 路径；不可为 .；多次时取最后一次。须对所有主机可见。
-rd path	覆盖 run_details 路径；不可为 .。
-w path	重定义当前工作目录。
-email dashboard	每当 icv.log 进度更新时通过 ICV Dashboard 发邮件。
-progress_script	进度更新时运行指定脚本。
-cpydb name path	追加错误分类数据库。
-elpc n ERROR_LIMIT_MAX	每条检查的错误上限，默认 100。

选项	说明
-slpc true false	启用/禁用每检查停止上限，覆盖 <code>error_options(stop_check_at_error_limit=)</code> 。
-ep prefix	覆盖 <code>write</code> 函数中错误 cell 名的前缀。
-hm	生成 violation heat map（必要时触发与 <code>error_limit_per_check</code> 相同整数的 <code>sampling</code> ）。
-hm_cell cell_name	另对匹配的 cell（及顶层）生成 heat map；须同时提供 <code>-hm</code> 。若 cell 因性能优化被 <code>explode</code> 则不生成。
-ned	覆盖 <code>error_options()</code> 的 <code>report_error_details</code> ：不向 <code>LAYOUT_ERRORS/TOP_LAYOUT_ERRORS</code> 写违例细节（仍存 <code>PYDB</code> ，文件仅写摘要）。
-errdm_file file	将运行中发现的全部违例写入可供 Fusion Compiler error browser 导入的文件。
-errdm_hierarchy	写 FC 错误时下层 cell 错误的处理： <code>INST</code> （默认，各错误一例映射到顶层）或 <code>TOP</code> （仅顶层违例）。
-errdm_resolution	FC 错误文件分辨率（微米）；当 NDM 设计分辨率非默认 0.0001 时必需。

1.2.9 调试选项（Debug）

Debug

表 1.10: IC Validator 调试命令行选项

选项	说明
-cpp outputfile	读入 <code>runset</code> ，写出经预处理器完全展开的版本后立即退出（宏/ <code>include</code> 展开，注释去除，加密段保留）。
-pcr output_file	写出经编译检查、函数内联与循环展开后的 <code>PXL</code> ；可用 <code>violation</code> 选择、 <code>run-only</code> 或 <code>-il/-iln</code> 剪裁。
-ro	启用 <code>run-only</code> 处理。见「仅运行执行」。
-rofl file:line	按文件与行号选择 <code>run-only</code> 命令。
-verbose	运行中向屏幕打印各命令输出（同 <code>summary</code> 信息，但按处理顺序而非 <code>runset</code> 顺序）。默认值 3。
-vfn name ALL	在 <code>summary</code> （及 <code>-verbose</code> 时屏幕）报告指定或全部函数的求值参数。
-layer_debugger	无需 <code>VUE</code> 即存储 layer debugger 输出；同时启用 <code>VUE</code> 输出，且按 <code>comment/name/function/run-only</code> 的并集做选择。
-debug	使用 IC Validator <code>PXL Debugger</code> 。
-debug_source file	调试器命令文件。见附录 D。

1.2.10 优化控制

Optimization Controls

表 1.11: IC Validator 优化控制命令行选项

选项	说明
-ndg	精简流程：跳过部分编译、依赖生成与依赖优化；不能分布式，函数在单机顺序执行。适合小 cell。调用内联 runset 函数的用户函数须带 inline；不可与双层次提取合用。
-nro	关闭 runset 优化。
-pec KEEP EXPLODE	覆盖 error_options() 的 partially_exploded_cells。
-prune_assign	仅读取/创建所需 runset 层。见「层剪枝」。
-rfc	减少后端 fill 压缩中的 fill cell 创建。
-prioritize_file_line	按文件与行号优先调度。
-prioritize_violation_comment	按 violation comment 优先调度。

1.2.11 许可选项

Licensing

表 1.12: IC Validator 许可命令行选项

选项	说明
-lic_token2023	强制基于 token 的许可方案。
-lic_apex	强制 Apex 许可。
-lic_base	强制 Base 许可。
-lic_elite	强制 Elite 许可。
-license_mode	多核许可不足时行为:run_available run_and_add exit exit_and_retry, 默认 run_and_add。
-license_wait_duration	等待许可的最长时间，默认 0（单位 d h m s）。
-keys key(s)	控制环境变量访问的密钥（逗号分隔、无空格）。
-nlc	关闭许可缓存。

1.2.12 通用选项

General

表 1.13: IC Validator 通用命令行选项

选项	说明
-exit	终止运行中的进程并给出部分结果；允许工具先清理。RESULTS 等文件会带退出横幅。
-copilot	执行 Knowledge Assistant System。
-time_limit hours	到达用户指定时限后带部分结果退出。
-usage	显示用法信息并退出。
-flat	启用扁平化运行，等效于在 runset 中加入 hierarchy_options(flatten={"*"}).
-recipe	从 recipe 文件加入 recipe，格式如 -recipe "/path/to/recipe:recipe name"。
-V	打印程序版本。
-Version	打印各组件及 Synopsys 软件库版本号。

参见

命令行选项与后文「编译器指令」「Runset 缓存」「层映射」等交叉引用；完整细节亦见原书 Table 2-13。

1.3 编译器指令

Compiler Directives

1.3.1 对 64 位坐标的支持

Support for 64-Bit Coordinates

IC Validator 工具会将坐标数据转换到有符号 32 位整数坐标空间。当芯片尺寸很大而数据库分辨率（database resolution）又很小时，可能需要更大的整数空间，否则会发生坐标溢出（coordinate overflow）错误。

要扩大坐标空间，可使用 -64 命令行选项。仅在确实需要时才使用 -64，因为启用后内存占用会增加。

32 位整数的取值范围为：

- 最大正值：2,147,483,647
- 最大负值：-2,147,483,648

在下列示例中：

- Rcoord 为实数 *xy* 坐标值；
- Icoord 为转换后的整数坐标。

假定：

- Rcoord = 25,000.1234 mm（距 0.0000 为 2.50001234 cm）

- 工作分辨率 (working resolution) = $2 \times$

当数据库分辨率为 0.0001 mm 时, Rcoord 不会溢出 32 位整数坐标空间:

```
Icoord = 25,000.1234*(1/0.0001)*2 = 500,002,468 < 2,147,483,647
```

当数据库分辨率为 0.00001 时, Rcoord 会溢出 32 位整数坐标空间:

```
Icoord = 25,000.1234*(1/0.00001)*2 = 5,000,024,680 > 2,147,483,647
```

不过,并非所有溢出情形都相同。在使用 -64 选项之前,应确认是否已充分利用可用坐标空间。如图 1.1 所示,若只使用正坐标值,会人为限制最大可容纳的芯片尺寸。

(原书 *Figure 1: Overflow of the 32-Bit Integer Coordinate Space*)
请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 1.1: 32 位整数坐标空间溢出示意

将芯片关于原点居中放置,可最大化可用坐标空间,如图 1.2 所示。此时无需使用 -64 选项即可避免溢出。

(原书 *Figure 2: No Overflow of the 32-Bit Integer Coordinate Space*)
请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 1.2: 32 位整数坐标空间无溢出示意 (芯片相对原点居中)

IC Validator 的错误数据库已增强为可变宽度坐标格式,可容纳 32 位或 64 位整数坐标。该能力同时适用于错误数据库 (PYDB) 与错误分类数据库 (cPYDB); 后者可导出用于 DRC 错误分类。

pydb_export 工具也已增强,可在多种坐标格式之间转换错误分类数据库 (cPYDB),以便在必要时保持兼容性。

注意

当前及后续版本的 IC Validator 工具仍支持读取采用旧式固定 32 位或 64 位坐标格式的错误数据库。但版本早于 L-2016.06 的 IC Validator 工具无法读取新的坐标格式。

1.3.2 仅运行（Run-Only）执行

Run-Only Execution

按下列步骤进行 run-only 执行。

1. 用下列指令标识 runset 中希望执行所需检查的区段：

```
#pragma runonly begin

#pragma runonly end
```

可在同一 runset 中定义多个 run-only 区段。

2. 调用 IC Validator 时加上 -ro（run-only）命令行选项，以：
 - a. 解析 runset；
 - b. 裁剪（prune）那些没有数据流进入 run-only 代码区段的 runset 函数；
 - c. 执行经裁剪后的 runset。

注意

若 IC Validator 在 runset 中未找到任何 run-only 区段，则不会执行该 runset。

3. 或者，若希望在为 run-only 执行完成解析与裁剪后即停止执行，可同时使用 -pcr 与 -ro。例如：

```
% icv -ro -pcr outfile
```

4. 当将已裁剪的 run-only 输出用作源 runset 时，应使用 -nro 命令行选项。若不使用 -nro，默认的裁剪优化可能会删除 run-only 代码区段内部的函数。

注意

所有 #pragma run-only 指令会保留在经裁剪的 run-only 输出中。正因如此，可用 icv -ro runonly.rs 命令行直接执行该输出文件 runonly.rs。

1.3.3 环境变量

Environment Variables

变量名前的 \$ 表示该变量为环境变量。例如：

```
$ENV_ABC
```

该变量必须已在环境中设置；否则会发生解析错误。所有环境变量均视为字符串。例如：

```
if( $ENV_ABC == "XYZ" ) {
    x = abc(XYZ);
}
else {
    x = abc(NOT_XYZ)
}
```

1.3.4 向编译器提供附加信息

Providing Additional Information to the Compiler

编译器指令 `#pragma envvar` 用于与环境变量交互，可：

- 打印指定环境变量的名称与取值列表：

```
#pragma envvar report variable ... variable
```

若任一变量不存在，则报错。例如：

```
error: #pragma envvar references undefined environment variable 'e'
```

- 探测环境中是否存在指定变量：

```
// input file
#pragma envvar default x "aa"
#pragma envvar default y "bb"
#pragma envvar default z $x "__" $y // Use $X notation to
                                     // reference defined
                                     // variables

#pragma envvar report x y z

// output from compiler
a.rs:4: x=aa
a.rs:4: y=bb
a.rs:4: z=aa__bb
```

- 若指定变量已存在，则不采取任何操作；
- 若指定变量不存在，则将其在环境中的值设为该行其余部分的拼接结果。拼接内容必须是字符串字面量、对其他环境变量的引用，或二者的组合。

```
#pragma envvar default variable ...
```

注意

若有多条 `#pragma envvar default` 语句定义同一变量，后续出现时会将该变量视为已由首次出现设置到环境中。

1.3.5 由环境变量创建预处理器定义

Creating Preprocessor Definitions From Environment Variables

预处理器指令 `#pragma PXL define_from_envvar` 可在环境变量已设置时，据此创建预处理器定义。语法为：

```
#pragma PXL define_from_envvar cpp_var env_var
```

名为 `cpp_var` 的预处理器定义会被赋予名为 `env_var` 的环境变量之值，效果如同执行了 `#define`。若该环境变量未设置，则不采取任何操作，从而使 `cpp_var` 保持未定义，或保留其先前定义。

`#pragma PXL define_from_envvar` 接受两个参数：

- 第一个参数：若环境变量已设置，则要重定义的预处理器定义名称；
- 第二个参数：取值来源的环境变量名称。

```
#include <icv.rh>

#define str(x) #x
#define qstr(x) str(x)

#if defined(FOO_DEF)
note("Before pragma, defined as: " + qstr(FOO_DEF));
#else
note("Not defined before pragma");
#endif

#pragma PXL define_from_envvar FOO_DEF FOO_ENV

#if defined(FOO_DEF)
note("After pragma, defined as: " + qstr(FOO_DEF));

#if FOO_DEF == 2
note("... and satisfies test for == 2");
#elif FOO_DEF == 3
note("... and satisfies test for == 3");
#endif
#else
note("Not defined after pragma");
#endif
```

当按如下方式运行时：

```
% export FOO_ENV=3
% icv -verbose -D FOO_DEF=2 define_from_envvar.rs
```

输出为：

```
define_from_envvar.rs:10: Before pragma, defined as: 2
define_from_envvar.rs:17: After pragma, defined as: 3
define_from_envvar.rs:21: ... and satisfies test for == 3
```

注意：由于命令行传入的预处理器定义，原始值为 2；`define_from_envvar pragma` 将 `FOO_DEF` 重定义为 3。若未设置 `FOO_ENV`，则 `FOO_DEF` 保持为 2。若命令行上也未定义 `FOO_DEF`，则在余下预处理过程中它保持未定义。

可将 `#if definition(identifier, string, case_sensitivity)` 函数与 `define_from_envvar pragma` 配合使用，以查询环境变量 `$envvar_name` 的值是否等于 `string`。

在用户环境中设置：

```
setenv BEOL_STACK 9M_3Mx_2Fx_2Dx_1Gx_1Iz_LB
```

`runset` 可写为：

```
#pragma PXL define_from_envvar d_ENV_BEOL_STACK BEOL_STACK
```

```
#if definition(d_ENV_BEOL_STACK, "9M_3Mx_2Fx_2Dx_1Gx_1Iz_LB", 0)
...
#endif
#if definition(d_ENV_BEOL_STACK, "8M_2Mx_2Fx_2Dx_1Gx_1Iz_LB", 0)
...
#endif
```

- 第三个参数 `case_sensitivity` 用于 `definition(identifier, string, case_sensitivity)`; 它在 `runset` 中被显式 `#define`, 无需借助 `envvar`。取值为 0 时, 对标识符取值与指定字符串做大小写不敏感比较; 取值为 1 时做大小写敏感比较。

1.3.6 裁剪层

Pruning Layers

`#pragma prune_assign` 编译器指令指示 IC Validator 仅从输入设计中读取所需的 `runset` 层, 并创建所需的 `runset` 层。可用该指令启用或禁用裁剪:

```
#pragma icv compiler [enable|disable] prune_assign
```

若未使用该编译器指令, 或将其设为 `disable`, 且未使用 `-prune_assign` 命令行选项, 则 IC Validator 会读取输入设计中的全部 `runset` 层。无论何种情况, 不必要的 `runset` 层都不会出现在 `summary` 文件中。`runset` 中 `prune_assign` 指令的设置优先于 `-prune_assign` 命令行选项。

1.3.7 提升函数调度优先级

Prioritizing the Scheduling of Functions

`#pragma prioritize` 指令会提高该函数及其所依赖函数的调度优先级。将该指令紧置于目标命令或代码块之前。该指令也可能为这些被优先的函数分配额外线程。例如:

```
#pragma prioritize
not_rectangles(M5);
```

如下例所示, `#pragma prioritize` 会提高两个 `not_rectangles()` 函数的优先级:

```
#pragma prioritize
ERR_M4 @= { @ "M4 non-rectangles";
  not_rectangles(M4);
  not_rectangles(M5);
}
```

相关命令行选项见表 15。

1.4 Runset 配置

Runset Configuration

Runset 配置用于更改 Options 与 Compare 相关函数的配置。更多信息见 IC Validator 命令行选项中的 `-runset_config`。

Runset 配置支持下列 IC Validator 函数：

表 1.14: 支持的 Runset 配置函数

OPTIONS 函数	Compare setup 函数	compare()
compatibility_options()	check_property()	compare()
drc_black_box_options()	check_property_off()	
debug_options()	filter()	
error_options()	filter_off()	
gds_options()	get_filtered_devices()	
hierarchy_auto_options()	map_capacitor()	
hierarchy_options()	map_gendev()	
incremental_options()	map_inductor()	
layout_drawn_options()	map_nmos()	
layout_grid_options()	map_np()	
milkyway_options()	map_npn()	
ndm_options()	map_pmos()	
oasis_options()	map_pn()	
openaccess_options()	map_pnp()	
pattern_options()	map_resistor()	
prototype_options()	match()	
resolution_options()	merge_parallel()	
run_options()	merge_parallel_off()	
text_net_global_options()	merge_series()	
text_options()	merge_series_off()	
equiv_options()	recognize_gate()	
lvs_black_box_options()	recognize_gate_off()	
lvs_options()	recaculate_property()	
net_options()	short_equivalent_nodes()	
waiver_options()	short_equivalent_nodes_off()	
milkyway_error_options()	short_equivalent_stack_nodes()	
	short_equivalent_stack_nodes_off()	

-runset_config 文件中的函数按下列准则处理：

- 所有 OPTIONS 函数实际上插入到 runset 中最后一个 option 之后；
- 所有 “compare setup” 函数插入到最后一个 “compare setup” 函数之后；
- compare() 自身中指定的任何选项用于更新 runset 中的 compare。

1.5 Runset 插入

Runset Insertion

Runset 插入允许你用次级 runset 定制主 runset，而无需修改主 runset 本身。若主 runset 包含必要的 pragma 语句以标明可插入代码的位置，即可在单独文件中提供附加 runset 代码。

主 runset 中的语法为 `#pragma icv insert <label>`。每个 `<label>` 只能使用一次。所有这些 pragma 必须独占一行。

指定插入 pragma 之后，可用 `#pragma icv label <label> begin` 与 `#pragma icv label <label> end` 标记代码块。这些 pragma 必须出现在对应的 insert pragma 之后；不可嵌套或重叠。允许多个代码块使用同一 `<label>`，它们按定义顺序插入到同一目标位置。

1.6 退出码

Exit Codes

表 1.15 列出通用退出码。

表 1.15: 通用退出码

编号	类型	说明
0	EXIT_COMPLETE	运行完成。
30	EXIT_COMPLETE_WERROR	运行完成，但在 <code>cell.LAYOUT_ERRORS</code> 文件中报告了错误，或在 <code>cell.LVS_ERRORS</code> 文件中报告了 LVS 失败。仅当使用 <code>-ece</code> 命令行选项时才启用此退出码。干净版图若仅报告已豁免（waived）错误或方法论（methodology）警告，不会发出此退出码。
32	EXIT_USER_INT	运行被用户以 Ctrl+C 中断。
33	EXIT_USER_ERR	由用户引起的错误，例如解析器错误或文件未找到。
34	EXIT_OS	系统错误，例如除零、总线错误、磁盘满、交换空间不足，或无法创建目录/文件。
35	EXIT_PANIC	发生内部致命错误。
44	EXIT_DATABASE_ERROR	表示发生数据库处理错误，且仅有部分结果可用。

表 1.16 列出与许可证管理器相关的退出码。

表 1.16: 许可证管理器相关退出码

编号	类型	说明
64	EXIT_LIC_TOO_MANY	检出的许可证过多。
65	EXIT_LIC_NOT_ALLOWED	本站点未获授权运行该产品。
66	EXIT_LIC_NO_REPLY	无法联系许可证服务器。
67	EXIT_LIC_DENIED	许可证被拒绝，未给出说明。

表 1.16: 许可证管理器相关退出码（续）

编号	类型	说明
(-1)	EXIT_NONE	未初始化的退出状态变量。

1.7 内联函数

Inlined Functions

当以 `-ndg` 命令行选项运行 IC Validator 时：

- 任何调用内联 `runset` 函数的用户函数，都需用 `inline` 限定符标记。内联 `runset` 函数列表见表 1.17。例如：

```
my_options : inline function ( void ) returning void
{
    . . .
    error_options(...);
    gds_options(...);
    . . .
}
```

- 调用带 `cell` 列表的 `runset` 函数、以及用于构造 `cell` 列表的 `runset` 函数的用户函数，也必须用 `inline` 限定符标记。相关函数列表见表 1.18。例如：

```
add_strings_to_list : inline function (string_list : in_out list of
    string, new_strings : list of string) returning void
{
    string_list = string_list.merge(new_strings);
}

get_cell_extent : inline function (layer1 : polygon_layer, cells :
    list of string) returning result : polygon_layer
{
    result = cell_extent(layer1, cells);
}

cells : list of string = {};

add_strings_to_list(cells, {"AX", "AY", "AZ"});
add_strings_to_list(cells, {"BX", "BZ"});

part_extent = get_cell_extent(part, cells);
```

若某用户函数需要 `inline` 限定符却未标记，运行时错误为：

```
error: unable to extract cell list for no explode options
```

表 1.17 列出带 `inline` 限定符标记的 `runset` 函数。

表 1.17: 带 inline 限定符标记的 Runset 函数

函数	函数
assign()	assign_edge()
assign_openaccess()	assign_openaccess_edge()
assign_openaccess_text()	assign_text()
equiv_options()	error_options()
exclude_milkyway_cell_types()	exclude_milkyway_net_types()
exclude_milkyway_route_guide_layers()	exclude_milkyway_route_types()
exclude_ndm_blockage_types()	exclude_ndm_design_types()
exclude_ndm_net_types()	exclude_ndm_shape_uses()
gds_options()	get_netlist_connect_database()
get_top_cell()	hierarchy_auto_options()
hierarchy_options()	incremental_options()
init_device_matrix()	instance_property_number()
layout_drawn_options()	layout_grid_options()
layout_integrity_by_cell()	layout_integrity_by_marker_layer()
layout_integrity_options()	library()
library_create()	library_import()
lvs_black_box_options()	lvs_options()
milkyway_merge_library_options()	milkyway_options()
ndm_options()	ndm_merge_library_options()
net_options()	oasis_options()
openaccess_options()	
pattern_options()	prototype_options()
read_layout_netlist()	resolution_options()
run_options()	text_options()

表 1.18 列出带 cell 列表的 runset 函数，以及用于构造 cell 列表的函数。

表 1.18: 带 Cell 列表及用于构造 Cell 列表的 Runset 函数

函数	函数
cell_extent()	cell_extent_layer()
compare()	copy_by_cells()
create_ports()	flatten_by_cells()
text_origin()	

1.8 分布式处理

Distributed Processing

可在多种分布式处理场景下运行 IC Validator，包括在单台主机上使用多个 CPU，以及使用多台主机。

以分布式处理运行 IC Validator 可提供下列能力：

- 可扩展性随所用 CPU 数量增加而提升；
- 可向已有 IC Validator 作业灵活添加 CPU；
- 同时多线程（simultaneous multithreading）。

分布式处理在下列各小节中说明：

- 指定主机名与 CPU 数量
- 许可证要求

1.8.1 指定主机名与 CPU 数量

Specifying Host Names and CPU Counts

IC Validator 会利用主机上的多个 CPU，并自动决定启动并行任务还是多线程。你必须告知工具希望使用该主机上多少个 CPU，可通过 `-host_init` 命令行选项完成。

例如，要在单台主机上使用 8 个 CPU：

```
icv -host_init 8 runset.rs
```

也可以指定在哪台主机上运行。上例省略了主机名，因此假定为本地主机。若指定其他主机名，IC Validator 会在该主机上启动。下例在 `hostA` 上使用 8 个 CPU：

```
icv -host_init hostA:8 runset.rs
```

由于指定了远程主机，IC Validator 必须建立到该主机的连接。默认情况下，工具尝试用 `rsh` 命令连接。若希望使用其他连接方式，可使用 `-host_login` 命令行选项（某些 IT 环境可能禁用 `rsh`，或要求使用其他连接方法）。下例在 `hostA` 上使用 8 个 CPU，并以 `ssh` 连接：

```
icv -host_init hostA:8 -host_login /bin/ssh runset.rs
```

注意

IC Validator 在可用时默认使用 `rsh` 作为主机登录程序。使用 `rsh` 可能被视为安全风险，因此常被劝阻或禁用。若环境中禁用了 `rsh`，IC Validator 会自动改用 `ssh`。在系统级禁用 `rsh` 是对网络上所有用户强制安全策略的推荐做法。也可将 `-host_login` 设为始终使用 `ssh` 启动 IC Validator（例如 `-host_login ssh`）。

可以列出多台主机。此时 IC Validator 会使用所列全部主机及其指定的 CPU 数量。下例共使用 32 个 CPU：本地主机 8 个；`hostA` 上 8 个；`hostB` 上 16 个（因未指定 `-host_login`，以 `rsh` 连接各主机）：

```
icv -host_init 8 hostA:8 hostB:16 runset.rs
```

可在支持同时多线程（SMT）的主机上启用额外线程。下例中，在每核有两个逻辑 CPU 的主机上，IC Validator 将 8 个 CPU 专用于命令，并另用 8 个 CPU 做线程：

```
icv -host_init 8 -host_thread_mode SMT
```

更多信息见 IC Validator 命令行选项中的 `-host_init` 与 `-host_login`。

注意

若指定了主机名但省略了 CPU 数量，则使用该主机上的全部 CPU。

作业运行后还可添加更多 CPU。主机与 CPU 的指定方式相同，使用 `-host_add` 命令行选项。注意：该作业必须与原始 IC Validator 作业在同一工作目录中运行。若初始运行用 `-rd` 指定了 `run_details` 目录，则本次运行也必须用 `-rd` 指定同一目录。

1.8.2 修改运行中作业的资源

Modifying Resources for a Running Job

可以向正在运行的 IC Validator 作业添加更多 CPU。主机与 CPU 用 `-host_add` 指定，方式与 `-host_init` 相同。

下例向运行中的作业在 `hostC` 上添加 16 个 CPU：

```
icv -host_add hostC:16
```

注意

使用 `-host_add` 时，多数其他命令行选项既不必要也不允许。更多信息见表 3 中的 `-host_add`。

也可以从运行中的作业移除主机。用 `-host_remove` 指定主机；只能移除整台主机。下例从运行中的作业移除全部 `hostC`（及其全部 CPU）：

```
icv -host_remove hostC
```

还可指示 IC Validator 自动向运行中的作业添加与移除主机：当排队命令过多时添加主机，主机空闲时移除。可在作业初始启动时指定，也可在作业运行后指定。必须提供 IC Validator 用于获取新主机的命令；该命令必须在所获取主机上包含 `icv -host_add`。

对于 `-host_init`、`-host_add` 与 `-host_elastic` 命令行选项，该 IC Validator 调用必须与运行中的作业在同一工作目录中执行。此外，若初始运行用 `-rd` 指定了 `run_details` 目录，则本次运行也必须用 `-rd` 指定。除下列例外外，所有命令行选项都会被忽略：`-rd` 会被上述三个选项使用；`-host_login`/`-host_disk` 会被 `-host_add` 使用。

1.8.3 在 Platform LSF 中运行

Running in Platform LSF

在 LSF 中运行时，前述语法与示例仍然适用。下例展示如何用 `bsub` 启动一个在单台主机上以 16 个 CPU 运行 IC Validator 的 LSF 作业：

```
bsub -n 16 -R "span[ptile=16]" <bsub options> icv -host_init 16
runset.rs
```

类似地，要用 `-host_add` 向运行中的 IC Validator 作业在一台主机上添加 16 个 CPU，例如：

```
bsub -n 16 -R "span[ptile=16]" <bsub options> icv -host_add 16
```

为方便起见，IC Validator 可自动确定主机配置，并将 `-host_login` 设为 `blaunch`。要启用该功能，将 `LSF` 作为 `-host_init` 的参数。下例用 `bsub` 在单台主机上请求 16 个 CPU，并通过 `-host_init` 的 `LSF` 参数加以识别：

```
bsub -n 16 -R "span[ptile=16]" <bsub options> icv -host_init LSF
runset.rs
```

该功能在请求多台主机时最有用。下例同样适用于多主机（2 台主机，各 16 个 CPU）：

```
bsub -n 32 -R "span[ptile=16]" <bsub options> icv -host_init LSF
runset.rs
```

`LSF` 参数也可与 `-host_add` 配合使用。下例向运行中的作业共添加 64 个 CPU：8 台主机，各 8 个 CPU：

```
bsub -n 64 -R "span[ptile=8]" <bsub options> icv -host_add LSF
```

要弹性地向作业添加主机，向 `host_elastic` 命令行选项指定一个 `bsub` 命令，用于获取机器并启动 IC Validator。下例请求单台主机、8 个 CPU。（视环境而定，可能还需向 `bsub` 提供额外参数。）

```
bsub -n 8 'span[hosts=1]' icv -host_add LSF
```

将整段字符串加入 `host_elastic` 命令行选项，或写在单独脚本中。完整的 IC Validator 命令可如下：

```
icv -host_init 16 -host_elastic "bsub -n 8 'span[hosts=1]' icv -host_add
LSF" runset.rs
```

1.8.4 在 Univa Grid Engine 中运行

Running in Univa Grid Engine

在 SGE 中运行时，前述语法与示例仍然适用。下例展示如何用 `qsub` 启动一个在单台主机上以 16 个 CPU 运行 IC Validator 的 SGE 作业：

```
qsub -pe mt 16 <qsub options> icv -host_init 16 runset.rs
```

类似地，要用 `-host_add` 向运行中的 IC Validator 作业在一台主机上添加 16 个 CPU，例如：

```
qsub -pe mt 16 <qsub options> icv -host_add 16
```

为方便起见，IC Validator 可自动确定主机配置，并将 `-host_login` 设为 `qrsh -inherit`。要启用该功能，将 `SGE` 作为 `-host_init` 的参数。下例用 `qsub` 在单台主机上请求 16 个 CPU，并通过 `-host_init` 的 `SGE` 参数加以识别：

```
qsub -mt 16 <qsub options> icv -host_init SGE runset.rs
```

该功能在请求多台主机时最有用。下例同样适用于多主机（2 台主机，各 16 个 CPU）：

```
qsub -pe dp16 32 <qsub options> icv -host_init SGE runset.rs
```

SGE 参数也可与 `-host_add` 配合使用。下例向运行中的作业共添加 64 个 CPU：8 台主机，各 8 个 CPU：

```
qsub -pe dp8 64 <qsub options> icv -host_add SGE
```

要弹性地向运行中的作业添加主机，对 `host_elastic` 命令行选项使用请求机器并启动 IC Validator 的 `qsub` 命令。例如，请求单台主机、8 个 CPU：

```
qsub -cwd -V -pe mt 8 icv -host_add SGE
```

将整段字符串作为 `host_elastic` 选项的参数，或写在单独脚本中。完整的 IC Validator 命令可如下：

```
icv -host_init 16 -host_elastic "qsub -cwd -V -pe mt 8 icv -host_add SGE"
runset.rs
```

1.8.5 许可证要求

License Requirements

对于 Elite 许可证，多核分布式处理所需许可证数量示例见表 1.19。

表 1.19: 多核分布式处理的 Elite 许可证

情形	所需许可证	说明
16 核上的 DRC	4 个许可证	第一个许可证启用 4 核，另加 3 个许可证各启用 4 核
4 核上的 LVS	1 个许可证	
9 核上的 Metal fill	3 个许可证	前两个许可证各启用 4 核，第三个许可证再启用 1 核
62 核上的 Pattern matching	16 个许可证	前 15 个许可证各启用 4 核，第 16 个许可证再启用 2 核

有关 IC Validator 许可证的更多信息，见 *IC Validator User Guide* 中的 Licensing and Resource Requirements 一章。

1.9 Runset 缓存

Runset Caching

IC Validator 的 `runset` 缓存功能可缩短工具启动时间。在 `runset` 完成解析与优化后，其内部数据结构会保存到 `runset` 缓存文件。后续使用同一 `runset` 的 IC Validator 运行会加载该缓存文件，而不再重新解析与优化 `runset`。

可用 `-norscache` 命令行选项关闭 `runset` 缓存；在调试某次运行时可能希望使用该选项。

注意

若对多 corner 的寄生参数提取（PEX）函数使用 runset 缓存，IC Validator 在运行时之前不会标示任何文件命名冲突。

参见

有关在启用 runset 缓存时使用命令行选项的说明，见第 69 页「将 IC Validator 命令行选项与 Runset 缓存配合使用」。

1.9.1 缓存文件

Cache Files

当 IC Validator 写入 runset 缓存文件时，会向屏幕输出如下消息，指出缓存文件路径：

```
Saved runset to cache: /remote/us54h1/myname/.icvrscache/
e3b32d5de337d888106672bbba370041-rod-3908.rscache
Parsing Finished
Runset Compile Time=0:00:03 User=3.37 Sys=0.16 Mem=0.182 GB
```

当 IC Validator 读取 runset 缓存文件时，会向屏幕输出如下消息，指出所读取的缓存文件路径：

```
Loaded runset from cache:
/remote/us54h1/myname/.icvrscache/
e3b32d5de337d888106672bbba370041-rod-3908.rscache
Cached Runset Compile Time=0:00:00 User=0.25 Sys=0.04 Mem=0.108 GB
```

你可以确定缓存目录中哪个缓存文件与特定 runset 关联。查看缓存文件内容时，会看到类似如下文本：

```
Cached runset file created from ICV runset:
/remote/seg-scratch/myname/S475264/px1.ev
. . .
```

第二行即为生成该缓存文件所用 runset 的完整路径。

1.9.2 配置 Runset 缓存

Configuring Runset Caching

可用若干环境变量配置 runset 缓存。

ICV_RUNSET_CACHE_DIR_SHARED 设置 IC Validator 使用的共享 runset 缓存目录。

```
% setenv ICV_RUNSET_CACHE_DIR_SHARED directory
```

该缓存目录不特定于某一用户；多名用户可指向同一共享缓存目录。启用后，IC Validator 执行下列检查：

- 在共享缓存目录中检查是否命中或未命中符合 runset 条件的缓存；
- 若未命中，则检查本地缓存目录；

- 若命中，则使用本地缓存文件；
- 若未命中，则编译并将缓存存入本地缓存目录（对用户本地）。

须手动将缓存文件移至共享目录，才能供多名用户使用。

ICV_RUNSET_CACHE_DIR 设置 IC Validator 存储缓存文件时使用的 runset 缓存目录。

```
% setenv ICV_RUNSET_CACHE_DIR directory
```

默认 runset 缓存目录为 `~/.icvrscache`。

Runset 缓存目录特定于用户。也就是说，多名用户不能通过将其 **ICV_RUNSET_CACHE_DIR** 设为同一目录来共享同一 runset 缓存目录。

ICV_RUNSET_CACHE_MAXFILES 限制 IC Validator 写入 runset 缓存目录的缓存文件数量。

```
% setenv ICV_RUNSET_CACHE_MAXFILES num_files_to_keep
```

运行 IC Validator 时，在达到 **ICV_RUNSET_CACHE_MAXFILES** 或 **ICV_RUNSET_CACHE_MAXSIZE** 限制之前，不会删除任何已有的 runset 缓存文件。任一限制达到后，工具按最近最少使用（least-recently used）顺序删除文件；即最近加载过的缓存文件会保留，较旧的文件会被删除。

ICV_RUNSET_CACHE_MAXSIZE 限制 IC Validator 可用于存储 runset 缓存文件的磁盘空间。默认情况下，工具将 runset 缓存文件所用空间限制为 200 MB。

```
% setenv ICV_RUNSET_CACHE_MAXSIZE max_size
```

max_size 值为数值，后可跟可选字母表示单位：K（千字节）、M（兆字节）、G（吉字节）或 T（太字节）。若不指定单位，则假定 **max_size** 以兆字节计。

ICV_RUNSET_CACHE_DEBUG 报告有助于定位配置问题或意外行为来源的消息。若 runset 缓存行为异常，可使用该变量。消息会显示配置参数，并为 runset 缓存目录中每个候选缓存文件给出加载或不加载的原因。

```
% setenv ICV_RUNSET_CACHE_DEBUG
```

例如：

```
...
RUNSET-CACHE: cache directory : ./RSCACHE/
RUNSET-CACHE: cache max size : 307200
RUNSET-CACHE: cache max files : -1
RUNSET-CACHE: version string : <platform> <ICV version> <date>
RUNSET-CACHE: opendir() opened cachedir : ./RSCACHE/
RUNSET-CACHE: readdir() read filename :
3cb51a5a7c3d1da800be5d6ee862da29-rod-16372.rscache
RUNSET-CACHE: 0 cache files matched runset MD5:
1de3cbc1c76cd3f7b05877718fa83cae
RUNSET-CACHE: no matching cache file found!
```

ICV_DISABLE_RUNSET_CACHE 禁用 runset 缓存。

```
% setenv ICV_DISABLE_RUNSET_CACHE
```

也可使用 `-norscache` 命令行选项。

1.9.3 将 Runset 与缓存文件匹配

Matching a Runset to a Cache File

仅当满足下列条件时，IC Validator 才会将 runset 与已有 runset 缓存文件匹配：

- 预处理指令必须与缓存文件中存储的指令匹配。其中部分指令由 IC Validator 内部生成，另一些则直接来自命令行的 `-D`、`-U` 与 `-I` 选项；
- 各次运行之间，`-svn`、`-uvn`、`-svc`、`-uvc`、`-sfm` 与 `-ufm` 命令行选项未发生变化；
- 经预处理的 runset 必须与缓存文件中的一致（由哈希算法判定）；
- 当前 IC Validator 版本必须与缓存文件一致；
- 保存缓存时与当前运行（尝试加载缓存）所用架构的字节序（endianness）必须一致。例如，AIX 与 Sun 均为 big endian 平台，因此可共享缓存文件；
- runset 中引用的 UNIX 环境变量取值必须与生成缓存文件时相同。

注意

使用 `-ln` 与 `-lnf` 覆盖 `read_layout_netlist()` 函数 `layout_file` 参数中指定的文件名与格式，不会导致缓存未命中。

1.10 使用 IC Validator 运行 Knowledge Assistant

Running Knowledge Assistant Using IC Validator

要从命令行启动 IC Validator Knowledge Assistant，使用 `icv -copilot` 命令。

运行该命令前，请确保正确设置了 `SNPSAI_COPILOT_HOME` 环境变量。若无法通过 `SNPSAI_COPILOT_HOME` 或系统搜索路径找到 `snpsai_copilot` 二进制文件，IC Validator 会显示下列错误消息并终止：

```
Error: 'snpsai_copilot' could not be found in env var SNPSAI_COPILOT_HOME
```

不应与 `-copilot` 选项一起使用其他命令行参数；任何额外参数都会被忽略。

可用 `icv -copilot` 在前台运行 Knowledge Assistant，或用 `icv -copilot &` 在后台运行。约 10–15 秒后，Knowledge Assistant 启动，如图 1.3 所示。

关闭 Knowledge Assistant GUI 后，IC Validator 会自动退出。

IC Validator Knowledge Assistant 会验证是否存在任何有效的 IC Validator 许可证，但不会检出 IC Validator 许可证；而是检出 `GenAI_KA_Copilot` 许可证以运行。

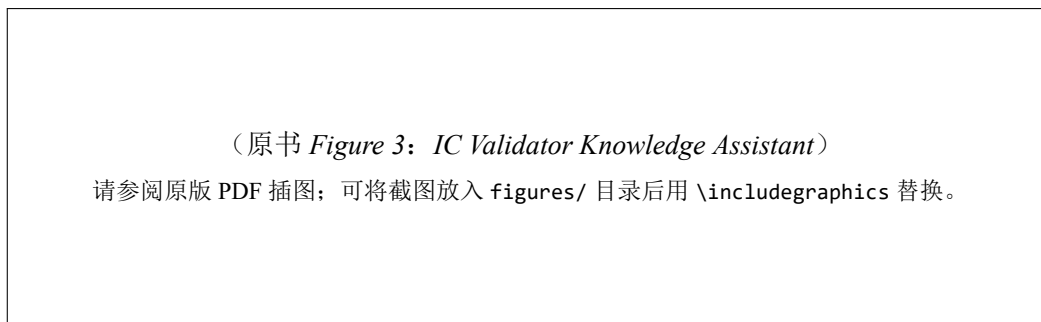


图 1.3: IC Validator Knowledge Assistant

1.11 将 IC Validator 命令行选项与 Runset 缓存配合使用

Using IC Validator Command-Line Options With Runset Caching

在启用 runset 缓存的情况下首次运行某 runset 后，后续对同一 runset 的运行通常会读取缓存文件，而非完整解析 runset。某些 IC Validator 命令行选项会改变 runset 内容，从而影响 runset 缓存的使用。这些更改会使 IC Validator 写入新缓存。只要 runset 被修改，runset 缓存就会受到影响。

注意

下列各节未特别提及的选项，一般与 runset 缓存兼容。

1.11.1 兼容的命令行选项

Compatible Command-Line Options

下列 IC Validator 选项与 runset 缓存兼容。可在命令行上添加、移除或修改这些选项；IC Validator 仍会使用先前生成的缓存：

- -c
- -g
- -i
- -p
- -f
- -rd
- -stc

1.11.2 影响 Runset 缓存的命令行选项

Command-Line Options That Affect Runset Caching

下列选项会改变经预处理的 runset 内容。因此，当在命令行上添加这些选项或其参数发生变化时，经预处理的 runset 不会哈希到同一缓存文件。runset 会正常解析并写入新缓存文件。若这些选项未改变，后续对同一 runset 的运行会哈希到该缓存文件：

- -64
- -D
- -U
- -il
- -iln
- -sfn
- -svc
- -svn
- -ufn
- -uvc
- -uvn
- -verbose

1.11.3 不兼容的命令行选项

Incompatible Command-Line Options

由于处理下列选项的流程特殊，使用它们时会禁用 `runset` 缓存：

- -debug
- -debug_source
- -ndg

下列选项因会更改解析器内部选项而影响 `runset` 缓存。使用该选项时，IC Validator 会生成单独的缓存文件：

- -nro

1.12 层映射

Layer Mapping

层映射有两种类型：

- **Runset 层映射：**修改你的 `runset` 以创建新的 `runset`；
- **数据层映射：**在读入数据时修改数据，通过更改数据的 `layer` 与 `datatype` 属性实现。

在多库上运行时，可同时使用这两种映射。

1.12.1 Runset 层映射

Runset Layer Mapping

Runset 层映射是将 runset 的 ASSIGN 段中指定的 layer/datatype 编号，替换为数据库中数据的实际 layer/datatype 编号的过程。例如，当数据库中的 layer/datatype 编号与晶圆厂提供的 runset 中指定的不一致时，就需要进行层映射。

用 `-lf` 命令行选项指定映射文件。见第 25 页 IC Validator 命令行选项。

映射文件的第一行必须始终为：

```
RUNSET(*,*) = Library(*,*)
```

该语句将每一层映射到其自身（恒等映射，identity mapping），从而确保每一层都有映射。映射文件的后续行针对选定层覆盖恒等映射。

例如：

```
Runset(*,*) = Library(*,*)
Runset(15,0) = Library(1,0)
Runset(16,0) = Library(2,0)
Runset(17,0) = Library(3,0)
```

文件中按下列格式指定映射：

注意

`Runset(i,j)` 指 runset 的 ASSIGN 段中的 layer/datatype 编号。

`Library(i,j)` 指数据库中数据的实际 layer/datatype 编号。

- `Runset(A,B) = Library(C,D)`

下例中，对 layer/datatype (1,5) 的 assign 引用会被替换为 (2,0)：

```
Runset(1,5) = Library(2,0)
```

- `Runset(A,B) = Library(C1...Cm,D1...Dn)`

下例中，对 layer/datatype (5,0) 的任何 assign 引用都会被替换为四个 layer/datatype 对 (10,0)、(10,4)、(12,0) 与 (12,4)：

```
Runset(5,0) = Library(10 12,0 4)
```

- `Runset(A1...Am,B1...Bn) = Library(C1...Cm,D1...Dn)`

注意

Runset 层元素个数与 Library 层元素个数必须相等。

Runset datatype 元素个数与 Library datatype 元素个数必须相等。

下例中进行了四次 layer/datatype 替换：

- assign (2,6) 替换为 (10,0)
- assign (2,8) 替换为 (10,4)

- `assign (4,6)` 替换为 `(12,0)`
- `assign (4,8)` 替换为 `(12,4)`

```
Runset(2 4,6 8) = Library(10 12,0 4)
```

映射文件的语法规则如下：

- 允许用通配符代替列表。例如：

```
Runset(1,*) = Library(2,*)
```

表示 `assign` 层 1、所有 `datatype`，替换为层 2、所有 `datatype`。

- `layer/datatype` 范围用短划线指定。例如：

```
Runset(2 5-8,*) = Library(10-12 17 20,*)
```

- 列表元素必须按升序排列。

当使用层映射文件执行 IC Validator 时，任何对 `assign layer/datatype` 编号的替换都会报告在 `cell.sum` 文件中相应 `assign` 语句之下。报告示例如下：

```
L1_all = assign({ { 11 } })
Function: assign
Inputs: L1_all.polygonlayer.0001
Outputs: L1_all.polygonlayer.0002
Layer list {{11}} was replaced with {{1}}
```

1.12.2 数据层映射

Data Layer Mapping

当合并多个库且对应数据位于不同 `layer` 或 `datatype` 上时，使用数据层映射。例如，若主设计库中 `metall` 在层 41，次级库中 `metall` 在层 31，则需用数据层映射合并这些库，使全部 `metall` 数据读入同一层——本例中为层 41。

通常用 `library()` 函数指定主设计库。`library()` 不对主库提供数据层映射能力；但在指定附加库时可使用数据层映射。因此，可将附加库中的数据映射到主设计库所用的对应 `layer` 与 `datatype`。

若使用 `library_create()` 而非 `library()`，则所有库都通过 `library_import()` 指定，并均可使用数据层映射。就数据层映射而言，可将任一库视为主库，并将其余库映射到该主库。

除数据层映射外，同一运行中还可使用 `runset` 层映射，将 `runset` 层映射到主库所用的层。下列示例说明该场景。

本例中，主库与次级库在 `library()` 函数中指定：

- 主库 `one.oasis` 由 `library()` 指定，其 `metall` 在层 41；
- 次级库 `two.oasis` 的 `metall` 在层 31；
- `runset` 假定 `metall` 在层 5。

读入时，`two.oasis` 中层 31 上的 `metall` 数据被映射到层 41，从而确保全部输入 `metall` 数据读入层 41。随后 `runset` 层映射将 `assign({{5}})` 变换为 `assign({{41}})`，以确保运行按预期执行。`runset` 片段如下：

```
library(library_name = "one.oasis",
        cell         = "A",
        format        = OASIS,
        merge_libraries = {{library_name = "two.oasis", format=OASIS,
                                layer_map_file="two_layer_map"}}});

metall = assign({{5}});
```

`two_layer_map` 文件包含 `two.oasis` 的数据层映射:

```
41  31
```

用 `-lf` 命令行选项指定的 `runset` 层映射文件内容为:

```
RUNSET(*,*) = LIBRARY(*,*)
RUNSET(5,*) = LIBRARY(41,*)
```

下例中, 主库由 `library()` 指定, 次级库由 `library_import()` 指定。当不希望发生 `cell` 合并时, 可用此方法导入多个库。各库彼此保持独立, 从而使 `assign` 函数可从某一个库导入数据而不从另一个导入。

- 主库 `one.oasis` 由 `library()` 指定, 其 `metall` 在层 41;
- 次级库 `two.gds` 的 `metall` 在层 31;
- `runset` 假定 `metall` 在层 5。

`two_layer_map` 文件与 `runset` 层映射与上一示例相同。`runset` 片段如下:

```
lib1 = library(library_name = "one.oasis",
               cell         = "A",
               format        = OASIS);

lib2 = library_import(library_name = "two.gds",
                      cell         = "A",
                      cell_prefix  = "TWO_",
                      format       = GDSII,
                      layer_map_file = "two_layer_map");

metall = assign({{5}});
```

下例中, 主库由 `library()` 指定, 次级库由 `milkyway_merge_library_options()` 指定。这些库既 not 合并也不彼此隔离; Milkyway `cell` 数据会按 `cell` 被等效 GDSII 数据替换。更多信息见 `milkyway_merge_library_opt` 函数。

- 主 Milkyway 库 `one` 由 `library()` 指定, 其 `metall` 在层 41;
- 次级 GDSII 库 `two.gds` 的 `metall` 在层 31;
- `runset` 假定 `metall` 在层 5。

`two_layer_map` 文件与 `runset` 层映射与第一个示例相同。`runset` 片段如下:

```
library(library_name = "one",
        cell         = "A",
```

```

        format      = MILKYWAY,
        library_path = ".");

milkyway_merge_library_options(
    libraries      = {{"one", ".", {"two.gds", GDSII,
                                   layer_map_file="two_layer_map"}}}},
    missing_cell = ABORT);

metal1 = assign({{5}});

```

1.13 使用 PXL Runset 加密

Using PXL Runset Encryption

Runset 加密是一种向最终用户隐藏 runset 内容的方法。使用加密可限制最终用户对晶圆厂工艺参数的了解，或对执行某些设计规则检查或变换所需步骤的了解。

1.13.1 操作

Operation

在以纯文本创建 runset 之后，可将部分内容转换为加密块。加密块内容可包含 PXL 构造以及任意标准预处理器指令。纯文本也称为明文（clear text）。

使用 `pxlencrypt` 工具加密 runset。它仅加密明文块。该工具包含在 IC Validator 发行版安装目录中。语法为：

```
% pxlencrypt cleartext_runset encrypted_runset
```

`pxlencrypt` 无法逆转加密过程。

随后将加密后的 runset 交付给最终用户，由其用 IC Validator 执行。

所有被加密的 runset 函数在 `summary` 文件中，以及在使用 `-verbose` 选项时的标准输出上，均报告为“encrypted function”。尽管函数列为加密函数，机器用量、生成的多边形数量，以及智能删除层（smart-deleted layers，即因不再有任何依赖而不再需要的层）的名称仍会报告。

1.13.2 加密指令

Encryption Directives

标记加密区段起点与终点的指令为：

```
#pragma PXL encrypt begin

#pragma PXL encrypt end
```

同一 runset 中可加密多个区段；但加密区段不可嵌套在另一加密区段之内。runset 中加密区段示例如下：

```
...
#pragma PXL encrypt begin
```



```
e2 @= { @ "e2 violation";
        external2( l1, l2, distance < 2.0, extension = NONE,
                   look_thru = COINCIDENT);}
#pragma PXL encrypt end
...
```

不可加密不完整的构造。例如，下列部分构造是不允许的：

```
...
external2( l1, l2, distance < 2.0, extension = NONE,
#pragma PXL encrypt begin
           look_thru = COINCIDENT);}
#pragma PXL encrypt end
...
```

pxlcrpt 会自动在被包含文件周围断开加密块。例如，若 runset 有如下代码：

```
#pragma PXL encrypt begin
...
#include "file1.rh"
...
#pragma PXL encrypt end
```

则加密后变为：

```
#pragma PXL encrypt begin
...
#pragma PXL encrypt end
#include "file1.rh"
#pragma PXL encrypt begin
...
#pragma PXL encrypt end
```

Chapter 2

许可与资源需求

Licensing and Resource Requirements

本章介绍 IC Validator 工具的设置说明。请确认目录与文件齐全、运行环境满足 IC Validator 要求，并已正确配置许可信息。

许可与资源需求包括以下各节：

- 许可方案（Licensing Schemes）
- 许可等待（License Waiting）
- 需要哪些 IC Validator 许可证（Which IC Validator Licenses Do I Need?）
- 资源需求（Resource Requirements）

关于 Synopsys 许可及 IC Validator 安装的更多信息，请参阅：

- *Synopsys Licensing QuickStart Guide*，见
<http://www.synopsys.com/Support/LI/Licensing/Pages>
- IC Validator 安装信息，见
<http://www.synopsys.com/Support/LI/Installation/Pages/default.aspx>

2.1 许可方案

Licensing Schemes

IC Validator 工具有三种许可方案，运行时使用其中之一：

- Apex 许可（Apex Licensing）
- Elite 许可（Elite Licensing）
- Base 许可（Base Licensing）

Base 许可为默认方案。若作业未使用许可等待，也未指定 `-lic_apex`、`-lic_elite` 或 `-lic_base` 命令行选项，则 IC Validator 首先检查该次运行所需的最低档（tier）许可证；若不可用，再检查更高

档。若运行可使用任一档，则先检查 **Base**；**Base** 不可用时检查 **Elite**，再检查 **Apex**。若均不可用，作业以许可拒绝（**license denied**）错误结束。

三种许可方案可同时作为系统上的可用方案存在，但一次 IC Validator 作业只使用其中一种，不能混用。

若多种许可证均可用，可用 **-lic_apex**、**-lic_elite** 或 **-lic_base** 为特定作业选择许可方案。更多信息见第一章「IC Validator 命令行选项」。

2.2 许可等待

License Waiting

IC Validator 按 Synopsys Common Licensing (SCL) 标准指南支持许可等待（**license waiting**）。

在启用许可等待时运行 IC Validator：

- 若仅安装了 **Apex** 许可，作业排队等待 **Apex** 许可证。
- 若仅安装了 **Elite** 许可，作业排队等待 **Elite** 许可证。
- 若仅安装了 **Base** 许可，作业排队等待 **Base** 许可证。
- 若安装了多种许可方案，作业按 **Base**、**Elite**、**Apex** 的顺序，排队等待其中第一种已安装的方案；其余方案被忽略。
 - 若使用 **-lic_base**，作业排队等待 **Base**，忽略 **Apex** 与 **Elite**。
 - 若使用 **-lic_elite**，作业排队等待 **Elite**，忽略 **Apex** 与 **Base**。
 - 若使用 **-lic_apex**，作业排队等待 **Apex**，忽略 **Elite** 与 **Base**。

2.2.1 用户控制

User Controls

当提交作业时并非全部多核（**multicore**）许可证都可用，IC Validator 提供如下用户控制：

License_wait_duration

指定工具等待许可证变为可用的最长时间。

License_mode

决定启动时并非全部多核许可证都可用时的行为。

- **run_available**：作业以当前可用的部分许可证运行。
 - 若同时设置了 **-license_wait_duration** 且当时没有任何许可证，作业排队直至有非零数量的许可证可用，然后用这些许可证运行。
- **run_and_add**：作业以当前可用的部分许可证运行，随后继续排队并在许可证变为可用时追加签出，尽量达到所请求的全部许可证数量。
 - 若同时设置了 **-license_wait_duration** 且当时没有任何许可证，作业排队直至有非零数量可用并以此启动；之后继续排队追加，尽量达到全部请求数量。

- **exit:** 若完整数量不可用，作业退出。
 - 若同时设置了 `-license_wait_duration`，作业排队并逐步获取许可证；若在时限内达到完整数量则启动，否则退出。
- **exit_and_retry:** 若完整数量不可用，作业退出、释放已获取的许可证并重试。
 - 若同时设置了 `-license_wait_duration`，作业排队并逐步获取；若在时限内达到完整数量则启动；否则退出、释放已获取许可证并重试。

2.3 许可缓存

License Caching

IC Validator 在获知各许可服务器上安装了哪些许可密钥后会缓存这些信息。在后续运行中，缓存信息可能用于调整作业使用的许可服务器列表顺序，或调整检查各许可方案的顺序，从而可能缩短成功签出许可证的时间。

缓存信息会周期性过期，并随工具进一步了解各服务器上的可用许可证而更新。当工具改变许可服务器或许可方案的优先级时，会记录到 `run_details/licmsg`。

默认情况下，许可缓存在下列目录：

```
$XDG_CACHE_HOME/Synopsys/icv/license
```

若未设置 `$XDG_CACHE_HOME`，则许可缓存在：

```
~/.cache/Synopsys/icv/license
```

可设置环境变量 `ICV_LICENSE_CACHE_DIR` 覆盖许可缓存位置。

使用命令行选项 `-nlc` 可禁用许可缓存。

2.4 需要哪些 IC Validator 许可证

Which IC Validator Licenses Do I Need?

`runset` 中使用的 IC Validator 功能决定所需许可证。以下各节说明所需许可：

- Apex 许可 (Apex Licensing)
- Elite 许可 (Elite Licensing)
- Base 许可 (Base Licensing)
- 按功能的许可需求 (Licensing Requirements by Function)
- 按技术的许可需求 (Licensing Requirements by Technology)
- 多核许可 (Multicore Licensing)
- 令牌许可 (Token Licensing)
- 超线程 (Hyperthreading)

2.4.1 Apex 许可

Apex Licensing

Apex 许可使用下列密钥：

- ICValidator-Manager-Apex
- ICValidator2-GeometryEngine
- ICValidator2-CompareEngine

每次运行 IC Validator 都会签出 ICValidator-Manager-Apex；该密钥采用分档许可：高档 Apex、中档 Elite、低档 Base，具体取决于所用技术或功能。

2.4.2 Elite 许可

Elite Licensing

Elite 许可使用下列密钥：

- ICValidator-Manager
- ICValidator2-GeometryEngine
- ICValidator2-CompareEngine

每次运行 IC Validator 都会签出 ICValidator-Manager；同样按技术或功能采用 Apex / Elite / Base 分档。

2.4.3 Base 许可

Base Licensing

Base 许可使用下列密钥：

- ICValidator-Manager-2020
- ICValidator2-GeometryEngine
- ICValidator2-CompareEngine

每次运行 IC Validator 都会签出 ICValidator-Manager-2020；同样按技术或功能采用 Apex / Elite / Base 分档。

2.4.4 按技术的许可需求

Licensing Requirements by Technology

技术分组可能需要一个或多个许可证，如表 2.1 所示。

表 2.1: 技术许可需求 (Technology Requirements)

技术	所需许可
DRC 功能	各档均可用
LVS 提取功能	各档均可用
LVS 比较功能	各档均可用。使用 ICValidator2-CompareEngine
Metal fill 功能	各档均可用
Antenna	各档均可用
与 IC Compiler II / Fusion Compiler 的 Fusion (In-Design)	各档均可用
Netlist Visualizer	各档均可用
Layer Debugger	各档均可用
LVS Short Finder	各档均可用
DRC Heat Map and Balanced Error Distribution	各档均可用
IC Validator Launch	各档均可用
错误数据库 (PYDB) 查询	各档均可用
PERC	各档均可用
Live DRC - Custom	各档均可用
Live DRC - Digital	各档均可用
16-Threaded Commands	各档均可用
VUE	各档均可用。使用 ICValidator2-GeometryEngine ¹
NetTran 及其他实用程序	各档均可用。使用 ICValidator2-GeometryEngine ²
Pattern matching 功能	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
Multiple-Patterning Technology (MPT) 功能	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
Explorer DRC	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
Explorer LVS	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
Elastic CPU Management	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
Blackbox DRC	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
3DIC	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
Unified fill	部分子组件限于 Elite Apex; 另有部分子组件限于 Apex
ML resource estimation	Apex (ICValidator-Manager-Apex)

¹ 仅检查是否存在有效许可证，不签出许可证。

² 仅检查是否存在有效许可证，不签出许可证。

2.4.5 按功能的许可需求

Licensing Requirements by Function

个别函数可能需要一个或多个许可证，如表 2.2 所示。

表 2.2: 功能许可需求 (Functions Requirements)

函数 (选项)	所需许可
adjust_color_graph()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
external2(look_thru blocking_layer)	各档均可用 Apex (ICValidator-Manager-Apex) Apex (ICValidator-Manager-Apex)
fill_to_text()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
four_color()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
layout_text_to_net_property()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
net_color_check()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
net_select_error()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
not_within_distance_by_property()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
optional_pattern_markers()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
pattern_extract()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
pattern_learn()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
pattern_library_compare()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
pattern_library_lock()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
pattern_library_merge()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
pattern_library_read()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
pattern_match()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
pattern_predict()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
pattern_predict_marker()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)

函数（选项）	所需许可
pex_color_layer_map()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
pex_qtf_layer_map()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
pex_qtf_layers()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
reduce_four_color_graph()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
reduce_three_color_graph()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
remove_vias()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
three_color()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
two_color()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
unified_fill(different_size_fill_spacing forbidden_layer merging_limit_exclude_layers merging_limit_exclude_layers_secondary min_fill_region_concave_distance projecting_layer_shrink remove_point_touch)	各档均可用 Apex (ICValidator-Manager-Apex) Apex (ICValidator-Manager-Apex) Apex (ICValidator-Manager-Apex) Apex (ICValidator-Manager-Apex) Apex (ICValidator-Manager-Apex) Apex (ICValidator-Manager-Apex) Apex (ICValidator-Manager-Apex)
unified_fill_region(deangle_max_deviation region_type = UF_BLOCKAGE_REGION)	各档均可用 Apex (ICValidator-Manager-Apex) Apex (ICValidator-Manager-Apex)
xref_to_double_property()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
xref_to_double_property()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)
xref_to_property()	Elite (ICValidator-Manager) 或 Apex (ICValidator-Manager-Apex)

2.4.6 多核许可

Multicore Licensing

IC Validator 的 Apex、Elite 与 Base 多核许可需求如下。

Apex 多核许可

Apex Multicore Licensing 工具可使用一个或多个 ICValidator-Manager 许可证运行 (Apex 档对应 ICValidator-Manager-Apex)。每个许可证最多允许使用四个 CPU。例如：

100 CPU 作业需要 25 个 Apex (ICValidator-Manager-Apex) 许可证

Elite 多核许可

Elite Multicore Licensing 工具可使用一个或多个 ICValidator-Manager 许可证运行。每个许可证最多允许使用四个 CPU。例如：

100 CPU 作业需要 25 个 Elite (ICValidator-Manager) 许可证

Base 多核许可

Base Multicore Licensing 工具可使用一个或多个 ICValidator-Manager-2020 许可证运行。每个许可证最多允许使用两个 CPU。例如：

100 CPU 作业需要 50 个 Base (ICValidator-Manager-2020) 许可证

2.4.7 令牌许可

Token Licensing

基于令牌 (token) 的许可描述签出多个许可证的方式。令牌许可与多核许可类似：根据 CPU 数量、按既定比例消耗多个 ICV Manager 许可证。

下列组成部分共同决定一次作业所需的许可证总数：

令牌许可证总数 = (Per-CPU Flow × Multicore) + Per-Session Functionality

- **Per-CPU Flow Functionality:** 视作业类型而定，需要一定数量的许可证，并与多核许可相乘。
- **Multicore:** 多核结果是 Per-CPU 功能的既定乘数。
- **Per-Session Functionality:** 部分功能不与多核相乘；无论多核如何，它们额外消耗固定数量的许可证。

2.4.8 超线程

Hyperthreading

IC Validator 签出许可证，直到 CPU 数量等于物理核数。例如：

100 CPU 作业需要 25 个 Apex (ICValidator-Manager-Apex) 许可证

超线程示例

Hyperthreading Example 一台启用超线程的 20 核主机呈现为 40 个 CPU，其中 4 个 CPU = 1 个许可证：

```
8 CPUs (-host_init 8) : Consumes 2 licenses
12 CPUs (-host_init 12) : Consumes 3 licenses
16 CPUs (-host_init 16) : Consumes 4 licenses
20 CPUs (-host_init 20) : Consumes 5 licenses
24 CPUs (-host_init 24) : Consumes 5 licenses (4 CPUs reserved for Threading)
28 CPUs (-host_init 28) : Consumes 5 licenses (8 CPUs reserved for Threading)
32 CPUs (-host_init 32) : Consumes 5 licenses (12 CPUs reserved for Threading)
36 CPUs (-host_init 36) : Consumes 5 licenses (16 CPUs reserved for Threading)
40 CPUs (-host_init 40) : Consumes 5 licenses (20 CPUs reserved for Threading)
```

2.5 资源需求

Resource Requirements

初始数据库文件以及 IC Validator 创建与使用的文件决定了系统需求。在对将要检查的设计有一定了解时，可估算物理内存、磁盘空间以及可能的交换空间（swap space）需求。

IC Validator 发行包中的 `contrib` 目录提供用于分析 IC Validator 性能的脚本。该目录位于发行安装目录顶层。这些脚本不受支持（unsupported）；更多信息见 `contrib` 目录中的 README 文件。

2.5.1 空间需求

Space Requirements

IC Validator 在由设计数据库生成并存放于 `group` 目录的 `group` 文件上执行版图检查。工具首先创建主 `group` 文件（primary group files）：runset 中每个 `assign` 函数对应的层各一个文件。每个主 `group` 文件是描述该层在整个设计中存在情况的数据库，并保持全部层次。随着运行推进，某些检查会生成输出，这些输出也以 `group` 文件形式写入。例如，`group` 目录中的文件可能包括：

HRCHY.ERROR	inst_indx.dat	pgate.group	toxcont.group
cont.group	met1.group	poly.group	via.group
gate.group	met2.group	psel.group	well.group
inst_data.dat	ngate.group	tox.group	

一般规则：若初始数据库为 GDSII 格式且设计层次风格良好，应为 `group` 文件创建预留至少相当于初始数据库两到三倍的空间。若设计接近扁平（flat），或存在不希望的数据交互，`group` 文件创建可能需要初始数据库四到六倍的空间。因此，对 10 MB 的 GDSII 数据库，若设计具有层次性，`group` 文件至少需要 20–30 MB；扁平设计则至少需要 40–60 MB。

除 `group` 文件外，IC Validator 运行的其他重要输出还包括运行摘要文件与输出数据库文件。摘要文件为 `cell.LAYOUT_ERRORS` 与 `cell.sum`，均为 ASCII 日志，记录 runset 中所命名 `cell` 的运行结果。

`cell.LAYOUT_ERRORS` 与输出数据库的大小取决于运行中遇到的错误数量。与 `group` 文件类似，若设计具有层次性，图形输出数据库会小于更扁平的设计：层次设计只需对重复模块内部的错误报告一次，而扁平设计则对模块的每个实例都报告错误。

2.5.2 内存使用

Memory Usage

可通过合理的错误定位策略管理错误输出报告所需的内存：先对设计中的小模块运行 IC Validator 并处理错误；修复后再对更高层次模块运行。这样可使 `cell.LAYOUT_ERRORS` 与输出数据库保持合理大小，并自下而上控制错误输出存储所需内存。

IC Validator 完成检查还需要物理内存与交换空间。工具尽量将当前检查所需的全部数据载入内存；物理内存溢出部分使用交换空间。一般来说，一旦需要访问交换空间，性能就会下降，尤其当交换空间分布在流量较大的网络上时。若物理内存足以载入全部数据并在不访问交换空间的情况下完成检查，可获得最大吞吐。

对层次设计，内存约为 GDSII 文件大小三到四倍的机器通常表现良好；扁平设计需要更多硬件。应设置足够的交换空间，使物理内存溢出时不至于耗尽交换空间而导致运行终止。

输出 `cell.sum` 文件列出完成各项检查所用的内存以及缺页（page faults）次数。若某些检查所需内存超过机器分配量并导致大量数据换出到磁盘，后续运行可能需要更大内存的机器。例 2.1 给出了内存使用报告示例。下一节「运行时间与内存使用报告」说明摘要文件中报告的运行时间与内存用法。

```
length_edge() at sf.rs:162
gate_gt_3_5 = length_edge(gate_e, distance > 3.5)
Function: length_edge
```

```

Inputs: gate_e.edgelaye.0001
Outputs: gate_gt_3_5.edgelaye.0001
8 unique edge sets written.
Check time = 0:00:00 User=0.00 Sys=0.00 Mem=0.015 GB
external2() at sf.rs:166
external2(gate_le_1_5, POLY, distance < 0.15, extension = RADIAL,
look_thru = COINCIDENT)
Comment: "R.POLY.G1: Min Space POLY to Gate for Gate Width <=1.5 must be > 0.15"
Function: external2
Inputs: gate_le_1_5.edgelaye.0001, POLY.polygonlayer.0002
WARNING - 6 spacing violations found.
Check time = 0:00:00 User=0.00 Sys=0.00 Mem=0.016 GB
external2() at sf.rs:169
external2(gate_gt_1_5_le_2_5, POLY, distance < 0.25, extension = RADIAL,
look_thru = COINCIDENT)
Comment: "R.POLY.G2: Min Space POLY to Gate for 1.5 < Gate Width <=2.5 must be > 0.25"
Function: external2
Inputs: gate_gt_1_5_le_2_5.edgelaye.0001, POLY.polygonlayer.0002
WARNING - 6 spacing violations found.
Check time = 0:00:00 User=0.00 Sys=0.00 Mem=0.016 GB
external2() at sf.rs:172
external2(gate_gt_2_5_le_3_5, POLY, distance < 0.35, extension = RADIAL,
look_thru = COINCIDENT)
Comment: "R.POLY.G3: Min Space POLY to Gate for 2.5 < Gate Width <=3.5 must be > 0.35"
Function: external2
Inputs: gate_gt_2_5_le_3_5.edgelaye.0001, POLY.polygonlayer.0002
WARNING - 3 spacing violations found.
Check time = 0:00:00 User=0.00 Sys=0.00 Mem=0.016 GB
external2() at sf.rs:175
external2(gate_gt_3_5, POLY, distance < 0.45, extension = RADIAL,
look_thru = COINCIDENT)
Comment: "R.POLY.G4: Min Space POLY to Gate for Gate Width > 3.5 must be > 0.45"
Function: external2
Inputs: gate_gt_3_5.edgelaye.0001, POLY.polygonlayer.0002
WARNING - 6 spacing violations found.
Check time = 0:00:00 User=0.00 Sys=0.00 Mem=0.016 GB

```

代码 2.1: cell.sum 文件中的部分内存使用信息示例

在网络系统上为获得最佳性能，应尽量将所需文件放在本地机器上以加快完成。运行作业的机器本地磁盘应足以容纳 Milkyway 库（或 GDSII / OASIS 文件）、正在创建的 group 文件，以及所有 ASCII 或物理错误文件。

2.5.3 运行时间与内存使用报告

Runtime and Memory Use Report

本节说明摘要文件中针对 IC Validator 运行各阶段报告的运行时间与内存使用。注意：部分处理不在累计内存报告范围内。

注意

一般而言，峰值或累计内存使用报告仅包含由 IC Validator 工具分配的内存。例如，Milkyway 内存由 Milkyway 库分配，因此不计入 IC Validator 峰值内存或机器累计内存报告。

表 2.3 给出摘要文件中时间与内存术语的定义。

表 2.3: 摘要文件中使用的术语

术语	定义
Time	墙上时间 (wall time)
User	计算所用 CPU 时间
Sys	I/O 所用 CPU 时间
Mem	最大内存使用

以下示例取自摘要报告的不同阶段：

1. Runset 编译阶段

- 未使用 runset 缓存时，报告类似：

```
Runset Compile Time=0:01:03 User=61.08 Sys=0.81 Mem=1.065 GB
```

- 使用 -rscache 启用 runset 缓存时，报告类似：

```
Cached Runset Compile Time=0:00:04 User=0.81 Sys=0.25 Mem=0.203 GB
```

2. 分布式处理设置阶段

```
Virtual Machine Init Time=0:00:02 User=1.09 Sys=0.12 Mem=1.065 GB
System Startup Time=0:00:02 User=0.00 Sys=0.52 Mem=1.065 GB
```

3. 预处理阶段

- 层次读取与优化：峰值内存报告不计 Milkyway 与 OASIS 内存。

```
Oasis memory used: 27.0M
Milkyway memory usage: 4714.789M
Reading hierarchy Time=0:00:10 User=8.29 Sys=2.87 Mem=0.207 GB
Updating hierarchy Time=0:00:00 User=0.07 Sys=0.00 Mem=0.056 GB
Exploding Time=0:00:00 User=0.44 Sys=0.02 Mem=0.057 GB
```

- 虚拟 cell (virtual cell) 配对阶段：

```
Pairing Iteration 1 Time=0:00:00 User=0.53 Sys=0.04 Mem=0.098 GB
Pairing Iteration 2 Time=0:00:00 User=0.54 Sys=0.05 Mem=0.097 GB
Pairing Iteration 3 Time=0:00:00 User=0.51 Sys=0.04 Mem=0.096 GB
Pairing Iteration 4 Time=0:00:00 User=0.45 Sys=0.03 Mem=0.098 GB
Pairing Time=0:00:01 User=2.03 Sys=0.16 Mem=0.098 GB
Combined VCELL Time=0:00:02 User=2.67 Sys=0.23 Mem=0.098 GB
Post VCell Time=0:00:00 User=0.12 Sys=0.00 Mem=0.060 GB
Prototype Pass 2 Time=0:00:00 User=0.04 Sys=0.00 Mem=0.061 GB
```

- Regioning 阶段：

```
Determine Region Time=0:00:00 User=0.02 Sys=0.00 Mem=0.059 GB
```

- Create Layer 阶段:

```
Create Layer Setup Time=0:00:00 User=0.06 Sys=0.01 Mem=0.058 GB
Hierarchy Cleanup Time=0:00:01 User=0.89 Sys=0.04 Mem=0.059 GB
```

4. 命令执行

- 命令统计报告:

```
Check Time=0:00:00 User=0.00 Sys=0.00 Mem=0.017 GB
```

- 读取错误层次:

```
Elapsed Time=0:00:00 User=0.08 Sys=0.00 Mem=0.024 GB
```

- text_net():

```
Total Text Time=0:00:00 User=0.05 Sys=0.00 Mem=0.064 GB
```

- extract_devices():

```
Device Connect Time=0:00:04 User=1.41 Sys=2.84 Mem=0.147 GB
```

- netlist():

```
Netlisting Time=0:00:03 User=1.63 Sys=0.68 Mem=0.069 GB
```

- write_milkyway(): 峰值内存报告不计 Milkyway 内存。

```
Milkyway memory usage: 4129.207M
Compression Time=0:05:07 User=275.34 Sys=27.80 Mem=3.529 GB
Writing Time=0:10:32 User=516.55 Sys=47.36 Mem=3.812 GB
```

- write_oasis(): 峰值内存报告不计 OASIS 内存。

```
Merging input layout from <input OASIS>
Merge Time=0:00:00 User=0.00 Sys=0.00 Mem=0.025 GB
Writing Time=0:00:00 User=70.00 Sys=0.00 Mem=0.025 GB
```

- write_gds():

```
Writing Time=0:00:00 User=0.00 Sys=0.00 Mem=0.025 GB
```

5. 主机摘要

- 每台主机的总运行时间与峰值内存（不计读写 Milkyway 与 OASIS 的内存）:

```
Host number 0 total check Time=0:04:40 User=182.60 Sys=37.42 Mem=0.791 GB
```

- 每台主机含预处理的总运行时间与峰值内存（不计读写 Milkyway 与 OASIS 的内存）:

```
Host number 0 total check and preprocess Time=0:04:56 User=195.38 Sys=40.62 0.791 GB
```

6. 错误写入摘要

```
Overall error storage time: User=0.06 Sys=0.11 Mem=0.014 GB
Generation Time=0:00:00 User=0.01 Sys=0.00 Mem=0.0147 GB
```

7. 总体内存与运行时间报告

- 每台机器的累计峰值内存（不计读写 Milkyway 与 OASIS）:

```
IC Validator Peak Machine Memory Report
igcae209      : Peak = 2.061 GB
igcaew008     : Peak = 3.813 GB
igcae191      : Peak = 1.272 GB
igcae177      : Peak = 1.396 GB
igcae188      : Peak = 1.403 GB
igcae205      : Peak = 1.269 GB
igcae178      : Peak = 1.399 GB
igcae174      : Peak = 1.404 GB
```

- 所有阶段与函数的总体运行时间与峰值内存（不计读写 Milkyway 与 OASIS）:

```
Overall engine Time=0:06:12 Highest command Mem=0.791 GB
```

- Master 内存使用（不计入峰值内存报告）:

```
Overall Master Mem=0.246 GB
```

2.5.4 操作系统

Operating Systems

IC Validator 支持多种硬件架构与操作系统。受支持操作系统列表见安装指南:

<http://www.synopsys.com/support/licensing-installation-computeplatforms/compute-platforms/release-specific-support.html>

2.5.5 系统限制

System Limits

若系统限制设置不当, IC Validator 运行中可能出错。例如, 下列命令为 Linux 上的 IC Validator 运行设置合适的系统限制（数值单位为 kilobytes）:

```
unlimit filesize
unlimit datasize
limit descriptors 1024
limit stacksize 8192
```

注意

`descriptors` 设置控制运行期间可同时打开的文件数量。

要验证主机上的限制, 请运行 shell 的 `limit` 命令。

2.5.6 网络环境

Networked Environments

在网络环境中访问文件（例如使用远程 `group` 目录）可能降低性能。您可能观察到 IC Validator 报告的墙上时间（`wall time`，总体时间）与总 CPU 时间（`user time + system time`）之间存在很大差异。

注意

这些差异也可能由下列原因造成：

- 同一系统上运行的其他进程；
- 系统开销（`system overhead`）。

NFS 挂载的读/写传输大小（`rsize` 与 `wsiz`）应至少为 32768。例如，对静态挂载设置：

```
% mount -t nfs -o rsize=32768,wsiz=32768 server:/path /mnt/nfs
```

Chapter 3

输出文件

Output Files

本章概述 IC Validator 设计规则检查（DRC）与版图对照原理图（LVS）的输出文件。

本章内容包括：

- 输出文件说明（Description of Output Files）
- 通用 DRC 与 LVS 输出文件（General DRC and LVS Output Files）
- LVS 专用输出文件（LVS-Specific Output Files）

3.1 输出文件说明

Description of Output Files

IC Validator 产生的输出文件取决于所执行的运行类型。

若执行 DRC，工具生成错误文件与摘要文件。错误文件列出设计中的 DRC 错误；摘要文件列出设计信息与已完成运行的信息。

若执行器件提取（device extraction），工具生成错误文件、摘要文件与版图网表（layout netlist）文件。版图网表在 LVS 比较中与原理图网表对照。DRC 与器件提取运行都会在 `run_details` 与 `run_details/hierarchy` 目录中生成树结构（tree structure）、虚拟 cell（virtual cell）与技术（technology）文件。

若执行 LVS 比较，工具生成错误文件、日志文件与等价性摘要（equivalence summary）文件。错误文件列出版图中与原理图不匹配的网络与器件。

错误、摘要与版图网表文件写入当前工作目录。另有位于 `run_details` 目录的附加文件，有助于调试 LVS 错误。

所有输出文件命名为 `cell.abbreviation`，其中 `cell` 为 `library()` 函数中 `cell` 参数所指定的名称；`cell` 之后的扩展名是文件用途的缩写。例如，`account` 文件名为 `cell.acct`。

3.2 通用 DRC 与 LVS 输出文件

General DRC and LVS Output Files

图 3.1 显示 DRC 与 LVS 运行共有的输出文件与目录。各文件说明见以下各小节。

(原书 *Figure 4: Output Files and Directories Common to DRC and LVS*)
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 3.1: DRC 与 LVS 共有的输出文件与目录

3.2.1 结果文件

Results File

IC Validator 创建 RESULTS 文件 `cell.RESULTS`，提供运行关键区域的结果以及对 `search/grep` 关键结果的摘要。该文件还包含本次运行所用的 IC Validator 命令行选项，是分析 IC Validator 运行结果的起点。

`cell.RESULTS` 文件分为下列各节：

- **Results Header**（结果头）

- 若 DRC 运行完成且无违例，工具返回单行：`RESULTS: CLEAN`。
- 若 DRC 运行完成但有违例，工具返回单行：`RESULTS: NOT CLEAN`。
- 若 LVS 运行完成，头节返回两行结果：
 - * 第一行：`LVS Compare Results: PASS | FAIL`；
 - * 第二行：`DRC and Extraction Results: CLEAN | NOT CLEAN`。
- 未完成的 DRC 或 LVS 运行返回单行：`RESULTS: RUN ABORTED`。

- **Execution**（执行信息）

本节分两部分：

- 第一部分打印 IC Validator 横幅与版权信息，以及工具的调用方式。
- 第二部分为基本运行信息，包括：输入库文件名、顶层 `cell` 名、输入库版图格式、`runset` 文件名、工作目录路径、`run details` 目录路径、LVS 错误文件名、`layout` 错误文件名、用户名，以及 IC Validator 运行的起止时间。

- **Results Summary: DRC Run**（DRC 运行结果摘要）

本节分两部分：

- 第一部分报告规则与 DRC 错误统计，包括：已运行规则总数、未执行规则数、含违例的规则数、违例总数。详细违例数据见 `cell.LAYOUT_ERRORS`。
- 第二部分为表格，列出各规则名及其违例总数。仅当 `run_options()` 的 `verbose_results_file` 参数包含 `ERRORS` 选项时才输出。规则名根据 `error_options()` 的 `rule_name_delimiter` 参数从规则注释中提取。若运行了 DRC 错误分类（`error classification`），违例计数中排除已豁免（`waived`）的违例。未执行的规则显示状态 `Not Executed`。规则下各函数的违例计数一般不报告，但无归属规则的函数除外：这些函数缩进两格，报告在默认规则名 `Violation` 之下。违例计数前缀为 `v =`，便于搜索特定违例数。

- **Results Summary: LVS Run**（LVS 运行结果摘要）

本节分两部分：

- 第一部分为 LVS 比较统计表，报告：LVS 比较结果、顶层等价点（`top equivalence points`）、已检查 `black-box cell` 数、通过的 `black-box cell` 数、失败的 `black-box cell` 数、已检查等价点数、成功等价点数、失败等价点数。详细比较违例见 `cell.LVS_ERRORS`。
- 第二部分列出所有等价点与 `black-box` 对的 LVS 结果：先列失败的等价点与 `black-box cell`，再列通过的。可用 `search/grep` 快速查找。

• Assign Layer Statistics (Assign 层统计)

本节分两部分：

- 第一部分报告四项：已用 assign 层总数、空的已用 assign 层数、非空的已用 assign 层数、被剪枝 (pruned) 的 assign 层数。
- 第二部分为表格，列出各层名、该层的层次与扁平多边形计数 (若被剪枝则显示 Pruned)，以及层或 datatype 赋值。仅当 run_options() 的 verbose_results_file 包含 ASSIGNS 时才输出。表格分为三部分：1) 非空已用层；2) 空的已用层；3) 剪枝层。每层的 assign 语句包含 layer/datatype 赋值及用户提供的其他赋值选项。

• Run Configuration (运行配置)

本节包含下列七部分：

- run_options(): 报告 verbose_results_file 参数。
- hierarchy_options(): 报告 delete 参数。
- error_options(): 报告 error_limit_per_check、match_errors、report_empty_violations、clean_classifications。
- drc_black_box_options(): 报告 black_box_cells、black_box_region_cells、keepout_layers_check、keepout_violation_cor
- Production Environment Variables: 列出所有已设置的生产环境变量，并在列表末给出总数。
- Debug Environment Variables: 列出所有已设置的调试环境变量，并在列表末给出总数。
- Distributed processing: 报告分布式核数与 turbo，随后列出各主机及其规格 (型号、CPU、核数、RAM、交换空间)；主机名旁还显示该主机上的分布式进程数与 CPU 数。

• Performance Statistics (性能统计)

本节报告五项：

- IC Validator 运行时间；
- 单命令峰值内存；
- 峰值磁盘用量；
- 分布式主机峰值内存 (表格：主机、该主机上的分布式处理与 CPU 数、主机峰值内存与 RAM)；
- 总体分布式利用率 (overall distributed utilization)。

3.2.2 错误文件

Error File

IC Validator 在 DRC 或网表提取运行中创建错误文件 cell.LAYOUT_ERRORS。这类错误称为 layout errors。LAYOUT_ERRORS 文件包含运行结果、错误摘要列表以及更详细的错误列表。

文件顶部标明 CLEAN 或 ERRORS，便于快速判断设计是否存在 layout 错误。

接下来是摘要节：给出函数注释、函数本身，以及若有错误则给出该函数的错误数。

最后是详情节。每条错误报告包含：标识所执行检查的错误描述；发生错误的结构 (structure) 名；错误位置或包围盒 (bounding box)；以及特定检查的其他信息。位置坐标按层次报告，即相对于所命名结构，且对每个被引用结构只报告一次。也可用 VUE 调试工具以图形方式查看这些错误。

运行期间，LAYOUT_ERRORS 会随时更新结果与摘要节。运行结束时写出最终版本，并按 error_options(report_error_details) 的规定包含详情节。

下列 LAYOUT_ERRORS 片段对应 metal1 (Met1) 最小面积规则。runset 中编码如下：

```
//R.Met1.A
{
  @"R.Met1.A:For Metal 1, minimum area must be 0.04";
  area( Met1, value < 0.04);
};
```

LAYOUT_ERRORS 对该规则的报告示例如下：

```

LAYOUT_ERRORS RESULTS: ERRORS

#####  #####  #####  #####  #####
#      #  #  #  #  #  #  #  #
#####  #####  #####  #      #  #####  #####
#      #  #  #  #  #      #  #  #      #
#####  #  #  #  #  #####  #      #  #####

=====

Library name:    ../../lab_data/INLIB_TRAINING.GDS
Structure name:  top
Generated by:    IC Validator <platform> <release> <date>

Runset name: icv runset.rs
User name:  juser
Time started: 2016/03/02 07:35:12AM
Time ended: 2016/03/02 07:35:51AM

Called as: icv runset.rs
                ERROR SUMMARY

R.Met1.A: Minimum area must be 0.04
area ..... 1 violation found.

                ERROR DETAILS

-----
R.Met1.A: Minimum area must be 0.04
-----

sf.rs:238:area
- - - - -
Structure ( lower left x, y ) ( upper right x, y ) Area
- - - - -
top      (37.2700, 4.0300)  (37.4800, 4.1700)  0.0294

```

3.2.3 摘要文件

Summary File

IC Validator 在 DRC 或网表提取运行中创建摘要文件 `cell.sum`。

若使用 `-verbose` 命令行选项，打印到屏幕的信息与写入摘要文件的信息相同。区别在于：屏幕输出按工具处理顺序打印，而摘要文件内容按 `runset` 的顺序排序。

3.2.4 规则文件

Rules File

待执行的规则写入 `run_details` 目录下的 `cell.rules`。在完成各类 `runset` 选择之后仍留在 `runset` 中的违例名与违例注释会报告到 `cell.rules`。`runset` 选择的例子包括：预处理器指令、可选规则（selectable rules）、增量层（incremental layers）、以及仅运行剪枝（run-only pruning）。

违例名与违例注释从最终解析树（parse tree）中的所有规则采集。由于运行时条件命令（if/then/else 以及 for/foreach/while 循环）可能仍保留，最终解析树中的规则不保证一定会执行。最终解析树中所有命令都会输出规则信息，包括因运行时条件而未执行的命令。

违例名与违例注释未必总能静态解析；此时输出文件提供文件名与行号，而非规则名。

若 `runset` 规则没有 `@=` 违例块，输出中仅出现违例注释。

默认注释为 `Violation`；规则使用默认注释时，输出中出现 `@ "Violation"`。

下列为 `cell.rules` 文件示例：

```

                                ICV_Master (R)
                                Copyright (c) 1996 - 2015 Synopsys, Inc.
                                This software and the associated documentation are proprietary to
                                Synopsys, Inc. This software may only be used in accordance with the terms
                                and conditions a written license agreement with Synopsys, Inc. All
                                other use, reproduction, or distribution of this software is strictly
                                prohibited.

Called as: icv runset.rs -i test.gds
-----
User name:      synopsoid
Layout format:  GDSII
Input file name: test.gds
Top cell name:  top
Time started:   2015/11/03 02:43:08PM
-----
                                Rules to be Executed
-----
                                Unresolved violation name/comment found at runset.rs:100
@ "Rule comment with no violation block"
@ "Violation"
VIOL_101 @= { @ "VIOL_101_comment" }
VIOL_102 @= { @ "VIOL_102_comment" }
VIOL_103 @= { @ "VIOL_103_comment" }
VIOL_104 @= { @ "VIOL_104_comment" }
VIOL_105 @= { @ "VIOL_105_comment" }
```

3.2.5 技术文件

Technology File

IC Validator 创建技术文件 `cell.tech`，显示本次运行所设置的选项，包括默认值与用户指定值。技术文件包含三节：exploded cells、deleted cells，以及 placement explode 统计。

3.2.6 树结构输出文件

Tree Structure Output File

IC Validator 在 DRC 或网表提取运行中创建名为 `cell.tree` 的树结构文件。树结构文件包含设计层次树信息，以及对设计树中每个 cell 的引用统计。`cell.tree0` 表示原始数据；`cell.tree1` 在所有 `explode` 选项完成后创建。使用 `hierarchy` 选项时会创建更多树文件。编号 N 最大的 `cell.treeN` 是本次运行实际处理的层次。若 `run_options()` 的 `print_preprocess_files` 参数指定 `TREE`，则打印树文件；`compress_tree_files` 参数决定写入磁盘时是否压缩。

3.2.7 虚拟 Cell 文件

Virtual Cell File

虚拟 cell 文件 `cell.vcell` 包含配对 (pairing)、创建集合 (sets) 以及虚拟 cell explode 的信息，具体取决于 `runset` 选项与设计数据。文件包含 `Pair`、`Set` 与 `Exclude` 各节。

Pair 节

Pair Section Pair 节报告每次虚拟 cell pass 迭代中创建的 cell 对。`hierarchy_options()` 中 `pairs` 参数的 `iterate_max` 选项设定最大迭代次数。

Set 节

Set Section Set 节报告每次虚拟 cell pass 迭代中创建的集合。`hierarchy_options()` 中 `sets` 参数的 `iterate_max` 选项设定最大迭代次数。

Explode 节

Explode Section Explode 节显示为将相互作用的 placement 汇集到共同父级之下而被 `explode` 的所有分支。更多信息见 *IC Validator Reference Manual* 中 `hierarchy_options()` 的 `explode` 参数。

3.2.8 分布式日志文件

Distributed Log File

分布式日志文件以 `runset` 名加 `dp.log` 扩展名命名，所含信息多于摘要文件：包括函数何时发送到某主机，以及主机完成该函数后产生的结果。分布式日志按执行顺序排列，而非按 `runset` 顺序。

3.2.9 分布式统计文件

Distributed Statistics File

分布式统计文件以 `runset` 名加 `dp.stats` 扩展名命名，包含分布式处理统计。

3.2.10 分布式工作目录

Distributed Work Directory

分布式运行会在 `run_details` 下创建名为 `dp_work` 的工作目录，内含分布式运行所用的子目录与文件。若该目录在运行前已存在，则连同其全部内容被删除并重新创建。因此请勿在某次运行的 `run_details` 中把个别目录命名为 `dp_work`。

3.2.11 图形错误结构

Graphical Error Structures

IC Validator 生成错误数据库 PYDB，可在版图编辑器（如 VUE）中查看。其中包含位于设计规则违例附近的错误多边形与坐标。不同检查产生的图形可按 `runset` 规定写入错误结构中的不同层与 `datatype`。关于错误数据库以及用于分类错误的 `pydb_report`、`pydb_export` 实用程序，见「DRC 错误分类」（DRC Error Classification）。

错误数据库包含违例块生成的全部错误数据,以及 VUE 调试会话所需的全部信息。默认位置为 `run_details/pydb`；可用 `error_options()` 的 `db_path` 参数更改。该数据库支持数据挖掘，并可据此创建自定义报告。

注意

可在 IC Validator 运行完成之前开始审查错误数据库。

IC Validator 生成的错误结构层次模仿正在检查的数据库层次。因此，数据库中某结构发现的错误会出现在错误层次中的对应错误结构中。仅对含有错误、或引用了含错误结构的数据库结构生成错误结构。若未发现任何错误，则不创建错误层次。

错误结构默认以前缀 `ERR_` 命名，置于结构名之前。例如，`cell CLOCK` 的默认错误结构为 `ERR_CLOCK`。可在写入函数（如 `write_gds()`）中用 `error_cell_prefix` 参数重定义该前缀。

例如：

Database hierarchy	Error hierarchy
CHIP { ALU { BIT0 BIT1 } CLOCK { CLK1 CLK2 CLK3 } }	ERR_CHIP { ERR_ALU { ERR_BIT0 ERR_BIT1 } ERR_CLOCK { ERR_CLK1 ERR_CLK3 } }

在错误层次中没有 `ERR_CLK2`：要么原始 `CLK2 cell` 在预处理中被 `explode`，要么结构 `CLK2` 本身不含错误且不含含错误的结构。若先前含错误的 IC Validator 运行经修正后重跑，先前的错误层次结构会保留。若新运行未发现错误，工具不会覆盖这些结构，但新错误层次不会再引用这些先前含错误的 `cell`。

3.3 LVS 专用输出文件

LVS-Specific Output Files

图 3.2 显示 LVS 运行的输出文件与目录。各文件说明见以下各小节。

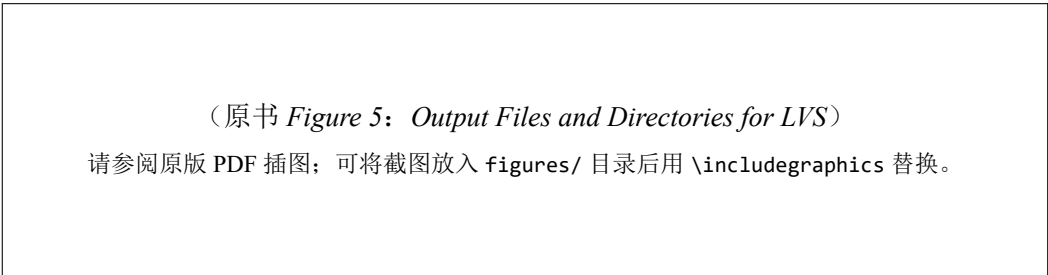


图 3.2: LVS 的输出文件与目录

3.3.1 LVS 调试文件

LVS Debugging File

`compare()` 函数创建调试文件 `cell.LVS_ERRORS`，指示原理图对照版图比较是否成功。该文件是调试 LVS 问题的起点。

根据比较是否成功，文件列出 PASS/FAIL 信息以及（若有）layout 错误。随后列出重要的失败等价点（按高/低优先级排序），并提供用于排查设计的诊断信息。

3.3.2 提取的版图网表文件

Extracted Layout Netlist File

每当 `runset` 中指定 `netlist()` 提取函数时，IC Validator 创建提取的版图网表文件 `cell.net`。该文件是版图数据库中所发现结构的层次化描述。

3.3.3 提取的版图 SPICE 网表文件

Extracted Layout SPICE Netlist File

每当 `runset` 中指定 `write_spice()` 时，IC Validator 创建提取的 SPICE 版图网表文件 `cell.sp`。该文件是版图数据库中所发现结构的层次化 SPICE 描述。更多信息见 *IC Validator Reference Manual* 中的 `write_spice()`。

3.3.4 Account 文件

Account File

IC Validator 创建 account 文件 `cell.acct`，列出层次中每个 cell 提取出的器件数量及其类型。所示累计计数（cumulative count）为该结构所含全部器件的总数。

3.3.5 包围盒文件

Bounding Box File

IC Validator 创建包围盒文件 `cell.bbox`，列出设计中 cell 总数以及每个 cell 的包围盒坐标。

3.3.6 等价文件

Equivalence Files

若未提供等价文件，LVS 比较运行会自动生成用于比较版图与原理图网表的等价文件。生成时，IC Validator 会识别某些多余、无助于比较的等价点。在全部分析完成后，工具生成 `equiv.run`，列出本次运行所用的全部等价 cell。更多信息见 *IC Validator Reference Manual* 中的 `equiv_options()`。

3.3.7 器件拉平摘要文件

Device Leveling Summary File

器件拉平摘要（device leveling summary，DLS）文件用于追溯器件按层次级别如何被提取。可通过 `run_details` 目录中的 `cell.dls` 诊断信息，确定器件被提取到错误层次（即被 leveled）的根本原因。

下列为 `cell.dls` 文件示例：

```
+-----+
|               ICV Device Leveling Summary               |
+-----+

Device Leveling Summary Results:

Body Layers          | Cell | Layer Interacted:
-----+-----+-----
ngt.polygonlayer.0001 A      nsd.polygonlayer.0001, NW.polygonlayer.0003
pgt.polygonlayer.0001 B      psd.polygonlayer.0001
...
```

器件拉平摘要文件显示：器件识别与属性计算之后，无法推回其原始 cell 的器件 body 层。成功推回的 body 层不出现在此文件中。文件显示：

- **Body Layers:** 被拉出原所在 cell 的器件 body 派生层。
- **Cell:** 被 leveled 器件的原始 cell。
- **Layer Interacted:** 与器件 body 交互从而导致 leveling 的层。若同一 body 层的多边形跨多个层次发生交互，则该列打印 body 层名。

3.3.8 比较日志文件

Compare Log File

IC Validator 在 LVS 运行中创建日志文件 `cell_lvs.log`。该比较日志包含与 `compare()` 函数相关的全部消息。

3.3.9 比较树文件

Compare Tree Files

`compare()` 产生两个树文件：`lay.tree` 给出版图网表的层次结构；`sch.tree` 给出原理图网表的层次结构。这些文件位于 `run_details/lvsflow` 目录。

3.3.10 比较目录输出文件

Compare Directory Output Files

LVS 比较的详细输出放在比较目录 `run_details/compare` 中。等价文件中的每个等价点在评估时，会在该比较目录下创建子目录，子目录名基于版图结构名。图 3.3 给出比较目录示例。



图 3.3: 比较目录结构示例

3.3.11 单个等价点摘要文件

Individual Equivalence Summary File

每个结构比较的结果写入摘要文件 `sum.schematic_module_name.layout_structure_name`, 通常称为 `sum.cell.cell`。该文件位于工作目录下的 `compare` 子目录中, 如图 3.3 所示。若 `compare()` 的 `remove_equiv_sum_files` 参数为 `NONE`, 或该等价点完成时无错误/警告, 则不生成此文件。摘要信息包括: 每种器件出现次数; 找到的网络总数; 以及因合并 (`merging`) 而移除或创建的器件与网络数。若无法匹配全部器件与网络, 摘要会列出差异。

`sum.cell.cell` 分为下列各节:

- Comparison Result (比较结果)
- Diagnostic Error (诊断错误)
- Unmatched Nodes (未匹配节点)
- Error Information (错误信息)
- Warning Information (警告信息)
- Preprocessing Information (预处理信息)
- Comparison Information (比较信息)
- Merged Node Information (合并节点信息)
- Cross-Referencing Information (交叉引用信息)
- Statistics Report (统计报告)

3.3.12 分区原理图网表

Partitioned Schematic Netlist

该部分比较所用的原理图数据写入 `sch.cell`, 位于工作目录下的 `compare` 子目录中 (见图 3.3)。若 `compare()` 的 `write_equiv_netlists` 为 `FAILED` 且该等价点完成时无错误/警告, 或该参数为 `NONE`, 则不生成此文件。该文件是 `explode` 非等价结构后的原理图网表, 通常对分析比较结果帮助不大。

3.3.13 分区版图网表

Partitioned Layout Netlist

该部分比较所用的版图数据写入 `lay.cell`, 位于工作目录下的 `compare` 子目录中 (见图 3.3)。若 `compare()` 的 `write_equiv_netlists` 为 `FAILED` 且该等价点完成时无错误/警告, 或该参数为 `NONE`, 则不生成此文件。该文件是 `explode` 非等价结构后的版图网表, 通常对分析比较结果帮助不大。

3.3.14 编辑错误

Editing Errors

错误结构写入数据库后, 即可开始编辑并修正错误。`LAYOUT_ERRORS` 列出每个错误的位置与结构名; 摘要文件 `cell.sum` 列出有错误的检查。由于 IC Validator 输出是层次化的, 无论结构被放置多少次, 每个错误只需修正一次。持续此过程直至消除全部错误。

Chapter 4

IC Validator Dashboard

IC Validator Dashboard

本章介绍 IC Validator Dashboard 的功能。

IC Validator Dashboard 包括以下各节：

- 简介
- 从 IC Validator 获取 Dashboard 电子邮件更新
- 独立启动 IC Validator Dashboard
- 初始目录设置
- IC Validator 图形用户界面

4.1 简介

Introduction

IC Validator Dashboard 可方便地查看运行进展、规则总数与已完成数、已报告的违例数量，以及 IC Validator 如何使用已分配的资源。

芯片临近完成时，物理验证运行处于流片关键路径上，确保按时完成愈发重要。可将 IC Validator Dashboard 与 IC Validator 的 `-host_elastic`、`-host_add` 和 `-host_remove` 命令行选项配合使用，以优化物理验证运行与硬件资源。

4.2 从 IC Validator 获取 Dashboard 电子邮件更新

Getting IC Validator Dashboard Email Updates From IC Validator

IC Validator Dashboard 可通过电子邮件发送更新。IC Validator 根据写入 `icv.log` 文件的已完成更新百分比自动发送。启动运行时在 IC Validator 命令行指定 `-email dashboard` 即可启用。例如：

```
icv ... -email dashboard ... runset.rs
```

邮件包含三页内容，其中 IC Validator Dashboard 显示按内存使用着色的 MRI 快照，以及违例密度热图。



图 4.1: IC Validator Dashboard 电子邮件内容

4.3 独立启动 IC Validator Dashboard

Starting IC Validator Dashboard Standalone

可用下列命令启动 IC Validator Dashboard:

```
icv_dashboard [-email ["user1,user2,..."]] [-V] [-h]
               [<run_details_path>|<dp_log_file>]
```

若不提供任何参数,Dashboard 会在 `./run_details` 目录中查找分布式处理日志文件,并打开图形用户界面(GUI)。命令行选项见表 4.1。

表 4.1: IC Validator Dashboard 命令行选项

选项	说明
-email ["user1,user2,..."]	向用户发送一次电子邮件更新后退出。
-V	打印产品版本后退出。忽略其他命令行选项。
-h	打印程序选项后退出。忽略其他命令行选项。
run_details_path dp_log_file	使用指定的 run details 目录或 DP 日志文件显示 GUI。默认值为 <code>./run_details/</code> 。指定 run details 目录时,使用该目录中的分布式处理日志文件。若该目录中存在多个分布式处理日志文件,则必须显式指定要使用的日志文件。

注意

以此方式启动 IC Validator Dashboard 时,请确保 PATH 变量包含 `$ICV_HOME_DIR/bin`,并且 `LM_LICENSE_FILE` 或 `SNPSLMD_LICENSE_FILE` 环境变量指向带有 IC Validator license 的有效 Synopsys license 服务器。

4.4 初始目录设置

Initial Directory Setting

IC Validator Dashboard 会自动创建初始目录以存放中间文件。可通过设置环境变量 `DASHBOARD_TMP` 指定初始目录路径。若该路径已设置且有效,则 Dashboard 在此路径下创建初始目录;否则依次尝试当前目录、用户主目录。若无法创建初始目录,将显示错误。

4.5 IC Validator 图形用户界面

The IC Validator Graphical User Interface

使用上一节所述命令行选项交互式启动 GUI 后，它会持续监视所找到或所指向的 IC Validator 运行。GUI 打开时显示 Summary 页面；从运行加载信息后，显示内容如图 4.2。

4.5.1 IC Validator Dashboard Summary 页面

IC Validator Dashboard Summary Page

Summary 页面包含五个面板，分别提供状态、CPU 使用、内存使用、运行进度与规则进度信息。将指针悬停在规则进度或状态“指示灯”上直至出现 tooltip，可查看更多信息。



图 4.2: IC Validator Dashboard Summary 页面



图 4.3: IC Validator Dashboard 状态面板

IC Validator Dashboard 状态

IC Validator Dashboard Status IC Validator Dashboard 有五种状态值：

- **Waiting:** Dashboard GUI 已启动，正在等待 IC Validator 运行开始以及分布式处理日志文件出现。
- **Running:** IC Validator 正在运行，Dashboard 正在等待更新。
- **Completed:** IC Validator 运行已成功完成。
- **Stopped:** IC Validator 运行因错误结束。
- **Unknown:** Dashboard 已读入不完整的分布式处理日志文件，但无法判断作业是否仍在运行。例如：加载了部分 DP 日志文件，而 IC Validator 已不再运行。

IC Validator 运行状态指示器

IC Validator Run Status Indicators 通常这些状态指示器为绿色。当 IC Validator 磁盘空间或内存不足时，相应指示器变为红色。若命令崩溃、主机被丢弃，或总运行时间中有相当大部分被延迟命令（deferred commands）占用，

则“Other”状态指示器变为红色。将指针悬停在指示器上可显示状态原因。

IC Validator 运行信息

IC Validator Run Information 状态面板的这一部分显示 IC Validator 的主要输入。

CPU 使用面板

CPU Usage Panel CPU 使用面板显示该 IC Validator 运行在所有主机上使用的 CPU 总数。以 elastic 模式运行时，作业运行期间可能释放部分 CPU；此时峰值使用量有可能超过已分配的 CPU 数。

(原书 *Figure 10: IC Validator Dashboard CPU Usage Panel*)
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 4.4: IC Validator Dashboard CPU 使用面板

内存使用面板

Memory Usage Panel Memory Usage 面板显示当前内存使用最多的主机的内存使用情况。当为 IC Validator 运行分配了多台主机时，该主机在运行过程中可能发生变化。

(原书 *Figure 11: IC Validator Dashboard Memory Usage Panel*)
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 4.5: IC Validator Dashboard 内存使用面板

运行进度面板

Run Progress Panel Run Progress 面板显示自运行开始以来的时长 (elapsed time) 以及 IC Validator 已完成操作的百分比。进度条以图形方式表示完成百分比。

规则进度面板

Rule Progress Panel Rule Progress 面板显示已执行的规则数以及已生成的错误数。如图 4.2 所示，将指针悬停在进度条上时，tooltip 会显示精确数值。

在 Explorer 模式下运行 IC Validator 时，Summary 页面的 Rule Progress 面板会显示最近的 Explorer Tier 消息。将指针悬停在消息上可显示 Tooltip，以查看更详细信息。

(原书 *Figure 12: IC Validator Dashboard Run Progress Panel*)
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 4.6: IC Validator Dashboard 运行进度面板

(原书 *Figure 13: IC Validator Dashboard Rule Progress Panel*)
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 4.7: IC Validator Dashboard 规则进度面板

4.5.2 IC Validator Dashboard CPU History 页面

IC Validator Dashboard CPU History Page

如图 4.8 所示，CPU 页面显示按主机着色编码的总 CPU 使用历史。上方面板是随时间变化的图：显示已分配的 CPU 总数以及各主机正在使用的 CPU 数。下方面板作为图例，并显示各主机的当前状态快照。将指针悬停在不同元素上会出现带有附加信息的 tooltip。

(原书 *Figure 14: IC Validator Dashboard CPU History Page*)
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 4.8: IC Validator Dashboard CPU History 页面

4.5.3 IC Validator Memory History 页面

IC Validator Memory History Page

如图 4.10 所示，Memory 页面显示按主机的内存使用历史。上方面板显示各主机随时间的内存使用。下方面板显示各主机内存使用的当前快照：实心条表示当前内存使用，彩色轮廓表示峰值内存使用，黑色轮廓表示主机的总物理内存。若使用 IC Validator 的 `-host_memory` 命令行选项指定了主机内存限制，则黑色轮廓中高于该限制的部分以灰色填充。将指针悬停在不同元素上会出现带有附加信息的 tooltip。

（原书 *Figure 15: CPU History Page When Job Completes*）
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 4.9: 作业完成时的 CPU History 页面

（原书 *Figure 16: The IC Validator Dashboard Memory History Page*）
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 4.10: IC Validator Dashboard Memory History 页面

4.5.4 IC Validator Dashboard 选项

IC Validator Dashboard Options

单击菜单栏上的邮件图标可打开 **IC Validator Dashboard – Options** 对话框。可在此选择最多三种不同的电子邮件通知选项。基于时间的邮件在 GUI 打开期间每 10 分钟发送一次。基于事件的邮件在每次打印新进度行时发送。若 IC Validator 以 `-email dashboard` 命令行选项启动，则该框为勾选状态。取消勾选可禁用基于事件的邮件；也可在运行中的 ICV 作业完成或失败时发送通知邮件。

（原书 *Figure 17: IC Validator Dashboard - Options Dialog Box*）
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 4.11: IC Validator Dashboard – Options 对话框

Dashboard 电子邮件包含 GUI 各选项卡所显示信息的快照。

Chapter 5

IC Validator Live DRC

IC Validator Live DRC

本章介绍在 Custom Compiler、IC Compiler II、Fusion Compiler 与 Cadence Virtuoso 工具中使用 IC Validator Live DRC 的方法。

IC Validator Live DRC 工具包括以下各节：

- 关于 IC Validator Live DRC 工具
- 在 Custom Compiler 中使用 IC Validator Live DRC
- 在 Virtuoso 中使用 IC Validator Live DRC
- 在 IC Compiler II 与 Fusion Compiler 中使用 IC Validator Live DRC
- 设置预编译缓存目录
- 在 IC Validator 中复现 IC Validator Live DRC 运行

5.1 关于 IC Validator Live DRC 工具

About the IC Validator Live DRC Tool

先进工艺节点与大型设计上的长 DRC 运行时间会影响生产力。IC Validator Live DRC 工具使您能够直接在版图编辑器中运行 DRC，从而主动修复违例，最终得到更干净的版图并缩短签核 DRC 运行时间。

IC Validator Live DRC 工具使用 IC Validator 签核规则文件，因此检查较全面。但由于检查范围限于视口（viewport）与可见层，运行时间显著缩短。IC Validator Live DRC 工具支持在 Custom Compiler™、IC Compiler™ II、Fusion Compiler™ 以及 Cadence® Virtuoso® 版图编辑器中使用。

5.2 在 Custom Compiler 中使用 IC Validator Live DRC

IC Validator Live DRC With Custom Compiler

Custom Compiler 中的 IC Validator Live DRC 允许设计人员对活动版图运行 IC Validator 物理验证检查。使用与 IC Validator 相同的 runset 与文件即可获得即时 DRC 结果。诸如 recipe 等高级选项允许创建规则子集，并可从下拉菜单中选择执行。为便于使用，违例直接显示在 Custom Compiler 中，无需单独窗口查看。以下各节说明如何设置环境、执行运行以及查看违例。

5.2.1 设置环境

Setting Up the Environment

要在 Custom Compiler 中运行 IC Validator Live DRC，请按下列步骤操作：

1. 设置环境
- 启动 Custom Compiler 前须完成部分 IC Validator 环境变量设置。要加载 IC Validator Live DRC 菜单，必须设置两个环境变量：
- 将 ICV_HOME_DIR 设为 IC Validator 安装目录
 - 将 PATH 设为包含 \$ICV_HOME_DIR/bin

例如：

```
setenv ICV_HOME_DIR </path/to/icv>
setenv PATH $ICV_HOME_DIR/bin:$PATH
```

要使 IC Validator Live DRC 工具栏可见，选择 **Window > Toolbars > DRC**。

2. 打开 IC Validator Live DRC Setup 窗口
- 要通过 Verification 菜单访问 IC Validator Live DRC，选择 **Tools > Verification > Live DRC**，或单击相应工具栏按钮。
3. 所需文件与设置
- 在 Live DRC Setup 对话框中，选择下列选项卡之一：**Main**、**Control Variables**、**Custom Options**、**Include Paths**。
- a. Main 选项卡

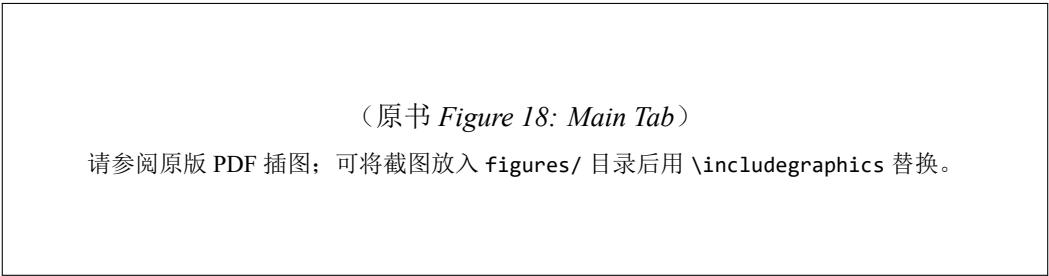


图 5.1: Main 选项卡

表 5.1: Live DRC Setup – Main 选项卡设置

设置	说明
Run Dir	选择运行 IC Validator Live DRC 的目录。输出文件将存放于此目录。
Area	执行 DRC 检查的区域：设计（design）、视口（viewport）或区域（region）。区域由矩形坐标（微米）按 [x1, y1, x2, y2] 顺序指定，例如 [-15.5, -20.0, 15.5, 20.0]。单击交互选择按钮可在设计中绘制选择区域。
Layout Format	对于 OpenAccess 格式，验证在当前 cell view 上运行。

表 5.1: Live DRC Setup – Main 选项卡设置（续）

设置	说明
	对于 Stream 与 OASIS 格式，工具导出 Stream 或 OASIS，并对生成的 GDSII 或 OASIS 文件运行 IC Validator Live DRC。请设置： • Format: OpenAccess、Stream 或 OASIS——将用作输入的版图 cellview 导出到您指定的 File。 • 单击 Config 按钮打开 Export Stream 或 Export OASIS 对话框。对话框选项设置见 <i>Custom Compiler Environment User Guide</i> 中 “Exporting Design Data in GDSII Stream Format” 或 “Exporting Design Data in OASIS Format” 两节。您指定的选项与 PVE（pdk.xml 配置中的 preferences）集成。
Job Parameters	选择下列项： • Runset: 用于 IC Validator Live DRC 的设计规则（runset）文件。要选择规则子集，单击相应按钮。若规则文件自加载后已修改，请重新打开对话框并单击相应按钮以加载修改后的文件。 • Arguments: 控制 IC Validator Live DRC 行为的命令行参数。 无需选择 Tool 与 Job Class。 Launch Debugger: 进程完成时启动（或更新已有）Error Viewer。 在 Background 模式下，可在 IC Validator Live DRC 于后台检查时继续使用 Custom Compiler。使用 preference xtICVLiveNotifyWhenComplete 可在检查完成时获得通知对话框。

要选择 runset（规则）文件中的特定规则，单击相应按钮。在 Live DRC Rule Selection 对话框中选择要用于规则检查的规则，然后单击 **OK**。

在过滤框中，可输入要选择的项或规则的完整或部分名称，支持简单字符串匹配、glob 风格模式或正则表达式；由 General Options 对话框中的 Filters 选项控制。要清除过滤框，单击文本字段中所示的 **X**。（见 *Custom Compiler Environment User Guide* 的 “Entering General Option Settings” 一节。）

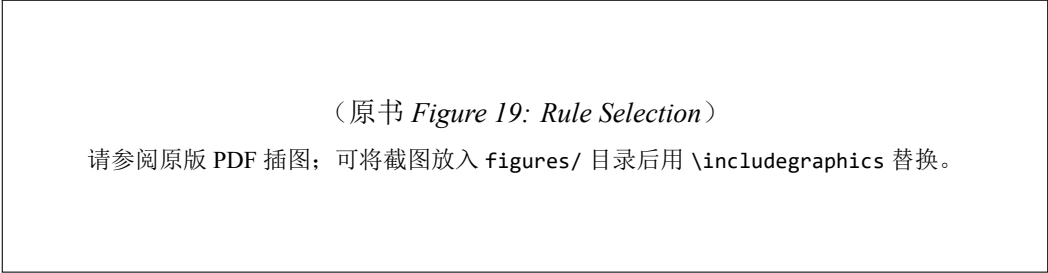


图 5.2: 规则选择

- b. **Control Variables** 选项卡
- 在 Control Variables 选项卡中，查看并按需修改本次 DRC 运行的 DRC runset 控制变量当前值。控制变量由 PDK 预置。要修改预置值，用非选择（备用）鼠标键单击，并编辑输入字段。
- 本对话框中的控制变量设置与 DRC Setup and Run 对话框中的设置类似。更多信息见 *Design Checking and Physical Verification User Guide* 的 “Control Variables Tab” 与 “Using the Integrated Tools for Design Rule Checking” 两节。
- c. **Custom Options** 选项卡
- 层映射文件将 OpenAccess（OA）层-purpose 名称映射到 GDSII 或 OASIS 的层号与数据类型。IC Validator Live DRC 在增量 DRC 中使用该文件。

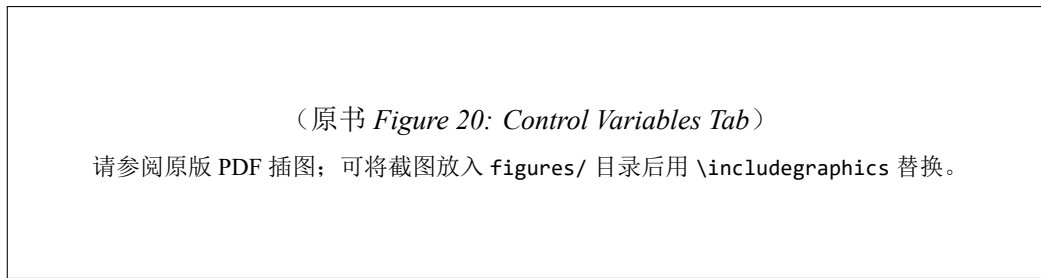


图 5.3: Control Variables 选项卡

可在 Custom Options 选项卡中指定 Layer Map 文件。此外，可添加 Options 以解析 IC Validator Live DRC runset。选项列表见 *Design Checking and Physical Verification User Guide* 的 “Using the Integrated Tools for Design Rule Checking” 一节。

可使用与 stream out GDS 或 OASIS 文件时相同的层映射文件。这些文件常可从先前定义的 preferences 中取得，在 console 中输入：

```
db::getPrefValue dbExportStreamLayerMapFile
```

或

```
db::getPrefValue dbExportOasisLayerMapFile
```

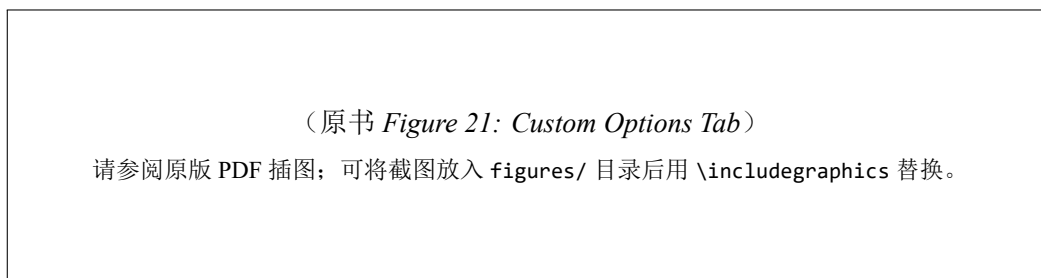


图 5.4: Custom Options 选项卡

d. Include Paths 选项卡

在 Include Paths 选项卡中，可指定包含在 DRC 操作期间额外传递给 IC Validator Live DRC 的文件的文件夹。

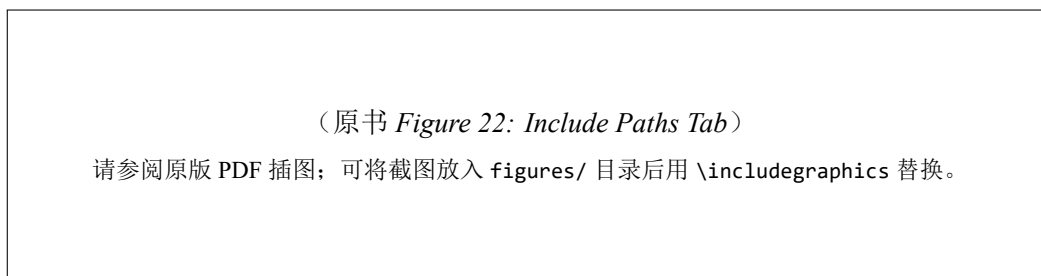


图 5.5: Include Paths 选项卡

5.2.2 查看作业进度

Viewing Job Progress

可在 job monitor 中查看 IC Validator Live DRC 作业，其中显示正确的 Status 与 Run Description。在选中的 IC Validator Live DRC 作业上右键单击，出现菜单，可：

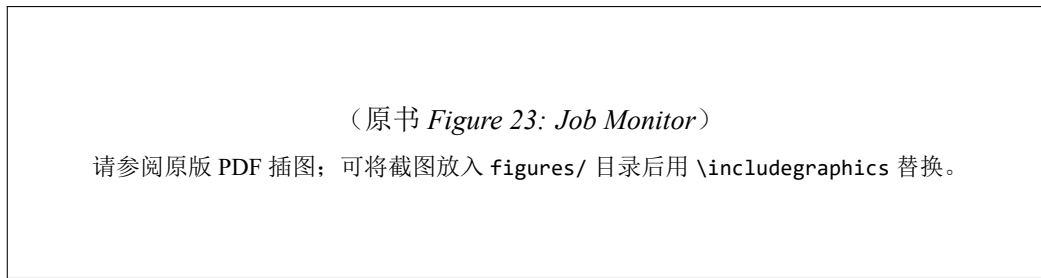


图 5.6: 作业监视器

- 使用 **Kill** 命令终止正在运行的作业
- 查看输出文件 (`icvl_manager.log`)

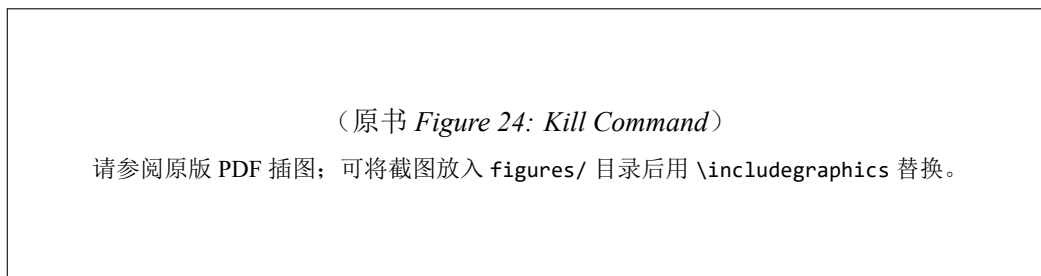


图 5.7: Kill 命令

若当前视图窗口中的 polygon 总数超过 50,000，将出现警告，因为超过该点后 IC Validator Live DRC 运行性能可能下降。若选择 “Do not show this message again”，后续运行将继续使用 IC Validator Live DRC，除非达到最大 polygon 数限制（当前为 2,000,000）。达到该限制时无法运行 IC Validator Live DRC。

5.2.3 运行 IC Validator Live DRC

Running IC Validator Live DRC

要启动 IC Validator Live DRC 运行：

1. 在 Layout Editor 中安排版图，使要检查的区域位于视口内。
2. 确保要检查的层可见。可用下列方法之一更改层可见性：
 - 在 Hierarchy 工具栏上设置 Stop Level。
 - 在 Object/Layer Panel 中显示或隐藏层。
3. 在 Live DRC 工具栏上单击运行按钮：DRC 进程启动。完成后，违例标记出现在画布上。违例也列于 Error Viewer 中。

OA 标记显示在 Error Viewer 中（选择 **Windows > Assistants > Error Viewer**）。

可通过 preference `xtICVLiveNotifyWhenComplete` 控制 IC Validator Live DRC 在检查完成时是否显示通知对话框。

5.2.4 查看与修复违例

Viewing and Fixing Violations

要查看 DRC 违例：

1. 显示 Error Viewer。
2. 单击 **ICV_Live** 选项卡以查看 DRC 错误列表。

(原书 *Figure 25: IC Validator Live DRC Console*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.8: IC Validator Live DRC 控制台

(原书 *Figure 26: Error Viewer Assistant*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.9: Error Viewer Assistant

5.2.5 附加选项

Additional Options

诸如 `halo` 与 `recipe` 等附加选项可更好地控制运行。例如，`halo` 将受检数据扩展到视口窗口外紧邻区域；`recipe` 允许创建并运行可从下拉菜单选择的规则子集。

Halo

Halos 要在紧邻视口窗口外的数据上运行 IC Validator Live DRC，可为运行添加 `halo`。添加 `halo` 会增大检查窗口，如图 5.10 所示。通过将受检数据扩展到包含视口窗口外紧邻的 `polygon`，可验证视口内的设计更改是否未与视口外紧邻的 `polygon` 产生新的 DRC 违例。

(原书 *Figure 27: IC Validator Live DRC Configuration - Halo Size*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.10: IC Validator Live DRC 配置 – Halo 大小

黑色窗口为视口，红色窗口为视口加 `halo`。若以 `halo` 设置执行 IC Validator Live DRC：

- 仅面积有一部分落在红色窗口内的 `polygon` 会纳入检查。（`Polygon 1` 纳入，`Polygon 2` 不纳入。）
- 仅与黑色窗口重叠的错误图形会纳入结果。（`Error 1` 纳入，`Error 2` 不纳入。）

可通过 `preference xtICVLiveHaloRange` 定义 `halo` 范围。可通过 `preference xtICVLiveDumpCheckingRange` 控制 IC Validator Live DRC 是否写出两个 OA 标记以指示检查范围：一个标记表示受检视口，另一个标记计入 `halo` 范围。

Recipe

Recipes Recipe 是包含签核 runset 规则子集的 JSON 文本文件。Recipe 允许按设计阶段运行特定规则。例如，包含间距规则或前端层相关规则的 recipe 可在设计极早期执行；含更高级规则或后端层相关规则的 recipe 可在设计周期后期执行。可通过设置 Recipe File 路径创建新 recipe 或修改现有 recipe；该路径可包含多个 recipe，并用 Recipe Editor 定义。

要调用 Recipe Editor：

1. 在 Live DRC Setup 对话框的 Main 选项卡中，选择现有 Recipe File 并单击相应按钮（如图 5.11），以加载 Recipe Editor。
2. 单击相应按钮（如图 5.11），从 Select Recipe File 对话框选择 recipe。单击 **OK**，将所选 recipe 加载到 Arguments 字段。



图 5.11: 带 Arguments 的 Recipe Editor

3. 选择 <No-Recipe>（如图 5.12），以从 Arguments 路径移除 recipe 设置并创建新 recipe。
4. 在 Rule Name 列表中勾选或取消勾选要纳入 recipe 的规则。单击相应按钮以保存新 recipe 或现有 recipe 的设置。单击相应按钮以删除指定 Recipe File 的现有 recipe。

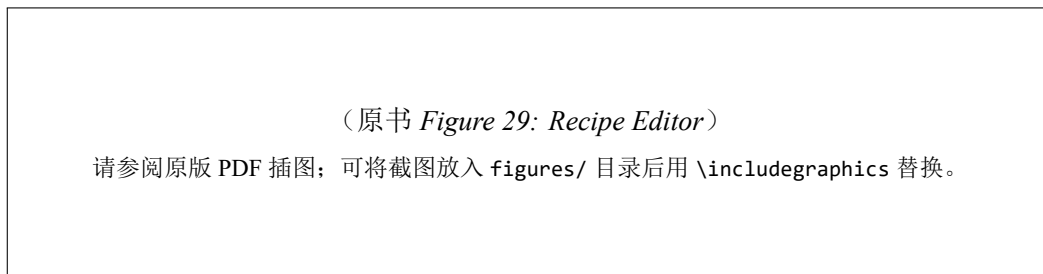


图 5.12: Recipe Editor

5.3 在 Virtuoso 中使用 IC Validator Live DRC

IC Validator Live DRC With Virtuoso

Virtuoso 中的 IC Validator Live DRC 允许对活动版图运行 IC Validator 物理验证检查。使用与 IC Validator 相同的 runset 与文件即可获得即时 DRC 结果。诸如 recipe 等高级选项允许创建规则子集，并可从下拉菜单中选择执行。违例直接显示在 Virtuoso 中，无需单独窗口查看。以下各节说明如何设置环境、执行运行以及查看违例。

5.3.1 设置环境

Setting Up the Environment

要在 Virtuoso 中设置 IC Validator Live DRC，须设置若干环境变量，以便 Live DRC 能定位 IC Validator 安装。

1. 设置环境

将 `ICV_HOME_DIR` 环境变量设为 IC Validator 的安装位置。

将 `SYNOPTSYS_SYSTYPE` 环境变量按系统适当设置；通常应设为 `linux64`。

启动 Virtuoso，并在 CIW 窗口中输入：

```
load( strcat( getShellEnvVar("ICV_HOME_DIR") \
"/etc/icvlive/virtuoso/icv1.il"))
```

也可将该命令加入 `.cdsinit` 文件，以便在 Virtuoso 中自动加载 IC Validator Live DRC 工具栏。

打开 Virtuoso 后，将显示 IC Validator Live DRC 工具栏，如图 5.13。可能需要拉伸工具栏才能看到全部按钮。

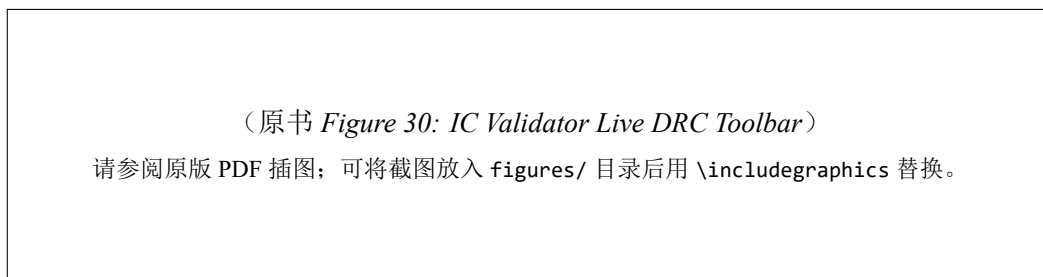


图 5.13: IC Validator Live DRC 工具栏

2. 打开 IC Validator Live DRC Configuration 对话框

单击工具栏上的相应按钮（或键盘快捷键 `Shift+F12`）。

Configuration 由三部分组成：

- **Run Settings:** 确定用于检查的参数。
- **Check Filter:** 确定将检查（或不检查）哪些层与规则。
- **Highlight Control:** 确定浏览 IC Validator Live DRC 结果时的行为。

若当前视图窗口中的 polygon 总数超过 50,000，将出现警告，如图 5.14。若选择“Do not show this message again”，后续运行将继续使用 IC Validator Live DRC，除非达到最大 polygon 数限制（当前为 2,000,000）。达到该限制时无法运行 IC Validator Live DRC。

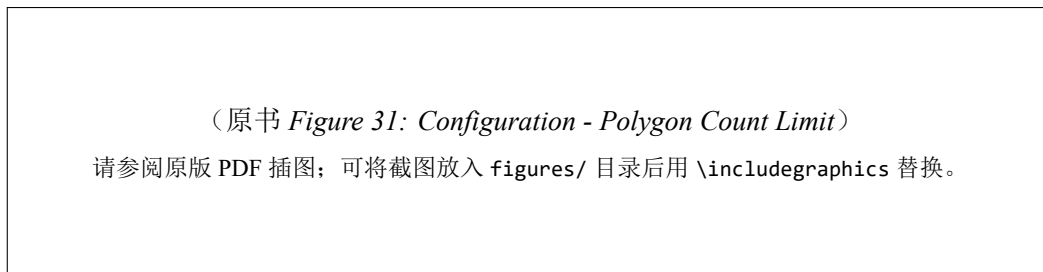


图 5.14: 配置 – Polygon 数量限制

5.3.2 启动 IC Validator Live DRC 检查

Starting the IC Validator Live DRC Check

单击相应按钮以启动 IC Validator Live DRC 检查（或键盘快捷键 **F12**）。检查期间出现模式对话框。单击 **Cancel** 可终止检查，如图 5.15。

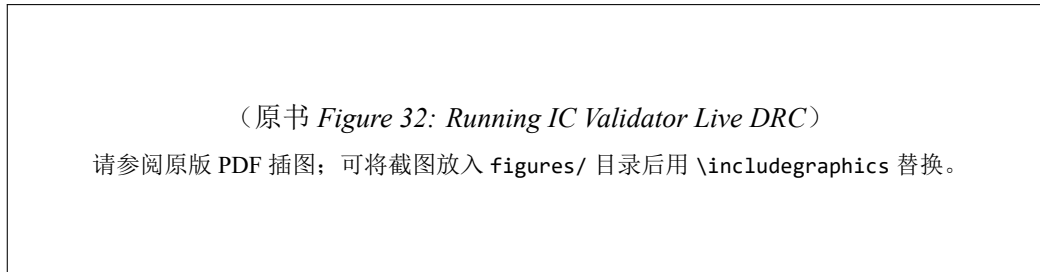


图 5.15: 正在运行 IC Validator Live DRC

Virtuoso 命令窗口中有相关消息。IC Validator Live DRC 消息以 `[ICV Live]` 为前缀，向用户说明 Live DRC 正在做什么，并给出作业可能终止的原因信息。部分示例如图 5.16。

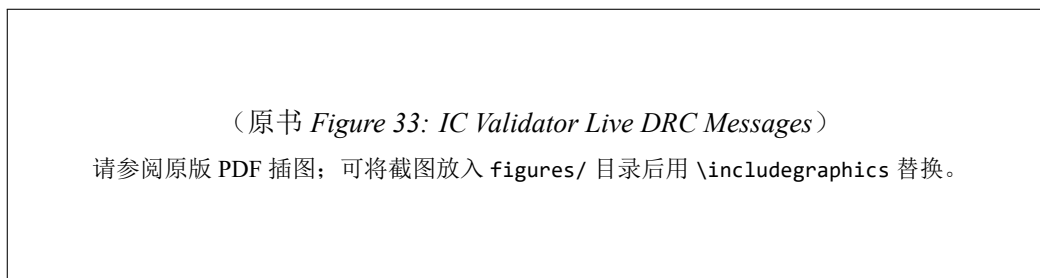


图 5.16: IC Validator Live DRC 消息

5.3.3 在 IC Validator Live DRC 中浏览结果

Browsing Results in IC Validator Live DRC

使用 IC Validator Live DRC 工具栏可：

- 选择违例规则（通过右侧的违例规则框）
- 浏览所选规则的错误（通过中间五个按钮）
- 缩放到视口（通过左侧的放大镜与坐标框）



图 5.17: IC Validator Live DRC 工具栏

IC Validator Live DRC 还提供弹出式违例查看器，含违例规则的详细说明以及更多高亮选项。IC Validator Live DRC Violation Viewer 如图 5.18。单击 IC Validator Live DRC 工具栏中的相应图标即可打开该查看器。

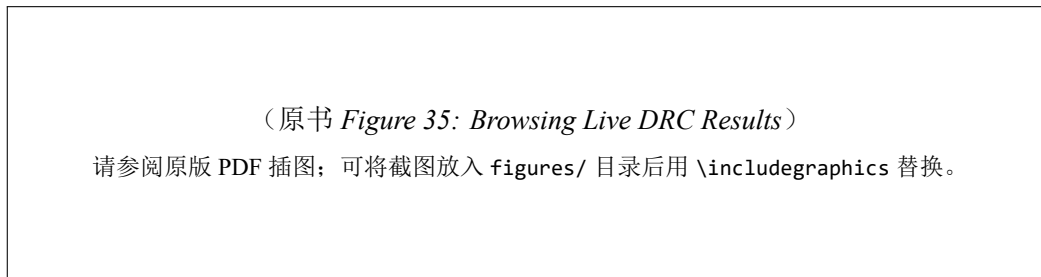


图 5.18: 浏览 Live DRC 结果

使用 Highlight Control 可：

- **Keep Existing Highlights**
若勾选，不清除先前高亮。
- **Follow Mode**
 - **Zoom with Factor:** 按因子缩放到高亮图形
 - **Pan:** 移到高亮图形中心而不改变缩放
 - **Off:** 不执行任何操作
- **Auto color change** (使用 Virtuoso 的 y0–y9 层进行高亮)
 - **Per Highlight:** 每次高亮更换颜色
 - **Per Rule:** 仅当前与先前高亮不属于同一规则时更换颜色
 - **Fixed:** 使用所选颜色进行高亮
- Highlight Control 中的设置与 Violation Viewer 中的设置同步。



图 5.19: Configuration 窗口中的 Highlight Control

5.3.4 附加选项

Additional Options

诸如 halo 与 recipe 等附加选项可更好地控制运行。例如，halo 将受检数据扩展到视口窗口外紧邻区域；recipe 允许创建并运行可从下拉菜单选择的规则子集。

Halo

Halos 添加 halo 会增大检查窗口，如图 5.20。通过将受检数据扩展到包含视口窗口外紧邻的 polygon，可验证视口内的设计更改是否未与视口外紧邻的 polygon 产生新的 DRC 违例。

(原书 *Figure 37: IC Validator Live DRC Configuration - Halo Size*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 5.20: IC Validator Live DRC 配置 – Halo 大小

黑色窗口为视口，红色窗口为视口加 halo。若以 halo 设置执行 IC Validator Live DRC:

- 仅面积有一部分落在红色窗口内的 polygon 会纳入检查。(Polygon 1 纳入，Polygon 2 不纳入。)
- 仅与黑色窗口重叠的错误图形会纳入结果。(Error 1 纳入，Error 2 不纳入。)

检查层

Check Layers Check Layers 选项根据可见层控制检查时纳入哪些规则。可用选项如图 5.21。

(原书 *Figure 38: IC Validator Live DRC Configuration - Check Layers*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 5.21: IC Validator Live DRC 配置 – Check Layers

各选项如下:

- **All Layers:** 纳入全部规则进行检查
- **Visible Layer Only:** 仅纳入使用可见层的规则
- **Visible Layer Involved:** 纳入至少涉及一个可见层的规则

例如，考虑下列三条规则，并假设仅 VIA 可见:

```
Rule1 @= {
    @ "Rule1";
    area(VIA, value = [2.0,2.0]);
}
Rule2 @= {
    @ "Rule2"; VIA and M1;
}
Rule3 @= {
    @ "Rule3"; M1 and M2;
}
```

选择 **All Layers** 时，纳入 Rule1、Rule2 与 Rule3。

选择 **Visible Layer Only** 时，仅纳入 Rule1。

选择 **Visible Layer Involved** 时，纳入 Rule1 与 Rule2。

遵循 P&R Boundary 可见性

Honor the P&R Boundary visibility 当 Check Layers 设为 **Visible Layer Only** 或 **Visible Layer Involved** 时，**Honor the P&R Boundary visibility** 选项生效。

未设置该选项时，P&R Boundary 对象始终视为可见。

设置该选项时，Live DRC 遵循 Virtuoso 编辑器中 P&R Boundary 对象的可见性。

(原书 *Figure 39: IC Validator Live DRC Configuration - Honor the P&R Boundary visibility*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 5.22: IC Validator Live DRC 配置 – 遵循 P&R Boundary 可见性

Recipe

Recipes Recipe 是包含签核 runset 规则子集的 JSON 文本文件。Recipe 允许按设计阶段运行特定规则。例如，包含间距规则或前端层相关规则的 recipe 可在设计极早期执行；含更高级规则或后端层相关规则的 recipe 可在设计周期后期执行。要在 IC Validator Live DRC 中创建 recipe，使用图 5.23 所示功能。

(原书 *Figure 40: Create New Recipe*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 5.23: 创建新 Recipe

- **Rule Recipe:** 定义 IC Validator Live DRC 运行中检查的规则。Live DRC 将 recipe 分为两类，如图 5.24:
 - **User recipes:** 位于 `$HOME/.icvlive/recipes` 的 recipe。
 - **Others recipes:** 位于 `$ICVLIVE_RECIPE_PATH` 的 recipe。
 - 单击相应按钮可刷新下拉列表内容。
 - 单击相应按钮可打开 Recipe Editor 并编辑当前所选 recipe。
- File 菜单有三个新操作: **New**、**Rename** 与 **Delete**。
- Recipes 菜单列出全部可用 recipe，并允许在 recipe 之间切换。
- Selected 与 Unselected 规则列表:
 - 每组规则总数显示在表头括号中。
 - 要在 Selected 与 Unselected 规则列表之间移动规则:
 - * 单击列表项以选择规则
 - * 支持连续选择 (按 **Shift**)、扩展选择 (按 **Ctrl**) 与全选 (**Ctrl+a**)
 - * 单击相应按钮以移动规则项

（原书 *Figure 41: Recipe Categories: User and Others*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.24: Recipe 类别: User 与 Others

（原书 *Figure 42: File Menu*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.25: File 菜单

（原书 *Figure 43: Recipes*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.26: Recipes

（原书 *Figure 44: Recipe Creating Rules*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.27: Recipe 创建规则

保存与加载配置

Saving and Loading Configurations 配置文件允许保存 IC Validator Live DRC 运行所用的 runset 及其他设置。保存设置后，即使关闭先前会话并打开新版图或会话，仍可恢复工作。配置文件是与 IC Validator Recipe 文件类似的基于 JSON 的文本文件。本节说明如何创建这些文件并将其加载到 IC Validator Live DRC 运行。要在运行前加载配置文件，设置 `ICV_LIVE_CONFIG_FILE`，后跟以冒号分隔的文件或目录。例如：

```
setenv ICV_LIVE_CONFIG_FILE file1:/path/to/dir1:file2:$cwd
```

对具有写权限的任何文件，可直接从 IC Validator Live DRC Configuration Window 编辑。

设置好配置后，可单击保存按钮并使用弹出对话框进行保存。

若该配置常用，自动加载会很有用。可通过环境变量 `ICV_LIVE_CONFIG_FILE` 实现：将其设为以冒号分隔的文件或目录列表。打开 IC Validator Live DRC 配置菜单时，Live DRC 读取 `ICV_LIVE_CONFIG_FILE` 路径中的配置文件。随后可从 Configuration 菜单顶部的配置下拉列表中选择配置。

（原书 *Figure 45: User Configuration*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.28: 用户配置

添加 runset 与层映射。Configuration 窗口顶部显示的配置下拉菜单可用于在不同配置设置之间切换。

保存配置

Saving Configurations 要保存配置：

1. 在 ICV Live Configuration 中，单击 **OK** 或 **Apply**。
2. 使用 Configuration 下拉列表切换到另一配置。

（原书 *Figure 46: Saving Configurations*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.29: 保存配置

3. 单击 **Create Configuration** 或 **Load Configuration** 文件。

IC Validator Live DRC 按如下方式检查配置的写权限：

- a. 若未保存的配置可写，Live DRC 将更改保存到可写的配置文件。
- b. 若未保存的配置为只读，则出现提示，要求将配置保存到新的可写配置文件，或覆盖现有可写配置文件。

配置文件内容

（原书 *Figure 47: Create Configuration*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.30: 创建配置

（原书 *Figure 48: User Configuration*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.31: 用户配置

（原书 *Figure 49: Loading an Existing Configuration File*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.32: 加载现有配置文件

Contents of a Configuration File 配置文件以 JSON 格式编写。IC Validator Live DRC 中的配置文件用于在后续新会话中记住初始用户设置。一个配置文件可包含多个配置。例如，某 N7 配置文件可能有两个配置——一个用于 BEOL 运行，一个用于 FEOL 运行。

例如：

```
%vi n7_configurations.icv1c
```

```
{
  "configurations": [
    {
      "checkFilter": {
        "checkLayers": "Visible Layers Involved",
        "excludeDensityRules": true,
        "loadedRecipeFiles": [
          "/path/to/beol_spacing.icvrpc",
          "/path/to/beol_density.icvrpc"
        ],
        "selectedRecipe": "Spacing Rules",
        "selectedRecipeFile": "/path/to/beol_spacing.icvrpc"
      },
      "name": "BEOL Configuration",
      "runSettings": {
        "clfFile": "/path/to/beol_clf.txt",
        "enableColoring": true,
        "exportLayoutToOasis": true,
        "haloSize": 1.5,
        "layerMappingFile": "n7.layermap",
        "objectMappingFile": "n7.objmap",
        "runsetFile": "n7.rs"
      }
    },
    {
      "checkFilter": {
        "checkLayers": "All Layers",
        "excludeDensityRules": true,
        "loadedRecipeFiles": [
        ],
        "selectedRecipe": "",
        "selectedRecipeFile": ""
      },
      "name": "All Layers ",
      "runSettings": {
        "clfFile": "",
        "enableColoring": false,
        "exportLayoutToOasis": false,
        "haloSize": 1,
        "layerMappingFile": "n7.layermap",
        "objectMappingFile": "n7.objmap",
        "runsetFile": "n7.rs"
      }
    }
  ]
}
```

```

    },
    {
      "checkFilter": {
        "checkLayers": "All Layers",
        "excludeDensityRules": true,
        "loadedRecipeFiles": [
        ],
        "selectedRecipe": "Custom Checks",
        "selectedRecipeFile": "/path/to/feol.icvrpc"
      },
      "name": "FEOL Configuration",
      "runSettings": {
        "clfFile": "/path/to/feol_clf.txt",
        "enableColoring": true,
        "exportLayoutToOasis": true,
        "haloSize": 1.5,
        "layerMappingFile": "n7.layermap",
        "objectMappingFile": "n7.objmap",
        "runsetFile": "n7.rs"
      }
    }
  ],
  "majorVersion": 1,
  "minorVersion": 0
}

```

Preference 设置

Preference Settings Live DRC 将与 preference 相关的设置存入每用户文件。这些设置不影响 Live DRC 检查本身，但影响 GUI 与高亮。

例如，Highlight Control 部分存为 preference。

默认情况下，该信息存于 `$HOME/.synopsys/icvlive/prefs.icv1c`。要更改存储文件，设置 `ICV_LIVE_PREF_FILE`，后跟文件名。例如：`setenv ICV_LIVE_PREF_FILE /path/to/my_prefs.icv1c`

运行目录

Run Directory 设置环境变量 `_ICV_LIVE_CLIENT_ENABLE_GUI_RUN_DIR` 可启用如图 5.33 所示的 Run Directory 字段。用该字段指定 Live DRC 写入中间文件的目录。未指定目录时，Live DRC 默认将中间文件保存到 `$HOME/.synopsys/icvlive`。

（原书 *Figure 50: Run Directory Field In Preference Settings*）
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.33: Preference 设置中的 Run Directory 字段

要覆盖 GUI 设置，设置环境变量 `ICV_LIVE_OUTPUT_DIR`。该环境变量优先级高于 GUI 中的 Run Directory 字段。

快捷键

Bindkeys IC Validator Live DRC 快捷键可快速访问常用功能。有两个默认可写在 `icv1.il` 文件末尾的快捷键，该文件位于 `$ICV_HOME_DIR/etc/icvlive/virtuoso/icv1.il`。

- **Shift+F12**: 打开配置窗口。
- **F12**: 运行 IC Validator Live DRC。

可通过 `hiSetBindKey()` 函数定义快捷键，以设置 Layout、Key 以及要调用的 SKILL 函数，如下所示：

- 填入 `icv1Setup(hiGetCurrentWindow())` 以打开配置窗口。
- 填入 `icv1Execute(hiGetCurrentWindow())` 以运行 IC Validator Live DRC。
- 填入 `icv1Browse(hiGetCurrentWindow())` 以打开违例查看器。

例如：

```
# Set hotkey for IC Validator Live DRC configuration dialog and execution
hiSetBindKey("Layout" "Shift<Key>F12" "icv1Setup(hiGetCurrentWindow())")
hiSetBindKey("Layout" "<Key>F12" "icv1Execute(hiGetCurrentWindow())")
```

5.4 在 IC Compiler II 与 Fusion Compiler 中使用 IC Validator Live DRC

IC Validator Live DRC With IC Compiler II and Fusion Compiler

可在 IC Compiler II 或 Fusion Compiler GUI 中使用 IC Validator Live DRC 工具执行交互式签核设计规则检查。

5.4.1 设置环境

Setting Up the Environment

要使用 IC Validator Live DRC 执行交互式签核设计规则检查：

1. 设置 IC Validator 环境。通过设置 `ICV_HOME_DIR` 环境变量指定 IC Validator 可执行文件的位置。可在 `.cshrc` 文件中设置该变量。使用类似下列示例的命令指定位置：

```
setenv ICV_HOME_DIR /root_dir/icv
set path = ($path $ICV_HOME_DIR/bin)
```

须确保指定的 IC Validator 可执行文件版本与所用的 IC Compiler II 或 Fusion Compiler 版本兼容。使用 IC Compiler II 或 Fusion Compiler 的 `report_versions` 命令可报告版本兼容性。

2. 设置交互式签核设计规则检查的应用选项。

至少须通过设置 `signoff.check_drc.runset` 应用选项指定用于设计规则检查的 foundry runset, 并通过设置 `signoff.physical.layer` 应用选项指定将技术文件层映射到 runset 层的层映射文件。

运行交互式设计规则检查前, 按表 5.2 设置交互式签核设计规则检查的应用选项以配置运行。使用 `set_app_options` 命令设置应用选项；使用 `report_app_options` 命令查看当前设置。

表 5.2: 交互式设计规则检查的应用选项

应用选项	默认值	说明
signoff.check_drc_live.default_layers	(none)	指定写入 IC Validator Live DRC 默认层的带 color mask 的层。
signoff.check_drc_live.default_halo	1	指定以微米为单位扩展 IC Validator Live DRC 执行检查之区域的距离。
signoff.check_drc_live.keep_enclosing_results	true	指定是否保留检查区域外的 DRC 违例。默认情况下, IC Validator Live DRC 仅保留完全位于检查区域内的违例。若将该应用选项设为 true, 则保留完全位于 trim 区域内的 DRC 违例。trim 区域为检查区域加上 signoff.check_drc_live.halo 应用选项所指定距离的一半。完全位于 trim 区域外的违例始终丢弃。
signoff.check_drc_live.run_dir	signoff_check_drc_live	指定运行目录, 其中包含交互式签核设计规则检查生成的文件。可指定相对路径(此时目录在当前工作目录下创建)或绝对路径。
signoff.check_drc_live.runset (required)	(none)	指定用于设计规则检查的 foundry runset。
signoff.check_drc_live.user_defined_options	(none)	指定 IC Validator 命令行的附加选项。该选项中指定的字符串会追加到用于调用 IC Validator 工具的命令行。IC Compiler II/Fusion Compiler 工具不对指定字符串执行任何检查。
signoff.physical.layer_map_file (required)	(none)	指定将技术文件中的层名映射到 foundry runset 中层名的层映射文件名。

- 确保版图窗口显示要对之执行设计规则检查的对象。
默认情况下, 交互式设计规则检查对每个 cell 使用 frame view。若需要比 frame view 更详细的内容, 可对特定 cell 使用 design 或 layout view。使用 gui_set_display_view 命令指定特定 cell 所用的 view。例如, 要对所有层次 cell 使用 layout view:

```
fc2_shell> gui_set_display_view \  
-cells [get_cells -hierarchical *] -view layout
```

关于控制用于设计规则检查的显示对象的信息, 见「为设计规则检查显示对象」。

- 在 GUI 菜单栏上右键单击并选择 **DRC Toolbar**, 以显示 DRC 工具栏, 如图 5.34。



图 5.34: DRC 工具栏

DRC Toolbar 图标见表 5.3。

表 5.3: DRC 工具栏按钮

功能	说明
运行检查	使用 IC Validator Live DRC 运行设计规则检查。
应用选项	打开对话框以查看并设置 IC Validator Live DRC 应用选项。
配置规则	打开对话框以配置用于设计规则检查的规则。
选择区域	选择执行设计规则检查的区域。
重置区域	将设计规则检查区域重置为先前运行所用区域。
清除/高亮全部违例	清除或高亮 IC Validator Live DRC 检测到的全部 DRC 违例。
遍历违例	遍历当前规则来自 IC Validator Live DRC 的 DRC 违例。
缩放/平移控制	控制遍历 DRC 违例时的缩放或平移。
显示当前规则	显示正在高亮的设计规则。

- 通过选择 **Edit > Options > ICV Live**，或单击 DRC 工具栏中的齿轮图标，配置用于设计规则检查的规则。
- 通过选择 **Edit > ICV > Run ICV-Live on Current View**，或单击 DRC 工具栏中的 ICV Run 图标，运行设计规则检查。
首次运行 IC Validator Live DRC（或在更改层或规则之后），工具会缓存 `runset`，可能需要几分钟。后续运行不再执行该缓存，因此快得多。

5.4.2 为设计规则检查显示对象

Displaying Objects for Design Rule Checking

运行交互式设计规则检查时，IC Validator 工具对版图窗口中显示的内容加上 1 微米扩展执行设计规则检查。
可通过设置 `signoff.check_drc_live.halo` 应用选项修改扩展大小。使用 View Settings 与 Hierarchy Settings 面板控制版图窗口中显示的层、层次与 cell 类型。

- 单击面板区的 View Settings 图标以显示 View Settings 面板。
 - 在 Level 字段中指定要显示的层级数。
 - 选择 Layers 选项卡并为所需层启用可见性。
- 单击 View Settings 面板或菜单栏中的 Hierarchy Settings 图标以显示 Hierarchy Settings 面板。
 - 在 “View level” 字段中指定视图层级数。
 - 在 “Expanding cell types” 部分选择 Standard、“Fill Cell” 与 Block。

图 5.35 显示用于控制设计规则检查之版图窗口显示的 View Settings 与 Hierarchy Settings 面板。

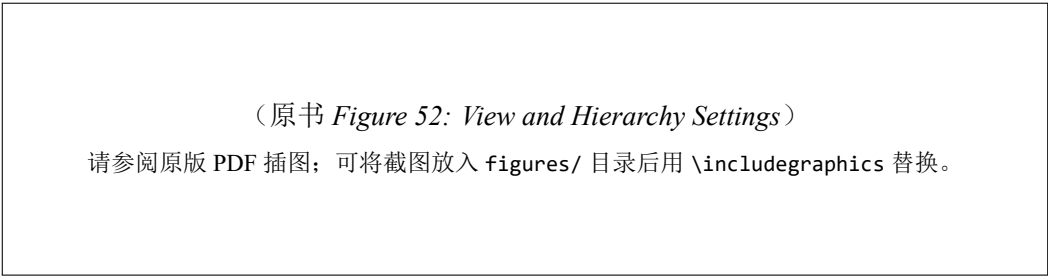


图 5.35: View 与 Hierarchy 设置

注意

关于交互式签核设计规则检查的更多信息，见 SolvNetPlus 上的 *Fusion Compiler™ User Guide* 或 *IC Compiler™ II Implementation User Guide*。

5.5 设置预编译缓存目录

Setting Up Pre-Compiled Cache Directories

每当 IC Validator Live DRC 在视图窗口中检测到新层或规则选择发生变化时，都会重新编译 `runset`。若频繁更改视图窗口与可见层，会导致频繁编译，从而减慢 Live DRC 运行时间。为提升性能，可准备包含设计过程中生成的缓存文件的缓存目录。当窗口日后更改时，若层、规则选择与设置与当前运行匹配，Live DRC 可从该目录搜索并重用较旧的缓存文件。这些缓存目录可与其他用户共享。要创建 `runset` 缓存目录并重用或共享文件：

1. 设置本地缓存目录并运行 IC Validator Live DRC，以不同配置编译 `runset`。

```
unset ICV_DISABLE_RUNSET_CACHE
setenv ICV_RUNSET_CACHE_DIR <first_user local>
# Start ICV Live
```

2. 将缓存文件从 `<first_user local>` 传输到 `<new_user>` 的共享缓存目录 `<new_user shared>`。
3. 在 `<new_user>` 中同时设置本地与共享缓存目录以运行 IC Validator Live DRC，使其既能拾取预编译文件，也能将新创建的缓存文件存入本地路径。

```
unset ICV_DISABLE_RUNSET_CACHE
setenv ICV_RUNSET_CACHE_DIR <new_user local>
setenv ICV_RUNSET_CACHE_DIR_SHARED <new_user shared>
# Start ICV Live
```

5.6 在 IC Validator 中复现 IC Validator Live DRC 运行

Reproducing an IC Validator Live DRC Run in IC Validator

要在 IC Validator 中复现 IC Validator Live DRC 运行结果：

1. 从 Console 捕获运行的坐标：

（原书 *Figure 53: IC Validator Live DRC Console*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.36: IC Validator Live DRC 控制台

2. 在运行目录中用下列内容创建 `config.rs` 文件，并将 `select_window` 中的参数替换为上一步 IC Validator Live DRC 运行的坐标：

```
hierarchy_options(  
    explode_all = {"*"}  
);  
incremental_options(  
    select_window = {{186.62, -33.817, 192.58, -31.087}},  
    clip_window = false,  
    window_error_filter = KEEP_ENCLOSED,  
    window_ambit = double  
);
```

IC Validator Live DRC 在扁平数据上运行。因此必须使用 `hierarchy_options()` 函数将输入扁平化到 IC Validator 运行。尽管 Live DRC 不在视口窗口边缘裁剪 polygon，但也不会报告视口外的任何违例。因此必须对 `incremental_options()` 函数使用 `clip_window` 与 `window_error_filter` 参数。

3. 通过设置 `-runset_config config.rs` 将该文件加入 IC Validator 运行。
4. 运行 IC Validator。

Chapter 6

IC Validator Explorer DRC

IC Validator Explorer DRC

本章说明 IC Validator Explorer DRC 的功能。

IC Validator Explorer DRC 包括以下各节：

- IC Validator Explorer DRC 简介
- Explorer DRC 模式类型
- 配置 Explorer DRC
- Explorer DRC 输出文件
- 使用 Explorer DRC 调试
- Explorer DRC 许可证

6.1 IC Validator Explorer DRC 简介

Introduction to IC Validator Explorer DRC

在设计早期调试 DRC 违例可能既复杂又耗时。例如，复杂的 DRC 规则（多重图形、delta voltage 等）运行时间很长，而在仍存在其他更简单违例时，这些检查往往并无实际收益。可用一套基本设计检查来调试典型的早期设计问题：查找 cell 重叠、不当 fill、布线问题等并不需要复杂规则。IC Validator Explorer DRC 会从晶圆厂 runset 中自动确定这些基本规则，并额外加入方法学规则与调试能力，从而显著缩短周转时间（turnaround time）。此外，当某些设计层触发过多违例时，IC Validator Explorer DRC 会检测并取消选择相关的慢速规则。这种对检查的动态选择与执行，可比传统的完整 DRC 运行获得更快的周转时间。

6.2 Explorer DRC 模式类型

Types of Explorer DRC Modes

Explorer DRC 有三种模式：

- Explorer DRC Standalone 模式
该模式运行从晶圆厂 runset 中自动选出的设计规则，以及 IC Validator 额外加入的方法学检查。
适用于存在违例、需要快速检查且计算资源有限的设计。

- Explorer DRC Tiers 模式
Explorer DRC tiers 提供系统化框架，优先执行并报告高优先级检查。在 tiers 模式下，检查会完整执行，不以性能为主要取舍：所有规则按 tier 顺序选择并执行，直至指定的 tier。
使用该模式时，性能并非主要考虑因素。
部分 tier 保留给 Explorer 优先级规则与方法学检查；多数 tier 可由用户定制。更多信息见表 6.1。
Tiers 按所赋优先级依次执行。编号较小的 tier 优先级最高。位于 Tier 0 与 Tier 10 之间、尚未定义归属的检查，在 Tier 10 之后作为「All remaining rules」tier 执行。每个 tier 完成后，结果会在运行过程中更新。
Tiers 模式运行示例
在全部 Tiers 上运行 Explorer DRC auto 模式，并指定用户定义的 Tiers，使用 `-explorer_tiers_file mytier.txt`：

```
% icv -explorer auto -explorer_tiers_file mytier.txt runsetfile
```

- Explorer DRC Auto 模式
与 tiers 模式类似，该模式同样使用 Explorer DRC tiers，但还具备停止慢速规则的能力。
在 auto 模式下，若发现存在违例的设计层，相关规则若其函数运行缓慢则会被停止。这样可利用较早 tier 的结果，阻止后续函数/规则运行过久（tier 0 与 tier 1 的规则绝不会被提前停止）。
该模式的性能取决于 DRC 错误数量。DRC 错误越多，auto 模式越可能取消选择规则并提前退出，从而加快周转。
当设计中的违例数量未知、并希望尽可能完成更多检查时，使用 auto 模式。若设计干净，auto 模式不会停止任何检查，并产生完整签核结果。

6.2.1 执行 Explorer DRC

Executing Explorer DRC

要执行 Explorer DRC，在命令行设置：

```
-explorer standalone|auto[:TierNumber]|tiers[:TierNumber]
```

其中 TierNumber 指定要执行到的最高 tier。将执行从最低到（含）TierNumber 的全部 tiers。若未指定 TierNumber，则所有检查均可作为执行候选。有关配置 IC Validator Explorer DRC 的更多信息，见下一节。

Explorer DRC 模式运行示例

Examples of Running in Explorer DRC Mode 运行 Explorer DRC standalone 模式：

```
% icv -explorer standalone runsetfile
```

运行 Explorer DRC auto 模式直至 Tier 4：

```
% icv -explorer auto:4 runsetfile
```

表 6.1 给出不同 Explorer DRC tiers、预含检查以及可定制类别。

表 6.1: Explorer Tiers

Tier	用户可否向该 tier 添加检查？	用户可否命名该 tier？	预含检查	可选检查
0	否	否	方法学警告、fill overlap 诊断、优先级规则	N/A
1	是	否	无	优先级规则（用户指定）
2	是	是	基于电压的诊断（集合 1）	用户指定规则
3	是	是	基于电压的诊断（集合 2）	用户指定规则
4–10	是	是	无	用户指定规则

Tier	用户可否向该 tier 添加检查?	用户可否命名该 tier?	预含检查	可选检查
N/A	N/A	N/A	全部剩余规则 (All remaining rules)	N/A

6.3 配置 Explorer DRC

Configuring Explorer DRC

Explorer DRC 模式的命令行选项见表 6.2。

表 6.2: Explorer DRC 模式命令行选项

选项	说明
<code>-explorer standalone</code> <code> auto[:TierNumber]</code> <code> tiers[:TierNumber]</code>	以 standalone、auto 或 tiers 模式运行 Explorer。TierNumber 对 auto 与 tiers 模式可选，用于将运行限制到指定的 TierNumber；未指定时，所有 tiers 均为执行候选。
<code>-explorer_tiers_file path_to_file</code>	指定用于定制用户定义 tiers 并传入 Explorer DRC 运行的文件。该文件中可使用 <code>ADD_RULES_TO_TIER</code> 与 <code>NAME_TIER</code> 指令。
<code>-explorer_script path_to_file</code>	指定在 Tier 0 与 Tier 1 完成后用于追加资源的文件。可先以较少资源启动 auto 运行，并在 Tier 0 与 Tier 1 完成后动态追加资源，以加快其余检查的周转。

命令行选项 `-explorer_tiers_file` 指向可选的脚本文件，用于定制 tiers。该文件示例如下：

```
ADD_RULES_TO_TIER 1 "*.1:" "*G1*"
ADD_RULES_TO_TIER 2 "*.S.*" "***DN*"
NAME_TIER 4 "Handoff Rules"
ADD_RULES_TO_TIER 4 "*delta*"
ADD_RULES_TO_TIER 4 "*LU*"
DISABLE_EXPLORER.OVERLAP
```

创建该文件时可使用下列指令：

- **ADD_RULES_TO_TIER TierNumber Rule comment ... Rule comment**
用于允许用户指定规则的 tiers。可按违例注释 (violation comments) 将规则加入各 tiers，支持字符串匹配。规则按优先级升序加入各 tiers。若某规则已加入更高优先级的 tier，则不会再加入任何更低优先级的 tier；该规则与 Explorer tiers 文件中的书写顺序无关。
允许跳过或留空的 tiers。当 Explorer DRC 遇到空 tier 时，执行会移至下一优先级 tier。报告按 tier 优先级顺序给出。
- **NAME_TIER TierNumber TierName**
用于允许用户指定名称的 tiers。该名称用于向 LAYOUT_ERRORS 文件、RESULTS 文件或 VUE 报告。
允许未命名的 tiers；报告时使用默认 Explorer tier 名称 TierNumber。
- **DISABLE_EXPLORER.DiagnosticName**
用于禁用 Explorer 专有诊断。支持的取值包括：
 - EXPLORER.OVERLAP: 禁用默认包含在 Tier 0 中的 fill overlap 诊断。
 - EXPLORER.ASSIGN: 禁用默认包含在 Tier 0 中的基于 assign 层的优先级规则。
 - EXPLORER.VOLTAGE: 禁用默认包含在 Tier 3 与 Tier 4 中的基于电压的诊断。

命令行选项 `-explorer_script path_to_file` 在 Tier 0 与 Tier 1 检查均完成后运行。可以少于完成整次运行所需的资源启动 DRC auto 模式；若后续 tiers 需要更多资源，该选项可在基本检查之后追加资源。脚本文件示例如下：

```
#!/bin/csh -f

echo "Explorer Tier 0 and Tier 1 checks are done. Adding resources and
running remaining Tiers"

# Following command adds host "A" to ongoing run
icv -rd run_details -host_add A
```

6.4 Explorer DRC 输出文件

Explorer DRC Output Files

Explorer DRC 会创建与 IC Validator DRC 运行类似的 ASCII 文件。有关 DRC 运行文件的更多信息，见「输出文件」一章。在 LAYOUT_ERRORS 与 RESULTS 文件中，Explorer 运行的错误按表 6.3 分类。

表 6.3: Explorer 错误分类

类别	说明
Methodology warnings	指定旨在标记潜在设计或性能问题的 Explorer 方法学警告。
Fill overlap diagnostic	指定被诊断为 fill overlap 问题的 DRC 错误（runset 选定的 fill overlap 规则）。
Priority rules	指定来自 Explorer 优先级检查的 DRC 错误（runset 选定的基本规则）。该类别中某些规则可能带有 EXPLORER 前缀，表示该规则已相对原始 runset 被修改（更快版本）。原始 runset 版本会作为「All remaining rules」类别中的 DRC Errors 运行。
Priority Rules (user-defined)	指定来自用户定义优先级规则的 DRC 错误。
Voltage dependent diagnostics	指定标记多边形或 net 对象上基于 text 的电压冲突的 Explorer 方法学警告；亦包含游离电压 text 与电压检查。
Explorer tier TierNumber	指定来自未命名 tiers 的 DRC 错误。默认将这些错误归类在 Explorer tier、TierNumber 下。可用文件中的 NAME_TIER 指令修改 tier 名称。
All remaining rules	指定来自归类为预定义或用户定义 tiers 之检查的可用 DRC 错误。

除表 6.3 中的错误分类外，DRC 错误还会按用户定义的名称分类（通过文件中的 NAME_TIER 指令定义）。

注意

错误类别取决于 Explorer 模式类型，以及该模式下所标记的诊断。

6.4.1 Explorer DRC RESULTS 文件

Explorer DRC Results File

Explorer DRC 创建 RESULTS 文件 cell.RESULTS。该文件提供运行结果，以及本次运行所用的 IC Validator 命令行选项。RESULTS 文件是分析 Explorer DRC 运行结果的起点。Results summary 部分按表 6.3 的分类报告 DRC 错误统计，包括不同 Explorer 类别中的规则总数与违例总数。

6.4.2 Explorer DRC Errors 文件

Explorer DRC Errors File

Explorer DRC 创建 LAYOUT_ERRORS 文件，提供运行期间标记的版图错误；这些错误按表 6.3 分类。为使错误报告简洁，LAYOUT_ERRORS 文件中会抑制错误坐标信息，等效于运行 `error_options(report_error_details = false)`。信息仍保存在错误数据库 (PYDB) 中，但仅将违例摘要写入 LAYOUT_ERRORS 文件。每个 tier/类别完成后即生成 LAYOUT_ERRORS 文件，以便在 Explorer DRC 运行过程中查阅。icv.log 会在 tier/类别完成时指示，例如：

```
ICV_Engine run is 5% complete. Elapsed Time=0:07:24
All Priority Rules are finished! 19103703 violations were found.
```

注意

在将错误标记存入错误数据库 (PYDB) 时，Tier 0 与 Tier 1 (Explorer 优先级检查与用户定义优先级检查) 会强制施加 `error_sampling_per_check` 参数所设限制 (默认 100 万)。其他 tiers 的限制则基于 `runset`。有关 `error_sampling_per_check` 参数的更多信息，见 *IC Validator Reference Manual*。

6.4.3 Explorer DRC VUE 文件

Explorer DRC VUE File

IC Validator Explorer 创建 VUE 文件 `cell.vue`，用于查看 Explorer DRC 错误详情与 Heat Map。

6.5 使用 Explorer DRC 调试

Debugging With Explorer DRC

分析 Explorer DRC 运行时：

1. 以 Explorer DRC 运行 IC Validator。
2. 审查 RESULTS 文件。
 - 若结果干净，进入签核 DRC；若不干净，进入步骤 3。
3. 将 Explorer DRC 的 .vue 文件加载到 VUE，并将版图编辑器附着到 VUE。
4. 点击 **Heat Map** 选项卡开始调试。Error Heat Map 是用于可视化错误分布的图形界面。Heat Maps 对 Explorer DRC 自动启用。更多信息见 *IC Validator VUE User Guide* 中的「Introduction to the IC Validator Error Heat Map」一章。
5. 与 RESULTS、LAYOUT_ERRORS 文件类似，Explorer DRC 错误默认按表 6.3 分类。一般按以下顺序开始调试：方法学警告、诊断、预定义优先级规则、用户定义 tiers，最后到全部剩余规则。此外，可在 Violation Category 窗口的菜单中选择 Clustered Errors 类别模式。该模式下相同错误会被聚类，从而可一次调试多条规则。有关 clustered errors 的更多信息，见 *IC Validator VUE User Guide* 中的「Introduction to the IC Validator Error Heat Map」一章。
6. 从各自类别选定规则后，定位错误计数很高的区域（热图中的红色区域），并放大这些密集区域。
7. 在这些区域中探测热图与版图错误，分析高错误计数的根本原因。探测可包括：
 - 放大区域
 - 高亮这些区域中的错误标记
 - 选择或取消选择特定规则
8. 重复步骤 6 与 7，直至穷尽所有类别/聚类。

6.6 Explorer DRC 许可证

Explorer DRC Licenses

IC Validator Explorer DRC 需要 Elite 许可。该功能需要下列 IC Validator 许可证：

- ICValidator-Manager
- ICValidator2-GeometryEngine

参见

更多信息见本手册「许可证与资源需求」一章。

Chapter 7

IC Validator Explorer LVS

IC Validator Explorer LVS

本章说明 IC Validator Explorer LVS 的功能。

IC Validator Explorer LVS 包括以下各节：

- IC Validator Explorer LVS 简介
- 配置 Explorer LVS
- 执行 Explorer LVS
- Explorer LVS 许可证

7.1 IC Validator Explorer LVS 简介

Introduction to IC Validator Explorer LVS

在签核 LVS 早期找到 LVS 错误的根因，对于缩短整体签核周期、满足流片进度至关重要。Explorer LVS 面向片上系统（SOC）全芯片，提供可在较短周转时间与高效机器资源下定位 LVS 问题根因的方案，从而降低整体签核时间与工作量。Explorer LVS 首先聚焦源于 SOC 集成的 LVS 问题，再逐步扩展分析，覆盖其他潜在 LVS 问题，例如来自器件/基础层或更低层次 cell 的 LVS 错误。为检查 SOC 集成相关问题，Explorer LVS 默认仅对构成 SOC Top 的 equivalence cells 与金属层执行基于 text 的 short 检查（例如构建主 SOC 结构的 P&R、PowerGrid、占位多边形 fill cells 等；相对地，宏与 IP 等则实例化在主 SOC 结构之下）。检查 LVS 问题的范围可按需通过 Explorer LVS 提供的灵活选项扩展。Explorer LVS 有助于提升签核 LVS 的生产力与效率，但并不取代传统 LVS 工具。最终签核 LVS 仍须使用传统 LVS 工具运行。

7.2 配置 Explorer LVS

Configuring Explorer LVS

表 7.1 描述可用于配置 Explorer LVS 的命令行选项。

表 7.1: Explorer LVS 命令行选项

语法	说明
<code>-explorer lvs</code>	运行 LVS 的 Explorer 模式。对 <code>-explorer_lvs_cells</code> 、 <code>-explorer_lvs_layers</code> 、 <code>-explorer_lvs_checks</code> ，分别采用 AUTO、METAL、SHORTS 默认值，并对 SOC Top cell 执行基于 text 的 short 检查（SOC Top cell 示例见「IC Validator Explorer LVS 简介」）。默认行为也可由本表后文所述这三个命令行选项更改。无论这些选项如何设置，Explorer LVS 均跳过器件提取。
<code>-explorer_lvs_cells [AUTO ALL]</code> AUTO（默认）：仅考虑与顶层相关的 cells。当 <code>-explorer_lvs_checks=SHORTS</code> 或 PG_SHORTS 时，删除在 <code>equiv_options()</code> 、 <code>lvs_black_box_options()</code> 中定义的、或由 <code>lvs_options(generate_user_equivs)</code> 生成的全部版图 cells。 ALL：考虑全部 cells。	可选。指定 Explorer LVS 要检查的 cells。
<code>-explorer_lvs_layers [METAL ALL user_custom_layer_list]</code> METAL（默认）：仅考虑金属层，或空的 FEOL 层（如器件端子层）。 ALL：考虑全部层。 <code>layer1, layer2, ...</code> ：仅考虑金属层或空 FEOL 层，但排除 <code>layer1</code> 、 <code>layer2</code> 等所列层。	可选。指定 Explorer LVS 要检查的层。
<code>-explorer_lvs_checks [SHORTS ALL PG_SHORTS]</code> SHORTS（默认）：仅报告 <code>text_short</code> 违例。 ALL：报告 <code>text_short</code> 违例与 LVS Compare 结果。 PG_SHORTS：报告电源与地的 <code>text_short</code> 违例。 PG shorts 定义：<power, power>、<power, ground>、<ground, ground>。 non-PG shorts 定义：<power, signal>、<ground, signal>、<signal, signal>。	可选。仅执行基于 text 的 short 检查，或同时执行基于 text 的 short 检查与 LVS compare。
<code>-explorer_lvs_preserve_cells [preserve_cell_list]</code>	可选。指定在 Explorer LVS 中应予保留、不被删除或 black-box 的 cells，以便将这些 cells 纳入 Explorer LVS。
<code>-explorer_lvs_preserve_cell_file "file path"</code>	指定 <code>-explorer_lvs_preserve_cells</code> 的文件输入形式。每行一个待保留的 cell。
<code>-explorer_lvs_preserve_hierarchy_cells "space separated cellnames" *</code>	指定在以 <code>-explorer lvs lvs:1 lvs:2</code> 运行时，不应被删除或 black-box 的 <code>equiv_cells</code> 及其子级 <code>equiv_cells</code> 。
<code>-explorer_lvs_preserve_hierarchy_cells "file path"</code>	指定 <code>-explorer_lvs_preserve_hierarchy_cells</code> 的文件输入形式。每行一个 cell。

Explorer LVS 命令行选项的全部可能用法组合如图 7.1 所示。

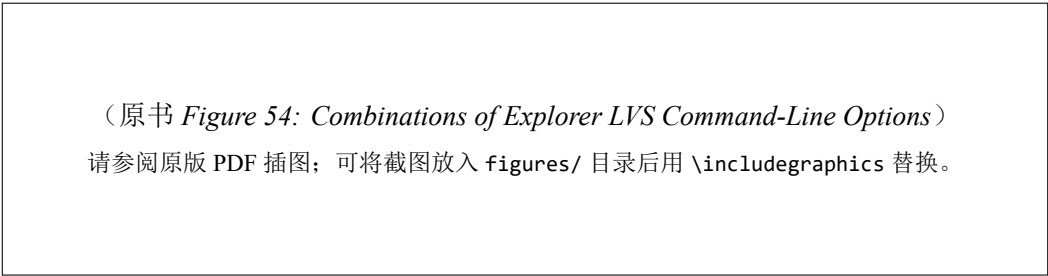


图 7.1: Explorer LVS 命令行选项组合

7.3 执行 Explorer LVS

Executing Explorer LVS

要执行 Explorer LVS,在命令行设置 `-explorer lvs`。该命令行选项对 `-explorer_lvs_cells`、`-explorer_lvs_layers`、`-explorer_lvs_checks` 分别采用 AUTO、METAL、SHORTS 默认值。Explorer LVS 仅对金属层上的 SOC Top cells 执行基于 text 的 short 检查。默认行为可通过三个 Explorer LVS 选项 `-explorer_lvs_cells`、`-explorer_lvs_layers` 与 `-explorer_lvs_checks` 更改，见表 7.1。用 `-explorer_lvs_preserve_cells` 指定的某些 cells 可在 Explorer LVS 中予以保留、不被删除或 black-box，从而纳入 Explorer LVS。执行 Explorer LVS 的示例见表 7.2。

表 7.2: 执行 Explorer LVS 的示例

模式	命令行	说明
默认	% icv -explorer lvs \ runsetfile	仅对金属层上的 SOC Top cells 做基于 text 的 short 检查
在默认基础上加入全部 cells	% icv -explorer lvs \ -explorer_lvs_cells ALL \ runsetfile	对全部 cells (仅金属层) 做基于 text 的 short 检查
在默认基础上加入全部层	% icv -explorer lvs \ -explorer_lvs_layers ALL \ runsetfile	对 SOC Top cells (全部层) 做基于 text 的 short 检查
在默认基础上加入全部 cells 与层	% icv -explorer lvs \ -explorer_lvs_cells ALL \ -explorer_lvs_layers ALL \ runsetfile	对全部 cells、全部层做基于 text 的 short 检查
在默认基础上加入 LVS compare 检查	% icv -explorer lvs \ -explorer_lvs_checks ALL \ runsetfile	对 SOC Top cells (仅金属层) 做基于 text 的 short 检查，并附带 LVS compare
在 Explorer LVS 中保留 CellA 与 CellB	% icv -explorer lvs \ -explorer_lvs_preserve_cells \ "CellA CellB" runsetfile	与默认行为相同，唯一区别是 Explorer LVS 保证保留 CellA 与 CellB

7.4 Explorer LVS 许可证

Explorer LVS Licenses

IC Validator Explorer LVS 需要 Elite 许可。该功能需要下列 IC Validator 许可证：

- ICValidator-Manager
- ICValidator2-GeometryEngine

参见

更多信息见本手册「许可证与资源需求」一章。

Chapter 8

PXL

PXL

本章介绍可编程扩展语言 PXL，以及在 IC Validator 工具中函数定义与函数使用的基础。

PXL 包括以下各节：

- PXL 体系结构 (PXL Architecture)
- 一般定义 (General Definitions)
- 变量与运算符 (Variables and Operators)
- 数据类型与类型系统 (Data Types and Typing)
- 流程控制 (Flow Control)
- 函数 (Functions)
- 程序结构 (Program Structure)
- 模块化代码示例 (Example of Modularized Code)

8.1 PXL 体系结构

PXL Architecture

PXL 完全可编程，具备下列一般可编程性特性：

- 变量 (Variables)
- 流程控制 (Flow Control)
- 带标准索引的列表 (Lists / arrays)
- 结构体 (Structures)
- 哈希表 (Hashes)
- 枚举类型 (Enumerated types)
- 用户定义函数 (User-defined functions)
- 数学、逻辑与字符串运算
- 支持命令行选项的宏 (Macros)
- 环境变量 (Environment variables)

- 静态作用域（Static scope）

这些特性可带来：

- 直观的 `runset` 开发
- 易于维护的 `runset`
- 简洁的代码
- 通过子程序实现代码复用
- 支持用户定义函数

图 8.1 给出 PXL 的基本结构。

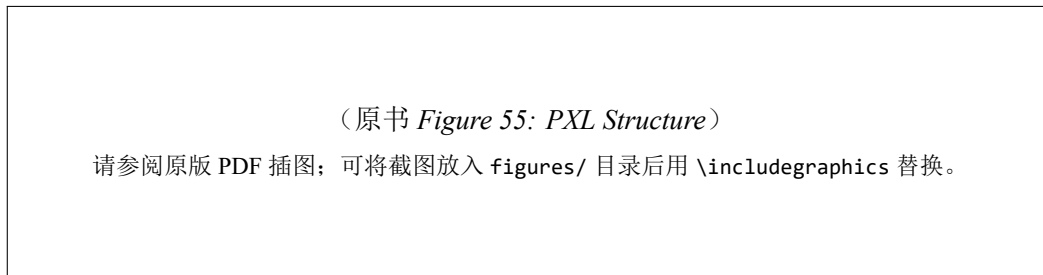


图 8.1: PXL 结构

8.2 一般定义

General Definitions

本节给出 PXL 中部分语法元素的定义与特征，例如标识符、字符串字面量、关键字、运算符与注释。理解这些语法元素的用途与作用，有助于编写 `runset`。

8.2.1 标识符

Identifiers

任意 PXL 组件的标识符均可按任意顺序包含字母字符、数字与下划线。PXL 区分大小写。下列标识符彼此不同：

- `abCD`
- `ABCD`
- `abcd`

标识符须遵守下列命名限制：

- 双下划线前缀（`__`）。不允许以两个下划线开头的标识符；保留给语言内部使用。
- 单下划线前缀（`_`）。不允许以一个下划线开头的标识符；保留给产品使用。
- 全大写。保留给应用程序编程接口（API）使用。不过，可将全大写标识符用作枚举类型元素。
- 保留关键字。不得将保留关键字用作标识符。PXL 保留关键字列表见第 8.2.4 节「关键字」。

下列名称也不可用作标识符：

- 函数名：这些函数定义于 *IC Validator Reference Manual* 与 IC Validator 头文件中。示例函数名包括 `capacitor`、`check_property`、`internal11` 与 `xor`。
- 容器类型名：列表、哈希与结构体的名称定义于 IC Validator 头文件中。容器类型名示例：
 - 列表：`coordinate_1` 用于定义 `property()` 函数

- 哈希: `layer_list_h` 用于定义 `density()` 函数
- 结构体: `check_property_s` 用于定义 `check_property()` 函数

合法标识符示例:

```
This
IsAnIdentifier
another_identifier_using_underscores
layer20
```

保留（非法）标识符示例:

```
__SYSTEMNAME
```

（双下划线前缀：保留给语言内部使用）

```
_productname
```

（单下划线前缀：保留给产品使用）

```
HIERARCHICAL
```

（保留关键字）

8.2.2 字符串字面量

String Literals

字符串字面量是以双引号（"）开始并结束的一组字符。

注意

文本字符串的第一个字符不能是整数，也不能是后跟整数的空格。
字符串字面量不得包含换行。若字符串字面量中需要换行，必须使用换行字符（`\n`）。

下列字符组均为字符串字面量示例:

```
"hello"
"\ "
"a string with\nmultiple lines"
"abc" "def"
```

字符串字面量会自动与相邻的字符串字面量合并（字符串拼接）。尤其是，它们会与包含字符串值的预处理器变量（如 `__FILE__`）以及环境变量引用（如 `$(id)`）合并。

可在字符串内使用转义序列，将部分字符串内容标记为给编译器的指令。PXL 可用的转义序列见表 8.1。

表 8.1: 转义序列（Escape Sequences）

转义	定义
<code>\n</code>	换行字符
<code>\"</code>	双引号字符
<code>\\</code>	单个反斜杠字符
<code>\NNN</code>	字符的八进制值
<code>\t</code>	制表符
<code>\xNN</code>	字符的十六进制值

8.2.3 数值常量

Numeric Constants

PXL 中的数值常量定义见表 8.2。

表 8.2: 数值常量 (Numeric Constants)

数值类型	条件	示例
double	含指数	4e5, 10.305e+10
double	含小数点	3.141592
十进制整数 (decimal integer)	以数字 1–9 开头	1023
八进制数 (octal number)	以零 (0) 开头, 且各位为 0–7	0346
十六进制数 (hexadecimal number)	以 0x 或 0X 开头, 且仅含十六进制数字 (0–9、a–f、A–F)	0xDEADBEEF

8.2.4 关键字

Keywords

关键字是语言识别的内置词。关键字为保留字, 因此不能用作标识符。
关键字的若干用途包括:

- 流程控制, 如 if、foreach 与 while
- 运算符, 如 in 与 contains
- 常量, 如 true 与 false

下列各表列出 PXL 中的保留字。

注意

有关命名限制的更多信息, 见「标识符」小节。

表 8.3 列出由系统保留、外部不可用的关键字。

表 8.3: 系统关键字 (System Keywords)

barrier	broadcast	builtin
deprecated	intrinsic	method
obsolete	preprocess	primary
proxy_out	published	required
shutdown	unsafe	unique
volatile		

表 8.4 列出 In-Design 流程使用的关键字。请勿在 IC Validator runset 中使用这些名称作为变量。

表 8.4: In-Design 流程关键字 (In-Design Flow Keywords)

INCREMENTAL_MILKYWAY_OPTIONS_INCLUDE_VIEW	INCREMENTAL_OPTIONS_SELECT_WINDOW
INDESIGN_DO_FLOATING_VIA_FILL_PURGE	INDESIGN_DONT_UPDATE_TECHNOLOGY_FILE
INDESIGN_ENABLE_MY_ASSIGN	METAL_FILL_BLOCKAGE_LAYERS
METAL_FILL_BLOCKAGE_VIEW	METAL_FILL_DATATYPES

METAL_FILL_DENSITY_DEBUG	METAL_FILL_DENSITY_CHECK
METAL_FILL_DENSITY_DELTA_WIN_SIZE	METAL_FILL_DENSITY_MIN
METAL_FILL_FLATTEN	METAL_FILL_INLIB
METAL_FILL_INLIB_CELL	METAL_FILL_INLIB_EXCLUDE_FILL
METAL_FILL_INLIB_PATH	METAL_FILL_INSTALL_ICV_RH
METAL_FILL_INSTALL_PRIMEYIELD_RH	METAL_FILL_LAYER_NUMBERS
METAL_FILL_OUTLIB	METAL_FILL_OUTLIB_APPLY_PREFIX
METAL_FILL_OUTLIB_CELL	METAL_FILL_OUTLIB_CELL_PREFIX
METAL_FILL_OUTLIB_MODE	METAL_FILL_OUTLIB_PATH
METAL_FILL_OUTLIB_TEMP_CELL	METAL_FILL_OUTLIB_VIEW
METAL_FILL_OUTLIB_RF_CELL_PREFIX	METAL_FILL_RUNSET2LIBRARY_LAYER_MAP
METAL_FILL_SELECT_WINDOW	METAL_FILL_USER_RUNSET
_METHODOLOGY_FUNCTIONS_RS_	SAVE_METAL_FILL_INLIB_EXCLUDE_FILL
_SMF_ICV_RH_	SNPSINDESIGN
VIA_FILL_ALLOWED_ONE_SIDE	VIA_FILL_ASSOCIATED_METAL
VIA_FILL_DATATYPES	VIA_FILL_LAYER_NUMBERS
VIA_FILL_MASK_NAMES	VIA_FILL_MET_ENCLOSURE
VIA_FILL_OUTLIB_RF_CELL_PREFIX	_WRITE_AREFS

表 8.5 列出不可用作标识符的保留应用程序标识符。

表 8.5: 保留应用程序标识符 (Reserved Application Identifiers)

binary	boolean	break
by	const	constraint
constraint_category	CONSTRAINT_EQ	CONSTRAINT_GE
CONSTRAINT_GELE	CONSTRAINT_GELT	CONSTRAINT_GT
CONSTRAINT_GTLE	CONSTRAINT_GTLT	CONSTRAINT_LE
CONSTRAINT_LT	CONSTRAINT_NE	continue
DELETE_TEXT	DELETE_AND_REPORT	defined
double	elif	else
elseif	enum	false
for	foreach	function
hash	if	in
in_out	integer	list
newtype	of	on
out	return	returning
step	string	struct
substrate	SUBSTRATE	thru
to	true	unary
undefined	violation	void
while		

枚举类型元素也不能用作标识符。表 8.6 列出 IC Validator 工具使用的枚举类型元素。

表 8.6: 枚举类型元素 (Enumerated Type Elements)

ABORT	ABSOLUTE	ABSTRACT_DESIGN
-------	----------	-----------------

ACCEPT	ACUTE	ADJUST
ALIEN	ALL	ALL_CELLS
ALL_POLYGONS	ALLOW	ALWAYS
AMBIT_ON_BLOCK	ANALOG_DESIGN	ANALOG_GROUND_NET
ANALOG_POWER_NET	ANALOG_SIGNAL_NET	ANNOTATION_FILE
ANTENNA_CELL	ANTIDISHING_BRIDGE	ANY_DEVICE_NAME
ANY_LEVEL	APPEND	APPEND_NEW_VERSION
AREA_FILL	AREA_FILL_USE	ARC
ARC_PLAN_CELL	ASSIGNS	ASYMMETRIC
ASYMMETRIC_INTERSECTING	AUTO	AUTO_DETECT
AUTOMATIC	AXIS	BACKSLASH
BALANCED	BASE	BISECTOR
BLACK_BOX_DESIGN	BLOCK_RADIAL	BLOCK_SQUARE
BLOCKAGE_OBJECT	BODY_EXTENTS	BODY_POLYGONS
BOTH	BOUNDARY	BOTTOM
BOTTOM_HEIGHT	BREAK_PATH	BRIEF
BULK	BUS	BV_CELL
CAPACITOR	CASE_INSENSITIVE_NAME	CATEGORY_BLOCKAGE
CDL	CELL_LEVEL	CELL_ROW
CENTER	CENTER_TO_CENTER	CENTERLINE
CHECK	CHIP	CHIP_CONTEXT
CHIP_CELL	CHIPWIDE_AVERAGE	CLASSIFY
CLEAN	CLIP	CLIPPED_DELTA_WINDOW
CLIPPED_CENTER	CLOCK	CLOCK_0_SKEW
CLOCK_AND_SPLIT_CLOCK	CLOCK_BUFFER_CELL	CLOCK_NET
CLOCK_RING	CLOCK_STRIPE	CLOSED_OUTER_BOUNDARY
CLOSEST	CMPSIM	COINCIDENT
COLLECTOR	COLOR_1	COLOR_2
COLOR_LAYER_TYPE	COLOR_LINE_END	COLOR_RING
COLOR_SIDE	COMBINE_COLOR	COMMENT
CONCAVE_TO_CONCAVE	CONCAVE_TO_EDGE	CONDUCTING_LAYER_TYPE
CONDUCTIVE_HEIGHT	CONNECT	CONFLICT
CONTACT	CONTACTARRAY	CONTINUE
CONVERT_TO_POLYGON	CONVERT_TO_RECT	CONVEX_TO_CONCAVE
CONVEX_TO_CONVEX	CONVEX_TO_EDGE	COORDINATES_AND_DEVICE_TYPES
CORE_AREA_OBJECT	CORE_OF_BLOCK	CORE_WIRE
CORE_WIRE_USE	CORNER	CORNER_PAD_CELL
CORNER_PAD_DESIGN	CORNER_TO_CORNER	CORNER_TO_CORNER_OR_EDGE
CORNER_TO_EDGE	COUNT	COVER_CELL
COVER_DESIGN	CREATED_PORTS	CROSS
CUMULATIVE_TOPOGRAPHY	CUSTOM	CUTTING
DATATYPE_ZERO_ONLY	DEEP_NWELL_NET	DEEP_PWELL_NET
DEFocus	DELTA_WINDOW	DENSITY
DENSITY_GRADIENT	DENSITY_NORMAL	DENSITY_PERIMETER
DENSITY_SMARTSTEP	DENSITY_TOTARGE	DENSITY_UNDER_1
DENSITY_UNDER_2	DEPTH_OF_FOCUS_COLUMNS	DEPTH_OF_FOCUS_COLUMNS_HIGH
DEPTH_OF_FOCUS_COLUMNS_LOW	DEPTH_OF_FOCUS_ROWS	DEPTH_OF_FOCUS_ROWS_HIGH
DEPTH_OF_FOCUS_ROWS_LOW	DESIGN	DESIGN_VIEW
DETAIL_ROUTE	DETAIL_ROUTE_USE	DEVICE_NAME

DEVICE_PIN	DIAMOND	DIELECTRIC_HEIGHT
DIFFERENT_NET	DIFFERENT_POLYGON	DIFFERENT_SIZE_RECTANGLES
DIODE_DESIGN	DISCARD	DONUT
DOT	DOUBLE	DOUBLE_HEIGHT_STANDARD_CELL
DOUBLE_LIST	DRC	DROP
DUAL	DYNAMIC_SHIELD	EDGE
EDGES	EDGE_TO_CONCAVE	EDGE_TO_CONVEX
EDIF	EIGHT	ELLIPSE
ELLIPTICAL	ELPC	EMITTER
EMPTY	ENCLOSE	END_CAP_DESIGN
EQUATE_NETS	ERROR	ERRORS
ERROR_CLASSIFICATION	ERROR_NO_ABORT	EXCLUSIVE
EXPANDED_EDGE	EXPLODE	EXTENDED
EXTENTS	EXTERIOR	EXTERIOR_INTERIOR
EXTRACT	FAIL	FAILED
FAILED_INSERTION	FC_BUMP_CELL	FC_DRIVER_CELL
FEEDTHRU_BLOCKAGE	FEMTO	FILL
FILL_BLOCKAGE	FILL_BLOCKAGE_USE	FILL_DESIGN
FILL_RINGS	FILL_TRACK	FILL_USE
FILLER_CELL	FILLER_DESIGN	FIRST_PATTERN_LAYER
FIXED	FLIP_CHIP	FLIP_CHIP_DRIVER_DESIGN
FLIP_CHIP_PAD_CELL	FLIP_CHIP_PAD_DESIGN	FLIP_FLOP_CELL
FLOATING	FOLLOW_PIN	FOLLOW_PIN_USE
FORTY_FIVE	FOUR	FRAM
FRAME_VIEW	FULL	FULL_NAME
FULL_NAME_CASE_INSENSITIVE	FULL_NAME_CASE_SENSITIVE	GALAXY_AND_XO_CELL
GALAXY_CELL	GDSII	GENERIC
GLOBAL	GLOBAL_REPLACE	GLOBAL_ROUTE
GLOBAL_ROUTE_USE	GROUND	GROUND_NET
GROW	GUIDANCE	HALF_WIDTH
HIERARCHICAL	HIGH	HIGHEST_TEXT
HOLDING_CELL	HORIZONTAL	HORIZONTAL_AXIS
HORIZONTAL_TRACK	HORIZONTAL_TRACK_OBJECT	HORIZONTAL_WIRE_TRACK
HORIZONTALWIRE	HYBRID	HYBRID_BAIL_EARLY
HYBRID_PATTERN_ONLY	ICV	ICV_LCC_AOI
ICV_LCC_BLOCKING	ICV_LCC_CO	ICV_LCC_CONTOUR_AREA
ICV_LCC_M1	ICV_LCC_M1_DUMMY	ICV_LCC_M1_OPC_DUMMY
ICV_LCC_M2	ICV_LCC_M2_DUMMY	ICV_LCC_M2_OPC_DUMMY
ICV_LCC_M3	ICV_LCC_M3_DUMMY	ICV_LCC_M3_OPC_DUMMY
ICV_LCC_M4	ICV_LCC_M4_DUMMY	ICV_LCC_M4_OPC_DUMMY
ICV_LCC_M5	ICV_LCC_M5_DUMMY	ICV_LCC_M5_OPC_DUMMY
ICV_LCC_M6	ICV_LCC_M6_DUMMY	ICV_LCC_M6_OPC_DUMMY
ICV_LCC_M7	ICV_LCC_M7_DUMMY	ICV_LCC_M7_OPC_DUMMY
ICV_LCC_M8	ICV_LCC_M8_DUMMY	ICV_LCC_M8_OPC_DUMMY
ICV_LCC_M9	ICV_LCC_M9_DUMMY	ICV_LCC_M9_OPC_DUMMY
ICV_LCC_M10	ICV_LCC_M10_DUMMY	ICV_LCC_M10_OPC_DUMMY
ICV_LCC_MAX	ICV_LCC_OD	ICV_LCC_OD_DUMMY
ICV_LCC_OD_OPC_DUMMY	ICV_LCC_PO	ICV_LCC_PO_DUMMY
ICV_LCC_PO_OPC_DUMMY	ICV_LCC_VIA1	ICV_LCC_VIA2

ICV_LCC_VIA3	ICV_LCC_VIA4	ICV_LCC_VIA5
ICV_LCC_VIA6	ICV_LCC_VIA7	ICV_LCC_VIA8
ICV_LCC_VIA9	IF_INSERTION_FAILS	IGNORE
IGNORE_CAP_LAYER_TYPE	IMAGE_CELL	IN
INCLUSIVE	INCLUSIVE_PARALLEL	INDIVIDUAL
INDUCTOR	INNER	INPUT_LAYOUT
INSERTION	INSIDE	INSIDE_TO_OUTSIDE
INSTANCE_NAME	INSTANCES	INTERACT
INTERACTING	INTERIOR	INTERNAL_DESIGNS
INTERSECTION	IO_PAD_CELL	KEEP
KEEP_ALL	KEEP_ENCLOSED	KEEP_INTERACTING
KEEP_ONE	KILO	LATCH_CELL
LAYER	LAYER_INTENT_BASE	LAYER_INTENT_DEVICE
LAYER_INTENT_FILL	LAYER_INTENT_INTERCONNECT	LAYER_INTENT_METAL
LAYER_INTENT_VIA	LAYER_MAPS	LAYER1
LAYER2	LAYOUT_UNTEXTED	LAYOUT_VIEW
LCC	LEVEL_SOURCE_DRAIN	LI_BASE
LI_FILL	LI_INTERCONNECT	LI_METAL
LI_VIA	LIB_CELL_DESIGN	LIB_CELL_PIN_CONNECT
LIB_CELL_PIN_CONNECT_USE	LIBRARIES_AND_VIEWS	LINE
LINE_END_DIFF_COLOR	LINE_END_SAME_COLOR	LINE_END_TO_LINE_END
LINE_END_TO_SIDE	LINE_TO_LINE	LINEAR_CONTEXT
LINK	LOCAL	LOGIC_0_NET
LOGIC_1_NET	LOOP	LOW
LOWER	LOWER_CELLS	LOWEST
LVS	LVS_NETLIST_FLOW_ICV	LVS_NETLIST_FLOW_SPICE
LVS_USER_UNIT_METER	LVS_USER_UNIT_MICRON	M1_ROUTE_GUIDE
M10_ROUTE_GUIDE	M11_ROUTE_GUIDE	M12_ROUTE_GUIDE
M13_ROUTE_GUIDE	M14_ROUTE_GUIDE	M15_ROUTE_GUIDE
M2_ROUTE_GUIDE	M3_ROUTE_GUIDE	M4_ROUTE_GUIDE
M5_ROUTE_GUIDE	M6_ROUTE_GUIDE	M7_ROUTE_GUIDE
M8_ROUTE_GUIDE	M9_ROUTE_GUIDE	MACRO_CELL
MACRO_DESIGN	MACRO_PIN_CONNECT	MACRO_PIN_CONNECT_USE
MAIN	MARKER_LAYER_TYPE	MASK_EIGHT
MASK_ELEVEN	MASK_FIFTEEN	MASK_FIVE
MASK_FOUR	MASK_FOURTEEN	MASK_NINE
MASK_ONE	MASK_ONE_HARD	MASK_ONE_SOFT
MASK_SEVEN	MASK_SIX	MASK_TEN
MASK_THIRTEEN	MASK_THREE	MASK_TWELVE
MASK_TWO	MASK_TWO_HARD	MASK_TWO_SOFT
MAX	MAX_AREA	MAX_CONDITION
MAX_CONDITION	MAX_INSERTION	MEEF
MEGA	MERGE	MERGED
MERGE_ALL	MERGE_CONNECTED	MERGE_CONNECTED_AND_TOP
MERGE_TOP_THEN_CONNECTED	MERGE_TOP_PORTS_THEN_CONNECTED	MERGED
METAL_HEIGHT	METER	MICRO
MICRON	MID_CELL	MILKYWAY
MILLI	MIN	MIN_AREA
MISSING_CELLS	MISSING_COLOR	MIXED

MODULE_CELL	MODULE_DESIGN	MORE_THAN_TRIPLE_HEIGHT_CELL
MUTUAL	MUTUAL_NON_ORTHOGONAL	NAME_MATCHED
NANO	NDB_ALL	NDB_ANY
NDB_BJT	NDB_CACHE	NDB_CAP
NDB_CELL	NDB_DIODE	NDB_IGNORE
NDB_IND	NDB_INST	NDB_MOS
NDB_NET	NDB_NETLIST	NDB_NMOS
NDB_NONE	NDB_NP	NDB_NPN
NDB_ONE_WAY	NDB_PIN	NDB_PMOS
NDB_PN	NDB_PNP	NDB_RES
NDB_SUBCKT	NDB_UNDEFINED	NDB_TWO_WAY
NDB_UNCACHED	NDM	NEGATIVE_45
NEIGHBORING_GAP	NEITHER	NET_NAME
NET_PORTPROPERTY	NETLIST_ANNOTATION_FILE_SPICE	NETLIST_ONLY
NETLIST_PEX_SPICE	NETLIST_XTR_SPICE	NETS
NEVER	NEW_AREF_ONLY	NINETY
NMOS	NO_COLOR	NO_EXPLODE
NODE	NOMINAL	NOMINAL_VARIATION
NON_CONT_PG	NON_ORTHOGONAL	NON_RECTANGULAR_SHAPES
NONE	NONE_DENSITY	NONE_DENSITY_PERIMETER
NONE_INCLUSIVE	NORMAL	NOT_A_DOUBLE
NOT_ADJACENT	NOT_CONTAINED	NOT_CORNER
NOT_DEFINED_LAYER_TYPE	NOT_TEXTED	NOTCH
NP	NPN	NULL_CONNECT_DATABASE
NWELL_NET	NULL_PARAMETRIX_LAYER	OASIS
OCTAGONAL	ODD_LOOP_ONE_LINK	ODD_LOOP_ALL_LINKS
ODD_LOOP_ALL_NODES	ODD_LOOP_ALL_LOOP	ODD_LOOP_INSIDE_RING
OFF	OLD_AREF_ONLY	ON
ONE	ONE_EIGHTY	ONE_OR_NEITHER
ONE_POLYGON_PER_LAYER	ONE_X	ONLY_COLOR_1
ONLY_COLOR_2	ONLY_COORDINATES	OPC
OPC_USE	OPEN_OUTER_BOUNDARY	OPENACCESS
ORTHOGONAL	OTHER_NETS	OUT
OUTPUT_CELL	OUTPUT_ERRORS	OUTSIDE
OVERLAP	OVER_EXPOSURE	OVERSIZE
OVERWRITE	OWNER_NET	OXIDE_HEIGHT
PAD_DESIGN	PAD_FILLER_CELL	PAD_SPACER_DESIGN
PARALLEL	PARALLEL_POINT_PROJECTION	PARTITION
PARTITION_CONTEXT	PATH	PATH_OBJECT
PATH_SEGMENT	PAUSE	PERCENT
PERIMETER	PERIMETER_DENSITY	PERPENDICULAR
PG_FOLLOW_PIN	PG_PIN	PG_PIN_ONLY_CELL
PG_RING	PG_STRIPE	PHYSICAL_ONLY_DESIGN
PICO	PIN	PIN_BLOCKAGE
PIN_NAME	PIN_OBJECT	PIN_TEXT
PINS	PLACEMENT_BLOCKAGE	PLACEMENT_HARD_BLOCKAGE
PLACEMENT_HARD_MACRO_BLOCKAGE	PLACEMENT_PARTIAL_BLOCKAGE	PLACEMENT_SOFT_BLOCKAGE
PM_EXACT	PM_EDGE_NONUNIFORM	PM_EDGE_UNIFORM
PM_EQ	PM_FUZZY	PM_GE

PM_GT	PM_LE	PM_LT
PM_MARKER_CENTER	PM_MARKER_EDGE	PM_NONE
PM_ONE_DIMENSIONAL	PM_TWO_DIMENSIONAL	PM_TWO_DIMENSIONAL_RUN_LENGTH
PM_UNIFORM	PMOS	PN
PNP	POINT	POINT_TO_POINT
POINT_TOUCH	POLY_ROUTE_GUIDE	POLYGON
POLYGON_OBJECT	POLYGON_TEXT	PORT_NAME
PORT_TEXT_MATCHED	PORT_TOPOLOGY_MATCHED	POSITIVE_45
POWER	POWER_AND_GROUND	POWER_NET
POWER_OR_GROUND	POWER_XOR_GROUND	PRE_COLOR_ALL_NODES
PRE_COLOR_ENVELOPE_PATH	PRE_COLOR_MIN_PATH	PRE_THREE_COLOR_ALL_NODES
PRE_THREE_COLOR_MIN_PATH	PREFIX_NAME	PRIMARY_AXIS
PRINT	PRINT_ALL	PRINT_RESET_ALL
PROJECTING	PROJECTION	PROPERTY
PROPERTY_ERRORS_ONLY	PROPERTY_TEXT	PWELL_NET
QCMP	QTF_LAYER_TYPE	RADIAL
RADIAL_INSIDE	RADIAL_INTERSECTION	RADIAL_OUTSIDE
RAM_CELL	REASSIGN	REASSIGNED_SHORTED
RDL_USE	RECT	RECTANGLE
RECTANGLE_OBJECT	REDUCTION	REGION
REGION_CONTEXT	REGULAR_TEXT	RELATED_COINCIDENT
RELATIVE	REMAINDER	REMOVE_LAYER_TYPE
REMOVE_MULTI_EQUIVS	RENAME	RENAMED
REPLACE	REPORT	RESET
RESET_NET	RESISTOR	RETAIN_45
RIGHT	RING	RING_USE
ROM_CELL	ROTATE_180	ROTATIONAL
ROUND	ROUTE_GUIDE_OBJECT	ROUTE_TYPE
ROUTING_BLOCKAGE	ROUTING_FOR_TOP_LEVEL_BLOCKAGE	RPGROUP_BLOCKAGE
SAME	SAME_COLOR	SAME_DEVICE_NAME_ONLY
SAME_MASK	SAME_NET	SAME_POLYGON
SCAN_NET	SCREEN_BLOCKAGE	SERIES
SERIES_PARALLEL	SHAPING_BLOCKAGE	SHIELD_ROUTE
SHIELD_ROUTE_USE	SHORT_BY_LONG	SHORTED
SHRINK	SIDE_DIFF_COLOR	SIDE_SAME_COLOR
SIDE_TO_SIDE	SIGNAL	SIGNAL_CONTEXT
SIGNAL_DETAIL	SIGNAL_GLOBAL	SIGNAL_LOCAL
SIGNAL_NET	SIGNAL_USER	SIMPLE
SIMULATE	SINGLE	SINGLE_DEVICE_EQUIVS
SINKS	SIXTEEN	SKIP
SKIP_PARENTS	SLASH	SLEEP_CONTROL
SLOT_BLOCKAGE	SOFT_MACRO_IO_FIXED_CELL	SOG_DESIGN_CELL
SOURCES_ON_SAME_NET	SPECIAL_POWER	SPECIAL_VIA_CELL
SPICE	SPICE_BJT	SPICE_CAP
SPICE_CELL	SPICE_DIODE	SPICE_IND
SPICE_MOS	SPICE_RES	SPLIT_CLOCK
SQUARE	SQUARE_CENTERED	SQUARE_EXTENDED
SQUARE_INSIDE	SQUARE_OUTSIDE	STACK_VIA_CELL
STANDARD_CELL	STANDARD_FILLER_CELL	START

STATISTICAL_COUNT	STOP	STRING
STRIPE	STRIPE_USE	SUBSTRING_NAME
SUM	SYMMETRIC	SYMMETRIC_NON_INTERSECTING
TAP_CELL	TECH	TERMINAL
TERMINAL_NAME	TERMINAL_TEXT	TEXT
TEXT_DISPLAY	TEXTED	TIE_HIGH
TIE_LOW	TOP	TOP_CELL
TOP_OF_NET	TOP_PORT	TOUCH
TREE	TRIANGLE	TRIANGLE_ONE_LINK
TRIANGLE_ALL_LINKS	TRIANGLE_ALL_NODES	TRIPLE_HEIGHT_STANDARD_CELL
TRUNCATE	TSV_CELL	TWO
TWO_SEVENTY	TWO_X	UF_ADJUSTABLE
UF_CELL	UF_CHECKERED	UF_DPT_SPACING
UF_EXPANDABLE	UF_LAYER	UF_LINE_END
UF_LINE_SIDE	UF_PATTERN	UF_POLYGON
UF_STACK	UF_STRIPE	UNCONNECTED
UNDER_EXPOSURE	UNDERSIZE	UNSET_NET
UNKNOWN_L_MODEL_CELL	UNMATCHED_PINS	UNMATCHED_TEXT
UNSPECIFIED	UNSPECIFIED_DOUBLE	UNUSED
UNUSED_CELLS	UNUSED_MARKERS	UPDATE
UPPER	USE	USE_ALL_EQUIV
USE_MILKYWAY	USE_OPENACCESS	USE_NDM
USED_CELLS	USER_DEFINED	USER_ROUTE
USER_ROUTE_USE	VCELL	VCMP
VERBOSE	VERILOG	VERTICAL
VERTICAL_AXIS	VERTICAL_TRACK	VERTICAL_TRACK_OBJECT
VERTICAL_WIRE_TRACK	VERTICALWIRE	VERTICES
VIA_BLOCKAGE	VIA_LAYER_TYPE	VIA_OBJECT
VIEWONLY_LAYER_TYPE	VIEWS	VIRTUAL_AMBIT
VIRTUAL_CORE	WAIVE	WARN
WARNING	WATCH	WELL_TAP_DESIGN
WINDOW	WIRING_BLOCKAGE	WITH_PORTS
X_BY_Y	XO_CELL	ZERO_SKEW
ZERO_SKEW_USE		

8.2.5 运算符

Operators

运算符是具有特定含义的符号。例如，+ 表示将两个数相加。运算符类型见表 8.7。

表 8.7: 运算符类型 (Operator Types)

运算符类型	操作数个数	示例
一元 (unary)	1	逻辑非 (!)
二元 (binary)	2	加法 (+)、幂运算 (**)
三元 (ternary)	3	条件运算符 (e1 ? e2 : e3)

8.2.6 注释

Comments

注释是用于说明程序的文本，便于向读者表明代码意图。注释按空白处理。表 8.8 按注释所占行数给出注释类型。

表 8.8: 注释类型（Comment Types）

注释类型	起始	结束	长度
行注释（line）	//	下一换行	一行
块注释（block）	/*	*/	任意长度

块注释不可嵌套。若在注释内看到 `*/`，预处理器会报告警告。例如，若 `runset` 含有：

```
/*      - 1st comment start
      /*      - 2nd comment start
      */      - 2nd comment end
*/      - 1st comment end
```

则「1st comment start」与「2nd comment end」配对；「1st comment start」不会与「1st comment end」配对。
注释示例如下：

```
/* this is a single-line block comment */

// this is a line comment, so /* is ignored

/* block comments /* cannot nest. Therefore this comment evokes a
warning from the preprocessor */

/*
 * block comments can span
 * several lines, such
 * as this one does.
 */

/*
  A leading * is not required on each line of a
  block comment but does make the comment stand out
  from the program.
*/
```

8.2.7 环境变量引用

Environment Variable Reference

环境变量是一组可用于控制进程运行方式的动态值。
环境变量引用由美元符（`$`）后跟任意合法标识符组成。若引用未定义的环境变量，将在编译期报错并拒绝。
环境变量会被替换为包含其值的字符串字面量。仅对形如 `$<id>` 的记号进行替换。具体而言，字符串字面量内部不会替换；由于相邻字符串字面量会拼接，通常也无需在字面量内替换。

注意

若环境变量的值为空字符串，则将其转换为值为空字符串的字符串字面量。

环境变量用法示例如下：

```
x : string = $y;

/*
 * assuming environment variable my_value contains the value abcd,
 * the following code evaluates to true; otherwise evaluates to false.
 */
res : boolean = ( $my_value == "abcd" );

/*
 * if environment variable aaa contains the value
 * /home/dir/path, the following code produces a string
 * containing a path to the file "cases" in that directory.
 */
path : string = $aaa "/cases";

/*
 * if environment variable bbb is defined but empty, the
 * following code produces the string abcdef.
 */
mystr : string = "abc" $bbb "def";
```

8.3 变量与运算符

Variables and Operators

本节介绍 PXL 中的变量与运算符概念。

8.3.1 变量

Variables

变量用于在程序中临时存储信息。声明变量时，变量名后须跟冒号（:）与变量类型。声明时也可给出初值。见下一小节示例。

注意

有关命名限制，见「标识符」小节。

初始化变量

Initializing Variables 可用两种方式将变量初始化为任意受支持的数据类型：

- 使用字面量值。

```
y : boolean = true;           // initialize by literal value
```

- 使用表达式。

```
z : boolean = !y;           // initialized by expression
```

下列示例展示不同的初始化方法：

```
x : integer;                // no initial value
pi : double = 3.141592;    // initialized by value
pi2 : double = pi * 2;     // initialized by expression
s : string = "abc" "def";  // concatenated
s1 : string = "abc" + pi;  // converted to string, then concatenated
c : constraint of double = (10.0, 200]; // initialized by value
```

变量作用域

Scope of Variables PXL 中的作用域（scope）指变量与表达式在程序若干上下文中的含义。下文通过示例说明与变量作用域相关的规则与条件。

局部作用域由一对花括号（{ }）定义。也就是说，用花括号定义的复合语句会引入新的变量作用域。花括号的用法见后文「复合语句」（Compound Statement）一节。

流程控制语句会创建新作用域。

```
if ( var == 1) {            // Start of new scope
    my_var: integer = 1;
    ...
}                           // End of scope.
```

- 在局部作用域中声明的变量在该作用域外不存在。也就是说，若变量在某一作用域内声明，则不能在该作用域外使用。例如：

```
{
    x = y + b;
}
z = x;    // x is not defined; it only exists in the local scope
```

- 变量从最外层作用域到最内层作用域全局可见。例如：

```
{
    x : integer = 1;
    {
        a = x + b; // x exists and can be seen within this scope
                  // (global from outer scope)
        x = y + b; // global value of x is overwritten
    }
    z = x;        // x is defined here and contains the value of y + b
}
z = x;           // x is not defined here - parser error
```

- 在外层作用域声明、又在内层作用域重新声明的变量会被遮蔽（shadowed），以避免值冲突。
 - 若发生变量遮蔽，且被遮蔽变量的作用域并非由函数声明定义，则程序生成警告。
 - 在函数调用内，遮蔽是预期行为，不会警告。例如：

```

y: integer = 1;
x: integer = 1;
{
  y : integer = 3;      // The original y value is shadowed.
                        // This is a new y.
  x = x + 1;            // x is equal to 2 here
  y = y + 1;            // y is equal to 4 here
}

                        // y is equal to 1 here - the shadowed value
                        // returns when exiting the inner scope
                        // x is equal to 2 here

```

下面是变量遮蔽的另一示例。函数声明：

```

fxy : function (
  x : integer;
  y : integer;
) returning z : integer {
  x = x + 1;
  y = y + 1;
  z = x + y;
};

```

函数调用：

```

x1 : integer = 1;
y : integer = 1;

z1 = fxy(x1, y);           // within the function y is shadowed
                           // y in the local scope is "2"

x2 = x1;                   // x1 is "1"
y1 = y;                    // y is still "1"
z2 = z1;                   // z1 is "4"

```

8.3.2 运算符

Operators

运算符是表达式中用于指定在求值时应执行某些操作的符号。

PXL 支持下列运算符：

- 赋值运算符（Assignment Operators）
- 关系运算符（Relational Operators）
- 逻辑运算符（Logical Operators）
- 条件运算符（Conditional Operator）
- 数值与按位运算符（Numeric and Bitwise Operators）
- 仅输出 double 的运算符（Operators With Double-Only Outputs）

赋值运算符

Assignment Operators 赋值运算符对表达式求值并将其保存到目标左值（lvalue）。左值是可容纳值、并可出现在赋值语句左侧的表达式。

涉及在约束值中使用 > 或 >= 的赋值，赋值运算符前后必须有空白。也可将约束值放在赋值 = 之后的括号内。
例如：

```
abc = >=3;
xyz = (>5);
```

注意

赋值运算符不可串联。下例不合法：
a = b = c

正确赋值示例如下：

```
x:      integer;      // declare a variable named x

point: newtype struct of { x: integer; y: integer; }; // declare a struct
                                     // of integers

p: point;
x  = 42;      // assign x a value of 42
p.x = 10;     // assign p.x a value of 10
p.y = 27;     // assign p.y a value of 27
```

关系运算符

Relational Operators 关系表达式将表达式求值为布尔值 true 或 false。表 8.9 定义关系运算符。

表 8.9: 关系运算符（Relational Operators）

语法	说明	适用于
e1 == e2	若 e1 等于 e2 则为 true，否则为 false	所有完全定义的数据类型
e1 != e2	若 e1 不等于 e2 则为 true，否则为 false	所有完全定义的数据类型
e1 < e2	若 e1 小于 e2 则为 true，否则为 false	数值
e1 <= e2	若 e1 不大于 e2 则为 true，否则为 false	数值
e1 >= e2	若 e1 不小于 e2 则为 true，否则为 false	数值
e1 > e2	若 e1 大于 e2 则为 true，否则为 false	数值

逻辑运算符

Logical Operators 逻辑表达式求值为布尔值。逻辑运算符要求全部参数均为 boolean 类型。表达式从左到右求值，并在结果已确定时停止。表 8.10 定义逻辑运算符。

表 8.10: 逻辑运算符（Logical Operators）

语法	说明
e1 && e2	若 e1 为 false，则结果为 false，否则为 e2
e1 e2	若 e1 为 true，则结果为 true，否则为 e2

语法	说明
<code>! e1</code>	若 <code>e1</code> 为 <code>true</code> ，则结果为 <code>false</code> ，否则为 <code>true</code>

条件运算符

Conditional Operator 条件表达式接受三个参数；第一个参数必须为 `boolean` 类型。表 8.11 定义条件运算符。

表 8.11: 条件运算符（Conditional Operator）

语法	说明
<code>e1 ? e2 : e3</code>	若 <code>e1</code> 为 <code>true</code> ，则为 <code>e2</code> ，否则为 <code>e3</code>

考虑带有三个参数 `e1`、`e2`、`e3` 的条件运算符。求值顺序如下：

- 1. 求值第一个参数（`e1`）。
- 2. 若第一个参数（`e1`）为 `true`，则求值第二个参数。
- 3. 若第一个参数（`e1`）为 `false`，则求值第三个参数（`e3`）。

数值与按位运算符

Numeric and Bitwise Operators PXL 支持全部标准数学运算符。按位运算符仅支持整数。表 8.12 定义数值与按位运算符。

表 8.12: 数值与按位运算符（Numeric and Bitwise Operators）

语法	说明	允许类型	注释
<code>e1 + e2</code>	加法	Integer 与 double	任一参数为 double 时返回 double
<code>e1 - e2</code>	减法	Integer 与 double	任一参数为 double 时返回 double
<code>e1 * e2</code>	乘法	Integer 与 double	任一参数为 double 时返回 double
<code>e1 % e2</code>	整数余数	Integer 与 double	任一参数为 double 时返回 double
<code>e1 << e2</code>	逻辑左移（移入零）	Integer	
<code>e1 >> e2</code>	逻辑右移（移入零）	Integer	
<code>e1 & e2</code>	按位 AND	Integer	
<code>e1 e2</code>	按位 OR	Integer	
<code>e1 ^ e2</code>	按位 XOR	Integer	
<code>! e1</code>	按位 NOT	Integer	

仅输出 `double` 的运算符

Operators With Double-Only Outputs 除法（`/`）与幂运算（`**`）将参数视为 `double`，并且始终只返回 `double`。表 8.13 定义这些运算符。

表 8.13: 仅输出 double 的运算符（Operators With Double-Only Outputs）

语法	说明	接受	注释
<code>e1 / e2</code>	除法	Integer 与 double	<code>e1</code> 与 <code>e2</code> 被强制为 <code>double</code> ；运算始终返回 <code>double</code>

语法	说明	接受	注释
e1 ** e2	幂运算	Double	e1 与 e2 被强制为 double; 运算始终返回 double

注意

任何含除法运算符（/）或幂运算符（**）的表达式，均不可用作列表或哈希表的索引。

若要执行整数运算，必须将该运算包在 `dtol()` 函数调用中。该函数将 `double` 转换为 `integer`。例如：

```
var : integer = dtol(10/3);
```

8.3.3 成员引用

Member Reference

成员引用表达式访问复合对象的一个成员。成员引用取该单一成员的类型。该成员是左值，并可作为赋值目标。表 8.14 定义成员引用表达式。有关左值的更多信息，见「赋值运算符」。

表 8.14: 成员引用（Member Reference）

语法	说明
e1 . e2	按名称访问结构体成员
e1 [e2]	按索引访问列表或哈希成员

8.3.4 优先级与结合性

Precedence and Associativity

表 8.15 列出 PXL 中全部运算符的优先级与结合性。表顶优先级最高，表底最低。

表 8.15: 运算符的优先级与结合性（最高优先级在前）

运算符	结合性
() [] .	从左到右
**	从右到左（幂运算）
! + -	从右到左（一元运算符）
== != < <= > >=	非法（一元运算符）
* / %	从左到右
+ -	从左到右
<< >>	从左到右
< <= > >=	非结合
== !=	非结合
&	从左到右
^	从左到右
	从左到右
&&	从左到右
	从左到右
?:	从右到左

运算符	结合性
= @= ?=	非法
用户定义运算符	非结合

表 8.16 定义各类结合性。

表 8.16: 结合性 (Associativity)

结合性	定义
从左到右 (Left to right)	表达式从左到右求值。
从右到左 (Right to left)	表达式从右到左求值。
非结合 (Nonassociative)	需要括号。
非法 (Illegal)	该运算符不可串联。运算符产生的类型与其消费的类型不同。

下列示例说明结合性如何工作：

```
/*
 * pairs of expressions show evaluation order
 */

A + B + C
( A + B ) + C

A ** B ** C
A ** ( B ** C )

A * B + C >> 2
( ( A * B ) + C ) >> 2

A >= B == C
( A >= B ) == C

A < B == C > D
( A < B ) == ( C > D )
```

8.4 数据类型与类型系统

Data Types and Typing

本节说明 PXL 支持的数据类型，以及决定 PXL 内类型如何工作的因素。
变量可显式声明类型。例如：

```
i : integer = 1 ;           // i is declared as an integer
```

变量也可根据运算符的返回类型隐式定型。例如：

```
a : double = 1.1203;
b : double = 2.0213;
c = a + b;                  // c is implicitly the type of "double"
```

8.4.1 受支持的数据类型

Supported Data Types

PXL 支持多数基本类型以及一些工具相关数据类型。

基本类型

Primitive Types 表 8.17 给出 PXL 支持的基本数据类型。

表 8.17: 基本类型（Primitive Types）

数据类型	说明
integer	整数；大小由实现定义，一般为 32 位有符号值。
double	双精度浮点；大小在实现时定义，一般为 64 位值。
boolean	true 或 false
string	字符序列；支持用转义序列包含特殊字符。
handle	不透明数据类型，用于跟踪 PXL 程序本身之外的数据对象。
constraint of double 与 constraint of integer	连续 double 值或 integer 值集合的简洁表示。
predicate_handle	布尔表达式，并对将层标识符求值为空/非空有特殊处理。

integer 整数声明示例如下：

```
i : integer;      // declared but not initialized
i : integer = 1;  // declared and initialized
```

string 定义零个或多个字符的字符串。string 类型由字符串字面量初始化。例如：

```
s : string = "xyzyz";
```

string 类型对任意 string 类型表达式支持表 8.18 所示操作。

表 8.18: 字符串类型操作（String Type Operations）

操作	e2 类型	返回	修改 e1	说明
e1.empty()	–	boolean	否	若字符串为空（即 ""）则返回 true；否则返回 false。
e1.size()	–	integer	否	返回字符串中的字符数。

handle 外部管理对象的内部表示。类型 handle 具有对每个外部管理对象唯一的字符串表示。

constraint of double 与 constraint of integer constraint of double 与 constraint of integer 是尺寸规则值的数学区间。这些约束为描述 DRC 间距距离等参数提供直观机制。

若未指定约束类型，则隐含该区间为 constraint of double。

约束数据类型对属于该类型的表达式支持表 8.19 所示操作。

表 8.19: constraint of double 与 constraint of integer 的操作

操作	e2 类型	返回	修改 e1	说明
e1.contains(e2)	item	boolean	否	若 e2 值落在 e1 约束指定的区间内则返回 true。
e1.category()	–	enum	否	返回该约束在预定义枚举 constraint_category 中的枚举元。见表 8.20。
e1.lo()	–	integer/double	否	对二元约束, 返回下界: constraint of double 为 double; constraint of integer 为 integer。 ¹
e1.hi()	–	integer/double	否	对二元约束, 返回上界: constraint of double 为 double; constraint of integer 为 integer。 ¹

¹ 对一元约束（如 == 与 >），e1.lo() 与 e1.hi() 均返回该单一界限。
约束数据类型只能通过从另一同类型约束赋值，或从字面常量创建，如表 8.20 所示。

表 8.20: 约束类别（Constraint Categories）

字面量	说明	约束类别
[a,b]	从 a 到 b 的闭区间，包含两端点	CONSTRAINT_GELE
(a,b)	从 a 到 b 的开区间，不含两端点	CONSTRAINT_GTLT
[a,b)	从 a 到 b 的半开区间，含 a 不含 b	CONSTRAINT_GELT
(a,b]	从 a 到 b 的半开区间，不含 a 含 b	CONSTRAINT_GTLE
>= a	大于等于 a 的全部数	CONSTRAINT_GE
> a	大于 a 的全部数	CONSTRAINT_GT
<= a	小于等于 a 的全部数	CONSTRAINT_LE
< a	小于 a 的全部数	CONSTRAINT_LT
== a	仅包含数 a 的区间	CONSTRAINT_EQ
!= a	包含除 a 以外全部数的区间	CONSTRAINT_NE

lo() 与 hi() 对所有约束均有效。对单端约束，两函数均定义为返回该单一端点 a。对双端约束，lo() 返回 a，hi() 返回 b。

示例 Examples

示例如下。

- Constraint of integer
 - [2,8] 包含值 2、3、4、5、6、7、8
 - (2,8) 包含值 3、4、5、6、7
- Constraint of double
 - [2,8] 包含该区间内全部实数（含 2.0 与 8.0）
 - (2,8) 包含除 2.0 与 8.0 外该区间内的全部实数
- 声明示例

```
a_value : constraint of double = [2.0,8.0];
```

• 赋值示例

```
distance <= 2.0           // All values less than or equal to 2.0
distance = <= 2.0         // All values less than or equal to 2.0
distance = [ 2.0, 8.0 )   // All values >= 2.0 and < 8.0
count = 5                 // Count is exactly equal to 5
count = ==5               // Count is exactly equal to 5
```

注意

诸如 == 与 <= 的符号是约束的一部分。对期望约束类型的变量，赋值运算符 = 可选。但若右侧是约束类型变量，则必须使用赋值运算符：

```
my_constraint: constraint of double = < 5.0;
external1(layer1, distance = my_constraint );
external1(layer1, distance <5);                // same as above
external1(layer1, distance = < 5);             // same as above
```

predicate_handle 括在双方括号 [...] 中的布尔表达式。允许任意包含至少一个层名或违例名的布尔逻辑。层名在谓词表达式内有特殊含义：运行时若该层有数据则求值为 **true**，若层为空则求值为 **false**。违例对象在存在违例时为 **true**，无违例时为 **false**。

在 runset 函数中，**predicate_handle** 参数根据表达式求值控制给定函数的行为：

- **true**: 函数按正常方式运行。
- **false**: 函数产生空输出层。

例如：

```
temp_layer = and(poly, diff, predicate = [[metal1]]);
```

若 **metal1** 为空，则跳过 **and()** 函数，且 **temp_layer** 为空。

8.4.2 用户定义结构体与类型

User-Defined Structures and Types

除基本类型外，PXL 还支持表 8.21 所示的结构体与类型。

表 8.21: 用户定义类型（User-Defined Types）

用户定义类型	主要特征
<code>newtype...</code>	定义类型。
<code>enum of...</code>	定义枚举元素列表。
<code>list of...</code>	定义元素列表。
<code>hash of...</code>	将一种数据类型的元素列表转换为另一种数据类型。
<code>struct of...</code>	定义由多个变量组成的复合类型。
<code>function</code>	定义可传递给另一用户定义函数、并可像普通变量一样调用的用户定义函数。

newtype...

newtype... newtype 声明一种新变量类型。它并不定义变量。每种类型都是唯一的。
合法类型定义示例如下：

```
/*
 * define a new type, direction.
 */

direction: newtype enum of {
    up, down, left, right, all // trailing comma is allowed
};

/* define two distinct types */
T1: newtype integer;
T2: newtype integer;

/* define objects of the two types */
o1: T1;
o2: T2;
```

非法类型定义示例如下：

```
/* given the definitions above, o1 and o2 are of
 * different types, therefore they cannot be compared
 * and this is an error
 */

if (o1 == o2) { ... }
```

下面是定义并使用名为 **direction** 的新数据类型的示例。它有四个允许取值。

```
direction : newtype enum of {
    NORTH,
    SOUTH,
    EAST,
    WEST
};

compass : direction; // Declare compass as type "direction"
compass = EAST;      // Assign a legal value to "compass"
```

enum of...

enum of... 定义一组与其他类型不同的名称（枚举字面量）。PXL 编码标准要求枚举字面量为全大写字符。
例如：

```
t3: newtype enum of {
    X,
    Y,
    Z,          // Trailing comma is not needed; silently discarded.
};
```

另一示例：

```
direction : newtype enum of {
    NORTH, SOUTH, EAST, WEST
};

compass : direction;           // Declare compass as type "direction"
compass = EAST;                // Assign a legal value to "compass"
```

枚举可与其他枚举类型共享名称而无错误:

```
t1_e : newtype enum of {
    NONE, TOP, BOTTOM, ALL
};
```

不允许跨类型（cross-type）操作。此外，在同一类型定义内，同一名称不能使用超过一次。
下例展示跨类型操作。该操作不合法:

```
v1 : t1_e;           // NONE, TOP, BOTTOM, ALL
v1 = NONE;           // Legal assignment
v1 = TOP;            // Legal assignment
v1 = RIGHT;          // Illegal assignment

v2 : t2_e;           // NONE, LEFT, RIGHT, ALL
v2 = NONE;           // Legal assignment

if (v2 == v1) // Illegal
```

list of...

列表（list）是对象的数组。

- 所有对象属于同一类型。该类型允许为空，也可包含重复对象。
- 可用 `foreach()` 循环结构进行处理。
- 列表可用方括号（`[]`）按索引号访问。

参见

关于列表的列表，见第 129 页「嵌套容器」。

该类型支持表 8.22 所示操作。

表 8.22: 列表操作

操作	e2 类型	返回	修改 e1	说明
e1.empty()	—	boolean	否	若列表不含任何元素则返回 <code>true</code> ，否则返回 <code>false</code> 。
e1.size()	—	integer	否	返回列表中的元素个数。
e1.contains(e2)	item	boolean	否	若 <code>e2</code> 是 <code>e1</code> 列表的元素则返回 <code>true</code> 。
e1.front()	—	item	否	返回列表第一个元素的副本。
e1.back()	—	item	否	返回列表最后一个元素的副本。

表 8.22: 列表操作（续）

操作	e2 类型	返回	修改 e1	说明
e1.chop_back()	—	list	否	返回去掉最后一个元素后的 e1 列表副本。
e1.chop_front()	—	list	否	返回去掉第一个元素后的 e1 列表副本。
e1.append(e2)	item	list	否	返回将 e2 追加到列表末尾后的 e1 列表副本。
e1.prepend(e2)	item	list	否	返回将 e2 插入到列表前端后的 e1 列表副本。
e1.push_front(e2)	item	void	是	将 e2 表达式的副本添加到列表前端。
e1.push_back(e2)	item	void	是	将 e2 表达式的副本添加到列表末尾。
e1.pop_front()	—	void	是	移除列表的第一个元素。
e1.pop_back()	—	void	是	移除列表的最后一个元素。
e1.merge(e2)	list	list	否	返回将 e2 列表拼接在末尾后的 e1 列表副本。
e1.remove(e2)	item	void	是	移除 e1 列表中所有等于 e2 表达式的值。

下列为合法代码示例：

```
x: layer = ... ;

MyPartition: list of layer = PartitionLayer(L1);

foreach (part in MyPartition) { ... }

if (MyPartition.contains(x)) { ... }

i : integer = MyPartition.size();
```

下例展示如何拼接列表：

```
list1 : list of integer = { 1, 2, 3 };
list2 : list of integer = { 10, 20, 30 };
list3 : list of integer = list1.merge(list2);
```

此时 list3 的值为 { 1, 2, 3, 10, 20, 30 }。

下面是初始化和使用字符串列表的示例：

```
// declare myList to be a list of strings, uninitialized
myList : list of string;

// declare myList to be a list of strings, initialized
myList : list of string = { "this", "is" , "a", "list" };
// myList[0] = "this"
// myList[3] = "list"
```

支持内置列表运算符函数。例如：


```

myList : list of string = { "this", "is" , "a", "list" };

s = myList.size();           // returns the size of a list: s is "4"
a = myList.front();          // a contains a copy of L1[0]: a is "this"
myList.pop_front();          // removes the first element of the list,
                             // the list is now { "is", "a", "list" }
myList.pop_back();           // removes the last element of the list,
                             // the list is now { "is", "a" }
myList.push_front("xyz");    // adds a new element containing "xyz"
                             // to the front of the list.
                             // the list is now { "xyz", "is", "a" }
myList.push_back("bar");     // adds a new element containing "bar"
                             // to the back of the list.
                             // the list is now { "xyz", "is", "a", "bar" }

```

hash of...

定义从一种数据类型到另一种数据类型的映射。源类型称为键类型 (key type); 目标类型称为值类型 (value type)。也就是说, hash 定义了键/值对数组。

该类型要求键类型为 `string`、`integer` 或 `enum`, 或由这三种类型之一派生的类型。不能是 `double`。值类型可以是任何先前已定义的类型。

下例创建将整数转换为字符串的 hash。此时键类型为 `integer`, 值类型为 `string`。该类型使用 `=>` 运算符初始化。

```

t1_h : newtype hash of integer to string;
t2_h : newtype hash of string to integer;

h1 : t1_h = {
  1 => "a",
  2 => "b",
  26 => "z"
};

h2 : t2_h = {
  "xyzy" => 1,
  "plugh" => 10,
  "help" => 100
};

```

下面是使用派生类型的示例:

```

t1 : newtype integer;
h1_h : newtype hash of t1 to string;
x : h1_h = { 1 => "a" };

```

hash of... 类型可使用下标按元素赋值:

```

x = h1[203];
h1[30] = "ad";

```

hash of... 类型支持表 8.23 所示成员操作。

表 8.23: Hash 操作

操作	e2 类型	返回	修改 e1	说明
<code>e1.contains_key(e2)</code>	hash 的键类型	boolean	否	若 <code>e2</code> 是 <code>e1</code> 哈希表的键则返回 <code>true</code> 。效果上与 <code>e1.keys().contains(e2)</code> 相同，但运行时性能可能更好。
<code>e1.contains_value(e2)</code>	hash 的值类型	boolean	否	若 <code>e2</code> 是 <code>e1</code> 哈希表中某项的值则返回 <code>true</code> 。效果上与 <code>e1.values().contains(e2)</code> 相同，但运行时性能可能更好。
<code>e1.empty()</code>	—	boolean	否	若 hash 不含任何元素则返回 <code>true</code> ，否则返回 <code>false</code> 。
<code>e1.keys()</code>	—	list of 键类型	否	返回所有键值的列表；该列表不含重复值。
<code>e1.size()</code>	—	integer	否	返回 hash 中的元素个数。
<code>e1.values()</code>	—	list of 值类型	否	返回所有值的列表；根据 hash 内容，该列表可含重复值。
<code>e1.merge(e2)</code>	与 <code>e1</code> 匹配的 hash 类型	hash	否	返回将 <code>e2</code> 的键与值拼接到末尾后的 <code>e1</code> hash 副本；若某键在 <code>e1</code> 与 <code>e2</code> 中均存在，则取其在 <code>e2</code> 中的值。
<code>e1.remove(e2)</code>	item	void	是	从 <code>e1</code> hash 中移除 <code>e2</code> 键。若 <code>e2</code> 键不存在则无效果。

再举一例：

```
alpha_h : newtype hash of string to integer;
alpha : alpha_h = {          // declare and initialize key => value pairs
    "a" => 1,
    "b" => 2,
    "c" => 3
};
y = alpha["b"];              // The value of y is "2"
keyList = alpha.keys();      // A list of the keys in alpha
```

struct of...

定义由用户提供名称标识的独立类型。必须至少有一个成员，且成员名必须唯一。各成员可以属于不同的类型。下面是一个用几何坐标定义点的结构示例：

```
point: newtype struct of { x: integer; y: integer; };
```

`struct of...` 类型可包含初始值，用作结构创建时的默认值。

```
t3_s : newtype struct of {
    x: integer;
    y: integer = 76;    // initializer
    z: string = "hi";   // initializer
};
```

```
t4_s : newtype struct of {
  a: integer = 106; // initializer
  b: string = "boo"; // initializer
};
```

`struct of...` 类型可从 C 风格初始化器赋值，也可从名-值对列表赋值。下例展示使用上例所定义类型的赋值。

```
s3 : t3_s = { 10, z = "bye" };           // note that 'y' defaults
                                         // to the value 76

// the defaults cause s4 to have a=106, b="boo"
s4 : t4_s = { };
```

再举一例：

```
shoebox : newtype struct of {
  style : string = "Tennis Shoe";
  shoe_size : double = 10.5;
};

box : shoebox;
shoe = box.style;           // shoe is "Tennis shoe", the default
box.style = "Dress Shoe"; // Reassign the member box.style
// create a new shoebox with a different kind of shoe
box2 : shoebox = {"Boat Shoe", 9};
```

function

允许将函数传递给另一个用户函数。被传递的函数可像全局函数一样使用。例如：

```
// input file
#include <icv.rh>
xyzy : function (void) returning void { note("abcd"); }
plugh : function ( f : function(void) returning void ) returning void {
  f(); }
plugh(xyzy); // summary file shows "abcd" in output when plugh runs
```

嵌套类型

Nested Types

类型可以组合形成嵌套类型，例如：

- list of struct
- list of enum

例如，使用上文 `struct of...` 中 `shoebox` 结构定义：

```
box : list of shoebox = {
  { "Walking", 12 },
  { "High Heel", 7 }
};
```

```
A = box[0].style;           // A is "Walking"
B = box[1].shoe_size;       // B is 7
```

8.4.3 类型分析

Type Analysis

本节说明 PXL 在特殊情况下如何处理数据类型，例如当参数传入一种数据类型但期望另一种数据类型时。

当 PXL 遇到一种数据类型而期望另一种时，只要合适就会执行自动类型转换。用于自动类型转换的规则在下列各小节中讨论。

标准算术转换

Standard Arithmetic Conversions

当表达式所要求的算术类型与实际找到的算术类型不匹配时，按表 8.24 进行转换。

表 8.24: 算术转换

期望 ...	允许 integer ^a	允许 constraint of integer ^b	允许 double	允许 constraint of double
integer	是	否	否	否
constraint of integer	是	是	否	否
double	是	否	是	否
constraint of double	是	是	是	是

^a 「是」表示允许该转换。

^b 「否」表示不允许该转换。该操作将导致错误。

这些转换仅在表达式类型要求时才应用。特别地，除非上下文要求，否则 `integer` 类型的表达式不会转换为 `double`。

将 `integer` 或 `double` 转换为 `constraint` 时，所得 `constraint` 看起来如同使用 `==` 一元约束 (`CONSTRAINT_EQ`) 书写一样。

```
f: function ( c: constraint ) returning void { ... }

f( (1.0, 26.0) );           // constraint is ok
f( 2.0 );                  // single value is ok also
f( 42 );                   // single integer is accepted
```

比较 double

Comparing Doubles

由于类型 `double` 是实数的近似，偶尔会出现累积误差，使表面上等价的运算产生不同结果。PXL 提供特殊函数，用于在考虑此类微小误差的情况下比较 `double`。这些特殊函数见 IC Validator Reference Manual 中「Utility Functions」一章 Math Functions 节的 Equivalence Comparison Functions 与 Double Comparison Functions。

注意

只要涉及与 `double` 相关类型之一的直接比较 (`==` 或 `!=`)，就会报告警告。

类型提升

Type Promotion

当期望列表，而找到的表达式类型与该列表所含元素类型相同时，该表达式会从单个项提升为仅含该项的列表。

参数传递

Parameter Passing

带有 output 或 input-output 限定的函数参数，其实参必须匹配。前述规则不适用。例如，当函数期望标记为 out 的 constraint 参数时，提供 integer 类型变量名不合法，因为所需转换不合法。

字符串拼接

String Concatenation

表 8.25 所示类型的任何表达式，均可使用标准加法运算符 (+) 追加到字符串表达式。

表 8.25: 字符串拼接

类型	追加到结果字符串的内容
integer	整数表达式的值
double	double 表达式的值
handle	handle 的文本表示；每个不同对象唯一
string	字符串内容
boolean	true 或 false
constraint of integer	等价的 constraint 字面量表达式
constraint of double	等价的 constraint 字面量表达式
user-defined enum	该表达式所表示的 enum 字面量名称

使用字符串时请记住：

- 当加法运算符 (+) 左侧为字符串时，结果为字符串。
- 对字符串值表达式，加法运算符 (+) 不对称：a + b != b + a。
- 后续追加到字符串的各项之间不添加空格，因此可能看到意外结果，如下例：

```
s : string;
s = "hello " + 10 + " " + (1 == 0);
// s now contains the string "hello 10 false"

c : constraint = ( 10, 40 ];
s = "" + c;
// s now contains the string "(10.,40.)", because the
// type of "c" is constraint of double

mye : newtype enum of { A, B, C, D, X };
t : mye = X;
s = "" + t + A + B;
// s now contains the string "XAB"

s = "" + 11 + 3.14 + (1 == 1);
```

```
// s now contains the string "113.14true"

s = "" + (11 + 3.14) + (1 == 1);
// s now contains the string "14.14true"
```

默认值注入

Default Injection

处理函数调用时，未给出值的参数若有默认值，则采用函数原型中的默认值。既无默认值也未指定值的参数会导致错误。

类似地，处理字面量结构时，未给出值的成员若有默认值，则采用 **newtype** 结构定义中的默认值。既无默认值也未指定值的成员会导致错误。

8.4.4 嵌套容器

Nested Containers

列表、hash 与结构也称为容器（container）。容器可以多种不同组合嵌套。嵌套容器的一个例子是结构的列表。

下面是 hash 的 hash 示例。runset 代码为（hash of a hash）：

```
hash1_h : newtype hash of string to integer;
hash2_h : newtype hash of string to hash1_h;

multi_hash : hash2_h = {};

multi_hash["A"] = {};           // Initialize hash of "A" to empty
multi_hash["A"]["ONE"] = 1;
multi_hash["A"]["TWO"] = 2;
multi_hash["B"] = {
  "THREE" => 3,
  "FOUR" => 4
};

foreach(key1 in multi_hash.keys() ){
  foreach(key2 in multi_hash[key1].keys()){
    note( "multi_hash[" + key1 + "][" + key2 + "] = "
      + multi_hash[key1][key2] + "\n");
  }
}
```

输出为：

```
runset.rs:123 multi_hash[A][ONE] = 1
runset.rs:123 multi_hash[A][TWO] = 2
runset.rs:123 multi_hash[B][FOUR] = 4
runset.rs:123 multi_hash[B][THREE] = 3
```

下面是列表的列表示例。runset 代码为（list of a list）：

```
list1_l : newtype list of integer;
list2_l : newtype list of list1_l;
```

```

list2 : list2_1 = {
  { 0, 1, 2, 3 },
  { 10, 11, 12, 13 }
};

list2.push_back({ 20, 21, 22, 23 });
list2[1].push_back( 14 );

for( j=0 to (list2.size() - 1) ){
  for( i=0 to (list2[j].size() - 1) ){
    note( "list2[" + j + "][" + i + "] = " + list2[j][i] + "\n");
  }
}

```

输出为:

```

runset.rs:143 list2[0][0] = 0
runset.rs:143 list2[0][1] = 1
runset.rs:143 list2[0][2] = 2
runset.rs:143 list2[0][3] = 3
runset.rs:143 list2[1][0] = 10
runset.rs:143 list2[1][1] = 11
runset.rs:143 list2[1][2] = 12
runset.rs:143 list2[1][3] = 13
runset.rs:143 list2[1][4] = 14
runset.rs:143 list2[2][0] = 20
runset.rs:143 list2[2][1] = 21
runset.rs:143 list2[2][2] = 22
runset.rs:143 list2[2][3] = 23

```

再举一个列表的列表示例。runset 代码为:

```

list3 : list of list of integer = {};

list3.push_back( { 0, 1, 2, 3 } );
list3.push_back( { 4, 5, 6 } );
list3.push_back( { 7, 8 } );
list3.push_back( { 9 } );

for( j=0 to (list3.size() - 1) ){
  for( i=0 to (list3[j].size() - 1) ){
    note( "list3[" + j + "][" + i + "] = " + list3[j][i] + "\n");
  }
}

```

输出为:

```

runset.rs:159 list3[0][0] = 0
runset.rs:159 list3[0][1] = 1
runset.rs:159 list3[0][2] = 2
runset.rs:159 list3[0][3] = 3
runset.rs:159 list3[1][0] = 4

```

```
runset.rs:159 list3[1][1] = 5
runset.rs:159 list3[1][2] = 6
runset.rs:159 list3[2][0] = 7
runset.rs:159 list3[2][1] = 8
runset.rs:159 list3[3][0] = 9
```

8.5 流程控制

Flow Control

流程可用下列方式控制：

- 条件语句（Conditional Statements）
- 循环（Loops）
- 执行终止（Termination of Execution）

8.5.1 条件语句

Conditional Statements

PXL 中的条件语句是一类构造，允许你根据指定条件求值为 `true` 还是 `false` 来执行不同动作。

`if` 语句用于仅在预定义条件满足时执行某动作。

`if` 语句至少与另外两个组成部分一起使用：

- 一个可求值为布尔值 `true` 或 `false` 的表达式。
- 一个在求值表达式返回 `true` 时执行的组成部分。

可选地，在必需组成部分之外，还可包含下列组成部分以构建更复杂的条件语句：

- `else` 构造，后跟在第一个表达式求值为 `false` 时执行的组成部分。
- `elseif` 构造，表示在父 `if` 条件语句内嵌套了循环式 `if` 条件或一个或多个 `if` 条件。PXL 也支持 `elif` 构造。

`if` 语句使用下列语法：

```
if (e1) s1;
if (e1) s1 elif (e2) s2;
if (e1) s1 elseif (e2) s2;
if (e1) s1 else s2;
```

`if` 语句按如下方式求值：

- 始终先求值第一个布尔表达式（`e1`）。
- 若 `e1` 求值为 `true`，则处理第一个组成部分（`s1`）。
- 若遇到 `else` 语句，则执行其后的语句（`s2`）。
- 若遇到 `elif` 或 `elseif` 语句，则执行其后的语句。

8.5.2 循环

Loops

与其他编程语言一样，可在 PXL 中使用循环将某动作重复若干次。

PXL 提供下列方法创建循环构造：

- **for** 循环
- **foreach** 循环
- **while** 循环

for 循环

for Loop

for 循环用于将特定动作重复指定次数。它至少包含一个整数（**e1**）以及必须重复的动作（**s1**）。也可包含附加整数（**e2**、**e3**）。

语法为：

```
for (var = e1 to e2) s1;           //s1 is a statement; it can be compound
for (var = e1 to e2 step e3) s1;
```

- 要求 **e1**、**e2** 与 **e3** 为整数值表达式。
- 将 **var** 定义为仅在语句 **s1** 内可见的整数变量。
- 在循环开始时对 **e1**、**e2** 与 **e3** 各求值一次；随后对这些表达式取值的赋值不影响循环。

例如：

```
for(i=0 to 10 step 2){           // Increment by 2: 0, 2, 4, 6, . . .
    if( i < 6) {
        a[i] = b[i] + c[i];
    } elif( ( i >=6 ) && ( i < 10)){
        a[i] = b[i] + c[i] + 1;
    } else {
        a[i] = b[i] - c[i];
    }
};
```

foreach 循环

foreach Loop

foreach 循环类似于 **for** 循环，但用于针对容器（如列表或 **enum**）的元素执行一组动作。它不使用变量计数器来决定动作的迭代次数。

它包含类型为列表的 **e1**，以及对 **e1** 中每个元素求值的组成部分 **s1**。

语法为：

```
foreach (var in e1) s1;           //e1 is of the type "list of"
```

- 对 **e1** 的每个元素求值一次 **s1**，同时将 **var** 从列表复制过来。
- 若输入列表 **e1** 为空，则跳过 **s1**。
- **var** 是类型正确的常量变量，仅在语句 **s1** 内可见。

while 循环

while Loop

while 循环用于重复特定动作直到满足某条件。它包含布尔表达式（**e1**）以及必须重复的动作（**s1**）。**while** 循环始终至少对布尔表达式 **e1** 求值一次。

语法为:

```
while (e1) s1;           // e1 is evaluated as a Boolean;
                        // loop continues until e1 is false
```

`while` 语句按如下方式求值:

- 每次 `e1` 求值为 `true` 时, 执行 `s1`。
- 当 `e1` 求值为 `false` 时, 该语句结束。

8.5.3 执行终止

Termination of Execution

`error("message")` 是诊断函数 (Diagnostics Function)。它写出所提供的消息, 然后终止执行。例如:

```
#ifdef ABC
#ifdef DEF
error("ABC and DEF are mutually exclusive. Please do not set both.");
#endif
#endif
```

8.6 函数

Functions

PXL 中的函数按用途分类。函数类型见表 8.26。

表 8.26: 函数类型

函数类型	说明
Runset function	IC Validator 发布的标准函数。
Remote function	由用户编写、可被 <code>runset</code> 函数调用的函数。IC Validator 工具也提供默认 <code>remote</code> 函数。
User function	由用户编写的函数。
Utility function	由 IC Validator 工具提供、用于在 <code>remote</code> 函数中访问原始数据的函数。

8.6.1 函数定义

Function Definitions

函数定义为一组与名称关联的操作。函数定义标识所有参数以及返回值的名称、类型与默认值。你可以创建自己的函数, 并与所提供的函数一起在 `runset` 中使用。如下例所示, 所有函数定义必须位于顶层。

```
// a simple function definition

myinversion: function ( arg: double ) returning rv : double
{
    rv = 1.0 / arg;
}
```

```
x = myinversion(2.0);           // simple function call
```

函数定义为所有处理提供单独作用域。作用域由书写位置的上下文决定（静态作用域，static scoping）。

参见

关于作用域，见「变量作用域」（原书第 197 页）。

在函数定义内部适用下列作用域规则：

1. 若在当前作用域找不到声明，则向上搜索更高作用域，直到搜遍整个函数定义，在匹配处停止。
2. 然后，如有必要，使用在函数定义之前声明且同名的全局变量。
3. 最后，若该变量正被赋值为函数调用的结果，或处于函数调用的输出参数位置，则在当前语句的作用域中隐式声明它。但是，若该变量处于函数调用的输入或输入-输出参数位置，即使它正被赋值为函数调用的结果，也不会当前语句的作用域中隐式声明。

在函数定义外部适用下列作用域规则：

1. 若在当前作用域找不到声明，则向上搜索更高作用域，直到搜遍整个 runset，在匹配处停止。
2. 然后，如有必要，使用先前声明且同名的全局变量。
3. 最后，若该变量正被赋值为函数调用的结果，或处于函数调用的输出参数位置，则在当前语句的作用域中隐式声明它。但是，若该变量处于函数调用的输入或输入-输出参数位置，即使它正被赋值为函数调用的结果，也不会当前语句的作用域中隐式声明。

8.6.2 函数原型

Function Prototypes

函数原型定义函数的参数以及可选参数的默认值。函数原型标识按引用调用（call-by-reference）的参数。函数原型不定义函数的操作。

下面是函数原型示例：

```
my_xor : function (
  a : polygon_layer,
  b : polygon_layer
) returning xorOut: polygon_layer;
```

下面是函数声明示例：

```
my_xor : function (
  a : polygon_layer,
  b : polygon_layer
) returning xorOut: polygon_layer {
  tmp1 = not(a,b);
  tmp2 = not(b,a);
  xorOut = or(tmp1, tmp2 );
};
```

下例展示调用前一例中定义的 my_xor() 函数：

```
a1 : polygon_layer;
b1 : polygon_layer;
xorA1B1 = my_xor(a1, b1);
```

8.6.3 函数调用

Function Calls

函数调用定义函数如何运作。函数调用由函数名后跟通过实参说明如何及在何处执行该函数的指令组成。

向函数调用的实参传递值可用两种不同风格：

- 仅按实参值（按值传递，`pass-by-value`）。值的顺序必须与函数原型中参数顺序匹配。使用此风格时，若你关心某个可选参数，则必须为其前面的所有可选参数指定值，因为值按顺序匹配到参数。
- 按 `argument_name = argument_value` 格式用名称（按名传递，`pass-by-name`）。参数顺序无关。使用此风格时，无需为其前面的所有可选参数指定值，因为值按名称匹配到参数。

两种风格可在同一次函数调用中使用。但是，在函数调用中，所有按值传递风格指定的实参值必须出现在所有按名传递风格指定的实参值之前。一旦在函数调用中使用了按名传递风格，其后的所有实参调用必须使用相同风格。

例如，`interacting()` 函数的语法为：

```
interacting : binary published function(
  layer1 : polygon_layer,
  layer2 : polygon_layer,
  count : count_constraint_t = >0,
  include_touch : more_include_touch_e = EDGE,
  processing_mode : processing_mode_e = HIERARCHICAL
) returning result : polygon_layer;
```

下面是以按名传递风格调用 `interacting()` 的示例：

```
y = interacting( layer1 = a, layer2 = b, include_touch = ALL );
// The count argument is skipped, and the value is the default.
// The value of the processing_mode argument is the default.
```

下面是以按值传递风格调用 `interacting()` 的示例：

```
y = interacting( a, b, 2 );
// The value of count is 2.
// The values of the include_touch and processing_mode arguments
// are the defaults.
```

下面是两种混合风格调用 `interacting()` 的示例：

```
y = interacting( a, b, include_touch = ALL );
// The count argument is skipped, and the value is the default.
// The value of the processing_mode argument is the default.

y = interacting( a, b, count = 2);
// The values of the include_touch and processing_mode arguments
// are the defaults.
```

函数调用有两种风格：

- 标准函数调用（Standard Function Call）

```
y = and(a, b);
y = and(a, b, CELL_LEVEL);
y = or(not(a,b),not(b,a));
```

- 运算符风格函数调用（Operator-Style Function Call）

```

y = a and b;
y = a and[CELL_LEVEL] b;
y = (a not b) or (b not a);

```

标准函数调用

Standard Function Call

对任何函数始终允许标准函数调用。

标准函数调用由函数名后跟括号括起、逗号分隔的至少一个表达式列表，再跟至少一个名-值对列表组成。

下例中函数有三个参数：

```

g: function ( x : integer, y : integer, z : integer)
    returning void;

```

下列四个函数调用中，有一个不合法：

```

g(x=3, y=10, z=10);           // allowed

g(z=3, y=10, x=4);           // allowed

g(10, 20, 30);                // allowed; x=10, y=20, z=30

g(z=3, 10, 20);               // ILLEGAL - expressions must not
                               // follow name-value pairs

```

若任一参数存在默认值，则不必指定这些参数。下例中函数有三个参数，其中两个有默认值。

```

f: function
    ( x : integer,
      y : integer = 27,
      z : integer = 42
    ) returning void { ... }
...

```

下列五个函数调用中，有两个不合法：

```

f(10, y=2);                    // allowed, z is 42

f(10);                          // allowed, y is 27, z is 42

f(z=3, x=4);                   // allowed, y is 27

f();                            // ILLEGAL - x requires a value

f(z=20);                       // ILLEGAL - x requires a value

```

运算符风格函数调用

Operator-Style Function Call

运算符风格函数调用要求二元函数在函数名左右两侧各有实参。其他实参须以名-值对形式写在紧随函数名之后的方括号中。

仅对以 `binary` 限定符声明或定义的函数允许运算符风格函数调用。使用运算符风格函数调用有助于防止在其他上下文（如变量名与成员名）中使用该函数名。

下列语法定义如何使用 `binary` 限定符调用函数：

```
length: binary function (l1: layer, c1: constraint of double) returning
result: layer {...}
...
LONGMET = MET1 length > 5;
```

通过仔细的接口定义，`binary` 可写出易读的程序。例如：

```
// A simple declaration that takes two arguments.
// The LHS is the layer to be selected from; the RHS is the
// constraint defining the criteria area.
...
// ... as is a variable of appropriate type.
c : constraint = (27.0, 42.5);
x : lyr1 area c;
```

实参绑定

Argument Bindings

实参绑定决定调用点的哪些实参绑定到函数原型中的哪些参数。

- 根据函数名以及上下文所要求的类型选择候选原型。若无法确定类型，则仅使用函数名。进行运算符风格函数调用时，外部实参绑定到原型中的参数。对二元函数，左侧实参绑定到第一个参数，右侧实参绑定到第二个参数。
- 每个名-值实参绑定到剩余的同名参数。
- 任何没有名称的实参从左到右绑定到剩余的无默认值参数。
- 若所有剩余未绑定参数都有默认值，则使用该函数。
- 若实参绑定未能恰好匹配一个函数原型，则为错误。

例如，给定如下定义的函数 `MyFunc`：

```
MyFunc: binary function (
  fromLayer: lhs layer,
  touching: rhs layer,
  corners: boolean = true,
  whole_polygon: boolean
) returning e: edge_layer;
```

下列函数调用全部等价：

```
T1 = MyFunc (metallayer, selectLayer, true);
T2 = MyFunc (metallayer, selectLayer, whole_polygon=true);
T3 = metallayer MyFunc[whole_polygon=true] selectLayer;
```

下列代码因 `whole_polygon` 实参没有默认值而产生编译时错误：

```
/* illegal function call */
Tbroken = metallayer MyFunc selectLayer;
```

默认值

Defaults

默认值在函数原型中指定为选项变量的右侧赋值。例如，`and()` 函数的语法为：

```
and : function(
    layer1 : polygon_layer,
    layer2 : polygon_layer,
    processing_mode : processing_mode_e = HIERARCHICAL,
    remove_hierarchical_overlap : boolean = true
) returning and_result : polygon_layer;
```

下列对 `and()` 的调用对 `processing_mode` 与 `remove_hierarchical_overlap` 实参使用默认值：

```
c = and( a, b );
```

下列对 `and()` 的调用对 `processing_mode` 使用可选设置，对 `remove_hierarchical_overlap` 使用默认值：

```
c = and( a, b, CELL_LEVEL );
```

重置默认值

Resetting Defaults 你可以覆盖任何函数实参的默认值。任何实参都可更改其默认值；可以是数值、布尔值或枚举量。

本例中设置了 `abc` 函数实参 `x` 的默认值：

```
abc : literal function ( x : double = 123) returning z : double;
A = abc(); // == abc(123)
```

注意

警告：重置默认值时，你是在重新定义该函数，必须列出其全部实参。

8.6.4 匿名函数

Anonymous Functions

若干 IC Validator 命令允许将用户函数作为实参传入。匿名函数为简单行为或组织更复杂行为提供了一种机制。可在期望函数名的地方使用匿名函数。编译器生成内部名称，并在紧临其调用之前的代码位置声明该函数。

传统上，可使用全局变量，并按需为不同调用设置不同值。例如：

```
x : integer;
y : integer;
calc_nmos_props : function (void) returning void { ... }

x = 0;
y = 5;
nmos(..., calc_nmos_props, ...);

x = 1;
y = 4;
nmos(..., calc_nmos_props, ...);
```

然而，匿名函数可使此类代码更紧凑、组织更好。下例中，在每次 NMOS 调用之前声明一个隐藏函数，随后使用它。

```
calc_nmos_props : function (x : integer, y : integer) returning void
{ ... }

nmos(..., function { calc_nmos_props(0, 5); }, ...);
nmos(..., function { calc_nmos_props(1, 4); }, ...);
```

8.6.5 函数重载

Function Overloading

在某些情况下，PXL 允许存在多个同名函数。若 PXL 能根据类型与参数名确定要调用哪个函数，则多个函数可以同名。若 PXL 无法确定要调用的函数，则导致错误。

若一个返回值而另一个返回 `void`，则可以有同名不同函数。

当函数出现在语句级时，假定为返回 `void` 的函数；当出现在表达式中时，假定为返回值的函数：

```
MyFunction(a,b,c);           // use the void-returning function
                             // "MyFunction"
x = MyFunction(a,b,c);       // use the value-returning function
                             // "MyFunction"
```

注意

PXL 仅支持基于不同返回类型的函数重载。

8.6.6 函数递归

Function Recursion

PXL 不支持递归函数调用。例如：

```
noCanDo : function( ... ) returning integer {
    ...
    x = noCanDo(...);
    ...
}
```

8.7 程序结构

Program Structure

PXL 程序是声明、函数定义与语句的序列。程序自上而下执行。

PXL 对程序有效性设定下列前提条件：

- 所有对象必须在使用前定义。
- 所有语句仅因副作用而求值。表达式必须是赋值的一部分。

```
10 + 5;    // illegal because the result (15) is not captured
```


8.7.1 复合语句

Compound Statement

复合语句定义可声明局部变量的嵌套作用域。复合语句可用在允许单条语句的任何地方。

参见

关于作用域，见「变量作用域」（原书第 197 页）。

- 复合语句允许局部变量隐藏其他作用域级别先前定义的变量。编译器报告警告。隐藏也称为遮蔽（shadowing）。
- 复合语句使用花括号（{}）标识。

```
{ ; }      // Smallest legal compound statement
```

- 复合语句遵循下列语法：

```
{
    s1;
    s2;
    ...
    sN;
}
```

8.7.2 违例

Violations

某些 IC Validator 函数可产生违例数据（也称为错误输出），替代或补充层输出。这些违例存储在 SQLite 数据库（错误数据库，PYDB）中，可供 VUE 用于调试，并在运行结束时写入 LAYOUT_ERRORS 文件。也有一些 PXL 函数以违例数据为输入，例如 write_gds() 与 violation_empty() 函数。

若希望直接访问 IC Validator 工具产生的错误数据，见附录 E「PYDB Perl API」，了解如何使用 IC Validator Perl 应用程序编程接口（API）。

参见

附录 E，PYDB Perl API。

违例注释

Violation Comments

所有违例输出都与一条注释字符串关联。该注释作为标题出现在 LAYOUT_ERRORS 文件以及 VUE 中。任意数量的函数可输出到同一违例注释。

当前及嵌套 PXL 作用域的违例注释使用 @ 运算符指定。若 runset 中未指定注释，违例获得默认注释 "Violation"。

下例展示如何为层集中的每一层编写唯一的违例注释。违例命名为 "Metal1 spacing <= 0.1um"、"Metal2 spacing <= 0.1um"，依此类推。

```
for (i=0 to metal_layers.size()-1) {
    @ "Metal" + (i+1) + " spacing <= 0.1um";
    external1(metal_layers[i], distance<=0.1);
}
```

下例展示违例注释相对于作用域的行为。

注意

通常，违例注释定义（@ 运算符）不应与其所适用的函数分开。下例突出说明违例注释相对于作用域的行为。

```
@ "First Error Message";
{ @ "Second Error Message";           // A new scope is created here
  local_var = abc(layer2);
  internal1(local_var, distance < 0.2); // Errors from internal1()
                                         // are stored in the database
                                         // under "Second Error Message"
} // The original scope is restored here

external1(layer1, distance < 0.1); // Errors from external1() are stored
                                   // in the database under "First Error
                                   // Message"
```

下例展示如何为层集合中的每一层编写唯一的违例注释。违例命名为“Metal1 spacing <= 0.1um”、“Metal2 spacing <= 0.1um”，依此类推。

```
for (i=0 to metal_layers.size()-1) {
  @ "Metal" + (i+1) + " spacing <= 0.1um";
  external1(metal_layers[i], distance<=0.1);
}
```

text_net()、texted_with() 与 not_texted_with() 函数可输出多种不同类型的违例。对这些函数，有办法为个别错误类型覆盖当前 runset 违例注释。本例中，text short 输出到违例注释“Rule 4.5.6: No Shorts”，而所有其他类型的 text 错误输出到“Rule 1.2.3: No text errors”。

```
@ "Rule 1.2.3: No text errors";

cdb1 = text_net(cdb1,
               { { M1, M1_text } },
               shorted_violation_comment = "Rule 4.5.6: No shorts");
```

IC Validator 工具使用诸如 -svc 等命令行选项处理违例注释。例如：

```
{@ "A";
  {@ "B";
    internal1(...);
  };
};
```

工具在选择时仅考虑较低层的违例注释。在此情况下，设置 -svc B 或 -uvc A 会执行 internal1() 函数；而设置 -svc A 或 -uvc B 则不会。

产生违例的函数**Violation-Producing Functions**

大多数 DRC 与数据生成函数是重载的；也就是说，它们有两个版本。返回多边形层的版本不产生错误输出。返回 void 的版本产生错误输出但不产生多边形层。下面是 external1() 函数两个版本的示例。

```
// This external1 outputs to a polygon layer, no errors are produced
t1 = external1(layer1, distance < 0.1);
```

```
// This external1 outputs error data to the comment "Ext1 Check"
@ "Ext1 Check";
external1(layer1, distance < 0.1);
```

也有一些 PXL 函数不返回 `void`，但也可产生错误输出。例如，`text_net()` 函数返回 `connect database`，同时也为 `text short`、`open` 与 `unused text` 产生错误输出。

违例变量

Violation Variables

内置 PXL 类型 `violation` 可用于保存错误数据的变量，以便稍后在 `runset` 中将其用作另一 PXL 函数的输入。例如，违例变量可用作 `write_gds()` 等函数的输入以输出错误多边形，或用作 `violation_empty()` 以判断违例是否包含任何错误。

如下例所示，某些 PXL 函数直接返回违例：

```
// These functions return violations for which there is not an associated
// PXL function
v1 = get_layout_grid_violation();
v2 = get_layout_drawn_violation();

// Write errors to GDSII
g = gds_library("err.gds");
write_gds(output_library = g,
          errors = { { v1, { 1 } },
                    { v2, { 2 } } });
```

注意

错误输出可受 `error_options()` 函数中 `error_limit_per_check` 等实参限制。这些选项可影响实际存入错误数据库的数据量。被丢弃的错误对 `write_gds()` 及其他函数不可用。

违例块

Violation Blocks

违例变量也可使用违例块赋值。违例块使用 `@=` 运算符指定。左侧为违例变量。`@=` 运算符后跟花括号括起的代码块。在该块内，所有产生错误输出的函数都与该违例变量关联。例如：

```
// v1 contains the error output from the external1() check
v1 @= {
  @ "Rule 1";
  l1 = a and b not c;
  external1(l1, distance < 2);
}

// v2 contains the error output from both internal1() checks and
// the external1() check
v2 @= {
  @ "Rule 2";
  internal1(l2, distance < 1);
  external1(l1, distance < 2);
}
```

```

    internal1(l3, distance < 1);

    @ "Rule 3";
    external1(l2, distance < 1);
}

tdb : connect_database; // Declared outside the violation block scope
                        // so it can be used after the violation block
// v3 contains all text errors found by text_net()
v3 @= {
    @ "Rule 4";
    tdb = text_net(cdb, {{ M1, M1_text }});
}

// These functions can be used to get only certain types of errors
// from a violation. The result is a subset of the input violation.
v4 = get_text_shorted(v3); // v4 contains only text shorts from v3
v5 = get_text_unused(v3); // v5 contains only unused text from v3

// Take some action if v4 is not empty
if (! violation_empty(v4)) {
    note("Found text shorts!");
}

```

8.7.3 执行模型

Execution Model

PXL 的执行模型要求资源可用。若某程序中操作的前提条件已满足，则该操作随时可供执行。强烈偏向尽早执行诊断，意味着在程序初始处理期间，所有已就绪的诊断尽可能立即执行。因此，你可能看到 PXL 程序中语句乱序执行。最终结果不受此重排影响；但是，中间结果可能在不同时间出现或同时出现。这种乱序执行也称为并行执行。

同一程序在不同次执行时始终生成相同结果。不过，这些结果可能因资源可用性而以不同顺序出现。可能的运行时错误包括：

- 试图使用尚未赋值的任何变量。
- 索引超出列表末尾，或以负索引索引。
- 以不存在元素的键值从 hash 读取。

8.7.4 预处理器

Preprocessor

PXL 使用预处理器使程序的开发与执行更简便、更快速。标准预处理器模块集成在 PXL 编译器中。表 8.27 展示预处理器能力。

注意

预处理器的行长度限制为 64K 字符。超出此限制通常可通过在空白或分隔符（如逗号，）处插入回车来规避。

表 8.27: 标准预处理器模块

典型预处理能力	所用语法
条件编译	<code>#if ... #ifdef、#ifndef、#elif、#else</code>
错误生成	<code>#error</code>
文件包含	<code>#include</code> 用于将附加文件内容插入 <code>runset</code>。文件名必须用双引号或尖括号括起。语法控制搜索路径。 用尖括号包含 IC Validator 文件： <pre>#include <icv.rh></pre> 按下列顺序搜索目录： 1. 命令行上用 <code>-I</code> 选项指定的目录，按其指定顺序 2. <code>ICV_INSTALL_DIRECTORY/include</code> 用双引号包含你自己的文件： <pre>#include "my_include.rh"</pre> 按下列顺序搜索目录： 1. 含有 <code>#include</code> 语句的文件所在同一目录 2. 命令行上用 <code>-I</code> 选项指定的目录，按其指定顺序 3. <code>ICV_INSTALL_DIRECTORY/include</code> 指定绝对路径时，仅搜索该路径。 可用 <code>#pragma once</code> 防止文件被包含超过一次。任何含有 <code>#pragma once</code> 的文件只会被包含一次，即使它出现在多个 <code>#include</code> 语句中。 例如，若 <code>runset</code> 目录结构如下： <pre>Runset -- rules.rs -- Include -- assign.rs -- other_assign.rs</pre> 要在 <code>rules.rs</code> 中包含 <code>other_assign.rs</code>，使用 <pre>#include "Include/other_assign.rs"</pre> 要在 <code>assign.rs</code> 中包含 <code>other_assign.rs</code>，使用 <pre>#include "other_assign.rs"</pre>
位置跟踪	<code>#line</code>
标准宏支持	<code>#define ... #undef</code>，包括记号构造（宏体中的 <code>##</code> 二元运算符）与字符串化（宏体中的 <code>#</code> 一元运算符）。也可用 <code>-D</code> 命令行选项定义变量。

参见

IC Validator 命令行选项（原书第 25 页）。
关于 IC Validator 主目录的更多信息，见 SolvNetPlus 上的 IC Validator Installation Notes。

使用预处理器指令移除大块代码；也就是说，将该区段包在一对 `#if 0` 与 `#endif` 语句内。例如：

```
#if 0
// everything through the matching #endif is treated as a comment.
if (something == true)
{
    a line comment.
    do_something(useful = true);
}
#endif
```

下面是使用指令强制单次包含的示例。首次遇到此文件时 `MYFILE_RH` 未定义。所含的 `#define MYFILE_RH` 将该变量设为已定义，并包含代码体。此后因 `MYFILE_RH` 已存在且 `#ifndef` 求值为 `false`，代码体不再被包含。

```
#ifndef MYFILE_RH
#define MYFILE_RH

// Body of code to include goes here

#endif           // end if the #ifndef MYFILE_RH
```

使用 `#if` 语句时：

- 使用标准比较运算符：`==`、`<=`、`>=`、`&&`、`||`
- 未定义的宏被解释为 `0`。例如，若 `SOMEMACRO` 未定义，则 `#if SOMEMACRO == 1` 被翻译为 `#if 0 == 1`。
- 仅比较整数值。不能比较字符串。

预定义标识符

Predefined Identifiers

预处理期间支持表 8.28 所示预定义标识符。

表 8.28: 预定义标识符

标识符	内容
<code>__LINE__</code>	当前源文件中的行号，为整数
<code>__FILE__</code>	当前源文件名，为字符串常量
<code>__DATE__</code>	编译日期，为 11 字符字符串字面量，例如 “Apr 25 2006”
<code>__TIME__</code>	编译时间，为字符串字面量 (“hh:mm:ss”)
<code>__PXL_VERSION_MAJOR__</code>	语言主版本号，为整数
<code>__PXL_VERSION_MINOR__</code>	语言次版本号，为整数
<code>__PXL_VERSION_PATCH__</code>	语言补丁级别，为整数
<code>__PXL_DATE__</code>	编译日期，为八位整数常量 <code>yyyymmdd</code> ，例如 20060425
<code>__PXL_TIME__</code>	编译时间，为四位整数常量 <code>hhmm</code> ，例如 0014 或 2300

例如，`__PXL_DATE__` 与 `__PXL_TIME__` 可用于编写基于当前日期改变行为的代码。因为这些变量是预处理器变量，也可用于 `#if` 与 `#define` 语句。

```
/**
 * define a new token, expire_20061231, that is be replaced
 * by the token deprecated before 2006.12.31 but expands
 * to obsolete thereafter.
 */
#if __PXL_DATE__ < 20061231
#define expire_20061231 deprecated
#endif

#ifndef expire_20061231
#define expire_20061231 obsolete
#endif
```

发布版本宏

Release Version Macros

已发布宏允许你将当前发布版本与其他发布版本比较。
例如，你可用年、月、service pack 与 hot fix 的整数值判断当前发布版本是否至少与指定发布版本一样新。

```
#if VERSION_GE(2009, 6, 1, 0)
. . .      // code that uses new features introduced in 2009.06.SP01
#else
. . .      // code that uses older method (pre 2006.06.SP01)
#endif
```

VERSION 宏定义见表 8.29。

注意

year、month、service_pack 与 patch 为整数值。

表 8.29: VERSION 宏

宏	定义
VERSION_GE(year, month, service_pack, patch)	测试当前发布版本是否大于或等于指定发布版本。
VERSION_LE(year, month, service_pack, patch)	测试当前发布版本是否小于或等于指定发布版本。
VERSION_GT(year, month, service_pack, patch)	测试当前发布版本是否大于指定发布版本。
VERSION_LT(year, month, service_pack, patch)	测试当前发布版本是否小于指定发布版本。
VERSION_EQ(year, month, service_pack, patch)	测试当前发布版本是否等于指定发布版本。

日期比较宏

Date Comparison Macros

日期比较宏允许你根据当前日期改变 `runset` 行为。
例如，可用新宏按当前日期启用或禁用某些 PXL 代码。

```
if (DATE_EQ(2015, 6, 4)) {
    note("Today == 2015-06-04");
} else {
    note("Today != 2015-06-04");
}
```

DATE 宏定义见表 8.30。

注意

year、month 与 day 为整数值。指定无效或越界值（例如 day 值大于 31，或 month 值大于 12）会导致未定义行为。

表 8.30: DATE 宏

宏	定义
DATE_EQ(year, month, day)	测试当前日期是否等于由 year、month、day 整数指定的日期。
DATE_GE(year, month, day)	测试当前日期是否大于或等于由 year、month、day 整数指定的日期。
DATE_GT(year, month, day)	测试当前日期是否大于由 year、month、day 整数指定的日期。
DATE_LE(year, month, day)	测试当前日期是否小于或等于由 year、month、day 整数指定的日期。
DATE_LT(year, month, day)	测试当前日期是否小于由 year、month、day 整数指定的日期。

8.7.5 限定符

Qualifiers

限定符通过改变其所标记项的行为或允许的操作来影响该项。表 8.31 定义各限定符。

表 8.31: 限定符

限定符	定义
const	每次对该对象赋新值都会导致错误。初始化不导致此错误。仅在尝试直接赋值或通过带 out 或 in_out 限定的函数参数传递时才会报错。
deprecated	每次定义或引用该对象都会导致警告。在函数原型中为函数参数赋初值，或在该参数所属函数体内使用该参数，不会生成此警告。
in_out	函数参数在每次函数调用中要求可赋值项 (lvalue)。不可赋值项或类型错误的项会导致错误。该项的值也会因函数调用而改变。关于 lvalue 的更多信息，见「赋值运算符」(原书第 199 页)。
inline	将函数标记为在使用 -ndg 命令行选项的 IC Validator 运行中需要展开。见 IC Validator 命令行选项 (原书第 25 页)。
literal	函数参数要求函数实参为字面量表达式。也就是说，编译器必须能在解析时解析函数实参的值。
obsolete	每次定义或引用该项都会导致错误。在函数原型中为函数参数赋初值不会生成此错误。
out	函数参数在每次函数调用中要求可赋值项 (lvalue)。不可赋值项或类型错误的项会导致错误。处理函数体时假定该参数无值。该项的值也会因函数调用而改变。关于 lvalue 的更多信息，见「赋值运算符」(原书第 199 页)。

限定符只能用于限定相关项。限定符见表 8.32。

表 8.32: 限定符适用项

限定符	变量	函数	参数	结构成员
binary		是		
const	是		是	
deprecated	是	是	是	是
in_out			是	
inline		是		
literal			是	

表 8.32: 限定符适用项（续）

限定符	变量	函数	参数	结构成员
obsolete	是	是	是	是
out			是	

8.8 模块化代码示例

Example of Modularized Code

• function.rh

```
// Use pragma once to prevent including
// this file multiple times
#pragma once

#include <icv.rh>

// Function prototypes
fx1 : function (
    lyr : polygon_layer,
    w   : double
) returning out: polygon_layer;

fx2 : function (
    lyr : polygon_layer,
    d   : double
) returning out: polygon_layer;

#include <functions.rs>
```

• function.rs

```
// Function definitions
fx1 : function (
    lyr : polygon_layer,
    w   : double
) returning out: polygon_layer {
    // code body
    out = f( lyr, w);
};

fx2 : function (
    lyr : polygon_layer,
    d   : double
) returning out: polygon_layer {
    // code body
    out = f( lyr, d);
};
```

- main.rs

```
#include <icv.rh>
#include <function.rh>

// assign(), library(), etc.

y = fx1( a, 0.1);
z = fx2( b, 0.3);
```

Chapter 9

动态链接库支持

Dynamic-Link Library Support

本章说明如何使用 IC Validator 工具的动态链接（dynamic-link）功能。

动态链接功能专用于器件抽取（device extraction）。该功能允许将测量数据传递给 IC Validator 工具外部的 C 库，再将 C 库的计算结果返回工具。这些 C 库必须是共享（动态）库。动态链接库支持包括以下各节：

- 动态链接函数
- 使用动态链接函数

9.1 动态链接函数

Dynamic-Link Functions

下列 IC Validator 函数支持动态链接库功能：

- `dev_dlink_library_close()` 函数
- `dev_dlink_library_open()` 函数
- `dev_dlink()` 实用函数
- `dev_dlink_library()` 实用函数
- 所有器件配置函数（例如 `capacitor()`）的 `dlink_libraries` 参数

9.2 使用动态链接函数

Using the Dynamic-Link Functions

使用动态链接的步骤如下：

1. 在 IC Validator runset 中：
 - (a) 使用 `dev_dlink_library_open()` 打开 C 库。
 - (b) 在器件配置函数中通过 `dlink_libraries` 参数指定库句柄。
 - (c) 使用 `dev_dlink_library_close()` 关闭库。

例如：

```

dlink_handle = dev_dlink_library_open("library_name");

nmos(
    property_function=my_mos_func,
    dlink_libraries = {dlink_handle}
);

dev_dlink_library_close(dlink_handle);

```

2. 使用实用函数定义器件测量数据的计算。动态链接远端函数通常按如下方式组织：

```

my_mos_func : function(void) returning void {
    //(1) 使用器件实用函数获取器件测量数据
    //(2) 将测量数据编码为一维数组
    //(3) 使用 dev_dlink_library() 实用函数获取动态链接库文件句柄
    //(4) 使用 dev_dlink() 实用函数调用用户定义的动态链接函数，
    //     并将必要信息传入外部共享库
        IC Validator data
}

```

3. 用户用 C 语言编写的包装函数典型流程如下：

```

customer_wrapper_function() {
    //(1) 将测量数据解码为客户格式
    //(2) 根据字符串参数决定函数指针
    //(3) 调用内部函数
    //(4) 返回计算结果
}

```

Chapter 10

统一填充

Unified Fill

借助 IC Validator 的函数及其编程语言 PXL，可编写简单或复杂的过程以满足填充需求。本章说明如何使用面向应用的填充插入函数 `unified_fill()`，并给出若干使用统一填充功能的 `runset` 用法模型。

注意

Unified Fill 函数的完整说明见 *IC Validator Reference Manual*。

`unified_fill()` 中定义的填充图案可包含单个填充层或多个填充层，即可输出多个多边形层。每个填充层定义可含单个矩形或任意数量的非矩形多边形。此外，每个图案可视为按间距约束在目标区域内重复的结构。

统一填充功能包括以下各节：

- 概述
- 填充图案类型
- 填充到填充间距
- 目标区域与填充到信号间距
- 范围与宽度相关间距
- 多边形分组
- 信号对齐填充
- 改进图案插入
- 节距对齐填充
- 判据分析
- 输出层
- 层压缩

10.1 概述

Overview

`unified_fill()` 在开发填充流程时提供高度自动化与灵活性。填充需求可以简单到在某一层内生成矩形图案阵列，也可以复杂到用多种图案填充设计以达到特定密度目标。

该函数支持多种可按需求组合配置的选项。主要高层特性包括：

- 定义不同类型图案的能力
- 定义使用从外部数据库读入的 fill cell 的填充插入序列
- 基于方程的判据分析
- 层着色（layer coloring）
- 信号对齐填充
- 自动创建目标层

表 10.1 定义统一填充相关术语。

表 10.1: 统一填充术语

术语	定义
设计层（Design layer）与信号层（Signal layer）	设计中已有的层
Fill cell	从外部版图数据库导入到 IC Validator runset 的填充图案定义
填充层（Fill layer）	由 unified_fill() 生成的多边形层输出
目标区域（Target region）与可填充区域（Fillable region）	可添加填充的区域
禁入区域（Keepout region）与排除区域（Exclusion region）	不得添加填充的区域
填充图案（Fill pattern）与填充结构（Fill structure）	由一个或多个填充层组成的填充配置。图案可定义为：单个填充层；或多个填充层
填充到信号间距（Fill-to-signal spacing）	从填充图案或填充层到设计层的间距
填充到填充间距（Fill-to-fill spacing）	填充图案之间或填充层之间的间距

图 10.1 汇总各类型填充图案可用的功能。



图 10.1: 填充图案概览

10.2 填充图案类型

Types of Fill Patterns

unified_fill() 支持下列填充图案定义类型：

- 多边形填充图案（Polygon Fill Patterns）
- 可调矩形填充图案（Adjustable Rectangle Fill Patterns）
- 堆叠填充图案（Stack Fill Patterns）

- Cell 填充图案 (Cell Fill Patterns)
- 可扩展 Fill Cell 图案 (Expandable Fill Cell Patterns)
- 条带填充图案 (Stripe Fill Patterns)

10.2.1 多边形填充图案

Polygon Fill Patterns

多边形填充图案是基于坐标的填充图案定义。对此类型，将 **type** 选项设为 UF_POLYGON。

填充图案可有任意数量的填充层。多层图案定义中，每个填充层有各自的层定义。如例 10.1 所示，填充层由 **layers** 选项中 **output_layer_key** 指定的名称标识。

单层单矩形图案定义 *Single Layer Pattern Definition With Single Rectangle*

```
DATA_M1_fill : list of list of coordinate_s = {
    { {0.0, 0.0}, {1.0, 0.0}, {1.0, 1.0}, {0.0, 1.0} }
};

SHAPE_M1_fill : polygon_layer_s = {
    layer_spec = {output_layer_key = "M1_fill"}
    polygons = DATA_M1_fill,
};

fill_patterns = {{
    type = UF_POLYGON,
    polygon_fill = {
        layers = {SHAPE_M1_fill}
    }
}}
```

代码 10.1: 单层单矩形图案定义

单层多矩形图案定义 *Single Layer Pattern Definition With Multiple Rectangles*

例 10.2 与图 10.2 所示图案定义有一个填充层 **M1_fill**，由两个矩形定义。虚线表示层定义的包围盒 (bounding box)。该图案按间距约束在目标区域内重复。因图案仅含一层，图案包围盒与层定义包围盒相同。

```
DATA_M1_fill : list of list of coordinate_s = {
    { {0.0, 0.0}, {1.0, 0.0}, {1.0, 2.0}, {0.0, 2.0} },
    { {2.0, 0.0}, {3.0, 0.0}, {3.0, 2.0}, {2.0, 2.0} }
};

SHAPE_M1_fill : polygon_layer_s = {
    layer_spec = {output_layer_key = "M1_fill"}
    polygons = DATA_M1_fill,
};

fill_patterns = {{
    type = UF_POLYGON,
    polygon_fill = {
        layers = {SHAPE_M1_fill}
    }
}}
```

```
}}
```

代码 10.2: 使用 UF_POLYGON 类型的单填充层示例

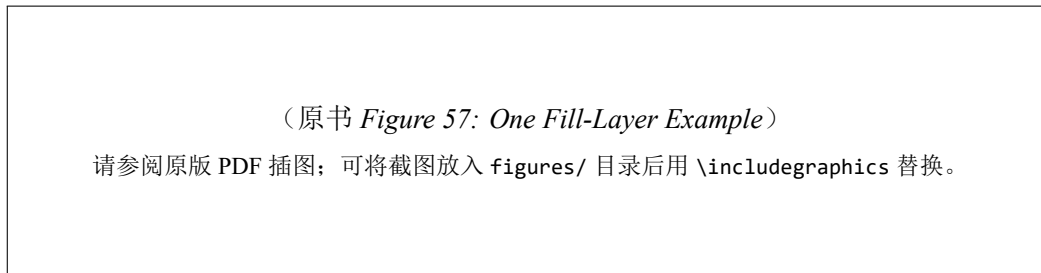


图 10.2: 单填充层示例

多层图案定义 *Multilayer Pattern Definition*

例 10.3 与图 10.3 所示图案定义有两个填充层 `M1_fill` 与 `M2_fill`。虚线表示图案定义的包围盒。该图案按间距约束在目标区域内重复。本例中图案包围盒与各层定义的包围盒不同。重复图案时，以整个图案的包围盒为准。

```
DATA_M1_fill : list of list of coordinate_s = {
  { {1.0, 0.0}, {2.0, 0.0}, {2.0, 3.0}, {1.0, 3.0} }
};

SHAPE_M1_fill : polygon_layer_s = {
  layer_spec = {output_layer_key = "M1_fill"}
  polygons = DATA_M1_fill,
};

DATA_M2_fill : list of list of coordinate_s = {
  { {0.0, 1.0}, {3.0, 1.0}, {3.0, 2.0}, {0.0, 2.0} }
};

SHAPE_M2_fill : polygon_layer_s = {
  layer_spec = {output_layer_key = "M2_fill"}
  polygons = DATA_M2_fill,
};

fill_patterns = {{
  type = UF_POLYGON,
  polygon_fill = {
    layers = {SHAPE_M1_fill,
              SHAPE_M2_fill}
  }
}}
```

代码 10.3: 使用 UF_POLYGON 类型的两填充层示例

多填充图案定义 *Multiple Fill Pattern Definition*

可在 `unified_fill()` 中指定多个图案定义。输入目标区域按指定顺序对每个填充图案定义进行处理。图 10.4 与图 10.5 展示的填充流程先插入多层图案 M1 与 M2，再在剩余目标区域插入单层图案 M1。

(原书 *Figure 58: Two Fill-Layer Example*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.3: 两填充层示例

```
DATA_M1_fill : list of list of coordinate_s = {
    { {1.0, 0.0}, {2.0, 0.0}, {2.0, 3.0}, {1.0, 3.0} }
};

SHAPE_M1_fill : polygon_layer_s = {
    layer_spec = {output_layer_key = "M1_fill"}
    polygons = DATA_M1_fill,
};

DATA_M2_fill : list of list of coordinate_s = {
    { {0.0, 1.0}, {3.0, 1.0}, {3.0, 2.0}, {0.0, 2.0} }
};

SHAPE_M2_fill : polygon_layer_s = {
    layer_spec = {output_layer_key = "M2_fill"}
    polygons = DATA_M2_fill,
};

fill_patterns = {
    {type = UF_POLYGON,
     polygon_fill = {
         layers = {SHAPE_M1_fill,
                   SHAPE_M2_fill}
     }
    },
    {type = UF_POLYGON,
     polygon_fill = {
         layers = {SHAPE_M1_fill}
     }
    }
}
```

代码 10.4: 使用 `unified_fill()` 的 `fill_patterns` 选项的多图案示例

(原书 *Figure 59: Multiple Patterns Example*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.4: 多图案示例

10.2.2 可调矩形填充图案

Adjustable Rectangle Fill Patterns

可调矩形填充图案是单层矩形图案定义，可自动拉伸或收缩矩形尺寸以最大化填充密度。对此类型，将 `type` 设为 `UF_ADJUSTABLE`。

可从特定矩形尺寸出发，选择沿目标区域边界增大或缩小所有形状。增大与缩小按用户指定的增量进行。仅对目标区域边界上的填充形状进行调整。

例 10.5 定义最小尺寸为 $1\mu\text{m} \times 1\mu\text{m}$ 的矩形。目标区域边界上的矩形沿 x 方向以 $0.05\mu\text{m}$ 为增量增长，以拟合最大可能形状。如图 10.5 所示，最大可能形状为 $4\mu\text{m} \times 1\mu\text{m}$ 。

```
fill_patterns = {{
  type = UF_ADJUSTABLE,
  adjustable_fill = {
    layer_spec = {output_layer_key = "M1_Fill"}
    width = 1.0,
    height = 1.0,
    width_bound = 4.0,
    width_delta = 0.05,
  }
}}
```

代码 10.5: 使用 `UF_ADJUSTABLE` 类型的可调矩形填充图案示例

(原书 *Figure 60: Adjustable Rectangle Fill Pattern Example*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.5: 可调矩形填充图案示例

10.2.3 堆叠填充图案

Stack Fill Patterns

堆叠填充图案类似于 `UF_POLYGON`，但在开发填充流程时自动化程度更高。对此类型，将 `type` 设为 `UF_STACK`。

当应用需要按序插入多层图案的多种变体、而又不必显式定义所有变体时,可使用 `UF_STACK`。通过 `unified_fill()` 的选项, 提供一个由多个填充层组成的单一图案定义。工具根据可填充区域尝试插入不同变体: 先从包含全部层的原始定义开始, 再尝试仅含层子集的变体。

目标是找到在给定可填充区域内能放入、且层数最多的图案变体。使用 `stack_fill` 选项的 `required_layer_keys` 定义填充层之间的依赖, 例如 `via1` 需要 `metal1` 与 `metal2`。例 10.6 给出示例。

```
SHAPE_M1_fill : stack_layer_s = {
    layer_spec = {output_layer_key = "M1_fill"}
    polygons = DATA_M1_fill,
};
SHAPE_M2_fill : stack_layer_s = {
    layer_spec = {output_layer_key = "M2_fill"}
    polygons = DATA_M2_fill,
};
SHAPE_M3_fill : stack_layer_s = {
    layer_spec = {output_layer_key = "M3_fill"}
    polygons = DATA_M3_fill,
};
SHAPE_V1_fill : stack_layer_s = {
    layer_spec = {output_layer_key = "V1_fill"},
    polygons = DATA_V1_fill,
    required_layer_keys = {"M1_fill", "M2_fill"}
};
SHAPE_V2_fill : stack_layer_s = {
    layer_spec = {output_layer_key = "V2_fill"},
    polygons = DATA_V2_fill,
    required_layer_keys = {"M2_fill", "M3_fill"}
};

fill_patterns = {{
    type = UF_STACK,
    stack_fill = {
        layers = {SHAPE_M1_fill,
                   SHAPE_V1_fill,
                   SHAPE_M2_fill,
                   SHAPE_V2_fill,
                   SHAPE_M3_fill},
        min_stack = 1,
        max_stack = 5
    }
}}
```

代码 10.6: 使用 `stack_fill` 选项的堆叠填充图案示例

`SHAPE_M1_fill`、`SHAPE_V1_fill`、`SHAPE_M2_fill`、`SHAPE_V2_fill` 与 `SHAPE_M3_fill` 是同一多层图案的各个层定义。`unified_fill()` 按下列顺序用这些图案处理可填充区域:

- M1-V1-M2-V2-M3
- M1-V1-M2
- M2-V2-M3
- M1

- M2
- M3

上述 UF_STACK 示例等价于编写多个 UF_POLYGON 类型的图案定义，如图 10.8 所示。

```
fill_patterns = {
    {type = UF_POLYGON,
      polygon_fill = {          // First pattern for insertion
        layers = {SHAPE_M1_fill,
                  SHAPE_V1_fill,
                  SHAPE_M2_fill,
                  SHAPE_V2_fill,
                  SHAPE_M3_fill}
        }
    },
    {type = UF_POLYGON,
      polygon_fill = {          // Second pattern for insertion
        layers = {SHAPE_M1_fill,
                  SHAPE_V1_fill,
                  SHAPE_M2_fill}
        }
    },
    {type = UF_POLYGON,
      polygon_fill = {          // Third pattern for insertion
        layers = {SHAPE_M2_fill,
                  SHAPE_V2_fill,
                  SHAPE_M3_fill}
        }
    },
    {type = UF_POLYGON,
      polygon_fill = {          // Fourth pattern for insertion
        layers = {SHAPE_M1_fill}
        }
    },
    {type = UF_POLYGON,
      polygon_fill =           // Fifth pattern for insertion
        {layers = {SHAPE_M2_fill}
        }
    },
    {type = UF_POLYGON,
      polygon_fill =           // Sixth pattern for insertion
        {layers = {SHAPE_M3_fill}
        }
    }
}
```

代码 10.7: 使用 UF_POLYGON 类型编写多个图案定义

各层定义的顺序决定所放置图案变体的顺序。若层定义如例 10.8 所示：

```
fill_patterns = {{
    type = UF_STACK,
```

```

    stack_fill = {
        layers = {SHAPE_M3_fill,
                  SHAPE_V2_fill,
                  SHAPE_M2_fill,
                  SHAPE_V1_fill,
                  SHAPE_M1_fill},
        min_stack = 1,
        max_stack = 5
    }
}

```

代码 10.8: 各层定义顺序示例

则图案插入顺序变为:

- M1-V1-M2-V2-M3
- M2-V2-M3
- M1-V1-M2
- M3
- M2
- M1

工具可构造的图案变体数量取决于 `min_stack` 与 `max_stack`:

- `min_stack` 指定任一变体所需的最少填充层数。
- `max_stack` 指定变体允许的最多填充层数。

图 10.6 给出填充层为 5 个金属与 4 个 via、且 `max_stack = 9`、`min_stack = 5` 时的全部图案变体示例。

(原书 *Figure 61: Example of All Pattern Variations for a Fill Application*)

请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.6: 填充应用中全部图案变体示例

取决于图案中的层定义, 图案包围盒可能大于各层定义的单独包围盒。此时, 图案较小变体的包围盒可能与包含全部层的完整图案定义不同。

默认情况下, `unified_fill()` 对所有变体保留完整图案的包围盒。所有自动变体默认与完整图案的包围盒对齐, 如图 10.6 所示。

例 10.9 与图 10.8 给出完整图案为含 `M1_fill` 与 `M2_fill` 两层结构 (见图 10.7) 的示例。有两个变体: 仅 `M1_fill` 与仅 `M2_fill`。但所有图案的包围盒都与完整图案相同。因此, `M1_fill` 上各矩形看起来可能比指定间距更远——因为间距基于包围盒。

```

fill_patterns = {{
    type = UF_STACK,
    stack_fill = {
        pattern_spec = {space_x = 0.1,
                        space_y = 0.1},
        layers = {SHAPE_M1_fill,

```

(原书 *Figure 62: Two Fill Layers Example*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.7: 两填充层示例

```
        SHAPE_M2_fill},
    min_stack = 1,
    max_stack = 2
}
}}
```

代码 10.9: 使用 `stack_fill` 的两填充层图案结构

(原书 *Figure 63: Pattern Structure With Two Fill Layers*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.8: 两填充层的图案结构

默认情况下, `unified_fill()` 对所有变体保留完整图案的包围盒。可将 `separate_patterns` 设为 `true` 以改变对齐行为。此时每个图案变体根据其所含层子集的范围拥有各自包围盒；插入变体时使用该变体自身的包围盒，而非完整图案的包围盒。在图 10.9 中，完整图案的两个变体各有包围盒，因此看起来比 `separate_patterns` 为 `false` 时放置得更近。

(原书 *Figure 64: Pattern Structure With Two Fill Layers With separate_patterns Option True*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.9: `separate_patterns` 为 `true` 时的两填充层图案结构

当 `separate_patterns` 为 `false` 时：

- `fill_context`、`iterations` 与 `shift_factor` 选项被忽略。
- `min_stack` 与 `max_stack` 被忽略；换言之，工具使用图案的全部可能变体。

10.2.4 Cell 填充图案

Cell Fill Patterns

Cell 填充图案允许从外部数据库导入图案，而无需在 runset 中手工定义。对此类型，将 **type** 设为 **UF_CELL**。

UF_CELL 提供额外接口，可从外部 GDSII 或 OASIS 库导入 cell，并将该 cell 中的层映射到填充层。其余功能与 **UF_POLYGON**、**UF_STACK** 相同。导入的图案定义可按 **UF_POLYGON** 方式原样使用，或按 **UF_STACK** 方式拆成更小图案。

Fill cell 或 **fill cell** 库可与填充 runset 一并维护。这些 **fill cell** 含 runset 可导入的填充层，并按指定约束插入目标区域。关于将 cell 导入 runset，见 *IC Validator Reference Manual* 中的 **import_gds_cell()** 与 **import_oasis_cell()**。

在例 10.10 中，**M1_fill**、**V1_fill**、**M2_fill**、**V2_fill** 与 **M3_fill** 层定义从 **Fill_Cell.gds** 文件中的 **FillCell** cell 导入。例如，**Fill_Cell.gds** 中的 **Layer (1;0)** 指定为 **M1_fill**。概念上等价的额外功能示例见“堆叠填充图案”。

```
Fill_Cell_lib = gds_library("./Fill_Cell.gds");
Fill_Cell = import_gds_cell(Fill_Cell_lib, "FillCell");

STACK_metal_fill : cell_fill_s =
{
    cell = Fill_Cell
    layers = {          // Map Individual layers of the cell to Fill layers
        {ldt_list = {{1, 0}}, layer_spec = {output_layer_key = "M1_fill"}},
        {ldt_list = {{2, 0}}, layer_spec = {
            output_layer_key = "V1_fill",
            required_layer_keys = {"M1", "M2"}},
        {ldt_list = {{3, 0}}, layer_spec = {output_layer_key = "M2_fill"}},
        {ldt_list = {{4, 0}}, layer_spec = {
            output_layer_key = "V2_fill",
            required_layer_keys = {"M2", "M3"}},
        {ldt_list = {{5, 0}}, layer_spec = {output_layer_key = "M3_fill"}}
    },
    min_stack = 1,
    max_stack = 5,
    rotation = NONE,
    reflection = false
};
```

代码 10.10: Cell 填充图案示例

注意

按需求，**UF_STACK** 相对 **UF_POLYGON** 在开发填充流程时提供更多自动化；类似地，**UF_CELL** 相对 **UF_STACK** 与 **UF_POLYGON** 提供更多自动化。

图 10.10 给出 **UF_CELL** 用法模型示例。

10.2.5 可扩展 Fill Cell 图案

Expandable Fill Cell Patterns

可扩展 **fill-cell** 图案允许插入沿填充目标区域边界扩展的 **fill cell**。对此类型，将 **type** 设为 **UF_EXPANDABLE** 或 **UF_EXPANDABLE_CELL**。

UF_EXPANDABLE_CELL 提供额外接口，可从外部 GDSII 或 OASIS 库导入 cell 并将层映射到填充层。其余功能与 **UF_EXPANDABLE** 相同。

(原书 *Figure 65: Usage Model for the UF_CELL Type*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 10.10: UF_CELL 类型的用法模型

使用可扩展 fill-cell 类型图案时，先用增量单元 (incremental unit cells) 定义基础 fill cell。例如图 10.11，简单基础 cell 由三个增量单元组成；本例中仅中间单元可重复。

(原书 *Figure 66: Repeatable Cell Example*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 10.11: 可重复 cell 示例

图 10.12 展示 fill cell 的扩展。上半部分为扩展前插入的基础 cell；下半部分为通过重复中间单元扩展后的基础 cell。

(原书 *Figure 67: Expansion of a Fill Cell*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 10.12: Fill cell 的扩展

例 10.11 说明如何编写 UF_EXPANDABLE 类型：

1. 定义每个增量单元所用的层。
2. 声明组成基础 cell 的三个实际增量单元 cell。
3. 调用 unified_fill() 并取回输出层。

```
// Define the layers used in each incremental unit
EXPAND_CELL_LEFT_LAYER_1 : polygon_layer_s = {
  layer_spec = {output_layer_key = "OUTPUT_EXPAND_CELL_LEFT_LAYER_1"},
  polygons = {{ {0.045, 0.193}, {0.261, 0.103}, {0.261, 0.287}, {0.058, 0.247} }}
};
EXPAND_CELL_LEFT_LAYER_2 : polygon_layer_s = {
  layer_spec = {output_layer_key = "OUTPUT_EXPAND_CELL_LEFT_LAYER_2"},
  polygons = {{ {0.045, 0.128}, {0.191, 0.128}, {0.191, 0.432}, {0.155, 0.232} }}
};
```



```

EXPAND_CELL_MIDDLE_LAYER_1 : polygon_layer_s = {
    layer_spec = {output_layer_key = "OUTPUT_EXPAND_CELL_MIDDLE_LAYER_1"},
    polygons = {{ {0.045, 0.193}, {0.261, 0.103}, {0.261, 0.287}, {0.058, 0.247} }}
};
EXPAND_CELL_MIDDLE_LAYER_2 : polygon_layer_s = {
    layer_spec = {output_layer_key = "OUTPUT_EXPAND_CELL_MIDDLE_LAYER_2"},
    polygons = {{ {0.045, 0.128}, {0.191, 0.128}, {0.191, 0.432}, {0.155, 0.232} }}
};
EXPAND_CELL_RIGHT_LAYER_1 : polygon_layer_s = {
    layer_spec = {output_layer_key = "OUTPUT_EXPAND_CELL_RIGHT_LAYER_1"},
    polygons = {{ {0.045, 0.193}, {0.261, 0.103}, {0.261, 0.287}, {0.058, 0.247} }}
};
EXPAND_CELL_RIGHT_LAYER_2 : polygon_layer_s = {
    layer_spec = {output_layer_key = "OUTPUT_EXPAND_CELL_RIGHT_LAYER_2"},
    polygons = {{ {0.045, 0.128}, {0.191, 0.128}, {0.191, 0.432}, {0.155, 0.232} }}
};
// Declare the actual three incremental unit cells that make up the base cell
cell_A_left : base_cell_s = {
    layers = {EXPAND_CELL_LEFT_LAYER_1, EXPAND_CELL_LEFT_LAYER_2 },
    repeatable = false
};
cell_B_middle : base_cell_s = {
    layers = { EXPAND_CELL_MIDDLE_LAYER_1, EXPAND_CELL_MIDDLE_LAYER_2 },
    repeatable = true,
    maximum = 10
};
cell_C_right : base_cell_s = {
    layers = { EXPAND_CELL_RIGHT_LAYER_1, EXPAND_CELL_RIGHT_LAYER_2 },
    repeatable = false
};
// Call unified_fill()
expand_fill_cell_output = unified_fill(
    fill_patterns = {
        {type = UF_EXPANDABLE,
         expandable_polygon_fill = {
             pattern_spec = {
                 space_x = 0.12,
                 space_y = 0.06,
                 stagger_x = 0.2 },
             base_cell = {cell_A_left, cell_B_middle, cell_C_right} }
        }
    },
    fill_boundary = { type = LAYER, layer = REGION_CELL2_CORE }
);
// Retrieve the output layers
expand_fill_cell_layer_1 =
    expand_fill_cell_output ["OUTPUT_EXPAND_CELL_LEFT_LAYER_1"][0]
or expand_fill_cell_output ["OUTPUT_EXPAND_CELL_MIDDLE_LAYER_1"][0]
or expand_fill_cell_output ["OUTPUT_EXPAND_CELL_RIGHT_LAYER_1"][0]

```

```

expand_fill_cell_layer_2 =
    expand_fill_cell_output ["OUTPUT_EXPAND_CELL_LEFT_LAYER_2"][0]
or expand_fill_cell_output ["OUTPUT_EXPAND_CELL_MIDDLE_LAYER_2"][0]
or expand_fill_cell_output ["OUTPUT_EXPAND_CELL_RIGHT_LAYER_2"][0]

```

代码 10.11: UF_EXPANDABLE 类型示例

10.2.6 条带填充图案

Stripe Fill Patterns

条带填充图案允许在目标区域插入条带形填充。对此类型，将 **type** 设为 UF_STRIPE。指定条带的方向、宽度与间距，如例 10.12 所示。

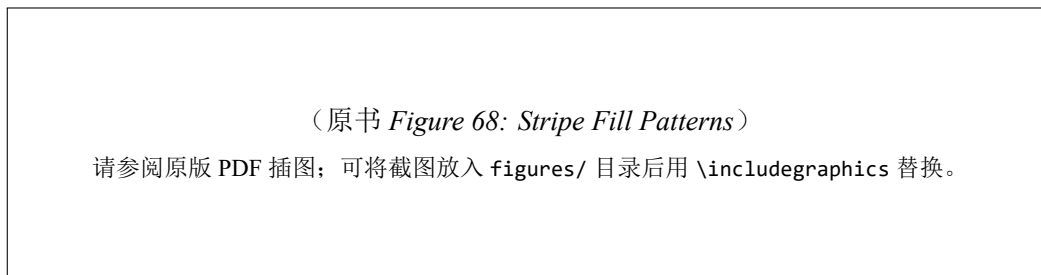


图 10.13: 条带填充图案

为条带填充指定对称 (symmetry) 时，unified_fill() 会：

1. 获取包围盒
2. 找到包围盒中心
3. 将第一条带的中心线放在包围盒中心
4. 以距中心条带等距的方式放置后续条带
5. 仅保留全宽条带

图 10.14 展示条带对称的结果。

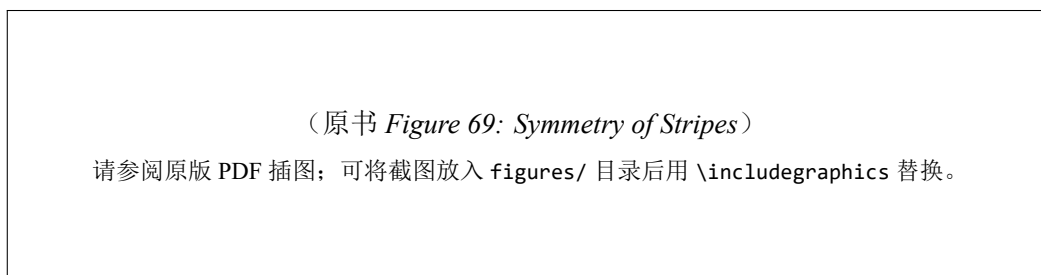


图 10.14: 条带对称

例 10.12 给出使用 UF_STRIPE 类型的 unified_fill() 调用。

```

result = unified_fill(
    fill_patterns = {
        {type = UF_STRIPE,
         stripe_fill = {
             layer_spec = {

```

```

        output_layer_key = "DM1_FILL",
    },
    direction = HORIZONTAL,
    width = 0.1,
    spacing = 0.2,
}
}
},

fill_boundary = {
    type = LAYER,
    layer = M1
},
extents_output = {
    {output_layer_key = "OUTPUT_DM_EXTENTS"}
}
);

```

代码 10.12: UF_STRIPE 类型示例

10.3 填充到填充间距

Fill-to-Fill Spacing

如前所述，可将 Unified Fill 函数配置为向指定目标区域插入不同类型图案。通过提供图案定义列表，指示工具按顺序重复每个图案定义以填充目标区域。

填充到填充间距功能允许定义：

- 相同图案实例之间的间距
- 不同图案之间的间距

10.3.1 相同图案实例之间的间距

Spacing Between Instances of the Same Patterns

相同图案实例之间的间距由 `pattern_spec` 中的 `space_x`、`space_y`、`stagger_x` 与 `stagger_y` 控制。

注意

在某些条件下这些值可为负，以支持图案放置重叠。

此外，对于位于不同目标区域内的填充多边形与图案之间的间距，可在 `pattern_spec` 内用 `pattern_spacing` 设置图案间距，或在 `layer_spec` 内用 `fill_to_fill_spacing` 设置各填充层之间的间距。相关选项为：

- `allowed_spacing_x`
- `allowed_spacing_y`
- `min_space_corner`
- `extension`

这些选项的说明见 Unified Fill 函数文档。可用它们在填充处理期间触发更复杂的层相关间距检查。`space_x`、`space_y`、`stagger_x` 与 `stagger_y` 按 `separate_patterns` 作用于整个图案定义的包围盒。

按设计, `unified_fill()` 确保任意两图案不会比 `space_x` 与 `space_y` 更近。例 10.13 为多层图案定义指定了 `space_x` 与 `space_y`。若落在目标区域相邻多边形中的图案过近, 则不插入其中之一以满足填充到填充间距约束, 如图 10.15。

```
SHAPE_M1_fill : polygon_layer_s = {
    layer_spec = {output_layer_key = "M1_fill"}
    polygons = DATA_M1_fill,
};
SHAPE_M2_fill : polygon_layer_s = {
    layer_spec = {output_layer_key = "M2_fill"}
    polygons = DATA_M2_fill,
};

fill_patterns = {{
    type = UF_POLYGON,
    polygon_fill = {
        pattern_spec = {space_x = 0.1,
                        space_y = 0.1},
        layers = {SHAPE_M1_fill,
                  SHAPE_M2_fill}
    }
}}
```

代码 10.13: 使用 `space_x` 与 `space_y` 的相同图案间距

(原书 *Figure 70: Spacing Between Same Patterns Example*)
 请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.15: 相同图案间距示例

例 10.14 在 `pattern_spacing` 中为多层图案定义指定 `allowed_spacing_x` 与 `allowed_spacing_y`, 以确保不同目标区域中图案实例之间有必要间距。如图 10.16, 因 x 、 y 方向间距满足允许值, 两个图案实例都被放置。

```
fill_patterns = {{
    type = UF_POLYGON,
    polygon_fill = {
        pattern_spec = {space_x = 0.1, space_y = 0.1,
                        pattern_spacing = {
                            allowed_spacing_x = {>= 0.02},
                            allowed_spacing_y = {>= 0.02}
                        }
        },
        layers = {SHAPE_M1_fill,
                  SHAPE_M2_fill}
    }
}}
```

代码 10.14: 允许间距小于 space 值时的多层图案间距

(原书 *Figure 71: Spacing Between Multilayer Patterns With Allowed Spacing Values Smaller Than Space Values Example*)

请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 10.16: 允许间距小于 space 值时的多层图案间距示例

例 10.15 在 M1_fill 填充层的 fill_to_fill_spacing 中指定允许间距。即使包围盒间距违反 space_x/space_y, 只要图案内各层满足允许间距, 工具也不移除图案, 如图 10.17。

```
SHAPE_M1_fill : polygon_layer_s = {
  layer_spec = {
    output_layer_key = "M1_fill",
    fill_to_fill_spacing = {
      allowed_spacing_x = {>= 0.02},
      allowed_spacing_y = {>= 0.02}
    }
  }
  polygons = DATA_M1_fill,
};
fill_patterns = [{
  type = UF_POLYGON,
  polygon_fill = {
    pattern_spec = {space_x = 0.1, space_y = 0.1},
    layers = {SHAPE_M1_fill,
              SHAPE_M2_fill}
  }
}]
```

代码 10.15: 在 fill_to_fill_spacing 中使用允许间距

(原书 *Figure 72: Using the Allowed Spacing Values in the fill_to_fill_spacing Option Example*)

请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 10.17: 在 fill_to_fill_spacing 中使用允许间距示例

10.3.2 不同图案之间的间距

Spacing Between Different Patterns

处理填充图案列表时，可用 `other_pattern_spacing` 指定不同图案之间的间距。该选项为整数到 `double` 约束的哈希，整数表示图案在列表中的顺序，0 表示第一个图案。例如，要将与第三个图案的间距设为 ≥ 0.5 ，写：

```
other_pattern_spacing = {2 => >= 0.5}
```

注意

间距始终作用于每个图案的包围盒。

图 10.18 给出使用图案 A、B、C 的应用，期望间距如下：

Pattern A:

- 到 Pattern B 的间距 = 2
- 到 Pattern C 的间距 = 1

Pattern B:

- 到 Pattern A 的间距 = 2
- 到 Pattern C 的间距 = 3

Pattern C:

- 到 Pattern A 的间距 = 1
- 到 Pattern B 的间距 = 3

(原书 *Figure 73: Spacing Between Different Patterns Example*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.18: 不同图案间距示例

在 `other_pattern_spacing` 中使用适当索引设置当前图案到其他图案的间距，如例 10.16。

```
fill_patterns = {
// First Pattern A
{type = UF_POLYGON,
 polygon_fill = {
   pattern_spec = {
     space_x = 0.1, space_y = 0.1,
     other_pattern_spacing = {1 => >= 2, 2 => >= 1}
   },
   layers = {SHAPE_A}
 }
},
// Second Pattern B
{type = UF_POLYGON,
```

```

    polygon_fill = {
        pattern_spec = {
            space_x = 0.1, space_y = 0.1,
            other_pattern_spacing = { 0 => >= 2, 2 => >= 3 }
        },
        layers = {SHAPE_B}
    }
},
// Third Pattern C
{type = UF_POLYGON,
  polygon_fill = {
    pattern_spec = {
        space_x = 0.1, space_y = 0.1,
        other_pattern_spacing = {0 => >= 1, 1 => >= 3}
    },
    layers = {SHAPE_C}
  }
}
}

```

代码 10.16: 使用 other_pattern_spacing 选项

10.3.3 间距选项的规则与限制

Rules and Restrictions for Spacing Options

间距选项适用下列规则与限制:

- allowed_spacing_x 与 allowed_spacing_y 可同时在 layer_spec 与 pattern_spacing 中设置。
- layer_spec 中的 allowed_spacing_x/allowed_spacing_y 不支持多个间距范围（列表只能有一个值）；仅支持 > 与 ≥ 约束。
- pattern_spacing 中的 allowed_spacing_x/allowed_spacing_y 支持多个间距范围；不支持 < 与 ≤。
- allowed_spacing_x、allowed_spacing_y 与 min_space_corner 不支持负值。
- 未指定允许间距时，有效填充到填充间距为任意 $\geq \text{space_x}/\text{space_y}$ 的距离；工具确保相邻填充区域多边形中的图案或填充层具有该最小间距。
- 指定允许间距时，工具确保相邻目标层多边形中的图案或填充层具有允许间距的最小值。
- 当 fill_context 为 PARTITION_CONTEXT 时，相邻分区中的图案或填充层使用已指定的允许间距。
- 当 type 为 UF_STACK 或 UF_CELL 时，other_pattern_spacing 不适用于图案变体；给定图案不同变体之间的最小间距基于 space_x/space_y。
- 默认情况下，unified_fill() 对填充区域中不共享任何公共投影的不同多边形之间的图案执行角落检查（corner checking）。
- 当 type 为 UF_STACK 时，工具不对不同置换的图案应用 pattern_spacing 或 fill_to_fill_spacing。
- 当 type 为 UF_POLYGON、UF_STACK、UF_CELL、UF_EXPANDABLE 或 UF_EXPANDABLE_CELL 时，space_x/space_y 可为 ≤ 0 。对 UF_STACK 与 UF_CELL，还须 separate_patterns 为 false 且 grouping 为 NONE。
- 若 space_x 或 space_y 为负，则使用 min_space_corner 与 extension 的角落检查被禁用。默认情况下，不同填充区域中填充图案之间的角落间距检查基于 $\max(\text{space_x}, \text{space_y})$ 。

10.4 目标区域与填充到信号间距

Target Region and Fill-to-Signal Spacing

`unified_fill()` 不要求在 `runset` 中显式推导目标区域。工具使用各种填充到信号间距约束自动构造目标区域。在单个填充层定义或填充图案定义中，用 `fill_to_signal_spacing` 指定这些间距值。

在例 10.17 中，`M1_Fill` 填充层相对 `aM1` 设计层保持 $0.1\ \mu\text{m}$ 。类似地：

- `M1_fill` 到 `aM1_fill` 的最小间距为 $0.5\ \mu\text{m}$
- `M2_fill` 到 `aM2` 的最小间距为 $0.1\ \mu\text{m}$
- `M2_fill` 到 `aM2_fill` 的最小间距为 $0.5\ \mu\text{m}$
- (`M1_fill`, `M2_fill`) 多层图案到 `BLKG_1` 的最小间距为 $0.3\ \mu\text{m}$
- (`M1_fill`, `M2_fill`) 多层图案到 `BLKG_2` 的最小间距为 $0.2\ \mu\text{m}$

```
DATA_M1_fill : list of list of coordinate_s = {
  { {1.0, 0.0}, {2.0, 0.0}, {2.0, 3.0}, {1.0, 3.0} }
};
SHAPE_M1_fill : polygon_layer_s = {
  layer_spec = {
    output_layer_key = "M1_fill",
    fill_to_signal_spacing = {
      {aM1, min_space = 0.1},
      {aM1_FILL, min_space = 0.5}
    }
  }
  polygons = DATA_M1_fill,
};

DATA_M2_fill : list of list of coordinate_s = {
  { {0.0, 1.0}, {3.0, 1.0}, {3.0, 2.0}, {0.0, 2.0} }
};
SHAPE_M2_fill : polygon_layer_s = {
  layer_spec = {
    output_layer_key = "M2_fill",
    fill_to_signal_spacing = {
      {aM2, min_space = 0.1},
      {aM2_FILL, min_space = 0.5}
    }
  }
  polygons = DATA_M2_fill,
};

fill_patterns = {{
  type = UF_POLYGON,
  polygon_fill = {
    layers = {SHAPE_M1_fill,
              SHAPE_M2_fill},
    fill_to_signal_spacing = {
      {aBLKG_1, min_space = 0.3},
      {aBLKG_2, min_space = 0.2}
    }
  }
};
```



```

    }
  }
}}
```

代码 10.17: fill_to_signal_spacing 选项示例

对 UF_POLYGON 类型，多层图案不会拆成更小变体。如图 10.19，图案在目标区域内按 fill_to_signal_spacing 指定的与设计层最小距离重复。

默认情况下，unified_fill() 从设计层边到图案包围盒测量间距。图案的禁入区域由该图案及其各层定义中的全部 fill_to_signal_spacing 约束构造。

测量填充与信号层间距有两种方法，由 spacing_context 控制：

- 设为 UF_PATTERN 时，从信号量到填充图案的包围盒。
- 设为 UF_LAYER 时，从信号量到图案内层的各个多边形。

注意

当 space_x 与 space_y 为负时，不支持将 spacing_context 设为 UF_PATTERN。

(原书 Figure 74: spacing_context = UF_PATTERN Example)
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 10.19: spacing_context = UF_PATTERN 示例

如图 10.20，当 spacing_context 为 UF_LAYER 时，工具从设计层边到单个层定义的包围盒测量填充到信号间距。

(原书 Figure 75: spacing_context = UF_LAYER Example)
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 10.20: spacing_context = UF_LAYER 示例

例 10.18 与图 10.21 说明 spacing_context 的行为。

```

SHAPE_M1_fill : polygon_layer_s = {
  layer_spec = {
    output_layer_key = "M1_fill",
    fill_to_signal_spacing = {
      {aM1, min_space = 5.0}
    }
  }
  polygons = DATA_M1_fill,
};
```

```

SHAPE_M2_fill : polygon_layer_s = {
    layer_spec = {
        output_layer_key = "M2_fill",
        fill_to_signal_spacing = {
            {aM2, min_space = 0.1}
        }
    }
    polygons = DATA_M2_fill,
};

fill_patterns = {{
    type = UF_POLYGON,
    polygon_fill = {
        layers = {SHAPE_M1_fill,
                  SHAPE_M2_fill}
    }
}}

```

代码 10.18: 填充层定义中的 fill_to_signal_spacing

(原书 *Figure 76: spacing_context Behavior*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 10.21: spacing_context 行为

对 UF_POLYGON、UF_STACK、UF_CELL、UF_EXPANDABLE 与 UF_EXPANDABLE_CELL，可在 layer_spec 与图案级同时指定 fill_to_signal_spacing；同时指定时取最严格值。例 10.19 中，M1_fill 相对 aM1 至少保持 0.5 μm 。

```

SHAPE_M1_fill : polygon_layer_s = {
    layer_spec = {
        output_layer_key = "M1_fill",
        fill_to_signal_spacing = {
            {aM1, min_space = 0.5}
        }
    }
    polygons = DATA_M1_fill,
};

fill_patterns = {{
    type = UF_POLYGON,
    polygon_fill = {
        layers = {SHAPE_M1_fill,},
        fill_to_signal_spacing = {
            {aM1, min_space = 0.1}
        }
    }
}}

```

```
}}
```

代码 10.19: 在 `layer_spec` 与图案中同时定义 `fill_to_signal_spacing`

例 10.20 与图 10.22 在填充层定义与填充图案定义的不同位置使用多个设计层与 `fill_to_signal_spacing` 值。

```
SHAPE_M1_fill : polygon_layer_s = {
  layer_spec = {
    output_layer_key = "M1_fill",
    fill_to_signal_spacing = {
      {aM1, min_space = 1.0}
    }
  }
  polygons = DATA_M1_fill,
};
SHAPE_M2_fill : polygon_layer_s = {
  layer_spec = {
    output_layer_key = "M2_fill",
    fill_to_signal_spacing = {
      {aM2, min_space = 2.0}
    }
  }
  polygons = DATA_M2_fill,
};

fill_patterns = {{
  type = UF_POLYGON,
  polygon_fill = {
    layers = {SHAPE_M1_fill,
              SHAPE_M2_fill},
    fill_to_signal_spacing = {
      {aBLKG, min_space = 5.0}
    }
  }
}}
```

代码 10.20: 带 `spacing_context` 与多个设计层的 `fill_to_signal_spacing`

(原书 *Figure 77: spacing_context Behavior With Multiple Design Layers*)

请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.22: 多设计层下的 `spacing_context` 行为

10.5 范围与宽度相关间距

Range and Width Dependent Spacing

除 `fill_to_signal_spacing` 中的固定最小间距外，还可指定填充层到设计层的间距范围。可指定下界与上界约束列表，表示填充层相对设计层的允许间距；填充图案仅放置在这些间距约束内。此外，可基于设计层多边形宽度指定间距约束列表。

在例 10.21 中，`M1_fill` 到 `aM1` 的间距为：

- `M1_fill` 到 `aM1` 的最小间距：
 - `aM1` 宽度 > 0.1
 - 间距 $= 0.1$
- `M1_fill` 到 `aM1` 的允许间距：
 - `aM1` > 0.3
 - $0.2 < \text{space} < 0.5$ 或 $\text{space} \geq 1.0$
 即不允许将 `M1_fill` 放在相对 `aM1` 间距 ≤ 0.2 或 $0.5 \leq \text{space} < 1.0$ 处。

```
layer_spec = {
    output_layer_key = "M1_fill",
    fill_to_signal_spacing = {
        {aM1, {
            {width > 0.1, {>= 0.1}},
            {width > 0.3, {(0.2, 0.5), >= 1.0}}
        }},
    }
    polygons = DATA_M1_fill,
};

fill_patterns = [{
    type = UF_POLYGON,
    polygon_fill = {
        layers = {SHAPE_M1_fill}
    }
}]
```

代码 10.21: 范围与宽度相关间距示例

10.6 多边形分组

Polygon Grouping

对 `UF_STACK` 与 `UF_CELL` 填充类型，`unified_fill()` 可将图案定义拆成更小置换。若图案定义有两个填充层（例如 `M1_fill` 与 `M2_fill`），可有三种置换：

- 完整图案
- 仅含 `M1_fill` 的图案
- 仅含 `M2_fill` 的图案

当填充层的单个层定义含多个多边形时，可配置工具进一步拆分层定义，使得在图案放置导致间距违例时，该填充层的多边形要么全部保留，要么全部不保留。默认情况下，当层定义由多个多边形构成时，每次图案重复中工具保留所有满足填充到信号间距的多边形。

如图 10.23，虚线所示多层图案在目标区域内重复，`spacing_context` 为 `UF_LAYER`。图案含两个填充层，各由两个矩形组成。基于填充层到设计层的间距，每个实例可能保留全部或部分多边形；导致间距违例的多边形被移除。

(原书 *Figure 78: Multilayer Pattern Repeated Inside a Target Region*)
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.23: 目标区域内重复的多层图案

也可配置为：若某次图案重复中填充层的任一多边形导致间距违例，则从该次重复中移除该填充层的全部多边形。为此在填充层定义中将 `grouping` 设为 `ALL`。例如例 10.22 与图 10.24，即使仅一个多边形违例，右上图案重复中 `M1_fill` 的两个多边形都被移除。

```
SHAPE_M1_fill : polygon_layer_s = {
  layer_spec = {
    output_layer_key = "M1_fill",
    grouping = ALL,
    fill_to_signal_spacing = {
      {aM1, min_space = 1.0}
    }
  }
  polygons = DATA_M1_fill,
};
```

代码 10.22: `grouping = ALL` 示例

(原书 *Figure 79: Layer Removed From Top Right Pattern Repetition Example*)
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.24: 右上图案重复中移除层示例

多边形分组适用下列规则与限制：

- 若 `spacing_context` 为 `UF_PATTERN`，则 `grouping` 的 `NONE` 不适用，因为填充到信号间距从图案包围盒而非各个多边形测量；此时 `grouping` 自动为 `ALL`。
- 当 `space_x` 与 `space_y` 为负时，不支持 `grouping` 设为 `ALL`。
- `grouping` 不适用于 `UF_POLYGON` 与 `UF_ADJUSTABLE` 类型。

10.7 信号对齐填充

Signal Aligned Fills

需要通过在设计层多边形旁放置填充形状以保护设计层时，使用 `unified_fill()` 的 `signal` 选项。该功能以环状形式在设计层形状周围插入填充，使填充形状主轴与设计层形状的边对齐。图 10.25 给出示例。

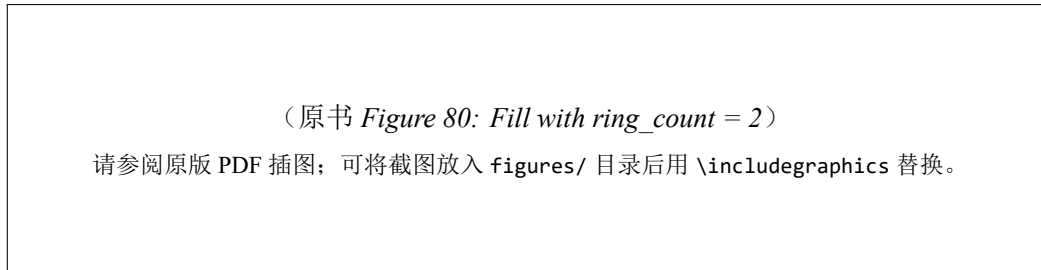


图 10.25: `ring_count = 2` 的填充

仅对 `UF_ADJUSTABLE` 与 `UF_POLYGON` 使用 `signal`。对 `UF_POLYGON`，图案必须是单层单形状。必须在 `fill_patterns` 定义中将 `fill_context` 设为 `SIGNAL_CONTEXT`。用 `signal` 的 `ring_count` 控制环绕每个设计层形状的填充图案环数。图 10.25 中环数为 2。图 10.26 与例 10.23 给出环数为 4 的示例。

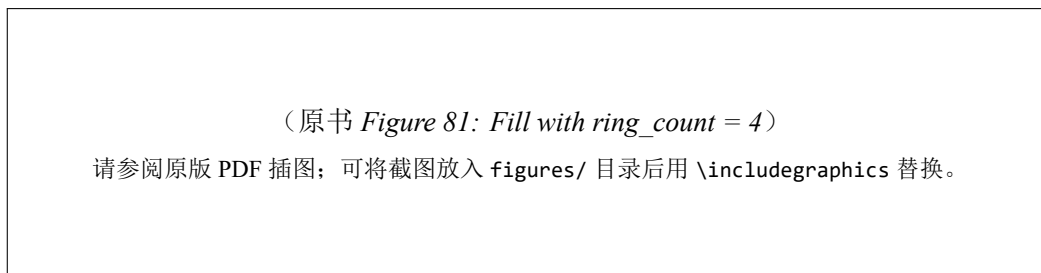


图 10.26: `ring_count = 4` 的填充

```
DATA_M1_fill : list of list of coordinate_s = {
  { {0.0, 0.0}, {2.0, 0.0}, {2.0, 1.0}, {0.0, 1.0} }
};
SHAPE_M1_fill : polygon_layer_s = {
  layer_spec = {
    output_layer_key = "M1_fill",
    fill_to_signal_spacing = {
      {aM1, min_space = 0.1},
    }
  }
  polygons = DATA_M1_fill,
};

fill_patterns = {{
  type = UF_POLYGON,
  polygon_fill = {
    pattern_spec = {space_x = 0.1,
                    space_y = 0.1},
    layers = {SHAPE_M1_fill},
  }
}
```

```

    fill_context = SIGNAL_CONTEXT,
    signal = {layer = aM1,
              ring_count = 4}
  }
}}
```

代码 10.23: ring_count = 4 的填充

可将 **signal** 与“范围与宽度相关间距”节所述的宽度相关间距配合使用，用于要求填充形状相对设计层形状按不同最小间距保持距离、且间距取决于设计层形状宽度的应用。填充图案环使用全部间距约束生成。

信号对齐填充适用下列规则与限制：

- **signal** 不能与 **UF_STACK** 或 **UF_CELL** 一起使用。
- 图案定义有多个填充层、或填充层定义有多个多边形时，不能使用 **signal**。
- 不支持 **stagger_y**；该选项被忽略。
- **fill_context** 必须为 **SIGNAL_CONTEXT**；此时忽略 **partition**、**insertion** 与 **pitch**。
- 对 **signal** 中指定的设计层不支持 **width_based_spacing**；用 **fill_to_signal_spacing** 中的 **min_space** 设置填充层与该设计层的最小间距。

10.7.1 为信号对齐填充定义填充图案

Defining the Fill Pattern for Signal Aligned Fills

用坐标列表定义填充图案时，确保图案在 *x*、*y* 方向的尺寸满足需求。**signal** 将 *x* 方向视为图案主轴，并将该边与设计层形状对齐。

- 要使填充图案矩形的长边与设计层形状对齐，确保图案 *x* 方向尺寸大于 *y* 方向。
- 要使短边与设计层形状对齐，确保图案 *x* 方向尺寸小于 *y* 方向。

图 10.27 说明这两种情形（为简洁未显示全部填充放置）。本例坐标数据为：

```
DATA_M1_fill_V : list of list of coordinates_s = {
  { {0.0, 0.0}, {1.0, 0.0}, {1.0, 5.0}, {0.0, 5.0} }
};
```

（原书 *Figure 82: Aligning Short or Long Edge of the Fill Pattern*）
请参阅原版 PDF 插图；可将截图放入 **figures/** 目录后用 **\includegraphics** 替换。

图 10.27: 对齐填充图案的短边或长边

10.8 改进图案插入

Improving Pattern Insertion

与 **fill_patterns** 参数类似，**unified_fill()** 中图案放置的默认起点是目标层多边形包围盒的左下角。当目标层多边形为复杂非矩形时，默认放置未必达到最大插入。图 10.28 展示若干图案因此无法放置的情况。

(原书 *Figure 83: Default Placement That Is Not Maximum Insertion*)
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.28: 未达到最大插入的默认放置

`unified_fill()` 提供若干可选配置以改变默认起点并提高图案放置灵活性，从而可能改善插入率。例 10.24 说明如何用 `iteration` 选项改善插入率。

```
fill_patterns = {{
  type = UF_POLYGON,
  polygon_fill = {
    pattern_spec = {space_x = 0.1,
                    space_y = 0.1},
    layers = {SHAPE_M1_fill},
    insertion = {iterations = 3}
  }
}}
```

代码 10.24: 使用 `iteration` 选项

`insertion` 选项的详细说明见 *IC Validator Reference Manual* 中的 `unified_fill()`。

10.9 节距对齐填充

Pitch Aligned Fills

在 `fill_pattern()` 函数中，可用 `grid_mode` 控制图案网格位置。

- 指定 **GLOBAL**：以整个数据库范围的左下角为参考放置填充形状。
- 指定 **LOCAL**：允许输入层上每个多边形有各自唯一的参考网格点。

参考网格点是输入层每个多边形包围盒的左下角。

对 `unified_fill()`，除 **GLOBAL** 与 **LOCAL** 外，还可通过引用另一层对图案网格位置施加更复杂控制。除目标层外还可指定上下文层 (`context layer`)。使用上下文层时，填充图案放在目标层多边形内，使填充图案包围盒的左下顶点与以上下文层各多边形包围盒左下顶点为参考网格点的网格点对齐。一个上下文层多边形可包含多个目标层多边形。

图 10.29 比较 `fill_pattern()` 的 `grid_mode` 与 `unified_fill()` 的 `fill_context`。

(原书 *Figure 84: Pitch Aligned Fills Comparison*)
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.29: 节距对齐填充比较

例 10.25 与图 10.30 展示 `pitch` 选项行为。本例仅对 y 方向网格点应用 `pitch`。给定目标层多边形中的填充图案从最接近该多边形包围盒左下顶点、且满足指定节距值的网格点开始放置。

```
fill_patterns = {{
    type = UF_POLYGON,
    polygon_fill = {
        pattern_spec = {space_x = 0.1,
                        space_y = 0.1},
        layers = {SHAPE_M1_fill},
        pitch = {y = 0.1,
                context_layer = aFILL_CONTEXT_LAYER}
    }
}}
```

代码 10.25: 仅在 Y 方向应用 `pitch` 选项

(原书 *Figure 85: Applying the Pitch Option Only in the Y-Direction*)
 请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 10.30: 仅在 Y 方向应用 `pitch` 选项

节距对齐填充适用下列规则与限制:

- 完全位于上下文层外的目标层多边形被丢弃; 不在这些多边形中插入填充图案。
- 若目标层多边形悬出上下文层多边形, 则仅插入完全被上下文层包围的填充图案多边形。
- 不在目标层与上下文层多个多边形交互的区域插入填充图案。
- 使用 `pitch` 时, 若 `fill_context` 为 `CHIP_CONTEXT` 或 `REGION_CONTEXT` 则被忽略 (此时这些设置不相关)。
- 节距值必须与图案尺寸及 `space_x/space_y` 相关。例如, 下列值必须是节距值的倍数, 否则结果填充可能缺失图案:
 $((\text{图案}x\text{ 方向尺寸}) + \text{space}_x)$

为提高插入率, 可增大 `insertion` 的 `shift_factor`。工具会自动调整目标层多边形内的填充图案, 使其与上下文层多边形的网格线对齐。目标层多边形为复杂非矩形时, `shift_factor` 很有用。例 10.26 说明用法。

```
fill_patterns = {{
    type = UF_POLYGON,
    polygon_fill = {
        pattern_spec = {space_x = 0.1,
                        space_y = 0.1},
        layers = {SHAPE_M1_fill},
        insertion = {shift_factor = 5}
        pitch = {y = 0.1,
                context_layer = aFILL_CONTEXT_LAYER},
    }
}}
```

代码 10.26: 使用 `shift_factor` 选项

10.10 判据分析

Criteria Analysis

用 `unified_fill()` 可按密度等指标定制填充流程。工具支持基于方程的判据分析，以满足密度范围、目标密度以及设计上均匀梯度密度等要求。`unified_fill()` 使用与 `density()` 相同的算法，确保填充结果满足填充后设计规则要求。

用下列参数配置填充流程：

- `criteria`
- `delta_window`
- `delta_x`
- `delta_y`
- `window_layer`
- `boundary`

`criteria` 选项支持下列子选项：

- `target`
- `gradient`
- `fill_layer_keys`
- `design_layers`
- `design_layers_hash`
- `window_function`

图案定义中 `output_layer_key` 指定的字符串名用于标识判据方程中的填充层。可在 `criteria` 参数中指定任意数量的判据方程。同一填充层有多个判据方程时，工具确保至少满足其中一个；不同填充层各有方程时（例如 `aM1_FILL` 与 `aM2_FILL` 各有一个），工具处理全部方程。

指定 `criteria` 但未给窗口函数时，工具对每个 `delta` 窗口求默认方程并与 `target`、`gradient` 比较；评估填充图案使计算值收敛到指定值。默认方程为：

$$\frac{(\text{填充层面积}) + (\text{设计层面积})}{(\text{delta 窗口面积})}$$

其中：

- 填充层面积是当前 `delta` 窗口内、`fill_layer_keys` 指定的全部填充层上多边形数据面积之和。
- 设计层面积是当前 `delta` 窗口内、`design_layers` 指定的全部设计层上多边形数据面积之和。

注意

图案定义必须生成能使判据有贡献的填充输出。例如，若图案定义由窄且间距很宽的矩形组成，工具无法达到较高的目标密度。

例 10.27 给出两个填充层 `M1_fill` 与 `M2_fill` 各有判据方程的填充规范。现有 `aM1` 设计层与 `M1_fill` 的累积密度应在 25%–45% 之间、目标 35%，在 $100\mu\text{m} \times 100\mu\text{m}$ 窗口内测量，沿 x 、 y 以 $50\mu\text{m}$ 步进。`M2_fill` 有类似规范与相同密度值。

```
DATA_M1_fill : list of list of coordinate_s = {
  { {1.0, 0.0}, {2.0, 0.0}, {2.0, 3.0}, {1.0, 3.0} }
};

SHAPE_M1_fill : polygon_layer_s = {
```

```

    layer_spec = {output_layer_key = "M1_fill"}
    polygons = DATA_M1_fill,
};

DATA_M2_fill : list of list of coordinate_s = {
    { {0.0, 1.0}, {3.0, 1.0}, {3.0, 2.0}, {0.0, 2.0} },
};

SHAPE_M2_fill : polygon_layer_s = {
    layer_spec = {output_layer_key = "M2_fill"}
    polygons = DATA_M2_fill,
};

fill_patterns_m1_m2 = {{
    type = UF_POLYGON,
    polygon_fill = {
        layers = {SHAPE_M1_fill,
                    SHAPE_M2_fill}
    }
}}

M1_density_criteria : fill_criteria_s =
{
    target = {range = [0.25, 0.45], target = 0.35},
    fill_layer_keys = {"M1_fill"},
    design_layers = {aM1}
};

M2_density_criteria : fill_criteria_s =
{
    target = {range = [0.25, 0.45], target = 0.35},
    fill_layer_keys = {"M2_fill"},
    design_layers = {aM2}
};

uf_fill_output_h = unified_fill(
    fill_patterns = fill_patterns_m1_m2,
    criteria = {M1_density_criteria,
                M2_density_criteria},
    delta_window = {100, 100},
    delta_x = 50,
    delta_y = 50,
    fill_boundary = {type = LAYER, layer = data_extent},
    window_layer = aWINDOW,
    grid = 0.001,
    boundary = ALIGN
);

M1_fill_output = uf_fill_output_h["M1_fill"][0];
M2_fill_output = uf_fill_output_h["M2_fill"][0];

```

代码 10.27: 带两个判据方程的填充规范

例 10.28 在同一判据方程中使用多个设计层与填充层。每个窗口中 M1、M2、M3 的设计与填充密度之和应 $\geq$

105%。

注意

为简洁，例 10.28 仅展示判据方程。

```
M1_M2_density_criteria : fill_criteria_s =
{
    target = {range = >= 1.05},
    fill_layer_keys = {"M1_fill", "M2_fill", "M3_fill"},
    design_layers = {aM1, aM2, aM3}
};
```

代码 10.28: 同一判据方程中含多层的填充规范

对特定填充层指定多个填充图案时，`unified_fill()` 按声明顺序依次插入。指定判据时，可能不会用到全部图案定义——一旦满足判据即停止插入，无论是否已用完所有图案。例 10.29 中，`M1_fill` 的流程由两个插入序列组成；插入第一个图案后若已满足判据，则不执行第二个序列。

```
fill_patterns_m1 = {
{
    type = UF_POLYGON,
    polygon_fill = {layers = {SHAPE_M1_fill_TYPE1}}
},
{
    type = UF_POLYGON,
    polygon_fill = {layers = {SHAPE_M1_fill_TYPE2}}
}}
M1_density_criteria : fill_criteria_s =
{
    target = {range = [0.25, 0.45], target = 0.35},
    fill_layer_keys = {"M1_fill"},
    design_layers = {aM1}
};
uf_fill_output_h = unified_fill(
    fill_patterns = fill_patterns_m1,
    criteria = {M1_density_criteria},
    delta_window = {100, 100},
    delta_x = 50,
    delta_y = 50,
    fill_boundary = {type = LAYER, layer = data_extent},
    window_layer = aWINDOW,
    grid = 0.001,
    boundary = ALIGN
);
M1_fill_output = uf_fill_output_h["M1_fill"][0];
```

代码 10.29: 图案定义序列

也可不使用默认判据方程，而通过定义窗口函数指定自定义判据，如例 10.30。

```
design_layers : layer_groups_h = {
    "aM1" => aM1
```

```

});
M1_density_criteria_func : function ( void ) returning void
{
    da_aM1 = uf_area(DESIGN, "aM1");
    fa = uf_area(FILL, "M1_fill");

    check_value : double = (da_aM1 + fa) / uf_window_area();
    uf_save_window(check_value);
};
M1_density_criteria : fill_criteria_s =
{
    target = {[0.25, 0.45]},
    fill_layer_keys = {"M1_fill"},
    design_layers_hash = design_layers,
    window_function = M1_density_criteria_func
};

```

代码 10.30: 自定义判据方程

参见

可在窗口函数中使用的实用函数，见 *IC Validator Reference Manual* “Utility Functions” 章中的 Unified Fill Utility Functions 一节。

适用下列规则与限制：

- 单个窗口函数中不能使用多个填充层。
- 未指定 **target** 时，工具用范围中点作为内部目标。
- 必须指定可收敛的有效约束。例如下列约束不允许：

```
target = {range = <0.50}
```

下列示例说明若干情形的行为：

- **target = {range = >0.50}**
工具尝试达到大于 50% 的目标密度。
- **target = {range = [0.45, 0.55]}**
工具假定内部目标密度为 50%。初始密度已大于 50% 的窗口不再添加填充；否则朝 50% 收敛但不超过 50%。
- **target = {range = [0.45, 0.55], target = 0.48}**
工具朝 48% 收敛。若窗口初始密度为 46%，则朝 48% 收敛但不超过 48%。
- **target = {range = >0.30, target = 0.50}**
允许范围为大于 30% 且小于等于 100%。工具朝 50% 收敛但不超过 50%。

10.11 输出层

Output Layers

`unified_fill()` 返回含层列表的哈希，每个列表有三个多边形层：

表 10.2: 输出层列表索引

索引	多边形层
0	未着色 (Uncolored)
1	Color 1 (须 color 为 true 才有此层)
2	Color 2 (须 color 为 true 才有此层)

取回多边形层列表时，指定填充图案定义 `layer_spec` 中使用的 `output_layer_key`。若 `color` 为 `false`，仅在索引 0 生成未着色数据；若为 `true`，则在索引 1、2 分别返回两个着色输出层。例如：

```
M1_fill_ALL = M1_fill_h["M1_fill"][0];
M1_fill_COLOR1 = M1_fill_h["M1_fill"][1];
M1_fill_COLOR2 = M1_fill_h["M1_fill"][2];
```

例 10.31 给出两个填充层从输出哈希访问的过程。

```
fill_patterns_m1_m2 = {{
    type = UF_POLYGON,
    polygon_fill = {
        layers = {SHAPE_M1_fill,      //output_layer_key is "M1_fill"
                  SHAPE_M2_fill}      //output_layer_key is "M2_fill"
    }
}}
uf_fill_output_h = unified_fill(
    fill_patterns = fill_patterns_m1_m2
);

M1_fill_output = uf_fill_output_h["M1_fill"][0];
M2_fill_output = uf_fill_output_h["M2_fill"][0];
```

代码 10.31: 从哈希访问填充层

例 10.32 给出 `M1_fill` 有两个图案插入步骤、且每个图案定义指定颜色的填充规范。每次插入步骤将填充形状作为独立集合着色。两个步骤在 `output_layer_key` 中使用相同字符串名指定填充层；工具将不同图案定义的全部多边形数据累积到该填充层输出哈希键的同一多边形层列表元素中。本例中 `M1_fill` 输出哈希键列表索引 0 含两个插入步骤的完整多边形数据；索引 1 含两个步骤的完整 `color 1` 数据；索引 2 含完整 `color 2` 数据。

```
SHAPE_M1_fill_TYPE1 : polygon_pattern_s = {
    layer_spec = {
        output_layer_key = "M1_fill",
    }
    polygons = DATA_M1_fill_TYPE1,
};
SHAPE_M1_fill_TYPE2 : polygon_pattern_s = {
    layer_spec = {
        output_layer_key = "M1_fill",
    }
    polygons = DATA_M1_fill_TYPE2,
};
M1_fill_h = unified_fill(
    fill_patterns = {
        {type = UF_POLYGON,
```

```

        polygon_fill = {
            layers = {SHAPE_M1_fill_TYPE1},
            color = true
        }
    },
    {type = UF_POLYGON,
        polygon_fill = {
            layers = {SHAPE_M1_fill_TYPE2},
            color = true
        }
    }
},

    fill_boundary = {type = LAYER, layer = data_extent}
);

M1_fill_ALL      = M1_fill_h["M1_fill"][0];
M1_fill_COLOR1   = M1_fill_h["M1_fill"][1];
M1_fill_COLOR2   = M1_fill_h["M1_fill"][2];

```

代码 10.32: 使用 color 选项的图案插入

除填充层外,还可通过指定一个或多个适当的 `output_layer_key`,取回一个或多个输出填充层的包围盒(extents)。例 10.33 与图 10.31 展示 `extents_output` 用法。

```

SHAPE_M1_fill : polygon_pattern_s = {
    layer_spec = {
        output_layer_key = "M1_fill",
    }
    polygons = DATA_M1_fill_TYPE1,
};

M1_fill_h = unified_fill(
    fill_patterns = {{
        {type = UF_POLYGON,
            polygon_fill = {
                layers = {SHAPE_M1_fill},
                color = true
            }
        }
    }},
    extents_output = {
        {output_layer_key = "M1_fill_boundary",
            fill_layer_keys = {"M1_fill"}
        }
    }
);

M1_fill      = M1_fill_h["M1_fill"][0];
M1_fill_E    = M1_fill_h["M1_fill_boundary"][0];

```

代码 **10.33**: 使用 `extents_output` 选项

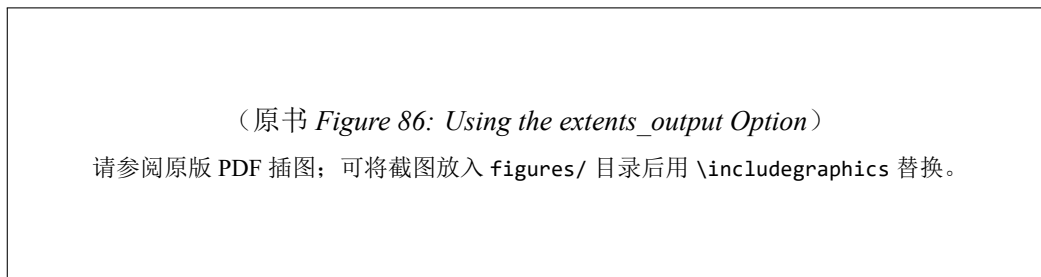


图 **10.31**: 使用 `extents_output` 选项

当 `extents_output` 中未指定 `output_layer_key` 时，图案包围盒由图案放置中全部填充层共同构造。对 `UF_STACK` 或 `UF_CELL` 生成的图案，`extents_output` 按 `separate_patterns` 工作。

10.12 层压缩

Layer Compression

`unified_fill()` 生成的输出多边形层上的几何数据可分层级分组，以减小 `write_gds()`、`write_milkyway()` 等函数创建的输出文件大小。IC Validator runset 中可通过两种方式实现层压缩：

- 在 `unified_fill()` 的图案定义中指定 `hierarchical_fill` 选项。
- 在 `write_gds()` 与 `write_milkyway()` 中使用 `compress_fill` 选项。

`unified_fill()` 中的 `hierarchical_fill` 类似于 `fill_pattern()` 中的 `output_aref`。但为获得最佳输出文件大小，建议对各个多边形层在 `write_gds()` 与 `write_milkyway()` 中使用 `compress_fill` 参数。

Chapter 11

处理边

Working With Edges

本章简要概述如何在 IC Validator runset 中使用边（edge）。边相关函数见 *IC Validator Reference Manual*。

IC Validator 工具提供：

- 将边层与父层关联
- 拓扑运算
- 逻辑运算
- 边操作

11.1 将边层与父层关联

Associating Edge Layers With a Parent Layer

边层可以有一个父多边形层（parent polygon layer）。

- 所有边选择运算都会保持边层与其派生自的多边形层之间的连接。
- 所有边创建运算会得到“孤立”（orphan）边层，即没有父多边形层的边层。
- 父多边形层通过影响边的尺寸类函数行为，使输出更直观。

例如，图 11.1 展示边与父层的关联：

```
red = edge_derived_from_blue;  
@ "red edge min width rule";  
internal1(red, spacing < x);
```

IC Validator 工具会检测从 e2 到 e3 的违例，并避免从 e1 到 e3 的违例。

11.2 拓扑运算

Topological Operations

拓扑运算使用所选的完整边，且不添加新端点。这些运算包括：

- adjacent_edge() 与 not_adjacent_edge()

(原书 *Figure 87: Edge Association With a Parent Layer*)
 请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 11.1: 边与父层的关联

- `angle_edge()` 与 `not_angle_edge()`
- `inside_point_touching_edge()` 与 `not_inside_point_touching_edge()`
- `inside_touching_edge()` 与 `not_inside_touching_edge()`
- `interacting_edge()` 与 `not_interacting_edge()`
- `length_edge()` 与 `not_length_edge()`
- `outside_point_touching_edge()` 与 `not_outside_point_touching_edge()`
- `outside_touching_edge()` 与 `not_outside_touching_edge()`
- `point_touching_edge()` 与 `not_point_touching_edge()`
- `touching_edge()` 与 `not_touching_edge()`
- `vertices_edge()` 与 `not_vertices_edge()`

图 11.2 给出 touching edges 示例:

```
green = touching_edge (layer1 = blue, layer2 = red);
```

(原书 *Figure 88: Touching Edges Example*)
 请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 11.2: Touching edges 示例

11.3 逻辑运算

Logical Operations

逻辑运算会生成新端点。这些运算包括:

- `coincident_edge()` 与 `not_coincident_edge()`
- `coincident_inside_edge()` 与 `not_coincident_inside_edge()`
- `coincident_outside_edge()` 与 `not_coincident_outside_edge()`
- `and_edge()`
- `not_edge()`

- `or_edge()`
- `xor_edge()`

图 11.3 给出 coincident edges 示例:

```
green = coincident_edge (layer1 = blue, layer2 = red);
```

(原书 *Figure 89: Coincident Edges Example*)

请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 11.3: Coincident edges 示例

11.4 边操作

Edge Manipulation

下列函数创建边: 创建的边始终为孤立边 (orphan edges):

- `move_edge()`
- `snap_edge()`
- `extend_edge()`
- `copy_edge()`

下列函数由边创建多边形:

- `edge_size()`

Chapter 12

关键面积分析流程

Critical Area Analysis Flow

本章介绍 IC Validator 关键面积分析（critical area analysis）流程。

IC Validator 关键面积分析（CAA）流程用于识别版图中对制造环境中随机污染物所致失效敏感的区域。关键面积定义为：给定尺寸缺陷的中心若落在该区域内，可能导致电路短路或开路失效。CAA 流程当前支持两类缺陷：OPEN 与 SHORT。

（原书 *Figure 90: Critical Area by Defect Types*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 12.1: 按缺陷类型划分的关键面积

关键面积分析流程可用下列示意图说明：

（原书 *Figure 91: Critical Area Analysis Flow*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 12.2: 关键面积分析流程

1. 晶圆厂缺陷数据：CAA 流程的起点是晶圆厂提供的制造缺陷数据及缺陷尺寸计算公式。
缺陷数据通常包含由三个参数定义的缺陷尺寸范围——最小与最大缺陷尺寸 $D_{\min}$ 、 $D_{\max}$ 、缺陷尺寸采样数，以及在 $(D_{\min}, D_{\max})$ 范围内计算缺陷尺寸的系数。
2. 查找版图关键面积：使用 `critical_areas()` 函数确定版图关键面积。更多信息见 *IC Validator Reference Manual* 中的 `critical_areas()` 函数。

3. 分析关键面积/生成报告：识别出版图关键面积并以多边形层表示后，可在 `runset` 中用这些数据计算加权关键面积（WCA）与平均质量（average quality）。

WCA 计算为

$$WCA = K \sum_i \frac{CA_i}{D_i^{fall_power}}$$

其中 **K-factor** 由用户在 `runset` 中提供。

平均质量通常按如下计算：

$$\text{Average Quality} = 1 - \frac{WCA}{\text{AREA}}$$

其中 WCA 为给定版图层的加权关键面积，AREA 为该层总面积。

计算结果可用于生成报告与热图。热图应使用 VUE 工具生成。

最简单的关键面积报告可由 `dfm_feature()` 函数生成，并输出到所分析层的 `LAYOUT_ERRORS` 文件。

```
chip area:                                = 1543.3206
Critical_Area[0]:                          = 0.0000
Defect Size[0]:                            = 0.0110
Critical_Area[1]:                          = 0.0000
Defect Size[1]:                            = 0.0120
Critical_Area[2]:                          = 0.0000
Defect Size[2]:                            = 0.0190
Critical_Area[3]:                          = 0.0000
Defect Size[3]:                            = 0.0380
Critical_Area[4]:                          = 0.0000
Defect Size[4]:                            = 0.0740
Critical_Area[5]:                          = 0.0000
Defect Size[5]:                            = 0.1350
Critical_Area[6]:                          = 1.4728
Defect Size[6]:                            = 0.2250
Critical_Area[7]:                          = 4.9703
Defect Size[7]:                            = 0.3500
Critical_Area[8]:                          = 17.1042
Defect Size[8]:                            = 0.5170
Critical_Area[9]:                          = 90.4452
Defect Size[9]:                            = 0.7320
Critical_Area[10]:                         = 200.9118
Defect Size[10]:                           = 1.0000
WCA:                                       = 0.0005
DIFF_CA_OPEN AVERAGE QUALITY:            = 1.0000
```

关键面积热图由 VUE 工具生成，如图 12.3 所示。

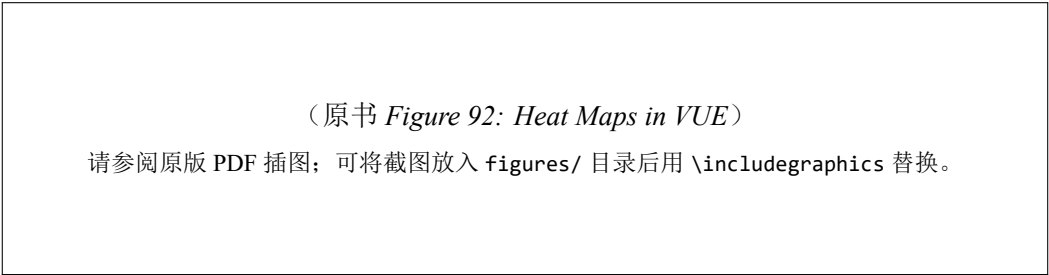


图 12.3: VUE 中的热图

12.1 IC Validator 关键面积分析器 Runset

IC Validator Critical Area Analyzer Runset

下列 PXL 结构给出在 CAA runset 中组织数据的一种可能方式:

```
ca_defect_range_s : newtype struct of {
    count : integer;
    min : double;
    max : double;
};
ca_defect_density_s : newtype struct of {
    na : integer;          // dummy
    fall_power : double;
    conversion_factor : double;
};
ca_config_s : newtype struct of {
    layer : polygon_layer;
    mode : critical_area_mode_e;
    modeling : integer     // dummy
    range : ca_defect_range_s;
    defect_density : ca_defect_density_s;
};
ca_layer_configuration_l : newtype list of ca_config_s;
```

典型 CAA runset 如下:

```
#include <icv.rh>
library("input_desing.gds", GDSII, "top");

// assign section
DIFF = assign({ { 3 } });
POLY = assign({ { 3 } });
METAL1 = assign({ { 3 } });

connect_layer = assign({ { 4 } });

// types to store defect sizes;
defect_sizes_list : newtype list of double;
defect_sizes_h : newtype hash of integer to defect_sizes_list;
defect_hash : defect_sizes_h = {};
layer_dr : list of double = {};
layer_tagname : string = "";
fall_power : double = 0.0;
ca_mode : critical_area_mode_e;

g_ca_layers : string_to_geometry_layer_h = {};

// the remote function to calculate the WCA and average quality and
// output report
get_area : function (void) returning void
```

```

{
    dfm_report_double("chip area: ", dfm_window_area());

    wca_tmp: double = 0;

    // calculate K-factor
    // put actual K-calculation code here
    K = 6.0e-07;

    // for each input layer the critical_areas() command creates
    // list of critical area layers - one layer per defect size

    for (idx = 0 to layer_dr.size() - 1) {
        this_area = dfm_sum_area(""+idx);
        dfm_report_double("Critical_Area["+idx+"]: ", this_area);
        // output critical area value

        dfm_report_double("Defect Size["+idx+"]: ", layer_dr[idx]);
        // output defect size

        // calculate sum of WCAs for all defect sizes for given input layer
        wca_tmp= wca_tmp + (this_area / (layer_dr[idx]**fall_power));
    }

    // final WCA calculation
    wca = K * wca_tmp;

    dfm_report_double("WCA: " , wca);          // report WCA

    average_quality : double = 1 - (wca / dfm_window_area());
    // calculate average quality
    dfm_report_double(layer_tagname + "_" + ca_mode + " AVERAGE QUALITY: ",
                      average_quality);
    // report average quality for one input layer

    dfm_save_data();
}

ca_defect_range_s : newtype struct of {
    count : integer;
    min : double;
    max : double;
};

ca_defect_density_s : newtype struct of {
    na : integer;          // dummy
    fall_power : double;
    conversion_factor : double;
};

ca_config_s : newtype struct of {

```

```

    layer : polygon_layer;
    mode : critical_area_mode_e;
    modeling : integer      // dummy
    range : ca_defect_range_s;
    defect_density : ca_defect_density_s;
};

cas : ca_layer_configuration_l = {
    {DIFF,CA_OPEN,MODELING,{10,0.022,2.000},{NA,2.5,1e-8}},
    {POLY,CA_OPEN,MODELING,{10,0.022,2.000},{NA,2.5,1e-8}},
    {METAL1,CA_OPEN,MODELING,{10,0.022,2.000},{NA,2.5,1e-8}},
};

get_defect_sizes : function      // function for defect size calculation
(
    min_size : double,
    max_size : double,
    steps      : integer,
    fall_power : double
) returning defect_list : defect_sizes_list
{
/*
    // put actual defect sizes calculation coded by user
*/
    defect_list = {0.011, 0.013, 0.017, 0.035, 0.073, 0.13, 0.24, 0.33,
    0.495, 0.74, 1.0};
}

di : integer = 0;
foreach (config in cas)
{
    // calculate defect size
    one_list = get_defect_sizes(config.range.min, config.range.max,
    config.range.count,
    config.defect_density.fall_power);
    defect_hash[di] = one_list;

    di = di + 1;
}

output_layers: list of write_layer_map_s = {};

di = 0;
layer_n : integer = 1;
foreach (config in cas)
{
    outs : list of polygon_layer;

    if(config.mode == CA_SHORT)

```



```

{
    // for SHORT mode connect_db is required
    cdb = connect({{config.layer}, connect_layer});
    outs = critical_areas( config.layer, defect_hash[di], config.mode,
cdb);
}
else
    outs = critical_areas( config.layer, defect_hash[di], config.mode);

    for(i = 0 to defect_hash[di].size()-1) {
        g_ca_layers[""+i] = outs[i];

        output_layers.push_back( {outs[i], {layer_n}} );
        layer_n = layer_n + 1;
    }

    g_ca_layers_keys = g_ca_layers.keys();

    layer_dr = defect_hash[di];
    fall_power = config.defect_density.fall_power;
    ca_mode = config.mode;

    layer_tagname = config.layer.tagname;

    {@ ""+config.layer; dfm_features(g_ca_layers, get_area);}
}

// write out CA layers
gds_file = gds_library("CA_out.gds");
write_gds
(
    output_library = gds_file,
    layers = output_layers
);

```

Chapter 13

DRC 错误分类

DRC Error Classification

本章介绍 IC Validator 工具的 DRC 错误分类能力及其用法。

DRC 错误分类能力分以下各节说明：

- 使用 DRC 错误分类
- DRC 错误分类流程
- 使用 PERC 错误分类

13.1 使用 DRC 错误分类

Using DRC Error Classification

DRC 错误分类允许你对 IC Validator 运行中发现的错误进行分类与标注，并将错误导出到数据库，供后续 IC Validator 运行使用。错误可单独分类、在 VUE 中按组分类，或使用 `pydb_report` 实用程序批量分类。

通过使用持久化错误分类数据库（cPYDB），错误分类与用户注释可在多次 IC Validator 运行之间保留。后续运行的错误数据以及分类数据均存储在 SQLite 数据库中。

此流程使用 `error_options()` 函数。该函数有若干专用于 DRC 错误分类的参数：

- **match_errors:** 指定要在 IC Validator 运行中导入以进行匹配的错误分类数据库列表。对列表中每个错误分类数据库，可控制是否抑制已匹配错误，以及是否报告未匹配错误。（未匹配错误指：存在于已导入的错误分类数据库中、但当前 IC Validator 运行未产生的错误。）
- **mustfix_errors:** 将某些违例注释与单元指定为必须修复（must-fix）。不能将这些注释与单元分类为 Ignore 或 Waive。此设置仅作用于当前运行，不会保留在错误分类数据库中。
- **comment_match_delimiter:** 指定在已导入错误分类数据库中进行违例注释匹配时的分隔符字符串。
- **match_errors_processing_mode:** 指定 IC Validator 工具使用单元级（cell level）还是层次化处理来匹配错误分类数据库中的已分类违例。默认为 CELL_LEVEL。
- **violation_options:** 允许按规则覆盖某些全局错误选项。例如，若大多数规则以 HIERARCHICAL 模式匹配，但少数规则需要以 CELL_LEVEL 模式运行，可用此参数实现。

使用 `pydb_report` 与 `pydb_export` 实用程序对错误分类，并复用分类数据。更多信息见 `pydb_report` Utility 与 `pydb_export` Utility。这些实用程序位于 IC Validator 安装目录的 `bin` 下。也可使用 VUE 对错误分类并导出。见 *IC Validator VUE User Guide*。

13.1.1 CELL_LEVEL 模式下的层次考虑

Hierarchy Considerations in CELL_LEVEL Mode

预处理期间，IC Validator 函数会进行层次优化。因此，在 CELL_LEVEL 模式下做错误匹配时，各次运行之间的错误可能不一致。为帮助防止运行间层次不一致，导入错误分类数据库时，数据库中定义的任何单元会在预处理期间自动受保护而不被展开（explosion）。然而，某些优化会创建层次，或对单元的个别实例展开/展平，因而仍可能造成一致性问题。

在 CELL_LEVEL 模式下运行 DRC 错误分类流程时，务必在 `hierarchy_auto_options()` 函数中将 `flow` 参数设为 `ERROR_CLASSIFICATION` 选项。

见 `hierarchy_auto_options()` 函数。

13.1.2 层次化 DRC 错误分类

Hierarchical DRC Error Classification

IC Validator 工具为错误分类匹配提供 HIERARCHICAL 模式，比 CELL_LEVEL 模式更稳健，对层次变化也更不敏感。该模式对层次优化不加限制。错误分类数据库（cPYDB）中含有已分类错误的单元允许被展开。

若 IC Validator 工具产生的错误之完整层次合成形状，与 cPYDB 中具有相同分类的错误之完整层次合成形状完全重叠，则视为与 cPYDB 中已分类错误匹配。若工具所发现错误与 cPYDB 中错误的完整形状有任何部分不同，则不是层次匹配。

不过，在此模式下，只要可能，错误也可单独匹配。也就是说，当单个错误（而非由多个错误组成的完整层次合成形状）与 cPYDB 中对应的单个错误完全重叠时，即使二者处于不同层次级别，也视为匹配。

注意

建议对所有使用层次错误分类的 IC Validator 运行（尤其是用于生成 cPYDB 的首次运行），将 `error_options()` 的 `partially_exploded_cells` 参数设为 `EXPLODE`，或使用命令行选项 `-pec EXPLODE`。该参数确保错误报告在 IC Validator 工具保留了原始设计层次中全部实例的单元内，并降低实例特有错误导致分类无法匹配的可能性。

13.1.3 错误分类最佳实践

Error Classification Best Practices

必须区分：错误分类数据库（cPYDB）中按规则或违例注释引用的已分类错误，与 IC Validator 工具由某条命令产生的错误。为确保正确匹配，推荐最佳实践是：用单条 IC Validator 命令输出某规则的全部违例。这样可避免同一规则内不同命令产生的错误在 cPYDB 中相互干扰，从而在后续运行按单条命令考虑时妨碍层次匹配。违例注释应对命令唯一，以确保这种对应关系。

清理无效的 cPYDB 数据也很重要。例如，若错误形状已改变，但新形状仍被视为有效且拟予 `waiver`，则应清除该违例与单元的 cPYDB 条目，并用新错误形状重新创建。在 cPYDB 中保留旧的无效形状会妨碍 IC Validator 工具找到层次匹配。

当前，错误分类最适用于 DRC 违例。对其他类型违例（包括 ERC、器件提取与密度）的先前已分类错误匹配尚不支持。

13.1.4 数据库命名约定

Database Naming Conventions

引用数据库时需指定数据库名与数据库路径。IC Validator 运行生成的错误数据库（PYDB）默认命名为 `PYDB_top-cell`，除非在 `error_options()` 的 `db_path` 参数中另行指定路径，否则位于 `run_details/pydb`。

例如，对设计 TOP 运行 IC Validator。目录 `run_details/pydb/PYDB_TOP` 包含错误数据库的数据文件。数据库路径为 `run_details/pydb`，数据库名为 `PYDB_TOP`。

SQLite 允许数据库名由字母数字字符与下划线（_）组成。数据库名按如下变换：

- 所有非字母数字字符替换为下划线（_）。例如，PYDB_A.1.2 变为 PYDB_A_1_2。
- 若数据库名仅含数字，则在首尾添加下划线。例如，123 变为 _123_。

可在 `runset`、`VUE` 以及 `pydb_report` 与 `pydb_export` 实用程序中继续使用原始数据库名，但磁盘上以修改后的名称存储。

13.2 DRC 错误分类流程

DRC Error Classification Flow

图 13.1 展示 DRC 错误分类流程。

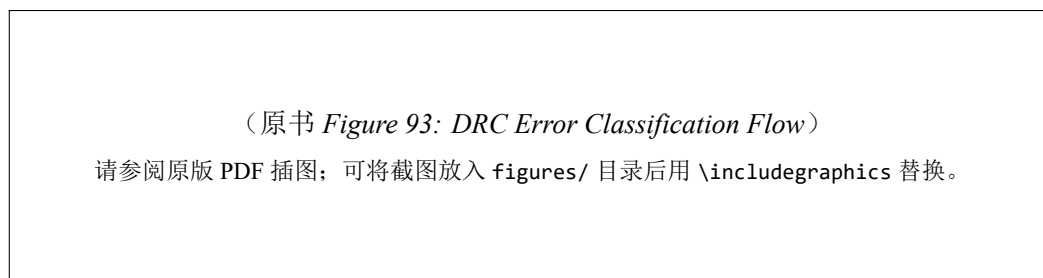


图 13.1: DRC 错误分类流程

1. 对设计运行 IC Validator 工具。随后可用 `VUE` 或 `pydb_report` 实用程序对所得 DRC 错误分类。
在后续运行中复用分类数据，使用 `VUE` 或 `pydb_export` 实用程序将已分类错误导出到错误分类数据库。
2. 再次运行 IC Validator 时，在 `error_options()` 的 `match_errors` 参数中指定错误分类数据库。若指定多个错误分类数据库，则按指定顺序搜索匹配错误。错误以最先匹配到的错误分类数据库中的数据预分类，并写入当前运行的错误数据库。该错误在 `LAYOUT_ERRORS` 文件与 `VUE` 中显示为已分类。
使用 `error_options()` 的 `match_errors` 参数并设为 `suppress_matched_errors` 选项时，可指定不写入错误数据库的错误。

注意

`error_options()` 的 `error_limit_per_check` 参数仅适用于新错误。本运行产生的全部错误都会对照已导入的错误分类数据库列表检查，所有匹配错误均写入错误数据库。未匹配任何错误分类数据库的错误则写入，直至达到指定的 `error_limit_per_check` 限制。

3. 最后，可将当前运行或其他运行中的新分类合并到现有错误分类数据库。新分类的错误追加到数据库；已有错误若有变化则更新。

13.2.1 DRC 错误分类流程示例

DRC Error Classification Flow Example

本节以 AD4FUL 设计为例走通 DRC 错误分类流程。使用 `pydb_report` 与 `pydb_export` 实用程序进行分类与数据库管理。（见 `pydb_report` Utility 与 `pydb_export` Utility。）

基本步骤为：

1. 首次运行 IC Validator 工具
2. 创建错误分类数据库

3. 再次运行 IC Validator 工具
4. 更新现有错误分类数据库

首次运行 IC Validator 工具

Running the IC Validator Tool Initially 首先对设计运行 IC Validator,产生 DRC 错误,并写出标准 LAYOUT_ERRORS 文件。

接着分类:

- 单元 INVH 中的全部错误为 Ignore。
- 违例 “Met1 Spacing must be >= 3” 中的全部错误为 Waive。
- 单元 TGATE 中某坐标窗口内的错误为 Watch。

使用 pydb_report 实用程序完成该分类:

```
pydb_report -pydb_name PYDB_AD4FUL \
-pydb_path ./run_details/pydb -classify classify.csv
```

pydb_report 的输入文件为标准逗号分隔格式。更多信息见 pydb_report Utility。本例所用文件为:

```
INVH,,Ignore,INVH is a known good cell,,,,
,Met1 Spacing must be >= 3,Waive,Done to fit tight routing,,,,
TGATE,,Watch,These need to be fixed,0,0,10,10
```

图 13.2 展示 Microsoft Excel 中的逗号分隔格式。

(原书 Figure 94: Example Classify File in Microsoft Excel)

请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 13.2: Microsoft Excel 中的分类文件示例

pydb_report 实用程序将日志文件 pydbreport.log 写到当前工作目录。相同信息也显示在屏幕上。本例输出见示例 13.1。

```
Connecting to DB: PYDB_AD4FUL
      path: ./run_details/pydb
Connected to DB PYDB_AD4FUL

Error database PYDB_AD4FUL summary
  Number of commands:      12
  Number of commands w/ errors:  12
  Number of dynamic tables:    5
  Total error polygons:      48
  DBU:                      0.00050
  Number of unique cells w/ errors: 6

Classifying Errors from classify.csv...
  Classifying all errors in violation/cell...
```

```

Violation: ALL
Cell:      INVH
Class:     Ignore
Comment:   INVH is a known good cell
6 new errors classified.
Classify time = 0:00:00 User=0.00 Sys=0.01 Mem=0.000 GB
Classifying all errors in violation/cell...
Violation: Met1 Spacing must be >= 3
Cell:      ALL
Class:     Waive
Comment:   Done to fit tight routing
3 new errors classified.
3 existing errors reclassified.
Classify time = 0:00:00 User=0.00 Sys=0.00 Mem=0.491 GB
Classifying all errors in window...
Violation: ALL
Cell:      TGATE
Window:    (0.0000, 0.0000) (10.0000, 10.0000)
Class:     Watch
Comment:   These need to be fixed
12 new errors classified.
Classify time = 0:00:00 User=0.01 Sys=0.00 Mem=0.000 GB
Overall Classify time = 0:00:00 User=0.01 Sys=0.01 Mem=0.000 GB
Done.

```

代码 13.1: 初始 pydb_report 实用程序 pydbreport.log

可用 pydb_report 获取当前运行错误数据库中已分类错误的报告:

```

pydb_report -pydb_name PYDB_AD4FUL \
-pydb_path ./run_details/pydb \
-show_classified file=classified.txt

```

示例 13.2 给出本例已分类错误报告 (classified.txt) 中的首条错误记录。

```

-----
                        Classified Error Report
-----
Report Violation: ALL
Report Cell:      ALL
Report Class:     ALL
=====
Violation: " toxcont must be enclosed by met1 by more than 1.0
without touching or overlapping"
-----
Cell: TGATE

Class Summary:

                Class      Errors
                -----
                Watch      6

```

```

Command: runset.rs:101:enclose
-----
( lower left x, y ) ( upper right x, y) Distance
-----
(3.0000, 4.0000)    (4.0000, 5.1180)    1.0000
* Class:           Watch
* Created by:      juser on 2008-10-21 08:36:19
* Last Modified by: juser on 2008-10-21 08:36:19
* Comment by:      juser on 2008-10-21 08:36:19
      These need to be fixed

```

代码 13.2: 已分类错误报告中的错误记录

创建错误分类数据库

Creating an Error Classification Database 要在后续运行中使用分类数据，运行 `pydb_export` 创建错误分类数据库（cPYDB）。例如：

```

pydb_export -pydb_name PYDB_AD4FUL \
  -pydb_path ./run_details/pydb \
  -cpydb_name CPYDB_AD4FUL -cpydb_path ./cpydb -create

```

本例在 `./cpydb` 目录创建名为 `CPYDB_AD4FUL` 的新错误分类数据库。该数据库可用于后续 IC Validator 运行以保留错误分类数据。`pydb_export` 在当前目录产生日志文件 `pydbexport.log`。本例运行输出见示例 13.3。

```

Called as: pydb_export -pydb_name PYDB_AD4FUL -pydb_path
./run_details/pydb -cpydb_name CPYDB_AD4FUL -cpydb_path ./cpydb
-create

Operation ..... Create New Database
Input PYDB Path ..... ./run_details/pydb
Input PYDB Name ..... PYDB_AD4FUL
Output cPYDB Path ..... ./cpydb
Output cPYDB Name ..... CPYDB_AD4FUL
Export Matched Errors ..... FALSE

Loading global database info
Loading database time = 0:00:00 User=0.00 Sys=0.01 Mem=0.000 GB
Exporting classified error data
Exporting violation " toxcont must be enclosed by met1 by more
than 1.0 without touching or overlapping"
  6 new errors exported
Exporting time = 0:00:00 User=0.00 Sys=0.00 Mem=0.000 GB
Exporting violation " toxcont must be enclosed by tox by more
than 1.5"
  6 new errors exported
Exporting time = 0:00:00 User=0.01 Sys=0.00 Mem=0.000 GB
Exporting violation "Met1 Spacing must be >= 3"
  6 new errors exported
Exporting time = 0:00:00 User=0.00 Sys=0.00 Mem=0.000 GB
Exporting violation "ngate must be exactly 2.0 by 6.0 rectangles"

```

```

2 new errors exported
Exporting time = 0:00:01 User=0.00 Sys=0.00 Mem=0.000 GB
Exporting violation "pgate must be exactly 2.0 by 12.0
rectangles"
1 new error exported
Exporting time = 0:00:00 User=0.00 Sys=0.00 Mem=0.000 GB
Export time = 0:00:01 User=0.01 Sys=0.01 Mem=0.803
Recreating indexes
Recreating indexes time = 0:00:00 User=0.00 Sys=0.00 Mem=0.000 GB
Overall export time = 0:00:01 User=0.01 Sys=0.02 Mem=0.000 GB

Export Successful.
PYDB_Export is done.

```

代码 13.3: 初始错误分类数据库

再次运行 IC Validator 工具

Running the IC Validator Tool Again 修复部分错误后，再次运行 IC Validator，并导入上次创建的错误分类数据库。使用错误分类数据库可避免重复劳动。要使用错误分类数据库，在 `error_options()` 中指定：

```

error_options(
    match_errors = {
        {db_name = "CPYDB_AD4FUL",
         db_path = "./cpydb"}
    }
);

```

IC Validator 运行后，注意新的 `LAYOUT_ERRORS` 文件中错误已用错误分类数据库中的数据预分类：

- 错误汇总节按分类标签给出错误计数。
- 错误详情节按分类标签对错误分组。

注意

可随时用 `pydb_report` 从错误数据库重新生成 `LAYOUT_ERRORS` 文件。也可用 `pydb_report` 生成已分类错误报告以显示详细分类信息，如示例 13.4 与示例 13.5 所示。

示例 13.4 展示错误汇总节。

```

ERROR SUMMARY

toxcont must be enclosed by met1 by more than 1.0 without touching or
overlapping
enclose ..... 9 violations found.
Unclassified Errors ..... 3 violations found.
Errors classified as Watch ..... 6 violations found.

```

代码 13.4: 错误汇总节

示例 13.5 展示错误详情节。

ERROR DETAILS


```

-----
toxcont must be enclosed by met1 by more than 1.0 without touching or
overlapping
-----

run2.rs:107:enclose
Structure ( lower left x, y ) ( upper right x, y ) Distance
- - - - -
TGATE      (21.5000, 23.5000) (22.0000, 24.9140) 0.5000
TGATE      (21.5000, 19.0860) (22.0000, 20.5000) 0.5000
TGATE      (21.5000, 20.5000) (22.0000, 23.5000) 0.5000

+ Errors classified as Watch
- - - - -
TGATE      (3.0000, 4.0000)      (4.0000, 5.1180)      1.0000
TGATE      (3.0000, 0.0000)      (4.0000, 1.0000)      1.0000
TGATE      (7.0000, 0.0000)      (8.1180, 1.0000)      1.0000
TGATE      (3.0000, 0.0000)      (4.0000, 1.0000)      1.0000
TGATE      (3.0000, 1.0000)      (4.0000, 4.0000)      1.0000
TGATE      (4.0000, 0.0000)      (7.0000, 1.0000)      1.0000

```

代码 13.5: 错误详情节

更新现有错误分类数据库

Updating the Existing Error Classification Database 第二次运行后，可对更多错误分类并重新分类现有错误：

```

pydb_report -pydb_name PYDB_AD4FUL \
            -pydb_path ./run_details/pydb -classify classify2.csv

```

图 13.3 展示所用 classify 文件。

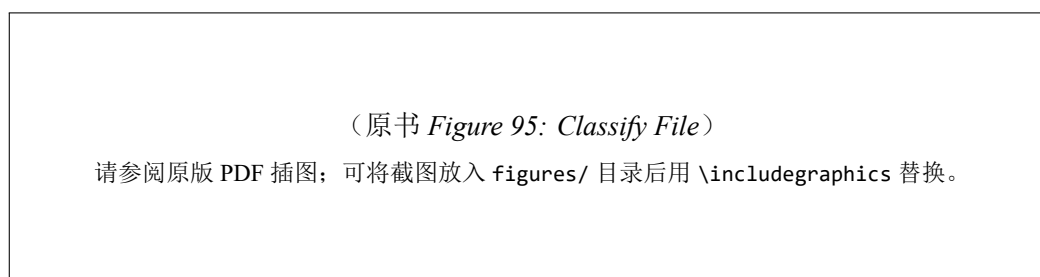


图 13.3: 分类文件

pydb_report 的日志文件见示例 13.6。

```

Connecting to DB: PYDB_AD4FUL
      path: ./run_details/pydb
Connected to DB PYDB_AD4FUL

Error database PYDB_AD4FUL summary
  Number of commands:      12
  Number of commands w/ errors:  12

```

```

Number of dynamic tables:      5
Total error polygons:         48
DBU:                          0.00050
Number of unique cells w/ errors: 6

```

Classifying Errors from classify2.csv...

Classifying all errors in window...

Violation: ALL

Cell: TGATE

Window: (0.0000, 0.0000) (10.0000, 10.0000)

Class: Fixed

Comment: These are fixed in the layout

12 existing errors reclassified.

Classify time = 0:00:00 User=0.01 Sys=0.01 Mem=0.000 GB

Classifying all errors in violation/cell...

Violation: Met2 width must be >=4.0, convex to convex corners
must be >= 4.5

Cell: ADFULAH

Class: Watch

Comment: These need to be fixed

3 new errors classified.

Classify time = 0:00:00 User=0.00 Sys=0.00 Mem=0.000 GB

Overall Classify time = 0:00:00 User=0.01 Sys=0.01 Mem=0.000 GB

Done.

代码 13.6: pydb_report 日志文件

使用 pydb_export 将变更合并到现有错误分类数据库 (cPYDB):

```

pydb_export -pydb_name PYDB_AD4FUL \
-pydb_path ./run_details/pydb -cpydb_name CPYDB_AD4FUL \
-cpydb_path ./cpydb -merge

```

合并步骤的 pydb_export 日志见示例 13.7。

```

Called as: pydb_export -pydb_name PYDB_AD4FUL -pydb_path
./run_details/pydb -cpydb_name CPYDB_AD4FUL -cpydb_path ./cpydb
-merge

Operation ..... Merge
Input PYDB Path ..... ./run_details/pydb
Input PYDB Name ..... PYDB_AD4FUL
Output cPYDB Path ..... ./cpydb
Output cPYDB Name ..... CPYDB_AD4FUL
Export Matched Errors ..... FALSE

Loading global database info
Loading database time = 0:00:00      User=0.00 Sys=0.00 Mem=0.000 GB
Merging classified error data
Exporting violation " toxcont must be enclosed by met1 by more
than 1.0 without touching or overlapping"
6 existing errors updated

```

```
Exporting time = 0:00:00 User=0.01 Sys=0.00 Mem=0.000 GB
Exporting violation " toxcont must be enclosed by tox by more
than 1.5"
  6 existing errors updated
Exporting time = 0:00:00 User=0.00 Sys=0.00 Mem=0.000 GB
Exporting violation "Met2 width must be >=4.0, convex to convex
corners must be >= 4.5"
  3 new errors exported
Exporting time = 0:00:01 User=0.00 Sys=0.00 Mem=0.000 GB
Export time = 0:00:01 User=0.01 Sys=0.01 Mem=0.000 GB
Overall export time = 0:00:01 User=0.01 Sys=0.01 Mem=0.000 GB
Export Successful.
PYDB_Export is done.
```

代码 13.7: pydb_export 日志文件

再次运行 IC Validator 时，LAYOUT_ERRORS 文件显示对数据库所做更改得以保留。
通过运行 pydb_report 查看详细信息：

```
pydb_report -pydb_name PYDB_AD4FUL \
-pydb_path ./run_details/pydb \
-show_classified file=classified2.txt
```

也可使用 VUE 生成报告。本例已分类错误报告的新输出与首次运行的错误报告类似。

13.2.2 pydb_report 实用程序

pydb_report Utility

使用 pydb_report 实用程序配合错误数据库可：

- 生成已分类错误的详细报告。
- 重新生成 LAYOUT_ERRORS 文件。
- 对错误分类。

对错误分类时，分类标签、UNIX 用户 ID、时间戳以及可能的注释会与该错误一并存储在当前运行的错误数据库中。

此外，可对错误分类数据库使用 pydb_report，以生成已分类错误的详细报告。

注意

见“数据库命名约定”一节。

该实用程序位于 IC Validator 安装目录的 bin 下。表 13.1 定义命令行选项。

表 13.1: pydb_report 实用程序命令行选项

命令行选项	定义
-64	以 64 位坐标支持运行该实用程序。若错误数据库中的坐标超出 32 位有符号整数容量（即 IC Validator 以 -64 命令行选项运行），可能需要此选项。
-classify <i>input_file</i>	根据给定输入文件对错误数据库中的错误分类。见下一节“分类文件格式”。

命令行选项	定义
<code>-classify_by_window</code> <code>[interacting enclosed]</code>	指定如何根据分类文件中的窗口坐标选择错误以进行分类。默认为 interacting 。 interacting : 与窗口坐标有任何相交的错误被选中分类。 enclosed : 完全被窗口坐标包围的错误被选中分类。所选错误可触及包围窗口。
<code>-cpydb_name name</code>	错误分类数据库（cPYDB）实例名。对错误分类数据库操作时必需。
<code>-cpydb_path path</code>	错误分类数据库（cPYDB）路径。对错误分类数据库操作时必需。
<code>-create_results_report</code> <code>output_file</code>	根据 PYDB 中的信息创建 RESULTS 风格文件。
<code>-dlpc <limit></code>	将所生成 LAYOUT_ERRORS 或 TOP_LAYOUT_ERRORS 文件中每个检查的错误详情（坐标）限制为给定数量。
<code>-help</code>	显示用法信息。
<code>-le output_file</code>	从当前错误数据库生成 LAYOUT_ERRORS 文件。
<code>-no_error_details</code>	不在 LAYOUT_ERRORS 或 TOP_LAYOUT_ERRORS 文件中报告错误详情（即坐标）。
<code>-pydb_name name</code>	错误数据库（PYDB）实例名；通常为 PYDB_top_cell。对错误数据库操作时必需。
<code>-pydb_path path</code>	错误数据库（PYDB）路径；通常为 ./run_details/pydb，除非在 error_options() 的 db_path 参数中另行指定。对错误数据库操作时必需。
<code>-show_classified [cell=cell]</code> <code>[violation=comment]</code> <code>[class=class1,...,classn]</code> <code>[file=file]</code>	为错误数据库与错误分类数据库生成详细的已分类错误报告。指定参数可按违例、单元与类别限制报告。若未指定输出文件，则输出到屏幕。
<code>suppress_classified_errors</code>	若指定，则 LAYOUT_ERRORS 或 TOP_LAYOUT_ERRORS 文件中不报告已分类错误。所有已分类错误均被抑制，无论其来自错误分类数据库、以编程方式分类错误的 IC Validator 选项，还是在 IC Validator 运行后由 VUE 或错误数据库分类。
<code>-tle output_file</code>	从 PYDB 数据库生成 TOP_LAYOUT_ERRORS 文件。
<code>-V</code>	打印实用程序版本。
<code>-verbose_results_file</code>	控制使用 pydb_report -create_results_report 时的输出。

分类文件格式

Classify File Format `-classify` 命令行选项输入文件的格式为逗号分隔文件，列依次为：Cell、Violation、Class、Comment、x1、y1、x2、y2。见图 13.2。

- 若给出单元但未给出违例，则对该单元内全部错误分类。
- 若给出违例但未给出单元，则对该违例内全部错误分类。
- 若给出窗口坐标，则必须给出单元，并对与该窗口相交的错误分类。默认为任意相交；可用 `-classify_by_window` 命令行选项更改此行为。

文件中记录的格式应遵循 RFC4180 推荐的 CSV 指南，且无列标题。Microsoft Excel 及其他电子表格工具支持此格式。

可选分类见表 13.2。

表 13.2: 分类文件格式

分类	说明
Error	未分类错误（默认分类）
Ignore	设计者暂不关心的错误：拟在流程后续修复
Waive	有意保留的设计错误。签收时仍将存在。
Watch	需在下一设计周期修复的错误。
Fixed	已在版图中修复的错误
Unmatched	存在于已导入错误分类数据库中、但当前运行未产生的错误

使用正则表达式匹配

Using Regular Expression Matching 在 `-classify` 命令行选项指定的输入文件中，现可用正则表达式匹配错误数据库中的违例注释与单元名。通过在首尾使用正斜杠（/），并在尾部斜杠后可选地加修饰字符来启用匹配。修饰字符见表 13.3。

表 13.3: 分类文件中可用的修饰字符

修饰字符	说明
m	区分大小写匹配。此为默认。
i	忽略大小写。

例如，分类文件中的下列行将单元 `Cell11` 中所有以 `M1`、`M2`、`M3` 或 `M4` 开头的违例注释分类为 `ignore`：

```
Cell11,"/^M[1-4].*/",Ignore,"comment"
```

下列行将含“`sram`”（忽略大小写）的所有单元中的违例予以 `waive`：

```
"/.*sram.*i","",Waive,"comment"
```

实际以斜杠开头的违例注释不能用字面字符串匹配，而必须使用 GNU 扩展正则表达式特殊字符。更多信息见 *IC Validator Reference Manual* 中 `text_options()` 函数关于这些特殊字符的表格。例如，分类文件中下列内容可匹配含“`/M4 rule 5`”的违例注释：

```
"/\\/M4 rule 5\\/"
```

示例

Examples 要从当前运行重新生成 `TOP.LAYOUT_ERRORS`，使用：

```
pydb_report -pydb_name PYDB_TOP -pydb_path ./run_details/pydb \
-l e TOP.LAYOUT_ERRORS
```

要对当前运行的错误数据库中的错误分类，使用：

```
pydb_report -pydb_name PYDB_TOP -pydb_path ./run_details/pydb \
-classify classify.csv
```

要在屏幕上显示单元 `TOP` 中全部 `Ignore` 与 `Waive` 错误的分类详情，使用：

```
pydb_report -pydb_name PYDB_TOP -pydb_path ./run_details/pydb \
-show_classified cell=TOP class=Ignore,Waive
```

要显示错误分类数据库中全部已 waive 错误的分类详情，使用：

```
pydb_report -cpydb_name CPYDB_TOP -cpydb_path ./cpydb \
-show_classified class=Waive
```

要将 `-verbose_results_file <value[s]>` 与 `-create_results_report` 一起使用，语法如下：

```
pydb_report -i run_details/pydb/PYDB_TOP -verbose_results_file
ERRORS,PRE_WAIVED_ERRORS,FLAT_VIOLATION_COUNT,TOTAL_VIOLATION_COUNT,ASSIGNS
-create_results_report TOP.report
```

注意

若 `<value[s]>` 含空白字符，需用双引号括起。

`verbose_results_file` 参数最多可有五个值，用于控制 IC Validator 生成的结果文件的详细程度。默认为 `ERRORS`。这些值为：

- **ERRORS**：报告每条规则名及其违例总数。

注意

使用错误分类时，已 waive 的违例数会从 `cell.RESULTS` 文件中报告的违例数中排除。例如，工具检测到 100 个违例，其中 60 个被 waive，因此报告的违例数为 40。

- **PRE_WAIVED_ERRORS**：在当前（waiver 后）违例计数表之后的单独表中报告 pre-waiver 违例计数。
- **FLAT_VIOLATION_COUNT**：在错误详情报告每条规则的平坦违例计数，在错误汇总节报告总平坦违例计数。
- **TOTAL_VIOLATION_COUNT**：在错误详情规则表顶部报告层次与平坦违例计数合计。
- **ASSIGNS**：报告每个 assign 层名、该层的层次与平坦多边形计数（若该层被剪枝则为“Pruned”），以及 `runset` 中 `assign` 函数对该层的附加赋值信息。

这些枚举值可与 `pydb_report` 的 `verbose_results_file` 参数一起使用，以定制结果文件输出。

以下为小型 PYDB 的 `TOP.report` 文件片段（在手动 waive “MyRule” 所属两个违例之一后）。

注意

已 waive 的违例计数不在此报告中显示。

```
...
...

- - - - -
Rule                Violations Found
TOTAL               v = 2 (2)
MyRule              v = 1 (1)
Violation           v = 1 (1)
  text_net:text_short    v = 1 (1)

- - - - -
Rule                Pre-Waived Violations Found
TOTAL               v_all = 3 (3)
MyRule              v_all = 2 (2)
Violation           v_all = 1 (1)
  text_net:text_short    v_all = 1 (1)
```

13.2.3 pydb_export 实用程序

pydb_export Utility

使用 `pydb_export` 实用程序从错误数据库导出错误分类数据，以便：

- 创建新的错误分类数据库（cPYDB）。见“错误分类数据库格式”。
- 合并到现有错误分类数据库：追加新错误并更新已有错误。

此外，`pydb_export` 可用于管理错误分类数据库：将多个错误分类数据库合并在一起，或从违例与单元中清除分类数据。

要使错误在各次运行间视为等价，违例注释必须完全匹配，或匹配到指定注释分隔符的首次出现为止，且错误多边形必须在单元级匹配。

注意

见“数据库命名约定”一节。

该实用程序位于 IC Validator 安装目录的 `bin` 下。表 13.4 定义命令行选项。

表 13.4: `pydb_export` 实用程序命令行选项

命令行选项	定义
<code>-64</code>	以 64 位坐标支持运行该实用程序。若错误数据库中的坐标超出 32 位有符号整数容量（即 IC Validator 以 <code>-64</code> 运行），可能需要此选项。
<code>-cell cell</code>	将导出限制为给定单元的错误。可多次给出以指定多个。支持正则表达式。
<code>-class class</code>	将导出限制为给定分类的错误。可多次给出以指定多个类别。支持正则表达式。
<code>-comment_match_delimiter delim</code>	匹配违例注释直至指定分隔符。导出错误数据库时，分隔符默认取 <code>runset</code> 中 <code>error_options()</code> 的 <code>comment_match_delimiter</code> 参数所指定者。合并多个错误分类数据库时，则应指定此命令行分隔符以使用部分违例注释匹配。
<code>-convert_to_64</code>	将 32 位或可变宽度坐标的错误分类数据库（cPYDB）转换为 64 位坐标错误分类数据库。与 <code>-copy_cpydb</code> 或 <code>-create</code> 一起使用时，在创建时转换为 64 位；否则就地转换现有数据库。
<code>-convert_to_32</code>	将 64 位或可变宽度坐标的错误分类数据库（cPYDB）转换为 32 位坐标错误分类数据库。与 <code>-copy_cpydb</code> 或 <code>-create</code> 一起使用时，在创建时转换为 32 位；否则就地转换现有数据库。注意：若错误分类数据库含有无法用 32 位整数表示的过大坐标，此操作会失败。
<code>-convert_to_variable</code>	将 64 位或 32 位坐标的错误分类数据库（cPYDB）转换为可变宽度坐标错误分类数据库。与 <code>-copy_cpydb</code> 或 <code>-create</code> 一起使用时，在创建时转换为可变宽度坐标；否则就地转换现有数据库。
<code>-copy_cpydb name=db_name path=db_path</code>	将错误分类数据库复制到目标错误分类数据库，创建新数据库或覆盖现有数据库。
<code>-cpydb_name name</code>	输出错误分类数据库（cPYDB）实例名。必需。
<code>-cpydb_path path</code>	输出错误分类数据库（cPYDB）路径。必需。
<code>-create</code>	导出错误数据库，创建新错误分类数据库或覆盖现有错误分类数据库。或者，合并多个错误分类数据库，创建新错误分类数据库或覆盖现有错误分类数据库。
<code>-exclude_cell cell</code>	从导出中排除给定单元的错误。可多次给出。支持正则表达式。 <code>-exclude_cell</code> 优先于 <code>-cell</code> 。

命令行选项	定义
<code>-exclude_class class</code>	排除给定分类的错误。可多次给出。支持正则表达式。 <code>-exclude_class</code> 优先于 <code>-class</code> 。
<code>-exclude_violation violation</code>	从导出中排除给定违例注释的错误。可多次给出。支持正则表达式。 <code>-exclude_violation</code> 优先于 <code>-violation</code> 。
<code>-export_matched</code>	包含已匹配错误。默认仅导出新分类的错误。合并到现有错误分类数据库时，此选项指定是否包含来自目标以外其他错误分类数据库的已匹配错误。
<code>-help</code>	显示用法信息。
<code>-host_init</code>	指定分布式操作的主机配置选项。当前，此选项的分布式处理仅支持 <code>-flatten</code> 。更多信息见第 1 章 IC Validator 基础中的 <code>-host_init</code> IC Validator 命令行选项。
<code>-flatten layout=layout cell=cell</code>	使用给定版图将输出错误分类数据库展平到给定顶层单元。应给出创建正在被展平的错误分类所用的版图与顶层单元作为 <code>layout</code> 与 <code>cell</code> 参数。版图格式自动检测，路径约定与 LVL 实用程序的 <code>library1</code> 、 <code>library2</code> 命令行选项相同。仅输出给定单元内及其层次下方的违例。对原始单元的版图网格与 <code>drawn</code> 检查不会被展平。更多信息见“展平错误分类数据库”。
<code>-format CPYDB OASIS PERC</code>	指定输出 waiver 数据库的格式。默认为 CPYDB。
<code>-i input db</code>	输入 PYDB 数据库、cPYDB 或 OASIS waiver 文件。
<code>-layer layer number</code>	指定 OASIS 格式 waiver 形状的层。Waiver 形状创建在指定层号上， <code>datatype</code> 基于规则的违例注释。默认为 87654。
<code>-merge</code>	将指定错误数据库合并到指定错误分类数据库，更新现有错误的分类或向数据库添加新错误。
<code>-merge_cpydb name=db_name path=db_path</code>	将错误分类数据库合并到目标错误分类数据库。可在命令行使用多个选项以合并多个错误分类数据库。
<code>-merge_layout main_library_path</code>	将通过 <code>-i</code> 指定的错误分类数据库中的 waivers 与本参数指定的设计版图合并到由 <code>-o</code> 指定的 OASIS 文件。这样单个 OASIS 文件可同时包含版图与 waivers。此参数需要 <code>-format OASIS</code> 。
<code>-o output db</code>	指定输出数据库路径。可用于指定 cPYDB 或 OASIS 格式输出数据库。
<code>-password password</code>	对现有错误分类数据库写访问的口令。指定 <code>prompt</code> 可从命令行读取口令。
<code>-perc_hierarchical_delimiter delimiter</code>	导出 PERC waivers 时使用的层次分隔符。默认为 “/”。
<code>-perc_holding_cell cell_name</code>	导出 PERC waiver 文件时，将给定单元名设为导出 waivers 的 holding cell。
<code>-perc_output_comment true false</code>	导出 PERC waiver 文件时，选择是否输出作为错误一部分的信息性注释。默认为 <code>true</code> 。
<code>-purge purge_file</code>	根据给定输入文件清除违例与单元的全部分类数据。文件格式见“清除文件格式”。若有错误数据库引用目标错误分类数据库，且错误匹配了正在清除的违例与单元，则这些错误将不再可查看。
<code>-pydb_name name</code>	输入错误数据库（PYDB）实例名；通常为 <code>PYDB_top_cell</code> 。导出时必需。
<code>-pydb_path path</code>	输入错误数据库（PYDB）路径；通常为 <code>./run_details/pydb</code> ，除非在 <code>error_options()</code> 的 <code>db_path</code> 中另行指定。导出时必需。

命令行选项	定义
<code>-read_only</code>	指定输出错误分类数据库（cPYDB）应为只读。与 <code>-create</code> 或 <code>-copy_cpydb</code> 一起使用时，创建后将 cPYDB 改为只读模式；否则将现有数据库改为只读。只读数据库不得以任何方式修改。每个连接到只读 cPYDB 的 IC Validator 组件有自己的数据库服务器进程。其他用户已连接时，不可将可写数据库转为只读。
<code>-rule_name_delimiter <i>regular expression</i></code>	指定用于在创建 OASIS waiver 文件中的规则 waiver 单元名时，将规则名与违例注释其余部分分开的规则名分隔符正则表达式。默认使用 <code>runset</code> 中 <code>error_options(rule_name_delimiter)</code> 指定的规则名分隔符。 <code>runset</code> 规则名分隔符默认为 <code>/[[:space:]]+:[[:space:]]*:[[:space:]]*/</code> 。
<code>-rule_name_length <i>integer</i></code>	指定用于创建 OASIS waiver 文件中规则 waiver 单元名的规则名最大长度。超过此值的规则名会被截断，若截断后冲突则添加后缀以保持唯一。默认为 15。
<code>-set_password <i>password</i></code>	为给定错误分类数据库设置口令。指定 <code>prompt</code> 可从命令行读取口令。
<code>-set_waiver_properties <i>user_property_file.json</i></code>	从给定 JSON waiver 属性文件设置 waiver 用户属性。
<code>-select_waiver_properties <i>select_properties_file.json</i></code>	仅包含与给定 JSON 属性选择文件中指定的 waiver 用户属性匹配的 waivers。
<code>-V</code>	打印实用程序版本。
<code>-violation <i>violation</i></code>	将导出限制为给定违例注释。可多次给出。支持正则表达式。
<code>-waiver_property_definition_file <i>property_definition_file.json</i></code>	指定用于验证 waiver 用户属性的 JSON 文件。此文件覆盖环境变量 <code>ICV_WAIVER_PROPERTY_DEFINITION_FILE</code> 。

错误分类数据库格式

Error Classification Database Format 错误分类数据库包含已分类错误的分类标签、用户 ID、时间戳、注释与错误多边形。在后续运行中使用错误分类数据库时，匹配的错误的数据库中用对错误分类数据库的引用加以标注。这些错误的实际分类数据仍留在错误分类数据库中。因此，要查看当前运行的错误，错误分类数据库需位于运行 IC Validator 时所指定的位置。此外，若从错误分类数据库清除数据或覆盖数据，则无法查看当前运行中的已匹配错误。

清除文件格式

Purge File Format `-purge` 命令行选项输入文件的格式为逗号分隔文件，两列：Cell 与 Violation。

- 若指定单元但未指定违例，则清除该单元内全部错误。
- 若指定违例但未指定单元，则清除该违例内全部错误。

按 RFC4180 推荐指南格式化文件中的记录，且无列标题。Microsoft Excel 及其他电子表格工具支持此格式。

示例

Examples 要从当前运行的分类数据创建新错误分类数据库（cPYDB），使用：

```
pydb_export -pydb_name PYDB_AD4FUL \
  -pydb_path ./run_details/pydb \
  -cpydb_name CPYDB_AD4FUL -cpydb_path ./cpydb \
  -create
```

要为错误分类数据库设置口令，使用：

```
pydb_export -cpydb_name CPYDB_AD4FUL -cpydb_path ./cpydb \
  -password oldpassword -set_password newpassword
```

要将分类数据合并到受口令保护的现有错误分类数据库，使用：

```
pydb_export -pydb_name PYDB_AD4FUL \
  -pydb_path ./run_details/pydb \
  -cpydb_name CPYDB_AD4FUL -cpydb_path ./cpydb \
  -merge -password password
```

要合并多个错误分类数据库并创建新错误分类数据库，使用：

```
pydb_export -cpydb_name CPYDB_MY_DESIGN -cpydb_path ./cpydb \
  -merge_cpydb name=CPYDB_CELLA path=./cpydb \
  -merge_cpydb name=CPYDB_CELLB path=./cpydb \
  -merge_cpydb name=CPYDB_TOP path=./cpydb \
  -create
```

要就地将 32 位坐标的错误分类数据库转换为 64 位坐标，使用下列语法。使用 `-convert_to_64` 时，需要对该数据库的独占访问；即数据库不能被 VUE、任何其他脚本或 IC Validator 作业打开。

```
pydb_export -cpydb_name db_name -cpydb_path db_path -convert_to_64
```

要复制错误分类数据库并转换为 64 位而不修改原数据库，使用：

```
pydb_export -cpydb_name db_name -cpydb_path db_path \
  -copy_cpydb name=input_db_name path=input_db_path \
  -convert_to_64
```

`pydb_report` 可提供现有错误分类数据库的坐标宽度信息：

```
pydb_report -cpydb_name CPYDB_AD4FUL64 -cpydb_path ./cpydb
...
```

(C) Copyright 2008-2014.
Synopsys, Inc. All rights reserved.

```
cPYDB Name ..... CPYDB_AD4FUL64
cPYDB Path ..... ./cpydb
Coordinate Width ..... 64 bit
```

要仅导出不以 “W” 开头的所有单元中以 “W” 开头的类别，使用：

```
pydb_export -class "/^W.*/" -exclude_cell "/^W.*/" ...
```

要导出除 “Ignore” 与 “Waive” 外的全部类别，使用：

```
pydb_export -exclude_class Ignore -exclude_class Waive ...
```

要仅导出顶层单元，使用：

```
pydb_export -cell TOP ...
```

展平错误分类数据库

Flattening an Error Classification Database 当使用 block 级 IC Validator 结果生成错误分类数据库，并计划在 block 被放置后用于芯片级层次错误分类时，若当前 block 与顶层设计中其他已放置 block 之间存在任何共享子单元，建议展平 block 级结果。作为层次 DRC 检查的一部分，IC Validator 工具可能将错误下推到子单元。若 block 级错误分类数据库未展平，它可能在共享子单元中包含仅在该 block 上下文中适用的 waivers，但在该 block 放入含有该共享子单元其他实例的更大设计后则无效。展平 block 级错误分类数据库可避免此情况。

`pydb_export` 实用程序展平错误分类数据库时，使用与创建违例的原始 IC Validator 运行相同的版图与顶层单元。必须提供版图以给出展平过程所需的层次信息。错误分类数据库中的违例被展平到所提供的顶层单元。仅输出所提供顶层单元内或其层次下方的违例。

版图以路径指定，约定与 LVL 实用程序的 `library1`、`library2` 命令行选项相同：

- GDSII: `/path/to/file.gds`
- OASIS: `/path/to/file.oas`
- Milkyway: `/library/path/LIBRARY_NAME`
- OpenAccess: `/path/to/lib.defs/LIBRARY_NAME`
- NDM: `/library/path/LIBRARY_NAME`

展平过程可用 `pydb_export` 的 `-host_init` 命令行选项进行分布式处理。支持下列用例。

从 PYDB 导出时展平：

```
pydb_export \
  -pydb_path path -pydb_name name \
  -cpydb_path output_path -cpydb_name output_name \
  -create \
  -flatten layout=layout cell=topcell \
  [-host_init number]
```

复制 cPYDB 并展平目标 cPYDB：

```
pydb_export \
  -copy_cpydb path=path name=name \
  -cpydb_path output_path -cpydb_name output_name \
  -flatten layout=layout cell=topcell \
  [-host_init number]
```

就地展平 cPYDB：

```
pydb_export \
  -cpydb_path path -cpydb_name name \
  -flatten layout=layout cell=topcell \
  [-host_init number]
```

13.3 使用 PERC 错误分类

Using PERC Error Classification

对于基于网表的 PERC 器件与网络错误支持错误分类。总体流程与 DRC 错误分类流程相同。首次 IC Validator 运行的错误使用 VUE 或某一 PYDB 实用程序分类，然后导出到错误分类数据库（cPYDB），以便在后续 IC Validator 运行中应用分类。应与 DRC 错误分类一样，在 `error_options(match_errors)` 参数中指定 cPYDB。

注意

分类仅支持基于网表的 PERC 流程，不适用于基于版图的流程。仅支持器件与网络错误。

13.3.1 匹配已分类错误

Matching Classified Errors

基于网表的错误与 DRC 错误不同：没有可用于唯一标识特定错误实例的形状或坐标。此外，基于网表的错误关联更多元数据。以下为器件错误示例。

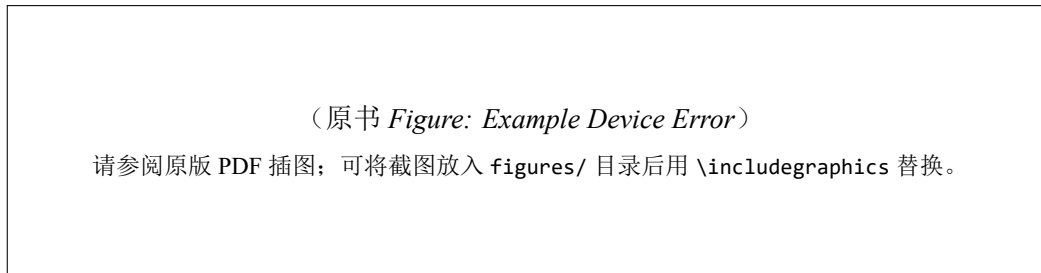


图 13.4: 器件错误示例

除单元名与器件实例及模型名外，错误实例还有可对每个错误实例唯一的 **Comment** 字段、其全部引脚名与关联网络、可能的一组参考对象（单元实例、器件实例、网络）、器件是否由合并产生、标签等。网络错误有类似的元数据集。错误在单元级按下列准则匹配：

器件错误：

- 违例注释 / 规则
- 单元名
- 器件名
- 器件类型
- 注释

网络错误：

- 违例注释 / 规则
- 单元名
- 网络名
- 注释

若新错误的上述全部准则与 cPYDB 中已分类错误匹配，IC Validator 将该分类应用于新错误。

13.3.2 层次化 PERC 错误分类

Hierarchical PERC Error Classification

IC Validator 提供层次模式：若错误在后续 IC Validator 运行中报告于更高层次，则可将原先在较低层次报告的错误分类应用于该错误。

在层次模式下检查器件错误的分类时，若器件具有实例路径（如上例），只要原始实例名与单元位于新实例路径底部，在更高层次发出的该器件错误仍应被分类。网络必须以相同方式匹配：原始实例名与单元应位于新实例路径底部。

分类匹配首先在单元级尝试。若未找到单元级匹配，IC Validator 沿层次向下搜索与实例路径某一部分匹配的已分类错误。错误可能匹配多个已分类错误。IC Validator 沿层次向下查找时应用找到的第一个匹配，忽略其他潜在匹配。

层次示例

Hierarchical Example 本例参考上例的器件错误。它在单元 `dwc_eusb2` 中报告并予以 waive，器件名为 `Xu_eusb/XI206/M7`。若对同一 block 重新运行 IC Validator，很可能会找到精确的单元级匹配。

若对更高层 block `dwc_top` 运行 IC Validator，且错误以器件名 `X1/X2/Xu_eusb/XI206/M7` 报告，则 IC Validator 按如下方式搜索匹配的已分类错误：

- IC Validator 在 `dwc_top` 中搜索单元级匹配，未找到匹配的已分类错误。
- `dwc_top` 中的 `X1` 是单元 `dwc_mid` 的实例。IC Validator 在单元 `dwc_mid` 中搜索器件名为 `X2/Xu_eusb/XI206/M7` 的已分类错误，未找到匹配。
- `dwc_mid` 中的 `X2` 是单元 `dwc_eusb2` 的实例。IC Validator 在单元 `dwc_eusb2` 中搜索器件名为 `Xu_eusb/XI206/M7` 的 waiver。这与上例错误匹配，因此将该分类应用于在 `dwc_top` 中报告的错误。

对网络错误的网络名层次匹配使用相同方法。

Chapter 14

DRC Waivers

DRC Waivers

本章介绍 IC Validator 工具的 DRC waivers 能力及其用法。

DRC waivers 能力分以下各节说明：

- 定义 Waivers
- 创建 Waiver 数据库
- 使用 Waiver 数据库

DRC waivers 允许你对 IC Validator 运行中发现的错误进行分类与标注，并将错误导出到 waiver 数据库，供后续 IC Validator 运行使用。DRC Waivers 流程分为三个独立步骤：

- 定义哪些错误可被 waive，并通过 VUE 或 `icv_pydb` 命令行实用程序对错误数据库加注
- 在磁盘上创建持久化 waiver 数据库
- 在后续 IC Validator 运行中应用 waivers

以下各节分别说明这些步骤。

（原书 *Figure 96: General Waiver Flow*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 14.1: 一般 Waiver 流程

14.1 定义 Waivers

Defining Waivers

可使用 VUE 或 ASCII 错误报告文件 `cell.LAYOUT_ERRORS` 审查写入错误数据库（PYDB）的设计错误。在许多设计中，存在可被 waive 的 DRC 违例，即这些错误无需在版图中修复。这可以是必须在 `tapeout` 前修复的临时 waiver，

也可以是晶圆厂批准的永久 waiver。可为 waivers 添加用户注释或预定义的 waiver 属性，以便为审查 waivers 的其他工程师提供更多上下文。

IC Validator SQLite 错误数据库允许许多用户并发访问；在审查错误并应用 waivers 时，访问该数据库的其他用户可以看到你已应用的更新。

IC Validator 支持若干 waiver 类别：

- 持久化 waiver 类别：Waive、Watch、Ignore
- 非持久化 waiver 类别：Error、Fixed

持久化 waiver 类别可导出到 waiver 数据库。见“创建 Waiver 数据库”。通过多种持久化 waiver 类别，可将已 waive 的错误划分到不同类别，使设计过程中的 waiver 管理更简便。这些 waiver 类别同等对待。例如，可在整个设计过程直至 tapeout 均拟 waive 该错误时选择 Waive，并选择 Watch 以标示可在下一两次设计迭代中修复的错误。

非持久化 waiver 类别向调试同一错误数据库的其他用户提供信息。例如，将错误定义为 Error 并附用户注释“this error needs to be fixed in conjunction with rule ABCD”，则在同一错误数据库中审查该错误的任何人都能看到此指导。

14.1.1 使用 VUE 定义 Waivers

Using VUE to Define Waivers

定义 waivers 最常见的方式是通过 VUE DRC 浏览器。有多种方式 waive 错误。

要 waive 某违例（规则）中的全部错误形状，在 Violation Browser 中选择该违例并右键单击以查看选项菜单。选择适当的 waiver 类别，如图 14.2 所示。

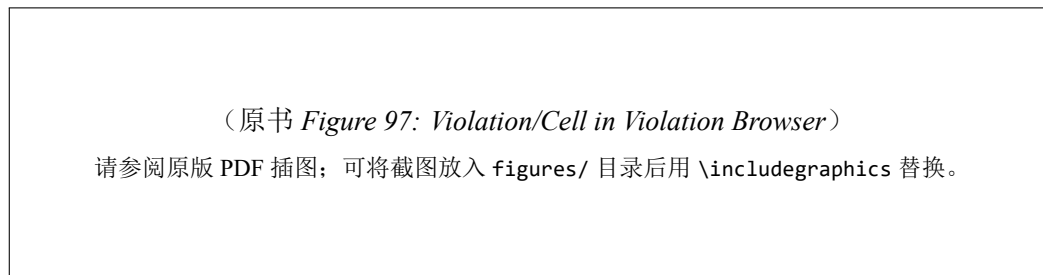


图 14.2: Violation Browser 中的违例/单元

本例选择了 Waive 类别。选择后出现窗口，允许提供可选用户注释并定义 waiver 上下文。

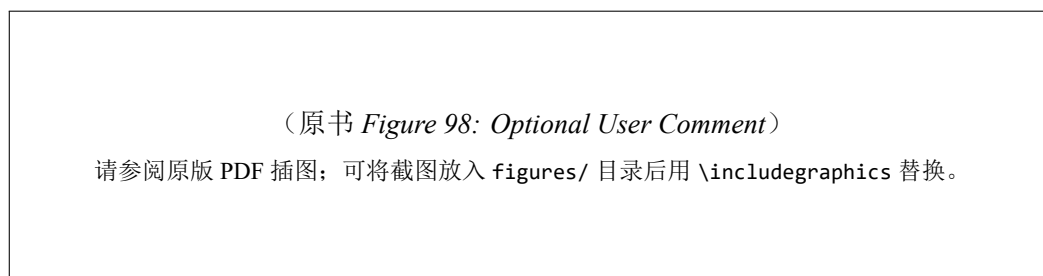


图 14.3: 可选用户注释

Context 选项决定这些 waivers 在未来 IC Validator 运行中何时适用。设为 ALL 时，若与错误匹配则始终应用该 waiver。设为顶层单元时（本例顶层单元名为“top”），仅当当前顶层 block 再次作为顶层单元运行时才应用该 waiver。

定义 waivers 后，错误浏览器中会新增一列以显示含已 waive 错误的违例。

每个错误现已用 waiver 信息加注。

状态现为 Waive。From 列显示“vue”，表示当前 waiver 由用户应用，并非 IC Validator 引用 waiver 数据库时找到的。

Error List 可用于仅 waive 某些错误，如图 14.6 所示。

（原书 *Figure 99: Error Browser Column*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 14.4: 错误浏览器列

（原书 *Figure 100: Waiver Information*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 14.5: Waiver 信息

（原书 *Figure 101: Error List*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 14.6: 错误列表

可通过按住 Shift-click 与 Ctrl-click 选择 R.M4.W 违例中的个别错误，然后右键单击查看 waive 选项菜单。另有选项 Waive In Area，允许 waive 特定区域内的全部错误，而无需从 Error List 中逐个选择。

错误被 waive 后，可在错误列表中过滤以隐藏特定 waiver 类别。在 DRC/Extraction VUE 选项中，使用 Enable/Disable filter functions 部分选择要在 Error List 中显示的 waiver 类别。

(原书 *Figure 102: Filter Settings*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 14.7: 过滤设置

本例中，分类为 Waive 的错误从 Error List 中过滤掉。显示所有其他 waiver 类别。

图 14.8 展示更新后的 Error List:

(原书 *Figure 103: Updated Error List*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 14.8: 更新后的错误列表

14.1.2 使用命令行实用程序定义 Waivers

Defining Waivers Using the Command-Line Utilities

可使用 icv_pydb 命令行实用程序定义 waivers。更多信息见第 十五 章“错误管理实用程序”。

14.1.3 自定义 Waiver 属性

Custom Waiver Properties

除用户注释外，还可为给定规则与单元定义额外的 waiver 属性。这些属性有助于添加信息，并提供跨设计或工艺节点管理 waivers 的额外方式。例如，可为 waivers 赋予定义特定跟踪号及其当前管理审批状态的 waiver 属性。若 waiver 未获批准，可使用 pydb_export 与 -select_waiver_properties 命令行选项，从 waiver 数据库中移除具有该跟踪号的任何 waiver。

允许的值定义在由 VUE 读取的 JSON 格式文件中。

```
{
  "waiver_property_definitions": [
    {
      "property_name": "WaiverTracking#",
      "property_values": []
    },
    {
```

```

    "property_name": "Valid_through",
    "property_values": ["milestone1", "milestone2", "milestone3"]
  },
  {
    "property_name": "Status",
    "property_values": ["pending", "approved", "not approved"],
    "default_value": "pending"
  }
]
}

```

代码 14.1: JSON 属性文件 (waiver_properties.json)

每个属性有 `property_name`、`property_values`，以及可选的 `default_value`。`property_name` 遵循 OASIS 属性标准，即使用可打印 ASCII 字符，排除空格、制表符、换行符与控制字符。属性值不受限制。

注意

WAIVER_REPORT 文件使用 | 作为多个属性值之间的分隔符。为避免混淆，请勿使用该字符。

设置下列环境变量后，VUE 会读取该 JSON 文件：

```
setenv ICV_WAIVER_PROPERTY_DEFINITION_FILE waiver_properties.json
```

然后创建含允许属性的自定义窗口。在 Violation Browser 中，右键单击选择规则与单元。此时可用 Add Waiver Properties 选项。

(原书 *Figure 104: Add Waiver Properties*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 14.9: 添加 Waiver 属性

选择该选项以定义用户属性。

该规则/单元的所有已 waive 错误继承这些属性。可在 Violation Detail 窗格中查看属性，如下所示。

14.2 创建 Waiver 数据库

Creating a Waiver Database

为当前设计定义全部 waivers 后，将信息导出到独立的 waiver 数据库。Waiver 数据库可以是 SQLite 格式 (CPYDB) 或 OASIS 格式。Waiver 数据库是自包含的，即 waiver 错误所需的全部信息（用户注释、用户 ID、日期、waiver 属性等）均包含在数据库内。

可使用 VUE 或 `pydb_export` 命令行实用程序导出 waiver 数据库。

要用 VUE 导出 SQLite waiver 数据库，选择 Classification > Export Classification Data (CPYDB)。

在此可定义数据库的详细信息。

输出数据库名写入启动 VUE 时的工作目录。要写到其他路径，在输出文件名中包含该路径。默认情况下，IC Validator 创建新的 waiver 数据库。可通过设置 Append Mode 将此信息追加到现有 CPYDB 数据库。导出期间，VUE 导

出错误数据库中定义的任何新 waivers。要包含全部 waivers（包括先前从另一 waiver 数据库匹配到的），勾选 **Include Matched Errors** 框。

SQLite 格式允许为 waiver 数据库设置口令。除非知道口令，否则无法修改该 waiver 数据库。

要从 VUE 导出 OASIS 格式 waiver 数据库，选择 **Classification > Export Classification Data (OASIS)**。

此处窗口提供 waiver 数据库的设置。

输出数据库名写入启动 VUE 时的工作目录。要写到其他路径，在输出文件名中包含该路径。输出的 OASIS 格式将 waiver 形状放在特殊命名的单元中，该单元为每组 waivers 提供规则与单元名 (**Rule_Name@cellname**)。Rule Name Delimiter 允许你定义工艺节点规则名的分隔符。在下列示例中，规则名使用 “:” 字符作为规则名与规则描述之间的分隔符。设置后，分隔符左侧的全部字符定义 waiver 单元名。若未设置，或规则名很长，则 **Max Rule Name Length** 将该 waiver 单元名截断到该长度。默认值为 "[[:space:]]+:[[:space:]]*:[[:space:]]"，可匹配大多数晶圆厂 runset 命名约定。

现可在 IC Validator WorkBench 中打开 OASIS 格式 waiver 数据库。在 waiver 数据库的树视图中（如下图所示），可以看到数据库有一个以 @ 为前缀的单元。这是 **holding cell**，包含为该单元定义的全部 waivers。该单元本身不含任何 waiver 形状；形状包含在放置于 **holding cell** 内的规则专用单元中。

每个 waiver 单元 (**rule@cell**) 在单元上附着属性。

这些属性在导出时设置。导出 waiver 数据库的用户会将其用户 ID 列在 **createID** 属性中，以及导出时间戳。**waiverChecksum** 属性在导出时对该单元计算。IC Validator 读取 waiver 单元时，会计算校验和并与存储的属性比较。若属性不同，表示数据库自导出后已被修改。**WAIVER_REPORT** 文件末尾包含校验和是否匹配的信息。

给定 **Violation/Cell** 的全部 waiver 形状写入任意层号。IC Validator 不使用层号，而是使用 **violationComment** 属性来了解 waiver 形状适用于哪条规则。因此不需要映射文件。

14.3 使用 Waiver 数据库

Using a Waiver Database

创建 waiver 数据库后，可将其用作其他 IC Validator 运行的输入。新运行不必是同一顶层单元。IC Validator 导入当前运行中存在的 Cell / Violation 的全部 waiver 形状。IC Validator waivers 是完全层次化的。

waiver_options() 函数提供对 IC Validator 运行期间如何应用与报告 waivers 的控制。该命令分为两个主要部分。第一部分是数据库管理：提供 waiver 数据库列表并定制每个数据库的处理方式。第二部分提供 IC Validator 对全部 waivers 与报告使用的全局 waiver 控制。

数据库管理由 **databases** 参数控制。IC Validator 自动检测数据库格式，同一 IC Validator 运行中可同时提供 CPYDB 与 OASIS 格式。使用 OASIS 格式 waiver 数据库时不需要层映射文件。

对每个 waiver 数据库，可定制控制以提供跨设计的灵活性。包括：

- 控制在找不到 waiver 数据库时如何处理。
- 用此数据库 waive 的错误是抑制还是分类。抑制意味着已 waive 的错误不写入 PYDB 或 **LAYOUT_ERRORS** 文件。
- 也可为 waiver 数据库给出单元前缀或单元后缀。若流程在将单元放入更大宏块时使单元唯一化，但基单元的 waivers 仍可应用于重命名后的单元，则此功能很有用。
- 生成特殊报告，列出 IC Validator 运行期间未匹配的数据库中全部 waiver 形状。

全局 waiver 控制决定如何将 waiver 形状与运行期间发现的错误形状比较，包括任何规则专用覆盖与报告生成。

14.3.1 匹配模式

Matching Modes

可控制 IC Validator 如何将 waiver 形状匹配到给定错误形状。所有匹配均在所有相交的 waiver 形状已合并、所有相交的错误形状已合并之后进行。在某些情况下，为允许合并发生，形状可能需要提升到共同父单元。匹配是层次化的，即层次合并后的 waiver 形状不必与层次合并后的错误形状在同一单元中即可找到匹配。

有三种匹配模式可用：**EXACT**、**ENCLOSE**（默认）与 **INTERACT**。

- **EXACT 模式**：层次合并后的 waiver 形状必须与层次合并后的错误形状完全匹配，才会应用 waiver。
- **ENCLOSE 模式**是默认匹配模式。Waiver 形状必须包围错误形状。
- **INTERACT 模式**：任何与 waiver 形状层次相交的错误均被 waive。使用 `include_touch` 参数可将外侧相触（outside-touch）交互视为 waive 的相交。

注意

使用 `tolerance` 参数可改善全角度（all-angle）形状的匹配。

在 `waiver_options()` 中设置的匹配模式适用于全部 `waiver_databases`。可用 `rule_overrides` 参数覆盖特定规则的匹配模式。例如，除 `R.MY.STRICT_RULE` 外的全部规则允许 **ENCLOSE** 模式；对该规则，仅当 waiver 与错误形状完全匹配时才应用 waivers。例如：

```
waiver_options(
  databases = { {"/path/to/waiver.oas"}},
  matching_mode = ENCLOSE,
  rule_overrides = {
    violation_comments = {"R.MY.STRICT_RULE"},
    matching_mode = EXACT,
    tolerance = 0.005
  }
);
```

14.3.2 Waiver 报告文件

Waiver Report File

IC Validator 将全部已分类（已 waive）的错误写入 IC Validator 工作目录中的 waiver 报告文件（`cell.WAIVER_REPORT`）。

LAYOUT_ERRORS 文件与 VUE run_summary 选项卡显示：

ERRORS (WAIVERS APPLIED)

默认生成 waiver 报告文件，含规则汇总（包括已应用的 waivers）以及输入 waiver 数据库列表。

```
-----
----
WAIVER SUMMARY
-----
----

R.M3.L: M3 can not have 3 or mor... | Waive : 5 | Ignore : 0 | Watch : 0
| WaiverConflict : 0 | Unmatched : 0
R.M4.W: M4 Minimum width >= 0.24 | Waive : 4 | Ignore : 0 | Watch : 0
| WaiverConflict : 0 | Unmatched : 0
```

表列为：规则名、Waive、Ignore、Watch 类别计数、WaiverConflict 计数，以及 Unmatched waiver 计数。

- 当具有不同 waiver 类别的相交 waiver 形状被层次合并在一起时，发生 waiver 冲突。
- Unmatched 是该规则在所有输入 waiver 数据库中未在当前设计中匹配的 waiver 形状计数。

也可通过设置下列内容生成详细 waiver 报告：

```
waiver_options(report_waiver_details = true)
```

这会为每个 waiver 添加表格，含单元、waiver 类别、匹配模式、上下文、匹配到的 waiver 形状所在的 waiver 数据库、任何 waiver 属性，以及错误详情（距离、包围盒等）。

注意

可用 `-runset_config` 命令行选项修改 `waiver_options()` 函数。

可在 IC Validator 运行完成后生成 waiver 报告文件。若不想在默认 IC Validator 运行中查看详情，但希望运行后再查看，此功能很有用。

```
%icv_pydb -i ./run_details/pydb/PYDB_top \
    -create_waiver_report top.DETAILED_WAIVERS \
    -include_waiver_details
```

也可用 `icv_pydb` 实用程序生成 JSON 格式的 waiver 报告文件。若需编写脚本处理给定设计上已应用的 waivers，这种结构化格式更易于解析。

```
% icv_pydb -i ./run_details/pydb/PYDB_top \
    -create_json_waiver_file top.waiver_file \
    -include_waiver_details
```

```
{
  "icv_run_info": {
    "library_name": ".././../lab_data/INLIB_TRAINING.GDS",
    "structure_name": "top",
    "generated_by": "IC Validator RHEL64
S-2021.06-SP3-BETA-20211123.7086978 2021/11/23",
    "runset_name": "../modular.rs",
    "user_name": "johndoe",
    "time_started": "2021/11/24 12:00:58PM",
    "time_ended": "2021/11/24 12:01:13PM"
  },
  "rules": [
    {
      "rule": "R.M3.L",
      "violation_comments": ["R.M3.L: M3 can not have 3
or more consecutive line segments be < 0.07000000000000001"]
    },
    {
      "rule": "R.M4.W",
      "violation_comments": ["R.M4.W: M4 Minimum width
>= 0.24"]
    }
  ],
  "waiver_databases": [
    {
      "waiver_database_id": "WDB1",
      "file":
"/path/to/johndoe/TRAINING/drc/lab_answers/lab5_modularize/mywaivers_db.oas",
      "checksum": "PASS"
    }
  ],
  "waiver_details": [
    {
      "rule": "R.M3.L",
      "command": ".././include/metal_rules.rs:78:vertices_edge",
      "cell": "top",
      "class": "Waive",

```

```
    "mode": "Enclose",
    "context": "ALL",
    "waiver_sources": ["WDB1:top"],
    "unique_id": "E.12.100.1",
    "bbox": [30.2400, 15.3700, 30.2400, 15.4000],
    "user_comments": ["waiving M3.L rule."]
  }
],
  "unmatched_waiver_details": [ ],
  "rule_overrides": [ ]
}
```

代码 14.2: JSON 格式 Waiver 报告文件

Chapter 15

错误管理实用程序

Error Management Utility

本章介绍 IC Validator 工具附带的错误管理实用程序。

错误管理实用程序分以下各节说明：

- 错误管理实用程序
- 拆分错误数据库
- 合并错误数据库
- VUE 文件支持
- 就地修改数据库
- 层次考虑
- XML 命令文件
- 将错误导出到版图
- 管理所引用的 cPYDB 路径

15.1 错误管理实用程序

Error Management Utility

使用错误管理实用程序（`icv_pydb`）可以：

- 将单次 IC Validator 运行的错误数据库（PYDB）拆分为多个数据库以便调试。
- 从输入 PYDB 选择错误，并仅用所选错误创建新 PYDB。
- 将多个 PYDB 数据库合并在一起。
- 通过重命名违例或单元、或对错误分类，来更改现有 PYDB 数据库。
- 将 PYDB 数据库中的错误导出到版图。
- 对 DRC 错误分组。

`icv_pydb` 实用程序提供命令行选项，可按单元（单独或层次）、违例注释、违例名、错误分类或单元级包围盒进行任意选择。

该实用程序位于 IC Validator 安装目录的 `bin` 下。表 15.1 定义命令行选项。

表 15.1: 错误管理实用程序命令行选项

命令行选项	定义
输入选项	
<code>-i input db</code>	指定输入数据库路径, 例如 <code>run_details/pydb/PYDB_top</code> 。必需。此选项必须出现在适用于它的任何输入选择选项之前。所有其他输入选项仅适用于命令行上位于其前的那个输入数据库。
<code>-svc violation_comment /</code> <code>-select_violation_comment</code> <code>violation_comment</code>	仅包含匹配给定模式的违例注释。
<code>-uvc violation_comment /</code> <code>-unselect_violation_comment</code> <code>violation_comment</code>	不包含匹配给定模式的违例注释。
<code>-select_cell cell</code>	仅包含匹配给定模式的单元。此命令行选项不包含匹配单元下方任意层次上放置的任何单元。
<code>-unselect_cell cell</code>	不包含匹配给定模式的单元。此命令行选项不排除匹配单元下方任意层次上放置的任何单元。
<code>-select_cell_hierarchical cell</code>	仅包含匹配给定模式的单元。此命令行选项也包含匹配单元的全部后代。
<code>-unselect_cell_hierarchical cell</code>	不包含匹配给定模式的单元。此命令行选项也排除匹配单元的全部后代。
<code>-select_interacting</code> <code>cell,x1,y1,x2,y2</code>	仅包含给定单元中与给定窗口相交的错误。此选项为单元级, 不适用于给定单元层次下方的任何错误。窗口选择会合并, 使得在某单元中被任一窗口选择包含、且不匹配任何取消选择的错误被包含。
<code>-unselect_interacting</code> <code>cell,x1,y1,x2,y2</code>	排除给定单元中与给定窗口相交的错误。此选项为单元级, 不适用于给定单元层次下方的任何错误。
<code>-select_enclosed cell,x1,y1,x2,y2</code>	仅包含给定单元中被给定窗口包围的错误。此选项为单元级, 不适用于给定单元层次下方的任何错误。窗口选择会合并, 使得在某单元中被任一窗口选择包含、且不匹配任何取消选择的错误被包含。
<code>-unselect_enclosed cell,x1,y1,x2,y2</code>	排除给定单元中被给定窗口包围的错误。此选项为单元级, 不适用于给定单元层次下方的任何错误。
<code>-select_classified</code> <code>Waive,Ignore,Error,...</code>	仅包含具有给定分类的错误。
<code>-unselect_classified</code> <code>Waive,Ignore,Error,...</code>	不包含具有给定分类的错误。
<code>-sln layer_name /</code> <code>-select_layer_names layer_name</code>	仅指定源自匹配给定模式的 <code>runset</code> 层名的错误。仅当 IC Validator 以 <code>-vue</code> 命令行选项运行时, 层信息才可用。
<code>-uln layer_name /</code> <code>-unselect_layer_names layer_name</code>	指定不包含源自给定层的错误。仅当 IC Validator 以 <code>-vue</code> 运行时, 层信息才可用。
<code>-sl / -select_layers</code> <code>layernum[-range][,datatype[-range]]</code> ...	仅指定源自给定层的错误。对 OpenAccess 版图, 层由版图层与 <code>purpose</code> 名指定, 而非层与 <code>datatype</code> 号。仅当以 <code>-vue</code> 运行时层信息才可用。
<code>-ul / -unselect_layers</code> <code>layernum[-range][,datatype[-range]]</code> ...	指定不包含源自给定层的错误。对 OpenAccess 版图, 层由版图层与 <code>purpose</code> 名指定。仅当以 <code>-vue</code> 运行时层信息才可用。
<code>-set / -select_explorer_tiers</code> <code>tier_number[-range] ...</code>	仅包含属于给定 Explorer tier 编号的错误。

表 15.1: 错误管理实用程序命令行选项（续）

命令行选项	定义
<code>-uet / -unselect_explorer_tiers tier_number[-range] ...</code>	不包含属于给定 Explorer tier 编号的错误。
<code>-svn / -select_violation_name violation_name</code>	仅包含匹配给定模式的违例名。违例名对应 runset 中 @= 的左侧。
<code>-uvn / -unselect_violation_name violation_name</code>	不包含匹配给定模式的违例名。违例名对应 runset 中 @= 的左侧。
<code>-exclude_heat_map</code>	不将热图复制到输出数据库。若输入与输出数据库中同一单元的热图网格不对齐，可用此选项以避免错误。
<code>-exclude_error_classification</code>	不将错误分类复制到输出数据库。
<code>-rename_cells rename_cells_file</code>	根据给定 CSV 文件更改输出中的单元名，第一列为旧单元名，第二列为新单元名。若未指定输出数据库，可用于就地修改输入数据库。就地修改时，要求无其他用户连接到该数据库。
<code>-rename_violations rename_violations_file</code>	根据给定 CSV 文件更改输出中的违例注释，第一列为旧注释，第二列为新注释。若未指定输出数据库，可用于就地修改输入数据库。就地修改时，要求无其他用户连接到该数据库。
<code>-new_top_cell orig_cell,new_cell[,x,y,angle,reflected]</code>	指定与源数据库不同的目标顶层单元，并可选地提供从源顶层单元到目标顶层单元的变换。若新顶层单元在层次中位于原顶层单元之下，则不需要变换。若新单元在层次中位于原顶层单元之上，则需给出原单元到新顶层单元的单实例变换。源数据库到顶层的实例变换会更新到新顶层单元上下文，以便在新顶层单元中高亮违例。可选地，可提供旧顶层单元在新顶层单元内的平坦放置计数，以在新顶层单元上下文中保持平坦错误计数准确。若未提供平坦放置计数，默认为一次放置。
<code>-classify classify file</code>	根据给定输入文件对输入数据库中的错误分类。此选项仅可用于就地修改 PYDB。更多信息见“分类文件格式”一节。
<code>-classify_by_window [interacting enclosed]</code>	指定如何根据分类文件中的窗口坐标选择错误以进行分类。默认为 interacting 。 interacting : 与窗口坐标有任何相交的错误被选中分类。 enclosed : 完全被窗口坐标包围的错误被选中分类。所选错误可触及包围窗口。
<code>-report_user_comments All most_recent</code>	指定 JSON waiver 报告中包含哪些用户注释。默认为 All。
<code>-set_waiver_context cellname ALL</code>	用 CSV 文件分类错误时，将给定单元设为正在分类错误的 waiver 上下文。这意味着仅当该单元是正在检查设计的顶层单元时才应用 waivers。这通常是仅适用于 block 级的 waivers 在 block 级运行中的顶层单元。若指定 ALL，则始终应用 waivers。
<code>-set_waiver_properties user_property_file.json</code>	从给定 JSON waiver 属性文件设置 waiver 用户属性。此操作仅可用于就地修改 cPYDB 或 OASIS waiver 数据库。
<code>-waiver_property_definition_file property_definition_file.json</code>	指定用于验证 waiver 用户属性的 JSON 文件。覆盖环境变量 ICV_WAIVER_PROPERTY_DEFINITION_FILE。见“自定义 Waiver 属性”。
<code>-select_waiver_properties</code>	过滤给定 waiver 数据库，仅包含与给定 JSON 属性选择文件中指定的 waiver 用户属性匹配的 waivers。此操作仅可用于就地修改 cPYDB 或 OASIS waiver 数据库。

表 15.1: 错误管理实用程序命令行选项（续）

命令行选项	定义
<code>-rule_name_delimiter</code>	指定规则名分隔符正则表达式，用于在按规则名而非完整违例注释选择时（例如处理 <code>waiver</code> 属性时）将规则名与违例注释其余部分分开。规则名分隔符默认为 <code>/[[:space:]]+:[[:space:]]*:[[:space:]]/</code> 。
<code>-replace_user_ids new_user_id</code>	将输入 cPYDB 或 OASIS waiver 数据库中的全部用户 ID 全局替换为给定值。此操作仅可用于就地修改 cPYDB 或 OASIS waiver 数据库，并要求无其他用户连接到该数据库。
<code>-delete_classification_comments</code>	删除输入 cPYDB 或 OASIS waiver 数据库中的全部分类注释。此操作仅可用于就地修改，并要求无其他用户连接到该数据库。
<code>-replace_db_path new_db_path</code>	将输入 cPYDB 中存储的数据库路径替换为给定值。此操作仅可用于就地修改 cPYDB，并要求无其他用户连接到该数据库。
<code>-rename_cpydb_paths pattern new_path</code>	将任何匹配给定模式的所引用 cPYDB 路径更改为指定的新 cPYDB 路径。仅更新所引用 cPYDB 的路径分量。此选项仅可用于就地修改 PYDB。
<code>-group_drc_errors</code>	为所选 DRC 错误确定唯一错误组。若违例标记形状与检查值相同（不论方向），则错误放入同一组。仅某些命令有资格分组。此过程替换任何现有分组。更多信息见“DRC 错误分组”。
<code>-minimum_errors_per_group</code>	调整构成一组所需的错误阈值。最小阈值为 3。见“DRC 错误分组”。
<code>-group_same_orientation</code>	将相同方向的错误分在一组。默认不论方向均分组。见“DRC 错误分组”。
输出选项	
<code>-o output db</code>	指定输出数据库路径。若未给出此选项，则就地修改输入数据库。若输出数据库不存在则创建；若存在，则除非指定 <code>-overwrite</code> ，否则将来自输入数据库的错误合并到其中。所有其他输出命令行选项适用于命令行上位于其前的输出数据库。对此数据库的输出仅来自自上次指定输出数据库以来、在命令行上位于其前的输入数据库。使用 <code>-format</code> 导出到版图时， <code>-o</code> 指定输出版图路径。
<code>-overwrite</code>	若输出数据库存在则覆盖。未指定时，默认行为是合并到现有输出数据库，或不存在时创建新输出数据库。
<code>-vue output VUE file</code>	创建指向输出 PYDB 的给定 VUE 文件。
<code>-vue_template input_vue_file</code>	使用给定输入 VUE 文件作为模板创建输出 VUE 文件。
<code>-dbu microns</code>	可选地以微米为单位指定输出数据库的数据库单位值。创建新数据库时使用该值；合并到现有数据库时可用于验证合并数据库的 DBU。未指定时，创建新输出数据库使用输入数据库的 DBU。导出到输出版图时，若给出此参数则用作输出版图的 DBU；若未给出，则使用原始输入版图的 DBU（可能比输入数据库的 DBU 更粗）。
导出选项	
<code>-format GDSII OASIS</code>	表示输出文件指从输入数据库导出的版图，并指定格式。
<code>-layout original_layout</code>	指定导出错误到版图时读取设计层次的原始版图。若未给出，版图路径取自 PYDB。无论是否给出此参数，布局导出流程均需要输入版图。
<code>-c top_cell</code>	指定版图顶层单元。若未给出，使用输入 PYDB 的顶层单元。

表 15.1: 错误管理实用程序命令行选项（续）

命令行选项	定义
<code>-violation_map violation mapping file</code>	指定将规则与命令映射到输出版图中层与 <code>datatype</code> 对的文件。此 CSV 格式文件包含下列列：“Violation Comment”, “Command”, “Layer”, “Datatype”。违例注释与命令列支持通配符。命令列可选，可留空；若存在，则仅当 <code>runset file:line:function</code> 匹配时选择错误。默认情况下，此文件中无条目的错误不输出。若未给出违例映射文件，则为规则分配任意层与 <code>datatype</code> 。
<code>-include_unmapped</code>	表示导出错误到版图时包含违例映射文件中无条目的错误。这些错误被分配尚未包含在违例映射文件中的任意层与 <code>datatype</code> 。
<code>-violation_map_output output violation mapping file</code>	写出包含导出中所用全部层/ <code>datatype</code> 对的违例映射文件。可在后续运行中复用该文件以再现相同的规则层与 <code>datatype</code> 选择。
<code>-command_property_number property number</code>	导出错误到版图时，将命令的 <code>runset file:line:function</code> 信息写到每个输出多边形的给定属性号。这可能显著增大输出版图尺寸。
<code>-density_property_number property number</code>	将密度违例属性以及未匹配密度 <code>waivers</code> 的属性约束写到每个输出多边形的给定属性号。这可能显著增大输出版图尺寸。
<code>-datatype_per_classification</code>	将不同分类的错误写到不同 <code>datatype</code> 。按下表将对应分类状态的值加到输出 <code>datatype</code> : Error/Unclassified=0, Ignore=1, Waive=2, Watch=3, Unmatched=4, Fixed=7, WaiverConflict=9。
<code>-host_init host1:#cores ... [hostN:#cores]</code>	为分布式操作分配主机。
一般选项	
<code>-list_cells</code>	显示输入数据库中的唯一单元列表。
<code>-list_cell_details</code>	显示输入数据库中的唯一单元列表，含平坦放置计数与实例变换等详情。
<code>-list_cell_descendants</code>	显示输入数据库中的唯一单元列表，含每个单元全部唯一后代的详细信息。
<code>-list_layers</code>	若 IC Validator 以 <code>-vue</code> 运行，显示数据库中定义的 <code>assign</code> 层列表。示例见表后。
<code>-list_violations</code>	显示输入数据库中的唯一违例注释列表。
<code>-include_layers</code>	若可用，在违例列表中包含层信息。示例见表后。
<code>-list_cpydbs</code>	显示用于匹配输入 PYDB 中错误分类的 <code>cPYDB</code> 数据库列表。
<code>-list_waiver_properties</code>	显示给定 <code>cPYDB</code> 或 <code>OASIS waiver</code> 数据库中已定义 <code>waiver</code> 属性的违例与单元对列表，并列 <code>waiver</code> 属性值。
<code>-check_waiver_database</code>	检查输入 <code>OASIS waiver</code> 数据库的合规性。
<code>-create_waiver_report</code>	为输入数据库创建 <code>WAIVER_REPORT</code> 文件。
<code>-include_waiver_details</code>	在 <code>WAIVER_REPORT</code> 文件中包含已 <code>waive</code> 错误的详情。覆盖 <code>runset</code> 设置 <code>waiver_options(report_waiver_details)</code> 。
<code>-include_unmatched_waivers</code>	在 <code>WAIVER_REPORT</code> 文件中包含未匹配 <code>waivers</code> 的详情。覆盖 <code>runset</code> 的 <code>report_waiver_details</code> 设置。
<code>-create_json_waiver_file</code>	创建 JSON <code>waiver</code> 详情文件。
<code>-password password</code>	指定对现有错误分类数据库写访问的口令。指定 <code>prompt</code> 可从命令行读取口令。
<code>-file command file</code>	使用给定 XML 命令文件控制工具操作。
<code>-help</code>	显示用法信息。
<code>-V</code>	打印实用程序版本。

-list_layers 示例:

```
icv_pydb -i run_details/pydb/PYDB_AD4FUL -list_layers
...
Layers defined in run_details/pydb/PYDB_AD4FUL:
tox (1,0)
poly (5,0)
well (31,0)
psel (14,0)
cont (6,0)
met1 (8,0)
met2 (10,0)
via (19,0)
```

-list_violations 与 -include_layers 示例:

```
icv_pydb -i run_details/pydb/PYDB_AD4FUL \
  -list_violations -include_layers
...
Violations in database run_details/pydb/PYDB_AD4FUL:
"Met1 Spacing must be >= 3"
  met1 (8,0)
"Met1 to Met2 spacing must be >= 1 and not touching "
  met1 (8,0)
  met2 (10,0)
"Met2 Spacing must be >= 3.0 and >5.0 for longedges >=50"
  met2 (10,0)
```

15.2 DRC 错误分组

DRC Error Grouping

错误数据库管理实用程序可将具有相同违例标记形状与属性的 DRC 错误放入组中。错误分组可在 VUE 中使用 Show Unique 与 Highlight Unique 菜单选项，仅显示每组的代表性错误。默认不论方向均进行分组。例如:

```
icv_pydb -i run_details/pydb/PYDB_TOPCELL -group_drc_errors
```

错误管理实用程序命令行选项见信息表 15.1。

15.3 拆分错误数据库

Splitting Error Databases

若设计中有多个 block:

TOP			
/		\	
Block1	Block2	Block3	
/ \	/ \	/ \	
A B	A C	B D	

结果可拆分为三个错误数据库（PYDB），每个子 block 一个。例如：

```
icv_pydb -i run_details/pydb/PYDB_TOP -select_cell_hierarchical "Block1"\
-o new_pydb/PYDB_Block1 -vue Block1.vue -vue_template TOP.vue

icv_pydb -i run_details/pydb/PYDB_TOP -select_cell_hierarchical "Block2"\
-o new_pydb/PYDB_Block2 -vue Block2.vue -vue_template TOP.vue

icv_pydb -i run_details/pydb/PYDB_TOP -select_cell_hierarchical "Block3"\
-o new_pydb/PYDB_Block3 -vue Block3.vue -vue_template TOP.vue
```

使用 `-select_cell_hierarchical` 命令行选项可自动拉入所选单元下方放置的任何后代单元。本例中，命令行选项拉入 Block1 下的 A 与 B、Block2 下的 A 与 C、Block3 下的 B 与 D。可用 `-select_cell` 或 `-unselect_cell` 在不包含子树的情况下单独包含或排除单元。或者，可用单条 `icv_pydb` 命令指定此操作：

```
icv_pydb -i run_details/pydb/PYDB_TOP -select_cell_hierarchical "Block1"\
-o new_pydb/PYDB_Block1 -vue Block1.vue -vue_template TOP.vue \
-i run_details/pydb/PYDB_TOP -select_cell_hierarchical "Block2" \
-o new_pydb/PYDB_Block2 -vue Block2.vue -vue_template TOP.vue \
-i run_details/pydb/PYDB_TOP -select_cell_hierarchical "Block3" \
-o new_pydb/PYDB_Block3
```

本例中，每个输出数据库的顶层单元仍为 TOP。要使较低层 block 成为顶层单元，可使用 `-new_top_cell` 命令行选项。

```
icv_pydb -i run_details/pydb/PYDB_TOP -select_cell_hierarchical \
-new_top_cell TOP,Block1 \
-o new_pydb/PYDB_Block1 -vue Block1.vue -vue_template TOP.vue
```

也允许按违例注释与包围盒选择。例如，要在无单元选择的情况下仅拆出 `metal1` 规则，可如下运行工具：

```
icv_pydb -i run_details/pydb/PYDB_TOP -svc "M1" \
-o new_pydb/PYDB_TOP_m1 -vue TOP_m1.vue -vue_template TOP.vue
```

但请注意，本例中位于顶层的任何热图不会被复制，因为 TOP 在输出数据库中不再存在。

由于 PYDB 不含单元放置数据，包围盒选择仅在单元内有效，不会沿层次向下查看。例如，当 `block1` 放置范围为 (100,200)–(300,400) 时，要包含 `block1` 上方的顶层数据，可如下运行：

```
icv_pydb -i run_details/pydb/PYDB_TOP \
-select_cell_hierarchical "Block1" \
-select_interacting TOP,100.0,200.0,300.0,400.0 \
-o new_pydb/PYDB_Block1 -vue Block1.vue -vue_template TOP.vue
```

注意，当设置 `error_options(flatten_violations)` 时，在整个层次可用的 IC Validator 运行期间，允许将违例展开到指定 block。将 `flatten_violations` 参数与包围盒选择结合使用，可对错误进行地理分区。

也可使用若干命令行选项按错误分类进行选择，以包含或排除某些分类的错误，以及从输出中丢弃错误分类数据。

15.4 合并错误数据库

Merging Error Databases

`icv_pydb` 实用程序可将多个错误数据库（PYDB）合并为一个，例如当子 block 或不同规则集分别运行时。在合并具有相同顶层单元的多个 PYDB 的简单情况下，可对每个输入数据库运行一次 `icv_pydb`。例如：

```
icv_pydb -i metal1run/run_details/pydb/PYDB_TOP -o merged_pydb/PYDB_TOP
icv_pydb -i metal2run/run_details/pydb/PYDB_TOP -o merged_pydb/PYDB_TOP
...
```

若 merged_pydb/PYDB_TOP 不存在则创建，后续运行将数据合并到现有数据库。或者，可在命令行上组合：

```
icv_pydb -i metal1run/run_details/pydb/PYDB_TOP \
-i metal2run/run_details/pydb/PYDB_TOP \
-o merged_pydb/PYDB_TOP
```

合并具有不同顶层单元的多个数据库时，提供到共同顶层单元的（单实例）变换，以便 VUE 中的“highlight in top cell”正常工作。可用 -new_top_cell 命令行选项完成。例如：

```
icv_pydb -i block1run/run_details/pydb/PYDB_Block1 \
-new_top_cell Block1,TOP,100,100,0,0 -o merged_pydb/PYDB_TOP
icv_pydb -i block2run/run_details/pydb/PYDB_Block2 \
-new_top_cell Block2,TOP,200,100,0,0 -o merged_pydb/PYDB_TOP
```

被合并的 PYDB 可能处于不同的 DBU 分辨率。合并到现有 PYDB 时，icv_pydb 工具始终使用目标数据库的 DBU。不过，可在命令行指定 DBU，以便创建新 PYDB 时使用指定 DBU，合并到现有 PYDB 时可验证现有数据库的 DBU。

-dbu 命令行选项允许以微米为单位的浮点值给出 DBU。若合并到现有数据库且其 DBU 与命令行给定值不同，则发出错误并退出工具。

注意

合并期间不合并 runset 函数。若同一违例中两个相似的 runset 函数被合并，它们将作为独立函数存在于目标数据库中。

15.5 VUE 文件支持

VUE File Support

创建新数据库时（无论通过拆分还是合并），可通过指定 -vue 命令行选项输出指向新数据库的 VUE 文件。也可使用对应的 -vue_template 命令行选项指定输入 VUE 文件作为模板，仅替换数据库名。否则，相关信息取自输入 PYDB。

15.6 就地修改数据库

Modifying Databases in Place

支持某些操作就地修改 PYDB 或 cPYDB，而无需目标数据库。这些操作为：

```
-rename_cells rename_cells_file
-rename_violations rename_violations_file
-classify classify.csv
```

对就地修改操作，仅用 -i 命令行选项指定输入 PYDB 或 cPYDB，没有输出数据库。

注意

-classify 命令行选项仅支持 PYDB，不支持 cPYDB。

15.7 层次考虑

Hierarchy Considerations

PYDB 为每个唯一单元存储单元后代信息，包括其全部后代的实例变换。这使得拆分数据库时能正确更新到顶层的实例变换，以支持 “Highlight In Top Cell” 功能。

常见用法是完全拆出子 block，并使其成为目标 PYDB 的新顶层单元。例如：

```
icv_pydb -i run_details/pydb/PYDB_TOP -o new_pydb/PYDB_Block1 \
  -select_cell_hierarchical "Block1" -vue Block1.vue -vue_template \
    TOP.vue \
  -new_top_cell TOP,Block1
```

在此层次示例中，创建以 Block1 为顶层单元的新 PYDB，并选择 Block1 及其下方的全部违例。由于 PYDB 具有全部单元后代信息，当新顶层单元位于原顶层单元之下时不需要变换。若新顶层单元位于原顶层单元之下，且单元选择包含新顶层单元之上的任何内容，则发出错误。

15.8 XML 命令文件

XML Command File

XML 命令文件允许使用命令行上可用的任何选择，以及按 ID 选择个别错误。与使用命令行选项类似，XML 文件可为每个源数据库指定多个源数据库与不同选择。文件可 gzip 压缩。

必须遵循全部标准 XML 格式规则。特别要注意：所有参数值必须用引号括起，尖括号、与号或引号等特殊字符必须转义。示例 15.1 展示 XML 命令文件的语法。

```
<?xml version="1.0"?>
<pydb>
  <pydb_operation>
    <input_pydb path="run_details/pydb/PYDB_AD4FUL"
      field_selection_mode="UNION|INTERSECTION">
      <select_cell pattern="cellA*"/>
      <unselect_cell pattern="cellB*"/>
      <select_cell_hierarchical pattern="cellA*"/>
      <unselect_cell_hierarchical pattern="cellB*"/>
      <select_violation_comment pattern="Rule1*"/>
      <unselect_violation_comment pattern="Rule2*"/>
      <select_enclosed cell="cellname" x1="1.0" y1="2.0" x2="3.0"
        y2="4.0"/>
      <unselect_enclosed cell="cellname" x1="1.0" y1="2.0" x2="3.0"
        y2="4.0"/>
      <select_interacting cell="cellname" x1="1.0" y1="2.0" x2="3.0"
        y2="4.0"/>
      <unselect_interacting cell="cellname" x1="1.0" y1="2.0" x2="3.0"
        y2="4.0"/>
      <select_classified>
        <class name="Waive"/>
        ...
      </select_classified>
      <unselect_classified>
```

```

    <class name="Waive"/>
    ...
</unselect_classified>
<select_layer_names pattern="m1*"/>
<unselect_layer_names pattern="m2*"/>
<select_layers range="10-11,0-55"/>
<unselect_layers range="10-11,0-255"/>
<select_explorer_tiers range="<range>"/>
<unselect_explorer_tiers range="<range>"/>
<rename_cell old="cellA" new="cellB"/>
<rename_cells file="rename_cells.csv"/>
<rename_violation old="Rule1" new="Rule2"/>
<rename_violations file="rename_violations.csv"/>
<top_cell old="top1" new="top2" dx="1.0" dy="2.0" angle="90"
  reflected="1" flat_placement_count="1"/>
<!-- Select individual errors by ID -->
<select_errors command_id="1" cell_id="2">
  <error id="1"/>
  <error id="2"/>
  ...
</select_errors>
<!-- No error IDs given == all errors selected -->
<select_errors command_id="2" cell_id="2"/>
<!-- No error IDs given, no cell ID given == entire command selected -->
<select_errors command_id="3"/>
<!-- Select errors by exact value of a field -->
<select_errors field="Instance Name" value="I123"/>
<select_errors field="Distance" value="4.5"/>
<!-- Select errors by wildcard pattern -->
<select_errors field="Instance Name" pattern="I4*"/>
<!-- Select errors based on a range of values -->
<select_errors field="Distance" min="1.0" max="2.0"/>
<!-- Selection conditional on another field for commands like density -->
<select_errors field="Value" min="1.0" max="2.0"
  when_field="Report" is="Area"/>
<!-- Classify errors -->
<classify_errors class="Waive" comment="Waive errors in IP block"
  field_selection_mode="UNION|INTERSECTION">
<!-- Any selection options may go here -->
</classify_errors>
<exclude_heat_map/>
<exclude_error_classification/>
</input_pydb>
...
<output_pydb path="/path_to_pydb" dbu="0.0001">
  <overwrite/>
  <vue file="output.vue" template="inblock.vue"/>
</output_pydb>
<output_layout file="output_layout_file" format="GDSII|OASIS">
  <violation_map file="mapping_file" out="output_file"/>

```



```

    <include_unmapped/>
    <input_layout file="input_layout_file" cell="top_cell"/>
    <command_property number="property_number"/>
    <host_init host="host1:#cores"/>
    <host_init host="hostN:#cores"/>
  </output_layout>
</pydb_operation>
...
</pydb>

```

代码 15.1: XML 命令文件语法

XML 命令文件语法定义如下:

- **<pydb>**
命令文件 XML 文档的顶层包围标签。必须有且仅有一个 **<pydb>** 标签，其余内容均包在其中。
- **<pydb_operation>**
单次数据库操作（例如拆分、合并或复制）的顶层包围标签。文件中可有多多个 **pydb_operation** 标签。
- **<input_pydb path="path/to/pydb" field_selection_mode="UNION|INTERSECTION">**
指定输入 PYDB 数据库，与命令行上的 **-i** 相同。此节内的全部选择与命令行选项仅适用于给定输入数据库。若未指定输出数据库，可就地修改输入 PYDB。单个 **pydb_operation** 中可指定多个输入数据库。
可选属性 **field_selection_mode** 控制基于字段值的 **<select_errors>** 标签如何组合。默认为 **UNION**，表示满足任一 **<select_errors field...>** 准则的错误均被包含。若指定 **INTERSECTION**，则仅包含满足全部 **<select_errors field...>** 准则的错误。
 - **<select_layer_names pattern="m1*"/>**: 仅包含匹配给定模式名（允许通配符）的层名。
 - **<unselect_layer_names pattern="m2*"/>**: 不包含匹配给定模式名的层名。
 - **<select_layers range="10-11,0-255"/>**: 仅包含匹配给定范围的层。
 - **<unselect_layers range="10-11,0-255"/>**: 不包含匹配给定范围的层。
 - **<select_cell pattern="cellA*"/>**: 仅包含匹配给定模式的单元。等价于 **-select_cell**，可多次指定。
 - **<unselect_cell pattern="cellA*"/>**: 不包含匹配给定模式的单元。等价于 **-unselect_cell**，可多次指定。
 - **<select_cell_hierarchical pattern="cellA*"/>**: 仅包含匹配给定模式的单元及其后代。等价于 **-select_cell_hierarchical**，可多次指定。
 - **<unselect_cell_hierarchical pattern="cellA*"/>**: 不包含匹配给定模式的单元或其后代。等价于 **-unselect_cell_hierarchical**，可多次指定。
 - **<select_violation_comment pattern="..."/>**: 仅包含匹配给定模式的违例注释。等价于 **-svc**，可多次指定。
 - **<unselect_violation_comment pattern="..."/>**: 不包含匹配给定模式的违例注释。等价于 **-uvc**，可多次指定。
 - **<select_enclosed .../>**: 在给定单元中，仅包含被给定窗口坐标包围的错误。等价于 **-select_enclosed**，可多次指定。
 - **<unselect_enclosed .../>**: 在给定单元中，不包含被给定窗口坐标包围的错误。等价于 **-unselect_enclosed**，可多次指定。
 - **<select_interacting .../>**: 在给定单元中，仅包含与给定窗口坐标相交的错误。等价于 **-select_interacting**，可多次指定。
 - **<unselect_interacting .../>**: 在给定单元中，不包含与给定窗口坐标相交的错误。等价于 **-unselect_interacting**，可多次指定。

- `<select_classified> ... </select_classified>`: 仅包含匹配给定分类列表的错误。等价于 `-select_classified`。
- `<unselect_classified> ... </unselect_classified>`: 不包含匹配给定分类列表的错误。等价于 `-unselect_classified`。
- `<rename_cell old="cellA" new="cellB"/>`: 重命名此输入数据库中的单个单元。可多次指定。
- `<rename_cells file="rename_cells.csv"/>`: 根据给定 CSV 文件重命名单元。等价于 `-rename_cells`。
- `<rename_violation old="Rule1" new="Rule2"/>`: 重命名此输入数据库中的单个违例。可多次指定。
- `<rename_violations file="rename_violations.csv"/>`: 根据给定 CSV 文件重命名违例。等价于 `-rename_violations`。
- `<top_cell old="top1" new="top2" dx="1.0" dy="2.0" angle="90" reflected="1"/>`: 指定与源数据库不同的目标顶层单元，并提供从源顶层单元到目标顶层单元的变换。若合并到具有不同顶层单元的 PYDB，此标签必需。源数据库到顶层的实例变换会更新到新顶层单元上下文，以便在新顶层单元中高亮违例。
- `<select_errors command_id="1" cell_id="2"> ... </select_errors>`: 使用源数据库中的数值 ID 选择要包含的个别错误。若 `<select_errors>` 记录内无 `<error>` 记录，则包含该命令/单元的全部错误。若未指定 `cell_id`，则包含该命令的全部错误。此标签可多次指定。
- `<select_errors field="Instance Name" value="I123"/>`: 包含给定字符串、整数或双精度字段等于给定值的错误。字段名应为 LAYOUT_ERRORS 文件中的列标题。基于字段的选择选项按其所在 `<input_pydb>` 或 `<classify_errors>` 节的 `field_selection_mode` 属性组合。
- `<select_errors field="Instance Name" pattern="I4*"/>`: 包含给定字符串字段匹配给定通配符模式的错误。
- `<select_errors field="Distance" min="1.0" max="2.0"/>`: 包含给定双精度或整数字段落在给定范围内的错误。
- `<select_errors field="Value" min="1.0" max="2.0" when_field="Report" is="Area"/>`: 包含满足给定选择准则的错误。添加 `when_field` 与 `is` 属性使此选择以该行中另一值为条件。对 `density()` 等函数很有用：单个错误可能有许多 Report 与 Value 对，选择必须仅针对特定 Report。上例选择 Area 在 1.0-2.0 范围内的密度错误。
- `<classify_errors class="Waive" comment="..." field_selection_mode="UNION|INTERSECTION"> ... </classify_error>`
按给定类别对此输入 PYDB 中的错误分类，并应用可选用户注释。可在 `<classify_errors>` 节内指定任何错误选择选项，并与节外选择选项组合。单个 `<input_pydb>` 节中可指定多个 `<classify_errors>` 节。可选 `field_selection_mode` 控制该节内基于字段的 `<select_errors>` 如何组合；默认为 UNION。
- `<exclude_heat_map/>`: 指定不从输入数据库向输出数据库复制热图数据。等价于 `-exclude_heat_map`。
- `<exclude_error_classification/>`: 指定不从输入数据库向输出数据库复制错误分类数据。等价于 `-exclude_error_classification`。
- `<output_pydb path="path/to/pydb" dbu="0.0001">`
指定输出 PYDB 数据库，与命令行上的 `-o` 相同。`dbu` 参数可选，等价于 `-dbu`。单个 `pydb_operation` 内最多可指定一次 `output_pydb`。
 - `<overwrite/>`: 若输出数据库存在则覆盖；否则合并到现有输出数据库。等价于 `-overwrite`。
 - `<vue file="output.vue" template="inlib.vue"/>`: 创建指向新输出数据库的给定 VUE 文件。等价于 `-vue`。可选 `template` 参数指定用作模板的现有 VUE 文件。
- `<output_layout file="output_layout_file" format="GDSII|OASIS">`
指定将来自输入 PYDB 的违例导出到由给定文件与格式指定的输出版图。
 - `<violation_map file="mapping_file" out="output_file"/>`: 为此输出版图指定违例映射文件或输出违例映射文件。
 - `<include_unmapped/>`: 若使用违例映射文件，指定映射文件中无条目的违例包含在输出中，位于任意分配的层与 datatype 上。
 - `<input_layout file="input_layout_file" cell="top_cell"/>`: 指定从中读取设计层次的输入版图。
 - `<command_property number="property_number"/>`: 指定将 `runset file:line:function` 以给定属性号添加到输出版图中的每个多边形。这可能显著增大输出版图尺寸。

– `<host_init host="host1:#cores"/>`: 为分布式操作分配主机。

15.8.1 用 XML 命令文件分类错误

Classifying Errors With the XML Command File

XML 命令文件提供使用 `icv_pydb` 工具分类错误的方式。可用于其他 `icv_pydb` 操作的任何选择选项也可用于选择待分类错误。示例 15.2 说明若干常见 waiver 场景。

```
<?xml version="1.0"?>
<pydb>
  <pydb_operation>
    <input_pydb path="run_details/pydb/PYDB_AD4FUL">

      <!-- Waive all the errors for an entire rule -->
      <classify_errors class="Waive" comment="Waiver for rule M1.R1">
        <select_violation_comment pattern="M1.R1*"/>
      </classify_errors>

      <!-- Ignore all the errors in particular cells -->
      <classify_errors class="Ignore" comment="Ignoring errors in IP cells">
        <select_cell pattern="MYIP*"/>
      </classify_errors>

      <!-- Classify errors in a window -->
      <classify_errors class="Watch" comment="Fix in next rev">
        <select_enclosed cell="TOP" x1="0.0" y1="0.0" x2="10.0"
          y2="5.0"/>
      </classify_errors>

      <!-- Waive metal1 spacing errors based on Distance -->
      <classify_errors class="Waive" comment="Waive these">
        <select_layer_names pattern="metal1*"/>
        <select_errors field="Distance" min="1.0" max="2.0"/>
      </classify_errors>

      <!-- Waive density errors based on Area -->
      <classify_errors class="Waive" comment="Accepted density values">
        <select_violation_comment pattern="DN*"/>
        <select_errors field="Value" min="0.0" max="0.5"
          when_field="Report" is="Area"/>
      </classify_errors>

      <!-- Waive PERC errors based on combination of device name and comment -->
      <classify_errors class="Waive" comment="Waiving M1 path errors"
        field_selection_mode="UNION">
        <select_violation_comment pattern="Rule_1*"/>
        <select_errors field="Device name (Type)" pattern="*/M1 */>
        <select_errors field="Comment"
          pattern="Gate has no path to power, ground, or top level port"/>
      </classify_errors>
```

```

<!-- Waive PERC errors with certain instance names -->
<classify_errors class="Waive" comment="Waiving specific instances">
  <select_violation_comment pattern="Rule_2*" />
  <!-- The default field_selection_mode is "UNION"-->
  <select_errors field="Reference Instance (Cell Name)"
    pattern="XI123 *" />
  <select_errors field="Reference Instance (Cell Name)"
    pattern="XI456 *" />
</classify_errors>

</input_pydb>
</pydb_operation>
</pydb>

```

代码 15.2: 常见 Waiver 场景

15.9 将错误导出到版图

Exporting Errors to a Layout

`icv_pydb` 实用程序可将一个或多个错误数据库 (PYDB) 中的错误导出到 GDSII 或 OASIS 版图文件。支持 `icv_pydb` 提供的全部选择选项。必须从用于 IC Validator 运行的原始输入版图读取设计层次。层与 `datatype` 号可由违例映射文件指定，或任意分配。

违例映射文件为 CSV 文件，含下列列：

- **Violation comment** (必需)：违例或规则注释的完整文本。支持通配符匹配。
- **Command** (可选)：给定规则内特定 IC Validator 命令的 `runset file:line:function`。支持通配符匹配。若此列为空，则匹配该违例或规则内的全部命令。
- **Layer** (必需)：用于输出匹配违例的层号。
- **Datatype** (必需)：用于输出匹配违例的 `datatype` 号。

若给出违例映射文件，则仅导出映射文件中列出的规则与命令。要包含映射文件中未列出的规则与命令，使用 `-include_unmapped` 命令行选项。

若未给出输入版图，`icv_pydb` 尝试使用 PYDB 中存储的原始版图路径读取设计层次。

```

icv_pydb -i run_details/pydb/PYDB_AD4FUL \
  -o errors.oas -format OASIS -layout AD4FUL.GDS -c AD4FUL \
  -violation_map violation_map.csv

```

代码 15.3: 导出到 OASIS 的示例

```

icv_pydb -i run_details/pydb/PYDB_AD4FUL \
  -svc "Met1*" -select_cell "AD4FUL" \
  -o errors.oas -format OASIS \
  -violation_map_output violation_map_used.csv

```

代码 15.4: 带选择并创建含所用层与 `datatype` 的参考违例映射文件

```

icv_pydb -i run_details/pydb/PYDB_AD4FUL \
  -violation_map_output violation_map.csv

```

代码 15.5: 从 PYDB 创建含任意分配层与 datatype 号的违例映射文件

15.10 管理所引用的 cPYDB 路径

Managing Referenced cPYDB Paths

当错误分类数据库（cPYDB）用于在 IC Validator 运行期间应用错误分类时，错误数据库（PYDB）包含指向该 cPYDB 的链接。分类注释与时间戳等部分详情不存储在 PYDB 中，而是在需要时由 VUE 等工具从所引用的 cPYDB 读取。若 cPYDB 不在原位置，这些详情可能缺失。icv_pydb 实用程序可编辑所引用 cPYDB 的路径，以便 VUE 能加载这些分类详情。

```
icv_pydb -i run_details/pydb/PYDB_AD4FUL -list_cpydbs
```

...

cPYDBs referenced by run_details/pydb/PYDB_AD4FUL:

Path: /global/cpydb1
Name: CPYDB1

Path: /global/cpydb1
Name: CPYDB1A

Path: /global/cpydb2
Name: CPYDB2

代码 15.6: 列出 PYDB 所引用的 cPYDB

```
icv_pydb -i run_details/pydb/PYDB_AD4FUL \  
-rename_cpydb_paths "*cpydb1" "/new/path/cpydb1" \  
-rename_cpydb_paths "*cpydb2" "/new/path/cpydb2"
```

...

Renamed cPYDBs:

Old Path: /global/cpydb1
New Path: /new/path/cpydb1
Name: CPYDB1

Old Path: /global/cpydb1
New Path: /new/path/cpydb1
Name: CPYDB1A

Old Path: /global/cpydb2
New Path: /new/path/cpydb2
Name: CPYDB2

代码 15.7: 使用 -rename_cpydb_paths 更新 cPYDB 路径

Chapter 16

图案匹配

Pattern Matching

本章介绍 IC Validator 图案匹配（**pattern matching**）。图案匹配相关函数的详细说明见 *IC Validator Reference Manual*。

IC Validator 图案匹配功能按下列各节说明：

- 概述
- 使用图案匹配流程
- 创建图案库
- 查看与锁定图案库
- 检查版图
- 在 IC Compiler 工具中检测并修复热点

16.1 概述

Overview

IC Validator 图案匹配比较两种几何构型（图案），以判定它们是否匹配。在图案匹配过程中，将来自设计版图的目标图案（**target pattern**）与来自既有图案库的参考图案（**reference pattern**）进行比对，以判定图案是否匹配。图 16.1 示意该步骤。

参考图案表示任意无法制造的几何形状组合（例如光刻热点），或难以用标准 DRC 规则准确解释的构型（例如复杂的二维几何度量）。参考图案通过稳健的验证与确认技术预先识别，并在匹配前被精确指定并加入图案库。

图案匹配作为基于 DRC 的流程，其优势在于远快于基于仿真的光刻检测——后者计算代价极高且耗时。因此，图案匹配能以更经济的方式在设计流程早期识别光刻或工艺引入的热点，并使实时验证与修复成为可能。

16.2 使用图案匹配流程

Using the Pattern Matching Flow

要使用图案匹配流程，需以设计图案层（**design pattern layers**）与位置标记层（**location marker layers**）的形式提供一组源图案。这些层可在制造现场识别为良率损害源（**yield detractors**），也可在无晶圆厂公司用于方法论检查。图 16.2 给出用于图案库创建与图案匹配的顶层 IC Validator 图案匹配流程。

（原书 *Figure 105: Pattern Matching Flow*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 16.1: 图案匹配流程

（原书 *Figure 106: Pattern Matching Flowchart*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 16.2: 图案匹配流程图

在图案库创建期间，从源图案提取图案信息并写入图案库。图案匹配步骤调用该图案库作为参考，以识别输入设计版图中相同或相似的图案。

`pattern_learn()` 函数创建图案库。`pattern_match()` 函数执行图案匹配。全部图案匹配函数的摘要见 *IC Validator Reference Manual* 第 1 章「Tables of Runset Functions」。

16.3 创建图案库

Creating the Pattern Library

本节给出使用 `pattern_learn()` 函数创建图案库的流程示例。

16.3.1 准备输入版图

Preparing the Input Layout

以 GDSII 或 OASIS 格式捕获的源图案，通过图案定义过程登记到图案库；该过程包含用于精确描述图案的规范。

必需输入

Required Input 创建图案库需要三项用户定义输入。这些输入是输入版图的一部分，或在调用 `pattern_learn()` 之前由 `runset` 操作生成。图 16.3 给出必需输入示例。

- 图案层（**Pattern layers**）。指定图案包含单层或多层图案层，例如由金属层与通孔层组成的图案。
- 图案标记（**Pattern marker**）。指定放置在每个源图案上的矩形多边形，用于标记关注位置，例如 `pinch/bridge` 位置或 DRC 违例。图案匹配时输出该图案标记，以报告图案库与设计之间的匹配。
- 图案范围（**Pattern extent**）。指定图案的包围区域。该区域内的图案层被处理并登记到图案库。图案范围也可通过 `pattern_learn()` 函数的 `ambit_mode` 与 `match_ambit` 参数生成。

(原书 *Figure 107: Required Input for Creating a Pattern Library*)
请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 16.3: 创建图案库的必需输入

可选输入

Optional Input 图案定义可能还需提供下列规范。图 16.4 给出可选输入示例。

- 边容差层 (**Edge tolerance layers**)。指定一组多边形层, 定义每个图案层边所允许的边放置变化。
- 忽略区域层 (**Ignore region layers**)。包含一个或多个直线多边形 (`rectilinear polygons`), 定义源图案上从匹配过程中排除的区域。
- 可选图案标记层 (**Optional pattern marker layers**)。包含可附着到图案、并在某些图案匹配后操作中取回的多边形。例如, 可为某一特定图案的用户定义修复指导 (`fix guidance`)。
- 图案文本 ID 层 (**Pattern text ID layer**)。包含源图案的文本 ID。若未提供文本 ID 层, 则使用工具生成的文本 ID。
- 图案文本属性层 (**Pattern text property layers**)。包含为源图案定义的文本信息, 例如图案 ID、图案类型或最小 CD。在图案库创建时将这些属性附着到图案, 并在图案匹配后操作中使用 `select_marker_by_double_property()` 与 `select_marker_by_string_property()` 函数取回。

(原书 *Figure 108: Optional Input for Creating a Pattern Library*)
请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 16.4: 创建图案库的可选输入

附加图案规范

Additional Pattern Specifications 在图案库创建期间可能还需设置附加图案规范, 例如:

- 图案模糊度 (**Pattern fuzziness**)。指定图案匹配模式。可选择精确匹配 (`exact`) 或模糊匹配 (`fuzzy`)。图案模糊度基于边, 并可分别按边与全局指定。
- 图案反射与旋转 (**Pattern reflection and rotation**)。指定当图案取向与原始形状不同时, 是否仍报告为匹配。
- 图案类型 (**Pattern type**)。指定源图案是一维还是二维图案。不支持混合图案类型。

参见

关于 `pattern_learn()` 函数的更多信息, 见 *IC Validator Reference Manual*。

16.3.2 创建图案库示例

Example of Creating a Pattern Library

创建标准 DRC runset 文件以创建图案库。该 runset 必须包含下列各部分：

- 版图数据库相关设置
- 通用选项
- 使用 `pattern_options()` 函数定义图案库路径
- 层赋值
- 使用 `pattern_learn()` 函数创建图案库

用于创建图案库的图案匹配 runset 按下列步骤描述：

1. runset 开头

```
#include <icv.rh>
```

2. 用户定义环境变量

runset 中的环境变量为可选，可自定义。

图案库创建定义了两个环境变量。PATTERN_LIB_PATH 环境变量指定图案库的保存位置；PM_LIB_NAME 环境变量指定图案库名称。例如：

```
#pragma envvar default PM_LIB_PATH "/pattern_library"
#pragma envvar default PM_LIB_NAME "MET"
```

3. 版图数据库规范

library() 函数定义设计库，在 runset 中为必需：

```
library (
    format      = GDSII,
    library_name = "TOP.gds",
    cell        = "TOP"
);
```

可使用 IC Validator 命令行选项覆盖 library() 函数中指定的参数。例如，用 `-i` 覆盖 library_name 指定的库名，或用 `-f` 覆盖 format 指定的库格式。更多信息见本用户指南「IC Validator 基础」章中「IC Validator 命令行选项」一节。

4. 通用选项

在 error_options() 函数中将 error_limit_per_check 设为 ERROR_LIMIT_MAX，以存储并报告 pattern_learn() 与 pattern_match() 函数报告的全部违例，直至工具定义的最大上限。默认值为 100。

```
error_options (
    error_limit_per_check = ERROR_LIMIT_MAX
);
```

将 error_limit_per_check 设为 ERROR_LIMIT_MAX 或其他极高值，在错误数量极大时可能导致性能下降与磁盘占用增加，因此不推荐。

5. 调用 pattern_options() 定义图案库路径

pattern_options() 函数为 pattern_learn() 与 pattern_match() 定义图案库路径。pattern_options() 在 runset 中只能调用一次。更多信息见 IC Validator Reference Manual 的 Runset Functions 各章。

```
pattern_options (
    pattern_library_path = $PM_LIB_PATH
);
```

注意

`pattern_library_path` 参数指定的图案库路径必须在图案库创建之前已存在，否则会发生运行时错误。

6. 层赋值

```
metal : polygon_layer = assign ( {{3,0}} );
pattern_marker : polygon_layer = assign ( {{4,0}, {5,0}} );
pattern_text_id : text_layer = assign_text( {{6,0}} );
error_list : list of write_error_map_s = {};
```

7. 调用 `pattern_learn()` 创建图案库

- 定义图案库创建参数。

`runset` 的这一可选部分提供对 `pattern_learn()` 任意参数取值列表的访问。关于 `pattern_learn()` 参数取值的更多信息，见 *IC Validator Reference Manual* 中该函数的说明。

```
match_ambit : ambit_values_s = {0.2,0.2,0.2,0.2};
ambit_mode : pattern_ambit_mode_e = PM_MARKER_CENTER;
pattern_fuzziness : pattern_fuzzy_e = PM_EDGE_UNIFORM;
uniform_fuzzy_size : double = 0.004;
pattern_reflect : boolean = true;
pattern_rotate : boolean = true;
```

- 图案库创建

调用 `pattern_learn()` 创建图案库。可在 `runset` 中多次调用该函数。下列示例调用 `pattern_learn()`，以边均匀匹配模式创建单图案层库。图案范围为 $0.4 \times 0.4 \mu\text{m}$ ，由工具从图案标记中心、以 $0.2 \mu\text{m}$ 的 `match_ambit` 生成。

```
tmp_violation @= {
    @"pattern_learn result";
    pattern_learn (
        pattern_library_name = $PM_LIB_NAME,
        pattern_layers = {metal},
        pattern_marker = pattern_marker,
        pattern_text_id = pattern_text_id,
        match_ambit = match_ambit,
        ambit_mode = ambit_mode,
        pattern_fuzziness = pattern_fuzziness,
        uniform_fuzzy_size = uniform_fuzzy_size,
        pattern_reflect = pattern_reflect,
        pattern_rotate = pattern_rotate
    );
}
```

生成以 `PM_LIB_NAME` 环境变量命名的图案库目录，并保存在 `PM_LIB_PATH` 环境变量定义的路径下。

`pattern_learn()` 的错误输出存储为 `pattern_learn result`。关于该错误输出的更多信息，见「检查图案库创建输出」。

8. 输出版图数据库

在此可选部分，可将 `pattern_learn()` 的输出写入 GDSII 或 OASIS 文件。

```
//DEBUG output only
error_list.push_back({tmp_violation,{0,0}});
output_lib=gds_library("out.gds");
write_gds(
    output_library=output_lib,
    errors=error_list
);
```

16.3.3 检查图案库创建输出

Examining the Output of Pattern Library Creation

IC Validator runset 为图案库创建产生的输出文件包括图案库，以及与执行 DRC 时相同的一组文件。更多细节见本用户指南「输出文件」章。

- 错误输出

DRC 运行完成后，IC Validator 工具创建 `cell.LAYOUT_ERRORS` 文件。对于图案库创建，该文件用作图案创建统计的快速摘要，而非违例报告。

下列 `TOP.LAYOUT_ERRORS` 文件报告 ERROR 结果：

```

                                LAYOUT ERRORS RESULTS: ERRORS

##### ##### #####      ### #####      ###
#      #  #  #  #  #      #  #      #  #
##### ##### ##### #      # #####      ###
#      #  #  #  #  #      #  #  #      #
##### #  #  #  #  #  #  #      # #####

=====
Library name:   ../../testcase/hotspot_clips.oas
Structure name: TOP
Generated by:   IC Validator RHEL64 Release I-2013.12.SP1.4010 2014/
01/09

                                ERROR SUMMARY

Runset name: icv runset.rs
User name: juser
Time started: 2016/03/02 07:35:12AM
Time ended: 2016/03/02 07:35:51AM

Called as: icv runset.rs
pattern_learn result
    pattern_learn ..... 6 violations found.

pattern_learn result/Invalid input: first pattern
layer has no corner within the pattern extent
    pattern_learn ..... 1 violation found.

pattern_learn result:pattern_1
    pattern_learn ..... 4 violations found.
```

```
pattern_learn result:pattern_5
pattern_learn ..... 2 violations found.
```

在该报告中，`pattern_learn()` 处理了六个源图案。其中两个识别为 `pattern_5`，四个识别为 `pattern_1`。因此，图案库捕获了两个唯一图案。

`pattern_learn()` 还将一个图案识别为无效二维图案，因为其在图案范围内没有角点。因此，无法将其加入二维图案库。图案库创建后应检查 `cell.LAYOUT_ERRORS` 文件，以确定是否存在无效图案。

- 图案库

可在 `pattern_options()` 函数的 `pattern_library_path` 参数所定义的同图案库路径下存储多个图案库。图案匹配时可调用一个或多个图案库。

`pattern_learn()` 将图案信息写入二进制 `pattern.dat` 文件，并在 `pattern_library_path` 参数定义的同目录中输出日志文件。

例 16.1 给出为图案库 MET 生成的日志文件。

```
[Tue Nov 5 15:19:22 2013]: Created pattern library.
version= 1.0
dbu=0.000500
learn_version=3
num_pattern_layers=1
num_optional_marker_layers=0
ambit_mode=PM_MARKER_CENTER
match_ambit.left=0.200000
match_ambit.bottom=0.200000
match_ambit.right=0.200000
match_ambit.top=0.200000
pattern_fuzziness=PM_EDGE_UNIFORM
pattern_grouping=PM_NONE
uniform_fuzzy_size=0.005000
edge_jog_size=0.000000
has_user_anchor=false
has_pattern_extent_layer=false
ignore_extra_polygons=false
pattern_type=PM_TWO_DIMENSIONAL
pattern_length=PM_GE
viewable=true
writable=true

[Tue Nov 5 15:19:22 2013]: Learned new pattern(s).
Registered 2 new unique pattern(s); total # of unique pattern(s)=2.
Created reflected/rotated pattern(s); total # of pattern(s)=16.
Derived 16 index pattern(s) using ambit = (0.195000, 0.195000,
0.195000, 0.195000).
viewable=true; writable=true.
```

代码 16.1: 日志文件

在日志文件开头，可看到首次生成该图案库时所应用的参数及其对应值。向既有图案库添加新图案时，必须使用同一组参数与取值，否则会发生错误。

在日志文件末尾，可看到每次 `pattern_learn()` 调用所登记的唯一图案数量。

日志文件最后一行表明该图案库可写，即可以向其中添加新图案。若该图案库可查看（viewable），则可转换为 GDSII 或 OASIS 格式以便查看。

16.4 查看与锁定图案库

Viewing and Locking a Pattern Library

图案库以二进制格式存储。要查看其内容，需将其转换为 GDSII 或 OASIS 文件。

转换需要 `pattern_library_read()` 函数。如何将图案库转换为 GDSII 文件的示例，见 *IC Validator Reference Manual* 的 Runset Functions 章。

默认情况下，图案库可查看且可写。`pattern_library_lock()` 函数为图案库添加读写保护；锁定后，既不能由 `pattern_learn()` 添加新图案，也不能由 `pattern_library_read()` 转换为图形格式文件。下列行会追加到已锁定图案库的既有日志文件：`viewable=false; writable=false`。

随后将锁定的图案库交付给最终用户。如何锁定图案库的示例，见 *IC Validator Reference Manual* 第 3 章「Runset Functions: J - Z」。

也可以使用位于 IC Validator contrib 目录中的 `pdb_utility.pl` 脚本，在不执行 runset 的情况下完成 `pattern_library_read()` 与 `pattern_library_lock()` 的功能：

```
Usage: pdb_utility.pl <pdb_lib_path> <pdb_lib_name> -read/-lock
      [output_name]

Options:
<pdb_path>.....pattern library path
<pdb_name>.....pattern library name
-read.....converting a pattern library to a graphical
      output.
-lock.....adds the write and view protection to a pattern
      library.
<output_name>.....optional, specifies the graphical output file
      name, [default: pdb_view.gds]
```

下列示例将位于 `/test/pattern_lib` 目录的 `met` 图案库转换为输出 GDSII 文件 `hs.gds`：

```
ICV_INSTALL_DIRECTORY/contrib/pdb_utility.pl /test/pattern_lib met -read
hs.gds
```

16.5 检查版图

Checking a Layout

本节给出使用 `pattern_match()` 函数的图案匹配流程示例。

16.5.1 图案匹配 Runset 示例

Runset Example of Pattern Matching

创建标准 DRC runset 文件，使 IC Validator 工具对设计版图执行图案匹配。runset 文件必须包含下列各部分：

- 版图数据库相关设置
- 通用选项

- 使用 `pattern_options()` 函数定义图案库路径
- 层赋值
- 使用 `pattern_match()` 函数执行图案匹配

采用图案匹配流程的 `runset` 具有下列组成部分：

1. `runset` 开头

在 `runset` 开头包含 `icv.rh`：

```
#include <icv.rh>
```

2. 用户定义环境变量

`runset` 中的环境变量为可选，可自定义。

```
#pragma envvar default PM_LIB_PATH "/pattern_library"
#pragma envvar default PM_LIB_NAME "MET"
#pragma envvar default PM_TOP_MET "10"
```

3. 版图数据库相关设置

`library()` 函数定义图案匹配检查所用的设计库。

```
library (
    format      = GDSII,
    library_name = "TOP.gds",
    cell        = "TOP"
);
```

若使用 Milkyway 库，将 `format` 设为 `MILKYWAY`，并将 `library_path` 指定为 Milkyway 库的父目录。

4. 通用选项

要将 `pattern_match()` 函数生成的全部违例存储并报告至工具定义的最大上限，在 `error_options()` 函数中将 `error_limit_per_check` 设为 `ERROR_LIMIT_MAX`。默认值为 100。

```
error_options (
    error_limit_per_check = ERROR_LIMIT_MAX
);
```

将 `error_limit_per_check` 设为 `ERROR_LIMIT_MAX` 或其他极高值，在错误数量极大时可能导致性能下降与磁盘占用增加，因此不推荐。

5. 调用 `pattern_options()` 定义图案库路径

```
pattern_options (
    pattern_library_path = $PM_LIB_PATH
);
```

`pattern_options()` 定义图案库路径，且在 `runset` 中只能调用一次。该函数指定访问单个或多个图案库所需的路径。

6. 层赋值

```
MET1 : polygon_layer = assign ( {{15,0}} );
MET2 : polygon_layer = assign ( {{17,0}} );
MET3 : polygon_layer = assign ( {{19,0}} );
MET4 : polygon_layer = assign ( {{21,0}} );
MET5 : polygon_layer = assign ( {{23,0}} );
```

```

MET6 : polygon_layer = assign ( {{25,0}} );
MET7 : polygon_layer = assign ( {{27,0}} );
MET8 : polygon_layer = assign ( {{29,0}} );
MET9 : polygon_layer = assign ( {{31,0}} );
MET10 : polygon_layer = assign ( {{33,0}} );

metal_layer_list : list of polygon_layer = {MET1, MET2, MET3, MET4,
MET5, MET6, MET7, MET8, MET9, MET10};
metal_error_list : list of write_error_map_s = {};

```

本例中，MET1 至 MET10 被赋值并接受图案匹配检查。

7. 调用 `pattern_match()` 执行图案匹配

必须调用 `pattern_match()` 以执行图案匹配。该函数可在 `runset` 中多次调用。下列示例显示 `pattern_match()` 对 `TOP.gds` 文件的金属层执行图案匹配。

```

for(i = 0 to strtol($PM_TOP_MET)-1) {
    layer_index : integer = i + 1;

    tmp_violation @= {
        @"METAL"+layer_index+" PM RESULT: ";
        pattern_match (
            pattern_library_name = $PM_LIB_NAME,
            pattern_layers = {metal_layer_list[i]}
        );
    }
    out_layer_index = layer_index;
    metal_error_list.push_back({tmp_violation,
{out_layer_index,0}});
}

```

8. 输出版图数据库

可选地将 `pattern_match()` 的输出写入 GDSII 或 OASIS 文件。

```

output_lib=gds_library("out.gds");
write_gds(
    output_library=output_lib,
    errors=metal_error_list
);

```

16.5.2 检查图案匹配输出

Examining Pattern Matching Output

DRC 运行完成后，IC Validator 工具创建 `cell.LAYOUT_ERRORS` 错误文件。文件开头说明本次运行行为 `CLEAN` 或存在 `ERRORS`。

在图案匹配中，`ERRORS` 结果表示识别出一个或多个匹配图案；`CLEAN` 结果表示未找到匹配图案。

下列 `TOP.LAYOUT_ERRORS` 文件报告违例（匹配）：

```

LAYOUT ERRORS RESULTS: ERRORS
#####
# # # # # # # #

```

```

##### ##### # # #####
# # # # # # # #
##### # # # #####
=====
Library name:      ./test.gds
Structure name: TOP
Generated by:      IC Validator RHEL64 Release H-2013.06.SP2.4035 2013/08/
28

                        ERROR SUMMARY

METAL2 PM RESULT:
  pattern_match .....873 violations found.

METAL3 PM RESULT:
  pattern_match .....69 violations found.

METAL4 PM RESULT:
  pattern_match .....2 violations found.

                        ERROR DETAILS
-----
METAL2 PM RESULT:
-----
/match.rs:38:pattern_match
-----
Structure (lower left x, y ) (upper right x, y ) Pattern      Orientation
-----
TOP   (8.1600, 528.2500) (8.4000, 528.7200)  pattern_78 R90
TOP   (2.1400, 528.6700) (2.3800, 529.1400)  pattern_57 R90
TOP   (100.1450, 529.3000) (100.3850, 529.7700) pattern_107 R90FX
TOP   (100.1450, 529.2300) (100.3850, 529.7000) pattern_108 R90
TOP   (107.9800, 531.7500) (108.2200, 532.2200) pattern_80 R0
TOP   (83.7900, 534.7600) (84.0300, 535.2300) pattern_7      R180
TOP   (86.0000, 534.8300) (86.2400, 535.3000) pattern_94 R90FY
TOP   (87.6800, 540.2900) (87.9200, 540.7600) pattern_78 R90
.....

```

错误摘要部分包含 `pattern_match()` 在每个金属层上产生的匹配图案数量。错误详情部分列出每个匹配图案的图案标记坐标与图案 ID。

可像查看其他 DRC 违例一样，在 VUE 中审阅图案匹配结果。

16.5.3 图案匹配后操作

Post Pattern-Matching Operations

在图案匹配后处理中，使用下列函数按属性选择匹配图案，或取回某些图案的可选图案标记，以供后续 DRC 操作：

- `marker_merge()`。将标记层合并到多边形层。标记层本身不合并；只能由若干 IC Validator runset 函数处理。必须将标记层转换为多边形层，才能由其他 DRC 函数处理。
- `optional_pattern_markers()`。取回附着到某图案的全部可选图案标记。可选图案标记层为多边形层。
- `select_marker_by_string_property()`。按指定的字符串属性取回图案标记。

- `select_marker_by_double_property()`。按指定的双精度属性取回图案标记。
- `milkyway_route_directives()`。将用户定义的修复指导传递给布线器，用于热点修复。
- `drc_features_marker()`。输出标记层。

参见

各函数的示例见 *IC Validator Reference Manual*。

16.6 在 IC Compiler 工具中检测并修复热点

Detecting and Repairing Hotspots Within the IC Compiler Tool

为满足先进工艺节点的可制造性设计 (DFM) 要求，IC Validator 工具在实现环境中提供 In-Design DFM 技术。该物理验证方法可在 IC Compiler 工具内尽早检测并修复 DRC 违例，且对物理与时序影响可忽略。关于 Automatic DRC Repair (ADR) 流程的更多信息，见 Synopsys 白皮书 *In-Design Physical Verification - Automatic DRC Repair (ADR)*。

ADR 流程在设计实现期间使用 IC Validator 图案匹配实现 In-Design 热点检测与纠正。用法模型与 DRC 类似：可使用经晶圆厂认证的 DFM 包（含图案库与匹配 `runset`），或使用自有的良率损害源图案库，在设计中搜索潜在制造相关热点，并对所有错误执行自动修复与再验证，从而提升设计的整体可制造性。

在 IC Compiler 工具内运行 IC Validator 图案匹配时，使用与执行签核 DRC 检查相同的一组 IC Compiler 命令。使用 `signoff_drc` 命令进行图案匹配，使用 `signoff_autofix_drc` 命令自动修复由 `signoff_drc` 检测到的违例（热点）。详细命令语法与用法见 SolvNetPlus 上的 *IC Compiler Implementation User Guide*，或 IC Compiler man pages。

下列各节说明如何使用 IC Compiler 工具中的签核 DRC 命令检测并纠正热点（不适用于 IC Compiler II 工具）。

16.6.1 设置物理签核选项

Setting Up the Physical Signoff Options

在使用 `signoff_drc` 与 `signoff_autofix_drc` 命令运行图案匹配之前，先设置适用于这些命令的物理签核选项。

参见

更多信息见 *IC Compiler Implementation User Guide* 中「Performing Signoff Design Rule Checking」一节。

16.6.2 运行图案匹配以检测热点

Running Pattern Matching for Hotspot Detection

使用 `signoff_drc` 命令对布线层执行热点检测。

参见

更多信息见 *IC Compiler Implementation User Guide* 中「Performing Signoff Design Rule Checking」一节。

16.6.3 自动修复热点图案

Automatically Fixing Hotspot Patterns

使用 `signoff_autofix_drc` 命令自动修复检测到的热点。

参见

更多信息见 *IC Compiler Implementation User Guide* 中「Performing Signoff Design Rule Checking」一节。

16.6.4 使用用户定义修复指导修复热点

Repairing Hotspots With User-Defined Fix Guidance

除默认 ADR 纠正外，IC Validator 工具还支持实现用户定义修复指导。在图案匹配流程中实现修复指导的步骤为：

1. 在图案库创建期间，将修复指导指定为可选图案标记。单个图案可有多个指导。

图 16.5 所示图案给出单个用户定义修复指导：在 L 形多边形末端添加矩形。

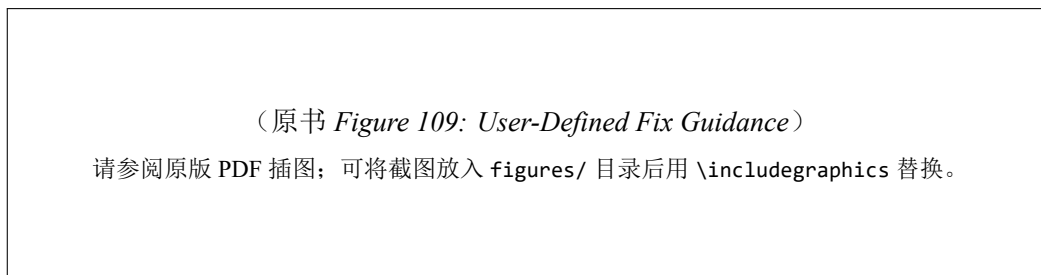


图 16.5: 用户定义修复指导

```
//Layer Assignment
metal : polygon_layer = assign ( {{3,0}} );
pattern_marker : polygon_layer = assign ( {{1,0}} );
pattern_extent : polygon_layer = assign ( {{2,0}} );
pattern_add : polygon_layer = assign ( {{4,0}} );
optional_markers: list of polygon_layer =
    {pattern_extent,pattern_marker,pattern_add};

//Create Pattern Library
pattern_learn (
    pattern_library_name = "met",
    pattern_layers = {metal},
    pattern_marker = marker_layer,
    optional_pattern_markers = optional_markers,
    pattern_text_id = text_layer,
    pattern_extent = pattern_extent,
    pattern_fuzziness = PM_EXACT,
    pattern_reflect = true,
    pattern_rotate = true
);
```

代码 16.2: 在图案库创建时附着修复指导

2. 在图案匹配后取回并传递修复指导。

使用 `optional_markers()` 函数取回修复指导，并使用 `milkyway_route_directives()` 函数将修复指导传递给布线器。

```
//Output hotspot marker layer
```

```

hotspot_marker = pattern_match (
    pattern_library_name = "met",
    pattern_layers = {metal_layer_list[i]});

//Retrieve all the optional pattern marker layers defined in step 1
optional_markers = optional_pattern_markers("met", hotspot_marker);

//Retrieve the add shape layer
pattern_add = optional_markers[2];

//Pass fixing guidance to IC Compiler
milkyway_route_directives(
    display_marker = optional_markers[1],
    group_marker = optional_markers[0],
    error_layers = { {optional_markers[1]} },
    route_directives = { {pattern_add, dummy_sub} } );

```

代码 16.3: 取回并传递修复指导

3. 在 IC Compiler 工具内运行 ADR，以进行热点检测与修复。

编写 IC Compiler TCL 脚本，在 `icc_shell` 中执行 ADR，或使用 IC Compiler GUI 执行命令。

```

open_mw_lib ./design
copy_mw_cel -from eco_route_routeopt -to pm_flow_add
open_mw_cel pm_flow_add

set_physical_signoff_options -exec icv -drc_runset {/runset/ \
match_add.rs}
signoff_drc -run_dir {./signoff_drc_run} -ignore_child_cell_errors

signoff_autofix_drc -run_dir {./signoff_autofix_drc_run} \
-init_drc_error_db signoff_drc_run

save_mw_cel -as pm_flow_add
close_mw_cel

```

代码 16.4: IC Compiler TCL 脚本

IC Compiler 工具按定义顺序从用户定义修复指导开始。在图 16.6 中，分别可见 `pattern_122` 由用户定义指导与默认 ADR 流程纠正的结果。

(原书 *Figure 110: Example of Hotspot Correction*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 16.6: 热点纠正示例

Chapter 17

图案库管理器

Pattern Library Manager

本章介绍 IC Validator 图案库管理器 (Pattern Library Manager)。

IC Validator 图案库管理器按下列各节说明：

- 入门
- 创建新图案库
- 编辑图案
- 导出与导入图案库
- 在图案库管理器中运行图案匹配
- 限制与局限

17.1 入门

Getting Started

IC Validator 图案库管理器 (Pattern Library Manager, PLM) 是图形用户界面 (GUI)，提供另一种方式为 IC Validator 图案匹配运行构建图案库，而无需使用批处理模式通过 `pattern_learn()` 函数生成图案库。

17.1.1 支持的 IC Validator WorkBench 版本

Supported Versions of IC Validator WorkBench

图案库管理器通过套接字或端口连接与 IC Validator WorkBench 工具通信。

IC Validator N-2017.12-SP2 及更高版本支持 IC Validator WorkBench J-2014.06 及更高版本。

17.1.2 启动图案库管理器

Invoking the Pattern Library Manager

确保已正确设置 `ICV_INSTALL_DIRECTORY` 安装目录。

可在 shell 命令行或 IC Validator WorkBench 版图编辑器中启动图案库管理器。

1. 命令行语法为：

```
icv_plm
```

注意

若分别从命令行启动图案库管理器与 IC Validator WorkBench，二者必须位于同一目录路径（同一台机器上），才能建立连接。关于如何将图案库管理器连接到 IC Validator WorkBench，见「与 IC Validator WorkBench 通信」一节。

2. 要从 IC Validator WorkBench 访问图案库管理器，在命令窗口中 source IC Validator 的 `plm_button.tcl` 文件：

```
% source $env(ICV_INSTALL_DIRECTORY)/contrib/plm_button.tcl
```

工具栏会添加 **ConnectPlm** 按钮。单击该按钮即可启动图案库管理器并建立连接，如图 17.1 所示。

（原书 *Figure 111: ConnectPlm Button*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.1: ConnectPlm 按钮

注意

也可将该 `plm_button.tcl` 文件放入 IC Validator WorkBench 启动文件，以免每次都重新设置连接。

设置 `ICVWB_USER` 环境变量后，IC Validator WorkBench 工具会自动连接到图案库管理器。

对于 IC Validator WorkBench M-2016.03-SP2 及更新版本：

```
setenv ICVWB_USER ICV_PLM
```

工具栏会添加一个黄色按钮。单击该按钮即可启动图案库管理器并建立连接。

（原书 *Figure 112: Pattern Library Manager Toolbar*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.2: 图案库管理器工具栏

17.1.3 与 IC Validator WorkBench 通信

Communicating With IC Validator WorkBench

若图案库管理器已启动，要与 IC Validator WorkBench 建立连接：

1. 在 IC Validator WorkBench 命令窗口中输入下列命令：

```
user_socket open 0
```

该命令返回一个数字。这是 IC Validator WorkBench 建立的打开套接字。



图 17.3: 打开套接字连接

2. 在图案库管理器中，选择 **Tools > Communication**。输入端口号并单击 **OK**。

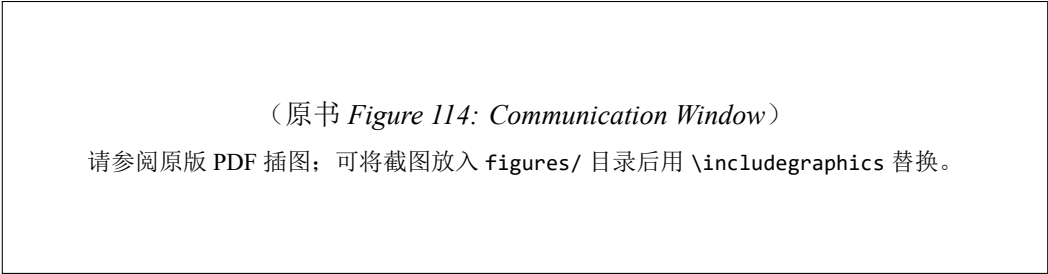


图 17.4: 通信窗口

3. 此时图案库管理器已连接到 IC Validator WorkBench。

17.1.4 图案库管理器 GUI

Pattern Library Manager GUI

主 GUI 如图所示。表 17.1 给出该 GUI 四个下拉菜单的主要功能。

表 17.1: 图案库管理器的下拉菜单

菜单	说明
File	创建、打开或保存图案库。导入 Pattern Database 格式的图案库，并导出 Pattern Library Manager 格式的图案库
Edit	在图案库中创建、添加或编辑图案
Tool	与 IC Validator WorkBench 建立通信，并在图案库管理器中执行图案匹配
Help	显示关于界面语言与用户指南的信息

17.2 创建新图案库

Creating a New Pattern Library

File 菜单允许创建用于添加新图案的新图案库。表 17.2 汇总可用选项。

表 17.2: File 菜单命令

选项	说明
New	创建新图案库
Open	打开既有图案库
Save	保存当前库
Export	将图案库输出为 Pattern Database 格式
Import	将 Pattern Database 格式的库读入图案库管理器
Close	关闭当前库
Exit	退出图案库管理器

17.2.1 创建新图案库名称

Creating a New Pattern Library Name

在向新图案库添加新图案之前，必须先创建图案库名称。

1. 选择 **File > New**。出现对话框。以默认名称 **New1** 创建新库名。
2. 必要时更改库名。
3. 指定保存图案库的路径。
4. 填写所需的图案库属性，例如图案模糊度（pattern fuzziness）、图案层数量，以及均匀模糊尺寸（uniform fuzzy size，仅用于 PM_EDGE_UNIFORM 模式）。
5. 单击 **More** 打开高级设置，例如图案类型与可选图案标记层数量。
6. 单击 **OK** 创建新图案库（名称）。IC WorkBench EV Plus 窗口中会创建对应的空版图（以图案库名称命名），并含多层。图案库管理器主窗口中会出现列出已定义图案库参数的摘要表，如图 17.5 所示。
7. 现在可以向新图案库添加图案。添加新图案时，每个图案都会出现在已创建的图案库下。



图 17.5: 创建新图案库

17.2.2 添加新图案

Adding New Patterns

可使用 **Edit** 菜单中的三个命令向图案库添加新图案，如表 17.3 所示。

表 17.3: Edit 菜单命令

命令	说明
New Pattern	创建图案版图并将其添加为新图案

表 17.3: Edit 菜单命令（续）

命令	说明
New Pattern By Clip	通过版图上的裁剪窗口生成图案
New Pattern By Layer	从版图选定区域提取多个图案
Copy	从图案库复制图案
Paste	将复制的图案粘贴到另一图案库
Rename	重命名图案
Delete	从图案库删除图案

通过裁剪捕获新图案

Capturing a New Pattern by Clipping 可从已定义图案层与图案标记的版图创建新图案。图案边界定义为裁剪窗口。

要捕获新图案，从包含多边形的版图开始。

1. 在图案库管理器中，打开既有图案库或创建新图案库条目。
2. 在 IC WorkBench EV Plus 中打开版图。
3. 浏览到图案位置，并缩放到合适显示，使所有待提取多边形显示在显示窗口中。
4. 如图 17.6 所示，在图案库管理器中选择 **Edit > New Pattern By Clip**。出现对话框。为该图案输入名称，并指定图案标记层与图案层的层号与数据类型。单击 **OK**。



图 17.6: 通过裁剪捕获新图案

注意

默认情况下，图案范围外的多边形不会被捕获到生成的图案中。若要在图案最终确定前保留图案范围外的周围区域以便进一步修改，请在单击 **OK** 之前选择 **Keep Context**。

5. 在 IC WorkBench EV Plus 中绘制裁剪窗口以定义图案范围。出现新版图窗口。每条多边形边从 0 开始标注编号。边标签作为文本层创建在层列表下。新图案被添加到图案库，如图 17.7 所示。

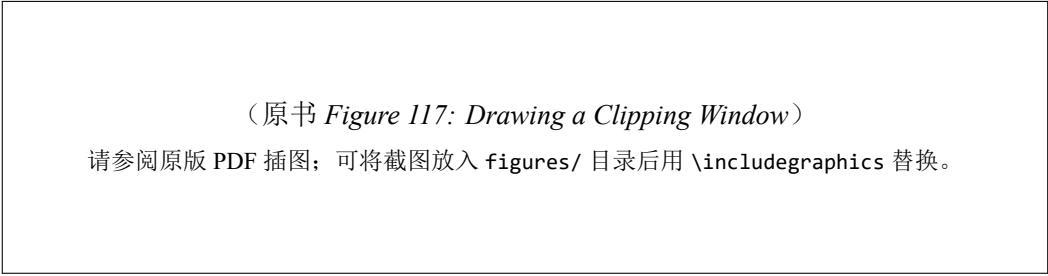


图 17.7: 绘制裁剪窗口

注意

在步骤 4 中单击 **OK** 之后，IC WorkBench EV Plus 中唯一允许的操作是绘制裁剪窗口以定义图案范围。

6. 如需要，在 **Pattern Property** 选项卡中定义图案属性。更多信息见「向图案附着图案属性」一节。
7. 如需要，在 **Tolerance** 选项卡中添加边约束。更多信息见「向图案添加边容差」一节。
8. 若同一版图中有多个位置，重复前述步骤以生成更多新图案。完成后，在图案库管理器中选择 **File > Save**，将更改保存到图案库文件。

由此生成包含来自版图的一个或多个图案的图案库。

在版图中提取图案

Extracting Patterns in a Layout 可从已定义图案层、图案标记与图案范围层的版图创建多个图案。

1. 在图案库管理器中，打开既有图案库或创建新图案库条目。
2. 在 IC WorkBench EV Plus 中打开版图。
3. 浏览到版图位置，并缩放到合适显示，使版图的正确部分显示在显示窗口中。
4. 如图 17.8 所示，在图案库管理器中选择 **Edit > New Pattern By Layer**。出现对话框。输入图案前缀名称，并指定图案范围、图案标记与图案层的层号与数据类型。若已定义可选忽略区域，也可指定。单击 **OK**。

（原书 *Figure 118: Extracting Patterns in a Layout*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.8: 在版图中提取图案

注意

默认情况下，图案范围外的多边形不包含在生成的图案中。若要保留图案范围外的周围区域，请在单击 **OK** 之前选择 **Keep Context**。

5. 在 IC WorkBench EV Plus 中绘制裁剪窗口，以选择用于生成新图案的版图部分。图案被提取并列在当前打开或选定的图案库下，如图 17.9 所示。

（原书 *Figure 119: Selecting Part of Layout for Generating New Patterns*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.9: 选择版图部分以生成新图案

注意

在步骤 4 中单击 **OK** 之后，IC WorkBench EV Plus 中唯一允许的操作是绘制裁剪窗口，以定义用于图案提取的版图区域。

6. 如需要，在 **Pattern Property** 选项卡中定义图案属性。更多信息见「向图案附着图案属性」一节。
7. 如需要，在 **Tolerance** 选项卡中添加边约束。完整信息见「向图案添加边容差」一节。
8. 完成后，在图案库管理器中选择 **File > Save**，将更改保存到图案库文件。

由此生成包含来自版图的多个图案的图案库，如图 17.10 所示。

（原书 *Figure 120: Pattern Library With Multiple Patterns*）
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.10: 含多个图案的图案库

从零生成新图案

Generating a New Pattern From Scratch 当没有既有版图可供图案提取时，可在 ICWBEV 中绘制图案版图以创建新图案。

1. 在图案库管理器中，打开既有图案库或创建新图案库条目。
2. 在图案库管理器中，选择 **Edit > New Pattern**。出现对话框。输入待生成图案的名称，然后单击 **OK**，如图 17.11 所示。

（原书 *Figure 121: Generating a New Pattern From Scratch*）
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.11: 从零生成新图案

3. 出现警告消息。这是因为 IC WorkBench EV Plus 中尚无与新创建图案名称关联的图形。单击 **OK** 关闭该消息。
4. 在 IC WorkBench EV Plus 中创建图案版图，如图 17.13 所示。
5. 关于版图创建与编辑的更多信息，见 Help 菜单中的 *IC WorkBench EV Plus User Guide*。见图 17.14。
6. 在 IC WorkBench EV Plus 中单击 **PlmRefresh**，用为该图案创建的全部层更新图案库。图案库管理器中出现对话框，如图 17.15 所示。单击 **Yes**。
图案被添加到图案库，如图 17.16 所示。
7. 如需要，在 **Pattern Property** 选项卡中定义图案属性。更多信息见「向图案附着图案属性」一节。
8. 如需要，在 **Tolerance** 选项卡中添加边约束。更多信息见「向图案添加边容差」一节。
9. 重复前述步骤，在同一图案库文件下生成更多图案。完成后，在图案库管理器中选择 **File > Save**，将更改保存到图案库文件。

（原书 *Figure 122: No Graphics Warning Message*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.12: 无图形警告消息

（原书 *Figure 123: Creating the Pattern Layout*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.13: 创建图案版图

（原书 *Figure 124: IC WorkBench EV Plus User Guide Help Menu*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.14: IC WorkBench EV Plus 用户指南 Help 菜单

（原书 *Figure 125: Dialog Box for Updating Pattern Library*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.15: 更新图案库对话框

（原书 *Figure 126: Pattern Added to Pattern Library*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.16: 图案已添加到图案库

17.3 编辑图案

Editing a Pattern

可在图案库管理器内部编辑图案：向该图案附着一个或多个文本属性，或为非均匀匹配模式指定边容差。
也可在 IC WorkBench EV Plus 中编辑图案版图：添加、删除或移动多边形、边与顶点。

17.3.1 向图案添加边容差

Adding an Edge Tolerance to a Pattern

图案库管理器允许为非均匀模糊匹配模式指定边容差（约束）。约束使您能够将图案的不同变体匹配到已定义图案。每条边不应有超过一个边约束。

要指定边容差：

1. 打开包含待编辑图案的图案库。
2. 单击图案名称以选择待编辑图案。图案库管理器中出现 **Tolerance** 与 **Pattern Property** 选项卡，图案版图显示在 IC WorkBench EV Plus 中。
3. 从下拉框中选择要指定容差的边。若所选边为水平边，则该边被限制为垂直移动；若所选边为竖直边，则该边被限制为水平移动。

注意

可使用 IC WorkBench EV Plus 中显示的边编号作为参考，以识别边编号。

4. 单击增大或减小按钮，以纳米为单位指定内侧与外侧限值。若容差区间包含该值，选择 **Inclusive**，如图 17.17 所示。

（原书 *Figure 127: Adding an Edge Tolerance*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.17: 添加边容差

5. 单击 **Add**，将容差添加到图案库。边容差出现在 IC WorkBench EV Plus 的层 (9001:0) 上。
6. 选择 **File > Save**，将更改保存到图案库。
7. 单击 **Update** 修改既有边容差，单击 **Delete** 删除边容差。
8. 也可在 **Edge2Edge** 窗格中通过选择两条边来指定边到边容差，如图 17.18 所示。

注意

要使边到边约束被识别，所选两条边中至少有一条必须已定义边容差。

（原书 *Figure 128: Adding an Edge-to-Edge Tolerance*）
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.18: 添加边到边容差

17.3.2 向图案附着图案属性

Attaching Pattern Properties to a Pattern

图案库管理器允许在图案库创建期间向图案附着属性。找到匹配时，这些属性会传递到输出图案标记。这使您能够使用其他 IC Validator `runset` 函数，基于属性名与属性值对输出图案标记进行后处理。

要向图案附着属性：

1. 打开包含待编辑图案的图案库。
2. 单击图案名称以选择待编辑图案。出现 **Tolerance** 与 **Pattern Property** 选项卡。单击 **Pattern Property** 选项卡以激活，如图 17.19 所示。

（原书 *Figure 129: Pattern Property Tab*）
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.19: Pattern Property 选项卡

3. 若属性为字符串类型，将属性名与值添加到 **String Property** 字段；若为双精度类型，则添加到 **Double Property** 字段。对于图 17.20 中的示例，`hs_type` 属性为字符串类型，赋值为 `space`；`min_cd` 属性为双精度类型，赋值为 `0.056`。

（原书 *Figure 130: Attaching Pattern Properties to a Pattern*）
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.20: 向图案附着图案属性

4. 单击 **Add**，将属性添加到图案库。
5. 选择 **File > Save**，将更改保存到图案库。
6. 单击 **Update** 修改既有属性，或单击 **Delete** 删除属性。

17.3.3 在 IC WorkBench EV Plus 中编辑图案版图

Editing a Pattern Layout in IC WorkBench EV Plus

可在 IC WorkBench EV Plus 中修改图案版图：为图案层、图案范围、图案标记与忽略区域层添加、移动或删除多边形与形状。关于如何编辑版图的更多细节，见 Help 菜单中的 *IC WorkBench EV Plus User Guide*。

移除细条（sliver）的示例：

1. 打开包含待编辑图案的图案库。
2. 单击图案名称以选择待编辑图案。图案 cell 被选中并出现在 IC WorkBench EV Plus 中。
3. 相应编辑该图案。例如，若要移除图 17.21 中 pat_2 的细条，执行下列步骤：

（原书 *Figure 131: Editing a Pattern Layout*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.21: 编辑图案版图

- a. 要选择形状，选择 **Tool > Select**，或单击 Tools 工具栏上的 **Select** 按钮并单击该形状。
- b. 要从 pat_2 移除所选形状，选择 **Edit > Delete**，或按键盘上的 Delete 键。
- c. 在 IC WorkBench EV Plus 中单击 **PlmRefresh**，以更新对 pat_2 所做的更改。出现消息，如图 17.22 所示。单击 **Yes**。

（原书 *Figure 132: Pattern Library Manager Refresh Button*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.22: 图案库管理器刷新按钮

4. 在图案库管理器中，选择 **File > Save** 保存图案库，如图 17.23 所示。

（原书 *Figure 133: Saving the Pattern Library*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.23: 保存图案库

17.4 导出与导入图案库

Exporting and Importing a Pattern Library

在 IC Validator 工具中，可使用基于 GUI 的图案库管理器创建图案库；也可通过运行 IC Validator 作业、使用 `pattern_learn()` 函数创建图案库。

在图案库管理器中生成的图案库由两部分组成：用于存储图形信息的 OASIS 目录，以及用于存储边容差与边到边约束的 XML 格式文本文件。

通过运行 IC Validator 作业、使用 `pattern_learn()` 函数创建的图案库由三部分组成：名为 `pattern.dat` 的二进制文件、名为 `log` 的文本文件（记录图案库创建参数与图案统计），以及在图案匹配模式设为 `PM_EDGE_NONUNIFORM` 时可选的、用于存储边容差与边到边约束的 XML 格式文本文件。

`pattern_match()` 函数在 IC Validator 运行中执行图案匹配。图案匹配运行前图案库必须就绪，且必须为二进制格式，也称为图案数据库（Pattern Database，PDB）格式。

表 17.4 给出这两种图案库格式的摘要比较。

表 17.4: 图案库格式摘要比较

模式	格式	导入/导出
使用 <code>pattern_learn()</code> 函数创建的图案库	Pattern Database 格式包括：(1) <code>pattern.dat</code> （二进制）；(2) <code>log</code> ；(3) 可选 XML 文件，在匹配模式为 <code>PM_EDGE_NONUNIFORM</code> 时存储边容差	必须导入图案库管理器才能编辑
在图案库管理器中创建的图案库	Pattern Library Manager 格式包括：(1) OASIS，用于存储图形信息；(2) XML 文件，用于存储边与边到边容差	必须从图案库管理器导出，才能在 IC Validator 图案匹配运行中使用该图案库

17.4.1 导入 Pattern Database 格式

Importing a Pattern Database Format

Pattern Database 格式的图案库必须导入后，才能使用图案库管理器进行编辑。

- 在图案库管理器中，选择 **File > Import**。
- 浏览到 Pattern Database 格式图案库的位置。浏览到将转换为 Pattern Library Manager 格式的图案库的保存位置，并为其输入名称，如图 17.24 所示。

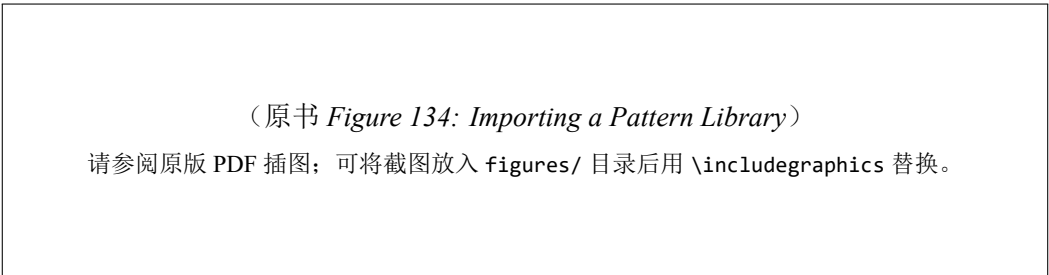


图 17.24: 导入图案库

注意

若 Pattern Database 格式的图案库已锁定（加密），导入将失败。

- 单击 **OK**。图案库被转换为 Pattern Library Manager 格式并加载到图案库管理器中。

17.4.2 导出 Pattern Library Manager 格式

Exporting a Pattern Library Manager Format

Pattern Library Manager 格式的图案库必须导出后，才能在 IC Validator 图案匹配运行中使用。

1. 在图案库管理器中，选择 **File > Open** 加载待导出的图案库，或单击选择已在图案库管理器中打开的图案库。
2. 选择 **File > Export**。
3. 浏览到将转换为 Pattern Database 格式的图案库的保存位置，并为其输入名称，如图 17.25 所示。
若不同取向的图案也应视为匹配，选择 **pattern reflect** 与 **pattern rotate**。
若在 PM_EXACT 匹配模式下允许额外多边形，选择 **ignore extra polygons**。



图 17.25: 导出图案库

4. 单击 **OK**。图案库被转换为 Pattern Database 格式，并按步骤 3 中定义的名称与路径保存到磁盘。出现 Export Result 窗口，如图 17.26 所示。单击 **OK** 关闭该窗口并完成导出过程。

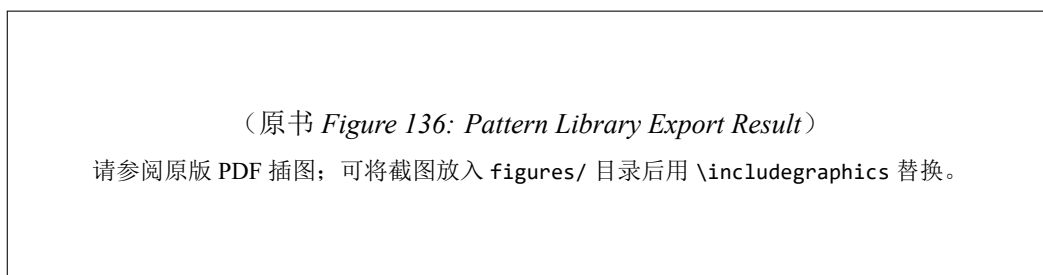


图 17.26: 图案库导出结果

该图案库已可在 IC Validator 运行中使用。

17.5 在图案库管理器中运行图案匹配

Running Pattern Matching in the Pattern Library Manager

IC Validator 工具使用 `pattern_match()` 函数在 IC Validator 运行中执行图案匹配。这意味着必须创建 IC Validator runset。关于如何创建 IC Validator 规则文件并在批处理模式下运行图案匹配的更多信息，见本用户指南「图案匹配」章。

除批处理模式外，图案库管理器还提供便捷方式：无需离开 GUI，即可使用已生成的图案库对设计版图运行图案匹配。

1. 在图案库管理器中，对选定的图案库单击 **Pattern Match**，如图 17.27 所示。
2. 弹出 Pattern Match 窗口，如图 17.28 所示。
 - a. 浏览到输入版图的位置。选择设计格式。输入顶层 cell 名称。指定图案层。

(原书 *Figure 137: Performing Pattern Matching on the Selected Pattern Library*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.27: 对选定图案库执行图案匹配

(原书 *Figure 138: Pattern Match Window*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.28: Pattern Match 窗口

- b. 可在 **Run Options** 列中指定 IC Validator 命令行选项。关于命令行选项的更多信息，见本用户指南「IC Validator 基础」章。
 - c. 指定输出目录路径。
 - d. 单击 **More** 显示附加选项。若希望按图案名对结果分组，选中 `group_errors_by_pattern` 复选框。
 - e. 单击 **OK**，对指定版图执行图案匹配。
3. 图案匹配结果出现在对话框中，如图 17.29 所示。详细信息保存在输出目录中。单击 **OK** 关闭该对话框。

(原书 *Figure 139: Pattern Matching Results Dialog Box*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.29: 图案匹配结果对话框

17.6 限制与局限

Restrictions and Limitations

下列功能目前在图案库管理器中不可用，但由批处理模式下的 `pattern_learn()` 函数支持：

- 指定由 IC Validator 工具根据用户定义的 `ambit` 值与 `ambit_mode` 生成图案范围。
- 将边容差指定为用户定义输入层。
- 指定可选区域层。
- 指定自动锚定 (`auto anchoring`)。

附录 A

IC Validator 体系结构

IC Validator Architecture

本附录展示 IC Validator 的基本体系结构。

A.1 基本体系结构

Basic Architecture

图 A.1 展示了 IC Validator 的基本体系结构。

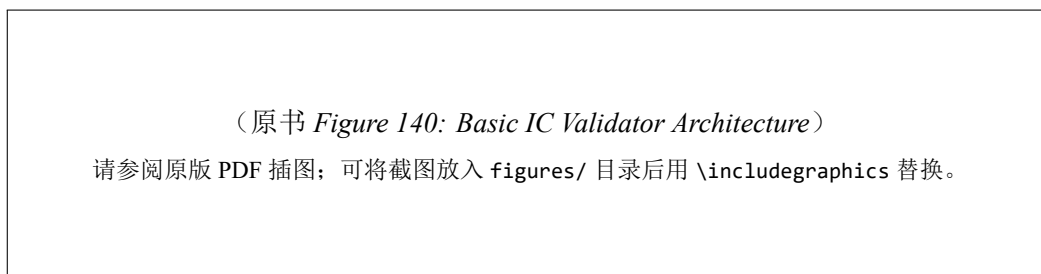


图 A.1: IC Validator 基本体系结构

附录 B

LVL 实用程序

LVL Utility

本附录介绍 IC Validator 工具提供的版图对版图（Layout-Versus-Layout，LVL）实用程序以及 QuickLVL 实用程序。

LVL 实用程序包括以下各节：

- 说明（Description）
- 命令行选项（Command-Line Options）
- 使用 OpenAccess 库格式（Using the OpenAccess Library Format）
- 层文件（Layer Files）
- 比较层子集（Comparing a Subset of Layers）
- 文本处理（Handling of Text）
- LVL 输出（LVL Output）
- 使用模型（Use Models）

QuickLVL 实用程序包括以下各节：

- QuickLVL 实用程序（QuickLVL Utility）
- 单元级 Quick LVL（Cell-Level Quick LVL）
- LVL 与 QuickLVL 的差异（Differences Between LVL and QuickLVL）
- 聚焦差异层与区域的自动增量 DRC（Auto-Incremental DRC Focused on Differing Layers and Regions）

B.1 说明

Description

LVL 实用程序可用于：

- 从单个库生成 assign 文件。
- 使用基于 XOR 的比较或基于 NOT 的比较，对设计的两个版图进行比较。发现的错误可通过 `cell11_vs_cell12.vue` 文件在 VUE 中查看。更多信息见 *IC Validator VUE User Guide*。

注意

LVL 实用程序支持除 NULL 字符外的任意文本字符。

对于基于 NOT 的比较：

- 基线层与检查库层做 NOT 运算；
- 检查库层与基线层做 NOT 运算；
- 各 NOT 运算的错误分别显示在 `cell1_vs_cell2.LVL_ERRORS` 文件中。

LVL 实用程序支持下列版图输入格式：

- GDSII
- Milkyway
- OASIS
- OpenAccess
- NDM

注意

LVL 实用程序支持除 NULL 字符外的任意文本字符。

B.2 命令行选项

Command-Line Options

LVL 实用程序的命令行语法为：

```
icv_lv1 library1 library2 -c cell | -c1 cell1 -c2 cell2 [options]
```

表 B.1 说明基本命令行选项以及可用于 LVL 分布式处理的命令行选项。

表 B.1: LVL 实用程序命令行选项

语法	说明
<code>-64</code>	以支持 64 位坐标的方式运行该实用程序。
<code>-af filename</code>	存放库中 assign 层的文件名。该文件由 LVL 实用程序生成，语法与 IC Validator runset 的 ASSIGN 节相同。若希望文件不在当前目录，请包含路径。 ^a
<code>-c cell</code>	两个库 <code>library1</code> 与 <code>library2</code> 的单元名。使用 <code>-af</code> 生成 assign 文件时也需要单元名。 ^a
<code>-c1 cell1</code>	第一个库 <code>library1</code> 的单元名。 ^a
<code>-c2 cell2</code>	第二个库 <code>library2</code> 的单元名。
<code>-cell_map_file</code>	支持将第二个输入库中的单元名映射到第一个输入库中的单元名。
<code>-elpc num</code> <code>-error_limit_per_check num</code>	设置每类检查的错误上限。默认值为 100。注意：将该选项设为 <code>ERROR_LIMIT_MAX</code> 时，将存储直至工具定义的最大上限的全部错误。也可给出具体数字，将上限设为高于工具定义的最大值；但对大量错误，提高上限可能导致性能下降与磁盘占用增加，因此不推荐。设为 0 则抑制存储全部错误。
<code>-flat</code>	将全部文本与几何展平到顶层单元，并在顶层单元中执行所有层比较。

语法	说明
<code>-flatten</code>	将全部输出展平到顶层单元。
<code>-h</code>	显示用法信息。
<code>-host_add hostname</code> <code> number_of_cpus </code> <code>hostname:number_of_cpus LSF </code> <code>SGE NB</code>	向已在运行的 <code>icv_lv1</code> 作业添加更多主机（含远程主机）。用法与 <code>-host_init</code> 相同。详见「指定主机名与 CPU 数量」。
<code>-host_init</code>	在单主机或多主机上启动 IC Validator 作业。详见「指定主机名与 CPU 数量」。登录远程主机所用远程登录协议的优先级见 <code>-host_login</code> 。更多信息见第一章中 IC Validator 的 <code>-host_init</code> 命令行选项。
<code>-host_init LSF</code>	通过 LSF 批处理排队系统获取主机，读取 <code>LSB_HOSTS</code> 或 <code>LSB_MCPU_HOSTS</code> 环境变量。远程登录协议优先级见 <code>-host_login</code> 。更多信息见第一章中 <code>-host_init LSF</code> 。
<code>-host_init SGE</code>	通过 SGE 批处理排队系统获取主机，读取 <code>PE_HOSTFILE</code> 环境变量。远程登录协议优先级见 <code>-host_login</code> 。更多信息见第一章中 <code>-host_init SGE</code> 。
<code>-host_disk</code>	为 <code>-host_init</code> 或 <code>-host_add</code> 指定的全部主机指定本地组目录。更多信息见第一章中 <code>-host_disk</code> 。
<code>-host_login</code>	使 IC Validator 能登录远程主机以启动分布式作业进程。请确保工具能找到提供给 <code>-host_login</code> 的二进制/脚本。默认为 <code>rsh</code> 。更多信息见第一章中 <code>-host_login</code> 。
<code>-host_memory Memory_Value M G T</code>	通过重调度与线程化，尝试使作业所用主机维持用户指定的目标内存。不能用该开关降低单个命令的内存。更多信息见第一章中 <code>-host_memory</code> 。
<code>-ignore_text_case</code>	指示实用程序在比较文本时忽略大小写。默认行为是文本比较区分大小写。
<code>-al</code>	与 <code>-layer_file</code> 参数一起使用。向 LVL 运行添加任何缺失的层。因需扫描输入文件以判断层文件是否缺失层，会有性能影响。仅当被检查文件为 GDSII 或 OASIS 时可用。
<code>-lf filename</code> <code>-layer_file filename</code>	两个库共用的层文件，用于层映射、层合并与层命名。必须包含有效的 PXL runset ASSIGN 节。若库不在当前目录，请包含路径。更多信息见「层文件」。 ^b
<code>-lf1 filename</code> <code>-layer_file1 filename</code>	第一个库的层文件。始终与 <code>-lf2</code> 一起使用。若库不在当前目录，请包含路径。更多信息见「层文件」。 ^b
<code>-lf2 filename</code> <code>-layer_file2 filename</code>	第二个库的层文件。始终与 <code>-lf1</code> 一起使用。若库不在当前目录，请包含路径。更多信息见「层文件」。 ^b

语法	说明
<code>-oa_color_mapping option</code>	指定如何使用锁定/未锁定锚定位 (anchor bit) 进行 OpenAccess 颜色层映射。<i>option</i> 可选: • ignore: 使用未指定 locked/unlocked 关键字的颜色映射, 忽略已指定者。 • strict: 使用已指定 locked/unlocked 的颜色映射, 忽略未指定者。 • full: 将读入时带颜色的全部形状设为 locked。映射与 compatibility 相同。 • compatibility: 遵循行业标准。忽略颜色规范之后出现 colorAnnotate 关键字的颜色映射。若无规则匹配彩色形状, 则使用无颜色的映射。 • lockOnly: 对可着色层, 若技术数据库中该层的 -fullColor 为 true (默认), 则仅映射已着色且锁定的形状; 未着色或未锁定的形状丢弃。若某层 -fullColor 为 false, 则对该层遵守层映射。
<code>-oa_color_mapping1 ignore strict full compatibility lockOnly</code>	针对 OpenAccess 库 1 的颜色层映射形式, 指定 locked/unlocked 锚定位用法。
<code>-oa_color_mapping2 ignore strict full compatibility lockOnly</code>	针对 OpenAccess 库 2 的颜色层映射形式, 指定 locked/unlocked 锚定位用法。
<code>-of options_file</code>	可选文件, 包含 IC Validator runset 的 OPTIONS 节。可含任意 OPTIONS 节函数, 包括 gds_options() 、 milkyway_options() 与 oasis_options() 。
<code>-rsi</code>	报告下列输入库信息。仅支持 GDSII 与 OASIS 输入格式。报告内容包括: GDSII 最后修改与最后访问时间; 用 md5sum -b file 计算的 md5sum ; 输入库格式; 输入库路径; 用于库输入的命令。枚举须为逗号分隔列表, 且整个列表放在引号内。见 run_options() 函数的 report_streamfile_information 参数。注意: -rsi 命令行选项会覆盖 run_options() 中的 runset 设置。
<code>library1</code>	基线库名。若不在当前目录请包含路径。当用 -af 只指定一个库时, LVL 实用程序生成 assign 文件。不能在同一次 LVL 运行中既生成 assign 文件又比较两个版图。各格式输入示例: • GDSII: /path/to/file.gds • OASIS: /path/to/file.oas • Milkyway: /library/path/LIBRARY_NAME • OpenAccess: /path/to/lib.defs/LIBRARY_NAME • NDM: /library/path/LIBRARY_NAME
<code>library2</code>	检查库名。若不在当前目录请包含路径。使用 -af 时不要指定该文件。不能在同一次运行中既生成 assign 文件又比较两个版图。输入格式示例同 <i>library1</i>。
<code>-mf filename</code>	应用于两个库的层映射文件。更多信息见「映射层」。
<code>-map_file filename</code>	
<code>-mf1 filename</code>	应用于第一个库的层映射文件。更多信息见「映射层」。
<code>-map_file1 filename</code>	

语法	说明
-mf2 <i>filename</i> -map_file2 <i>filename</i>	应用于第二个库的层映射文件。更多信息见「映射层」。
-nd1 <i>label</i> -ndm_design_label1	两个库 library1 与 library2 的设计标签。 ^a
-nd11 <i>label</i> -ndm_design_label11	第一个库 library1 的设计标签。 ^a
-nd12 <i>label</i> -ndm_design_label12	第二个库 library2 的设计标签。
-not	指示实用程序做两次基于 NOT 的比较，而非基于 XOR 的比较。
-oa_dm4	使用与 DM4 插件兼容的 OpenAccess 共享库。
-outside	指示实用程序做两次基于 OUTSIDE 的比较，而非基于 XOR 的比较。执行 not_interacting() 时，任何接触的数据不纳入该运算。可与 -report_touch 一起使用，使接触被视为不交互。
-quick	执行单元级 Quick LVL。更多信息见「QuickLVL 实用程序」。注意：Quick LVL（-quick）仅在单主机上工作。若从批处理排队系统获取了多主机，Quick LVL 将报错退出。
-resolution	允许在 LVL 实用程序内指定 IC Validator 运行的内部分辨率。
-rt ALL POINT -report_touch ALL POINT	指定如何处理点接触。与 -outside 一起使用。 - ALL：将多边形线接触与边点接触视为 outside。 - POINT：将点接触视为 outside。
-sel "layer1[-layer1max][,dtype1[-dtype1max],dtype2[-dtype2max],..." [...]" -select_edge_layers	指定比较所用的边层与 datatype 。可使用范围。未指定 datatype 时默认为全部 datatype 。 ^c
-small_cell	强制小单元模式，使 LVL 比较部分在小设计上尽可能快。若工具判定输入设计过大，为避免在大设计上变慢，将以错误退出。
-sn <i>layer_name1</i> ... -select_names <i>layer_name1</i> ...	按名称指定实用程序所运行的层。
-spl "layer1[-layer1max][,dtype1[-dtype1max],dtype2[-dtype2max],..." [...]" -select_polygon_layers	指定比较所用的多边形层与 datatype 。可使用范围。未指定 datatype 时默认为全部 datatype 。要选择文本层进行比较，应使用 -st1 。 ^c
-split_output	与 -not 一起使用时，将 not12 结果写入 Cell11_vs_Cell12_not12 ，将 not21 结果写入 Cell11_vs_Cell12_not21 。not12 对应对基线库（命令行第一个库）各层与检查库对应层执行 (<i>BaselineLayer</i> not <i>Checklayer</i>)；not21 对应 (<i>CheckLayer</i> not <i>BaselineLayer</i>)。
-st1 "layer1[-layer1max][,dtype1[-dtype1max],dtype2[-dtype2max],..." [...]" -select_text_layers	指定比较所用的文本层与 datatype 。可使用范围。未指定 datatype 时默认为全部 datatype 。要选择多边形层，应使用 -spl 。 ^{cd}

语法	说明
<code>-sub_cell</code>	指示对较大 OASIS 版图内的小下级单元运行，避免读取整个版图。
<code>-suppress_empty_layer_errors</code>	抑制报告空层错误多边形。
<code>-text</code>	指示实用程序比较两个版图之间的文本。更多信息见「文本处理」。未使用该选项时，仅比较几何，不比较文本。
<code>-top_cell_only</code>	仅比较各库顶层单元中的数据，丢弃全部下层单元数据。
<code>-ucl</code> " <code>layer1[-layer1max][,dtype1[-dtype1max] ...]</code> "	指定从比较中排除的边层与 <code>datatype</code> 。可使用范围。未指定 <code>datatype</code> 时默认为全部 <code>datatype</code> 。 ^c
<code>-unselect_edge_layers</code>	
<code>-un layer_name1 ...</code>	按名称指定运行时排除的层。
<code>-unselect_names layer_name1 ...</code>	
<code>-upl</code> " <code>layer1[-layer1max][,dtype1[-dtype1max] ...]</code> "	指定从比较中排除的多边形层与 <code>datatype</code> 。可使用范围。未指定 <code>datatype</code> 时默认为全部 <code>datatype</code> 。 ^c
<code>-unselect_polygon_layers</code>	
<code>-utl</code> " <code>layer1[-layer1max][,dtype1[-dtype1max] ...]</code> "	指定从比较中排除的文本层与 <code>datatype</code> 。可使用范围。未指定 <code>datatype</code> 时默认为全部 <code>datatype</code> 。 ^{c,d}
<code>-unselect_text_layers</code>	
<code>-transform</code> [RX,][0 90 180 270 DX,DY]	指示按命令行描述变换检查版图的顶层单元。RX: 绕 X 轴反射；0 90 180 270: 逆时针旋转角；dx dy: X/Y 方向平移（微米）。示例： <code>-transform 90,0,0</code> 、 <code>-transform RX,90,1,-1.2</code> 、 <code>-transform RX</code> 。更多信息见「QuickLVL 实用程序」。
<code>baseline_layout</code>	基线库名（QuickLVL）。若不在当前目录请包含路径。示例：OASIS /path/to/file.oas; GDSII /path/to/file.gds。更多信息见「QuickLVL 实用程序」。
<code>check_layout</code>	检查库名（QuickLVL）。若不在当前目录请包含路径。格式示例同基线。更多信息见「QuickLVL 实用程序」。
<code>-V</code>	打印实用程序版本。

^a 若两库单元名相同，用 `-c` 提供单元名；若不同，必须用 `-c1` 与 `-c2` 分别提供。

^b 层文件可以是完整 `runset`；LVL 实用程序忽略 `ASSIGN` 节以外的全部内容。

^c 若比较两个 OpenAccess 库，可用层名与 `purpose` 名代替层号与 `datatype` 号。

^d 仅当位置与文本字符串均匹配时文本才匹配。文本差异在 VUE 中显示为 X。文本比较仅为单元级操作。层次不匹配时，先比较层次与版图，再比较文本；若层次与版图不匹配，则无法准确比较文本。一般而言，若两个单元的名称、尺寸与放置不同，其文本不会被比较。

B.3 使用 OpenAccess 库格式

Using the OpenAccess Library Format

比较 OpenAccess 格式版图时，下列命令行选项很有用：

- `-oalm / -oa_layer_map filename`
- `-oalm1 / -oa_layer_map1 filename`

- -oalm2 / -oa_layer_map2 *filename*
- -oacm / -oa_cell_map *cellmap*
- -oacm1 / -oa_cell_map1 *cellmap1*
- -oacm2 / -oa_cell_map2 *cellmap2*
- -oa_color_mapping ignore | strict | full | compatibility
- -oa_color_mapping1 ignore | strict | full | compatibility
- -oa_color_mapping2 ignore | strict | full | compatibility
- -oav / -oa_view *viewname*
- -oav1 / -oa_view1 *viewname*
- -oav2 / -oa_view2 *viewname*

若比较两个 OpenAccess 库，无需指定层映射文件；LVL 实用程序按 **layer/purpose** 对匹配并运行比较。但若将 OpenAccess 库与 GDSII、OASIS、Milkyway 或 NDM 库比较，则需要层映射文件，以将 OpenAccess 的 **layer/purpose** 映射到 GDSII/Milkyway/OASIS/NDM 的 **layer/datatype**。例如：

```
% icv_lvl lib.defs/OA_LIB gds_lib.gds -c top -oalm layer_map.txt
```

在本例中：

- 先列出的 OpenAccess 库（OA_LIB）与 GDSII 库（gds_lib.gds）比较。
- lib.defs 文件列出设计所需的全部库（含 OpenAccess 库）。文件名必须为 **lib.defs**。若不在当前目录，请给出路径并接文件名，例如 **datapath1/testcase1/lib.defs/OA_LIB**。

lib.defs 文件内容示例：

```
DEFINE analogLib ./PDK/lib/analogLib
ASSIGN analogLib libMode shared
DEFINE basic ./PDK/lib/basic
ASSIGN basic libMode shared
DEFINE device_lib ./PDK/lib/device_lib
ASSIGN device_lib libMode shared
DEFINE OA_LIB ./OA_LIB
```

B.3.1 OpenAccess 层映射文件

OpenAccess Layer Mapping File

若比较两个 OpenAccess 数据库，无需提供层映射文件；LVL 按 **layer/purpose** 对匹配并比较。

但若将 OpenAccess 数据库与 GDSII、OASIS、Milkyway 或 NDM 库比较，则需提供层映射文件，将 OpenAccess 的 **layer/purpose** 映射到目标文件的 **layer/datatype**。

若两个 OpenAccess 库使用同一层映射文件，指定：

```
-oalm filename
```

若各输入 OA 数据库有独立层映射文件，分别指定：

```
-oalm1 filename1
-oalm2 filename2
```

更多信息见 *IC Validator Reference Manual* 中 **openaccess_options()** 函数的 **layer_mapping_file** 参数。

OpenAccess 层映射格式示例：

#OA Layer Name	OA Purpose Name	Stream layer	Stream Datatype
Layer1	drawing	1	0
Layer2	drawing	2	1
Layer3	drawing	3	2
Layer4	drawing	4	0
Layer4	boundary	4	1

B.3.2 单元映射

Cell Mapping

将 OpenAccess 数据库与其他格式比较时，可能需要提供单元映射以防止展平。OpenAccess 库用 library/cell/view 三元组标识唯一单元。IC Validator 与 LVL 必须为每个唯一三元组分配唯一单元名。LVL 会展平单元名不匹配的层次，直到两层次匹配。使用单元映射文件可保证被比较两库的单元命名一致，从而避免展平。

更多信息见 *IC Validator Reference Manual* 中 `openaccess_options()` 的 `cell_mapping_file` 参数。

若两个 OpenAccess 库使用同一单元映射，指定：

```
-oacm cellmap
```

若各输入数据库有独立映射，分别指定：

```
-oacm1 cellmap1
-oacm2 cellmap2
```

单元映射文件示例：

#Library Name	Cell Name	View Name	Stream Name
des_lib	nand2	layout	nand2_1
sdes_lib	nand2	layout	nand2_2
des_lib	cell_nu	layout	cell_nu
sdes_lib	cell_nu	layout	cell_new

B.3.3 OpenAccess 基线库的错误输出格式

Error Output Format for OpenAccess Baseline Library

错误输出写入当前工作目录下名为 `cell11_vs_cell12` 的 OpenAccess 库。

输出库中的错误数据尽可能使用基线库的层名。

未给出层文件时，层名一般格式为：

```
LlayerPpurpose
```

输出 `purpose` 名为 `Xor`、`BaselineNotCheck` 或 `CheckNotBaseline`，取决于所选 LVL 方法。

输出示例：

```
Checking LxxxPyyy_Baseline XOR LaaaPbbb_Check, output to (LxxxPyyy; Xor)
Checking LxxxPyyy_Baseline NOT LaaaPbbb_Check, output to (LxxxPyyy; BaselineNotCheck)
Checking LaaaPbbb_Check NOT LxxxPyyy_Baseline, output to (LaaaPbbb; CheckNotBaseline)
```

B.3.4 设置单元视图名

Setting the Cell View Name

在命令行指定 OpenAccess 库时，可能需要将顶层单元的视图名设为默认 `layout` 以外的值。

若两个 OpenAccess 库使用同一视图名，指定：

```
-oav viewname
```

若各输入数据库顶层单元视图名不同，分别指定：

```
-oav1 viewname1
-oav2 viewname2
```

B.4 层文件

Layer Files

默认比较版图数据库的全部层。通过命令行选项，可对层子集运行实用程序；也可指定映射文件以替代 Milkyway 与 NDM 库内部包含的映射文件。

层先经映射，再经命令行选项选择或过滤。

例如，若同时使用 `-lf layerfile` 与 `-mf mapfile`，则先用 `mapfile` 映射层号与 `datatype`，再解释 `layerfile`。

B.4.1 选择层子集

Selecting a Subset of Layers

使用 `-lf`，或同时使用 `-lf1` 与 `-lf2` 指定层文件时，仅比较层文件中出现的层；未出现在任何 `assign` 语句中的层号不做比较。

层文件限制如下：

- 当同一 `layer/datatype` 对多次使用时，冗余 `assign` 被忽略。下例中第三个赋值 `L3D0` 被忽略：

```
L1D0 = assign({{1,0}});
L2D0 = assign({{2,0}});
L3D0 = assign({{1,0}});
```

- 若层文件含同一层名的多个定义，LVL 报错停止。下例中层 `A` 定义两次，须删除其一：

```
A = assign({{1,0}});
B = assign({{2,0}});
A = assign({{10,0}});
```

B.4.2 映射层

Mapping Layers

映射文件可作为 Milkyway 与 NDM 库的内部部分。默认使用内部映射文件。用 `-mf`、`-mf1`、`-mf2` 可通过外部文件映射层；命令行指定的映射文件将替代内部映射文件。

注意

要对 OpenAccess 库用 `-mf/-mf1/-mf2` 映射层，还必须用 `-oalm/-oalm1/-oalm2` 将 LayerName/Purpose 字符串对转换为 LayerNumber/Datatype 数字对。

映射文件可为下列三种格式之二：Milkyway 格式、NDM 格式或 IC Validator 映射文件格式。Milkyway 与 NDM 格式见 SolvNetPlus 上的 *Milkyway Database Application Note* 与 *Library Data Preparation for IC Compiler User Guide*。IC Validator 映射文件格式见「层映射」。

`-lf`、`-lf1`、`-lf2`、`-spl`、`-upl`、`-sel`、`-uel`、`-stl`、`-utl`、`-sn` 与 `-un` 在层映射之后操作。

若不希望使用内部映射文件，可用映射文件命令行选项关闭：`-mf1 ""` 关闭第一个库；`-mf2 ""` 关闭第二个库；`-mf ""` 关闭两个库。

例如，两个输入库：

- `one.gds`：数据在层 1、datatype 0；
- `two.gds`：数据在层 2、datatype 0，与 `one.gds` 层 1 数据相同。

`mf2` 映射文件含：

```
D 2:0 1 0
```

LVL 命令为：

```
icv_lvl one.gds two.gds -mf2 mf2 -c top
```

`-mf2` 将 `two.gds` 中层 2 数据映射到层 1，使两库数据落在同一层，LVL 比较干净。

也可用层文件达到同样效果：

- `file1` 含 `Metal1 = assign({{1, 0}});`
- `file2` 含 `Metal1 = assign({{2, 0}});`

```
icv_lvl one.gds two.gds -lf1 file1 -lf2 file2 -c top
```

若已有可用于 LVL 的 IC Validator 层映射文件，第一种（映射文件）方法更简便。

B.5 比较层子集

Comparing a Subset of Layers

可用过滤命令行选项比较多边形与边层的子集（可有或无层文件）。规则：

- 不能将 `select` 选项与对应的 `unselect`（排除）选项同时使用；
- 不能将按名过滤与按号过滤同时使用。

过滤选项见表 B.2。

表 B.2: 过滤选项

数据过滤类别	按名称	按编号
Select	<code>-sn</code>	<code>-spl</code> 、 <code>-sel</code> 、 <code>-stl</code>
Unselect	<code>-un</code>	<code>-upl</code> 、 <code>-uel</code> 、 <code>-utl</code>

注意

同一次运行中不能同时使用 `-sn` 与 `-un`；不能同时使用 `-spl` 与 `-upl`；不能同时使用 `-sel` 与 `-uel`；不能同时使用 `-stl` 与 `-utl`。

B.5.1 按名称比较**Comparing by Name**

在用 `-lf` 或同时用 `-lf1`/-`lf2` 提供层文件时，可用命令行选项限制所比较的多边形与边层。此为按名过滤。
按名限制层的选项为：

- `-sn/-select_names layer_name1 ...`
- `-un/-unselect_names layer_name1 ...`

示例：在 POLY、NWEELL、METAL1 上运行：

```
-sn POLY NWEELL METAL1
```

在除 METAL2 外的全部层上运行：

```
-un METAL2
```

B.5.2 按编号比较**Comparing by Number**

未提供层文件时，可用命令行选项限制所比较的层。此为按号过滤。多边形、边与文本层分别过滤。
按号限制层的选项为：

- `-sel/-select_edge_layers "layer1[-layer1max][,dtype1[-dtype1max]] [...]"`
- `-uel/-unselect_edge_layers "..."`
- `-spl/-select_polygon_layers "..."`
- `-upl/-unselect_polygon_layers "..."`
- `-stl/-select_text_layers "..."`
- `-utl/-unselect_text_layers "..."`

下例在下列多边形层上运行：层 1-4 全部 datatype；层 5 datatype 0-7；层 6 全部 datatype；层 7-8 datatype 0-7：

```
-spl 1-4 5,0-7 6 7-8,0-7
```

下例在除多边形层 1、2、3 的 datatype 5、6、7 外的全部多边形层与 datatype 上运行：

```
-upl 1-3,5-7
```

边层示例：层 1-5 datatype 0-7；层 6-8 datatype 5-7：

```
-sel 1-5,0-7 6-8,5-7
```

排除边层 2，以及边层 4 的 datatype 5-6：

```
-uel 2 4,5-6
```

B.6 文本处理

Handling of Text

-text 命令行选项指示 LVL 比较两个版图之间的文本。若一个或两个数据库在同一坐标 (x1,y1) 有多个文本，实用程序在 (x1,y1) 处尽可能匹配两库间的文本字符串。错误报告方式如下：

若每个数据库在 (x1, y1) 仅有一个未匹配文本：

```
Violations Found:TEXT_XOR
- - - - -
Structure (Position x, y) BaselineText CheckText
- - - - -
A      (x1, y1)      base_text    check_text
```

若有一个未匹配基线文本且无未匹配检查文本（或相反）：

```
Violations Found:TEXT_XOR
- - - - -
Structure (Position x, y) BaselineText CheckText
- - - - -
A      (x1, y1)      base_text
```

若基线数据库在 (x1,y1) 有多个未匹配文本，且检查库至少一个未匹配文本：

```
Violations Found:TEXT_XOR
- - - - -
Structure (Position x, y) BaselineText CheckText
- - - - -
A      (x1, y1)      base_text1    check_text1
A      (x1, y1)      base_text1    check_text2
A      (x1, y1)      base_text2    check_text1
A      (x1, y1)      base_text2    check_text2
A      (x1, y1)      base_text3    check_text1
A      (x1, y1)      base_text3    check_text2
```

上例中有三个未匹配基线文本与两个未匹配检查文本。一般报告未匹配文本的全部可能组合。

B.7 LVL 输出

LVL Output

LVL 实用程序错误保存在「错误文件」所述文件中。

输出库格式与基线库相同。默认输出单元来自基线库。VUE 将差异叠加到基线库上。

- 若输入为 GDSII 或 OASIS 且 LVL 使用层文件运行，差异报告在 LVL_ERRORS 所列层上，可能与基线层或检查版图不同。例如，基线层 {31,0} 与检查层 {131,0}（经 LVL 选项映射到同一层）的差异可报告在输出层 {1,23}。
- 若输入为 GDSII 或 OASIS，使用层映射文件且无层文件，差异报告在由映射文件名称确定的层上。例如层名为 L10D15 时，差异报告在层 {10,15}。
- 若输入为 GDSII 或 OASIS，且既无层映射也无层文件，两层差异报告在输出库的同一层上。例如基线与检查均为 {31,0} 时，报告在输出层 {31,0}。

若某一输入文件（例如 OpenAccess 或 Milkyway）不是 GDSII 或 OASIS，差异报告在 LVL_ERRORS 所列层上，可能与基线或检查版图不同。例如基线 {31,0} 与检查 {131,0}（经选项映射到同一层）的差异可报告在输出层 {1,23}。

B.7.1 退出状态

Exit Status

icv_lv1 可执行文件的退出状态见表 B.3。

表 B.3: 退出状态值

退出状态值	定义
0	成功运行且未发现差异。
1	成功运行且发现差异。
负数	发生错误。

示例：成功且无差异：

```
% icv_lv1 lib1 lib2 -c cell
% echo $?
0
```

B.7.2 错误文件

Error Files

表 B.4 说明生成并存放在当前工作目录的错误文件。

注意

可用 `-elpc` 指定每类检查的最大错误数。见表 B.1。

LVL 还提供屏幕输出，包含每次 XOR 或 NOT 运算的实时结果。

表 B.4: 错误文件

文件名	说明
cell1_vs_cell2.LVL_ERRORS	按层分隔的全部比较错误的错误文件。
cell1_vs_cell2.LVL_RESULTS	结果文件，简要描述运行各步并指出是否有错误。有错误时，结果文件指向失败运行的 <code>run_details</code> 目录中特定的 <code>cell.RESULTS</code> 文件。关于指定 <code>cell.RESULTS</code> ，见 <i>IC Validator Reference Manual</i> 中 <code>run_options()</code> 。
cell1_vs_cell2.gds cell1_vs_cell2.oasis cell1_vs_cell2/	含比较差异的输出库（GDSII、OASIS、Milkyway、OpenAccess、NDM 格式）。对 OpenAccess 库，会创建或更新当前目录的 <code>lib.defs</code> 以加入输出库定义。
lvl_details/pydb	SQLite 数据库（错误数据库 PYDB），按 LVL 执行的每次 XOR 或 NOT 运算分隔存放比较错误。
lvl_details/cell1_vs_cell2.lvl_sum	摘要文件，含每次 LVL 运行的： <code>icv_lv1</code> 命令行；成功或失败指示；运行目的；耗时与峰值内存。
cell1_vs_cell2.vue	将差异叠加到基线库的文件，便于用 VUE 查看错误。

B.7.3 输出目录

Output Directories

表 B.5 说明 LVL 在当前工作目录创建的目录。

表 B.5: 输出目录

目录名	说明
lvl_details/compare_layers/	含基线库与检查库比较过程的额外诊断信息。
lvl_details/extract_layers/	含检查库预处理的摘要信息。
lvl_details/layer_list1/	若无层列表，为基线库自动生成的层列表；含层列表创建摘要供诊断。
lvl_details/layer_list2/	若无层列表，为检查库自动生成的层列表；含层列表创建摘要供诊断。

B.8 使用模型

Use Models

本节给出 LVL 实用程序的两个简单使用模型。

- 用两次 NOT 运算比较基线库与检查库：library1 的全部层与 library2 的全部层比较：

```
icv_lvl library1 library2 -c cell -not
```

- 以 cell 为顶层单元，从输入库生成层文件；输出文件对单元子树中发现的每个 layer/datatype 含一条 assign 语句：

```
icv_lvl library -af output_file -c cell
```

B.9 QuickLVL 实用程序

QuickLVL Utility

本节描述 Quick LVL 实用程序两种使用模型，可快速读取两个版图并按单元逐一比较版图元素。

- 单元级 Quick LVL：Quick LVL 的独立运行。
- 自动增量 DRC：IC Validator 先调用 Quick LVL 相对基线确定差异，结果以工具可读的内部格式传递，无需用户干预。

Quick LVL 支持下列版图输入格式：

- OASIS
- GDSII

注意

Quick LVL 实用程序支持除 NULL 字符外的任意文本字符。

图 B.1 展示 Quick LVL 流程。

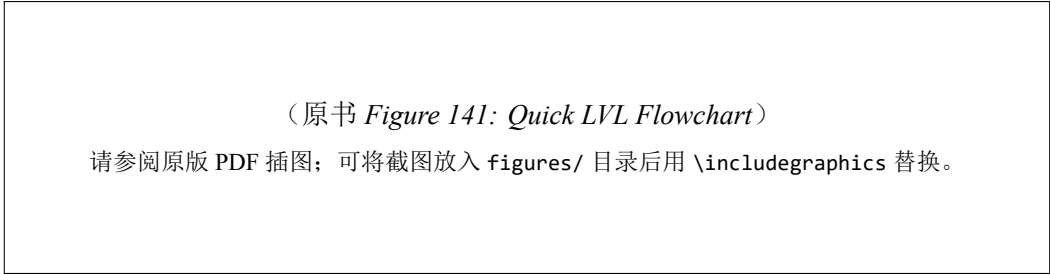


图 B.1: Quick LVL 流程图

B.9.1 单元级 Quick LVL

Cell-Level Quick LVL

Quick LVL 读取两个版图，按单元逐一比较版图元素。结果存入错误数据库 PYDB。该模式生成 `LVL_ERRORS` 与 `VUE` 格式输出文件。

命令行语法为：

```
icv_lvl baseline_layout check_layout -c top_cell [other options] -quick
```

表 B.1 说明 QuickLVL 的命令行选项。

B.9.2 LVL 与 QuickLVL 的差异

Differences Between LVL and QuickLVL

单元级 Quick LVL 给出 `baseline_layout` 与 `check_layout` 之间的差异列表。与传统全版图 LVL 的差异如下：

- 结果按单元级报告。
- 报告元素级差异，而非几何差异。例如某一矩形在一个版图中更大时，报告两个完整矩形元素，而非其几何差。
- 若命令行定义了变换，工具在尝试匹配基线版图前，先变换检查版图顶层单元中的坐标。
- 若同一元素在两个版图中以不同格式表示（例如 `path` 对 `rectangle`，或 `AREF` 对一组 `SREF`），即使几何上相同也不匹配，报告为“缺失”与“新增”元素。
- QuickLVL 在内部尝试匹配版图间元素。有一定相似性的元素报告为基线与检查之间的“不同”；其他元素报告为检查版图中缺失，或基线中的新形状。
- 可能出现一些误报（`false-positive`），以换取更高速度。
- QuickLVL 仅在单主机上工作。

该工具从 `LVL_ERRORS` 文件提供如下输出示例：

```
Path Mismatch (5:0)
  Baseline shapes different in Check ..... 25 violations found.

Placement Mismatch
  Baseline shapes missing in Check ..... 20 violations found.
  New shapes in Check not in Baseline ..... 150 violations found.

Path Mismatch (10:0)
  New shapes in Check not in Baseline ..... 75 violations found.
```

B.9.3 聚焦差异层与区域的自动增量 DRC

Auto-Incremental DRC Focused on Differing Layers and Regions

使用 `-idrc_i` 命令行选项时，IC Validator 先运行单元级 Quick LVL，将当前版图与基线版图比较。这些差异用于让工具聚焦含差异的层与窗口，从而改善运行时间。窗口 `ambit` 可通过设置 `incremental_options(window_ambit = ...)` 调整。

在自动增量 DRC 流程中，不推荐使用 `incremental_options()` 中除 `window_ambit` 以外的选项。该流程要求基线版图与检查版图具有相同的顶层单元名。自动增量 DRC 的结果可能无法完全替代完整的签核（signoff）运行。

自动增量 DRC 流程支持下列格式的基线版图：

- OASIS
- GDSII

图 B.2 展示自动增量 DRC 流程。

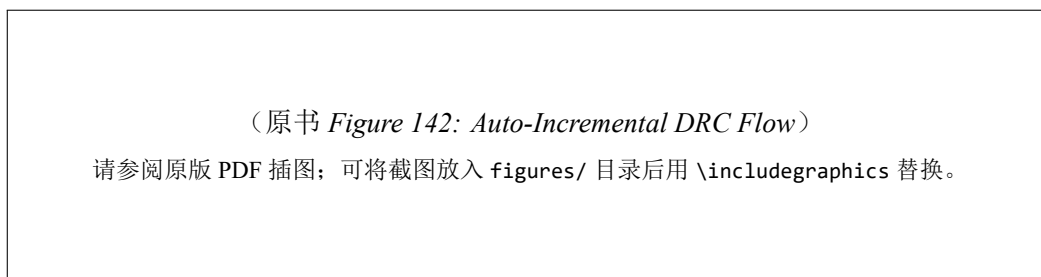


图 B.2: 自动增量 DRC 流程

附录 C

版图完整性管理

Layout Integrity Management

本附录介绍 IC Validator 工具提供的版图完整性管理（layout integrity management）功能。

版图完整性管理包括以下各节：

- 使用版图完整性管理（Using Layout Integrity Management）
- 版图完整性管理示例（Layout Integrity Management Examples）
- `icv_lidb` 实用程序（The `icv_lidb` Utility）
- `icv_lidb_report` 实用程序（The `icv_lidb_report` Utility）

C.1 使用版图完整性管理

Using Layout Integrity Management

版图完整性管理用于验证黄金（golden）知识产权（IP）、标准单元版图或任意其他版图单元的完整性。它提供一种便捷方式，在不做完整版图对版图比较的情况下，验证某些版图单元是否未发生变化。`icv_lidb` 实用程序从输入版图创建版图完整性数据库（LIDB）；随后可在 IC Validator 运行中使用该 LIDB，在读取设计时检查版图单元。`icv_lidb_report` 实用程序用于查看 LIDB 内容。

注意

尽管误匹配（false match）极不可能，该流程并不能替代流片前的完整验证运行。

对每个单元，LIDB 包含：

- 每层几何数据与文本的一个校验和；
- 每个唯一子单元名的全部引用的一个校验和。

LIDB 中每个单元可有多组校验和。若设计单元与该单元在 LIDB 中存储的任一组校验和匹配，则视为匹配。这使得单个 LIDB 可为具有不同校验和的等价单元服务。例如，不同版图格式可能生成略有不同的校验和，或流程中的工具可能改变数据（如合并相邻或重叠形状）。在这些情况下，可将新的一组校验和合并进现有 LIDB，使该单元的任一版本都能通过版图完整性检查。

设计中单元的版图完整性检查有三种方式：

- 使用 `icv_lidb` 对照现有 LIDB 验证输入版图，而无需完整 IC Validator 运行。实用程序按单元名将版图单元与 LIDB 中的一组校验和匹配。

- 在 IC Validator runset 中使用 `layout_integrity_by_cell()`，按名称检查单元。若设计单元计算出的校验和与 `layout_integrity_options()` 中任一输入 LIDB 里同名单元的任一组校验和匹配，则视为匹配。
- 在 runset 中使用 `layout_integrity_by_marker_layer()`，按标记层检查单元。若设计单元计算出的校验和与任一输入 LIDB 中在标记层上有数据的单元的任一组校验和匹配，则视为匹配。

IC Validator 运行中发现的版图完整性不匹配存入错误数据库，报告在 `LAYOUT_ERRORS` 文件中，并可在 VUE 中查看。这些错误无特定坐标，因此 VUE 中不会高亮显示。

版图完整性检查结果可用于控制随后在这些单元中发现的 DRC 错误的处理：可丢弃、按正常方式报告，或使用 DRC 分类。

C.2 版图完整性管理示例

Layout Integrity Management Examples

以下各节给出版图完整性管理功能的若干示例：

- 基本示例（Basic Example）
- 在不同版图格式上运行（Running on a Different Layout Format）
- 合并 LIDB 的不匹配报告（Reporting of Mismatches From a Merged LIDB）
- 标记层检查（Marker Layer Check）
- 混合 Milkyway 与 GDSII 流程（Mixed Milkyway and GDSII Flow）

C.2.1 基本示例

Basic Example

下例展示按单元名的基本版图完整性检查。首先从输入版图创建 LIDB：

```
icv_lidb -i EX_ADDER_3.GDS -o orig.lidb
```

然后在 runset 中加入下列内容，IC Validator 即可按计划运行：

```
layout_integrity_options(
  databases = {
    { db_name = "orig.lidb" }
  }
);

{
  @ "Layout Integrity Check";
  layout_integrity_by_cell(
    cells = {"*"}
  );
}
```

本次运行因输入版图与生成 LIDB 的版图相同，`LAYOUT_ERRORS` 中不会报告版图完整性不匹配。但可通过检查 `run_details/lidb.log` 确认已执行完整性检查：

```
-----
Layout Integrity Processing Log
11/08/2010 08:12:50
-----
```

```
Processing layout cell VIA
Cell check:
  Result      = Matched
  Matched Cell = VIA
  Database     = /home/user/orig.lidb

Processing layout cell INV
Cell check:
  Result      = Matched
  Matched Cell = INV
  Database     = /home/user/orig.lidb
...

```

每个被检查单元的版图完整性状态也可见于 `run_details/hierarchy/cell.tree0` 文件：

```
CELL = AD4FUL
      Cell Status           = USED
      GDSII Library         = MAINLIB
      Layout Integrity Check = Passed Completely

```

`cell.tree0` 中版图完整性检查状态有三种可能值：

- **Passed Completely**: 该单元及其下放置的全部单元均已检查并匹配。
- **Passed**: 该单元本身通过检查，但至少一个子单元未检查或检查失败。
- **Failed**: 未找到该单元的版图完整性匹配。

C.2.2 在不同版图格式上运行

Running on a Different Layout Format

下例展示对同一设计的不同版图格式运行的结果。使用不同格式可能导致不同的校验和数据。常见原因包括：

- 某些格式（如 Milkyway）可有单元边界层，而另一些（如 GDSII）没有。
- 某些格式以不同方式表示单元放置。例如 OASIS 不直接支持 AREF 概念，阵列放置表示为带 `repetition` 的多次 SREF；而 IC Validator 内部可能将这些放置解释为 AREF。
- 几何数据在格式间可有多种差异但仍物理等价，例如点序、重叠形状合并、共线点去除等。版图完整性管理按原样检查原始输入版图，不做任何变换。

当上一示例的 IC Validator 运行改用等价的 Milkyway 输入时，会因单元边界层而报告版图完整性不匹配：

```
Layout Integrity Check
  layout_integrity_by_cell:layer_mismatch ..... 9 violations found.
...

runset1.rs:17:layout_integrity_by_cell:layer_mismatch
- - - - -
Structure Layer    LIDB      Golden Layout
- - - - -
AD4FUL    (255;0) orig.lidb
VIA       (255;0) orig.lidb
ADFULAH   (255;0) orig.lidb
POLYHD    (255;0) orig.lidb

```

INV	(255;0) orig.lidb
INVH	(255;0) orig.lidb
DGATE	(255;0) orig.lidb
CONT	(255;0) orig.lidb
TGATE	(255;0) orig.lidb

在等价 OASIS 版图上运行时，会因单元放置被解释为 AREF 而报告不匹配：

```
Layout Integrity Check
  layout_integrity_by_cell:placement_mismatch .... 6 violations found.
...

runset1.rs:17:layout_integrity_by_cell:placement_mismatch
- - - - -
Structure Child Cell LIDB      Golden Layout
- - - - -
AD4FUL    ADFULAH    orig.lidb
AD4FUL    VIA        orig.lidb
ADFULAH   INV        orig.lidb
ADFULAH   INVH       orig.lidb
DGATE     TGATE      orig.lidb
DGATE     VIA        orig.lidb
```

由于这三种版图等价，可合并为单个 LIDB；该 LIDB 在三种格式上均能通过：

```
icv_lidb -i EX_ADDER_3 EX_ADDER_3.GDS EX_ADDER_3.oas -o merged.lidb
```

C.2.3 合并 LIDB 的不匹配报告

Reporting of Mismatches From a Merged LIDB

由于单个单元可有多组校验和，某些校验和可匹配而另一些不能。仅当某一组 LIDB 校验和中的全部层与放置校验和均匹配时，单元才视为匹配。下例展示合并 LIDB 中不同场景下的错误报告差异：

```
validate_layout.rs:14:layout_integrity_by_cell:layer_mismatch
- - - - -
Structure Layer LIDB      Golden Layout
- - - - -
TOP      (1;0) merged.lidb 2.gds
TOP      (2;0) merged.lidb 1.gds
TOP      (3;0) merged.lidb
```

上例中单元 TOP 无完整匹配。层 1 匹配版图 1.gds 中的 TOP，但不匹配 2.gds，故不匹配相对黄金版图 2.gds 报告；层 2 匹配 2.gds 而不匹配 1.gds，相对 1.gds 报告；层 3 不匹配任何黄金版图，该列留空。

同一场景下的单元放置不匹配：

```
validate_layout.rs:15:layout_integrity_by_cell:placement_mismatch
- - - - -
Structure Child Cell LIDB      Golden Layout
- - - - -
TOP      A      merged.lidb LIB2
TOP      B      merged.lidb LIB1
TOP      C      merged.lidb
```

上例中 TOP 无完整匹配。子单元 A 的放置匹配 LIB1 中 TOP 的放置而不匹配 LIB2，故相对 LIB2 报告；子单元 B 匹配 LIB2 而不匹配 LIB1，相对 LIB1 报告；子单元 C 不匹配任何黄金版图，该列留空。

C.2.4 层次模式

Hierarchical Mode

IC Validator 的版图完整性检查有两种操作模式：

- 单元级（**Cell level**）：单独考虑每个单元，不考虑层次中任何子单元是否通过或失败。此为默认模式。
- 层次（**Hierarchical**）：考虑层次中子单元的状态。子单元的任何版图完整性不匹配也会作为父单元中的放置不匹配报告，并一直向上传播到整个层次。

不匹配仅在检查函数所指定的单元中报告：

- `layout_integrity_by_cell()`：由 `cells` 参数指定单元。
- `layout_integrity_by_marker_layer()`：单元为在 `marker_layer` 指定层上含多边形的单元。

C.2.5 层次单元检查示例

Hierarchical Cell Check Example

本例展示 `layout_integrity_by_cell()` 在层次模式下的检查行为。设计层次见图 C.1。

（原书 *Figure 143: Design Hierarchy for Hierarchical Cell Check Example*）
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 C.1: 层次单元检查示例的设计层次

本例中，单元 A 的某层形状与版图完整性数据库中的校验和不同。

注意

尽管示例使用放置单元且单元名匹配，层次模式并不要求如此。检查单元下放置的单元须有相同内容，但不必有相同单元名。

下列代码在层次模式下检查全部单元：

```
layout_integrity_by_cell(
    cells = { "*" },
    processing_mode = HIERARCHICAL
);
```

差异报告在 A、C 与 TOP。LAYOUT_ERRORS 输出为：

```
test.rs:15:layout_integrity_by_cell:layer_mismatch
- - - - -
Structure Layer LIDB      Golden Layout
- - - - -
A      (31;0) test.lidb
```

```
test.rs:15:layout_integrity_by_cell:placement_mismatch
```

```
-----
Structure Child Cell LIDB      Golden Layout
-----
TOP      A      test.lidb
TOP      C      test.lidb
C        A      test.lidb
```

下列代码检查单元 C 与 TOP，但不检查单元 A:

```
layout_integrity_by_cell(
    cells = { "*", "!A" },
    processing_mode = HIERARCHICAL
);
```

差异仍报告在 C 与 TOP。层次模式规定：被检查的特定单元之下的一切也会被考虑。输出为:

```
test.rs:15:layout_integrity_by_cell:placement_mismatch
```

```
-----
Structure Child Cell LIDB      Golden Layout
-----
TOP      A      test.lidb
TOP      C      test.lidb
C        A      test.lidb
```

run_details/lidb.log 含额外细节,指示层次中每个单元的状态。本例中单元 A 被处理,但因未在 layout_integrity_by_cell() 中指定,没有“Cell check”条目:

```
-----
                        Layout Integrity Processing Log
                        04/23/2014 06:13:36
-----

Processing layout cell A
  Database: test.lidb
    All Layers: No hierarchically equivalent cells found

Processing layout cell B
  Database: test.lidb
    All Layers: Hierarchically equivalent cells found
  Cell check (hierarchical):
    Result      = Matched
    Matched Cell = B
    Database     = test.lidb

Processing layout cell C
  Database: test.lidb
    All Layers: No hierarchically equivalent cells found
  Cell check (hierarchical):
    Result      = No Match Found

Processing layout cell TOP
  Database: test.lidb
```


All Layers: No hierarchically equivalent cells found
Cell check (hierarchical):
Result = No Match Found

C.2.6 层次标记层检查示例

Hierarchical Marker Layer Check Example

本例展示 `layout_integrity_by_marker_layer()` 在层次模式下的检查行为。设计层次见图 C.2。



图 C.2: 层次标记层检查示例的设计层次

本例中，仅单元 M1–M6 在标记层上有数据。单元 B 在层 3 上有差异；单元 M5 在层 4 上有差异。

注意

尽管示例使用放置单元且单元名匹配，层次模式并不要求如此。检查单元下放置的单元须有相同内容，但不必有相同单元名。

若干层次模式检查示例如下：

```
// 报告 M1、M2、M4、M5 中的不匹配
layout_integrity_by_marker_layer(
    marker_layer = {1},
    processing_mode = HIERARCHICAL
);

// 仅报告 M2、M5：被检查且有差异的层仅为层 4
layout_integrity_by_marker_layer(
    marker_layer = {1},
    check_layers = { {2}, {4} },
    processing_mode = HIERARCHICAL
);

// 仅报告 M1、M4：被检查且有差异的层仅为层 3
layout_integrity_by_marker_layer(
    marker_layer = {1},
    check_layers = { {3} },
    processing_mode = HIERARCHICAL
);

// 无报告：被检查层中无差异
layout_integrity_by_marker_layer(
```

```

marker_layer = {1},
check_layers = { {2} },
processing_mode = HIERARCHICAL
);

```

不匹配在 LAYOUT_ERRORS 中报告:

```

test2.rs:17:layout_integrity_by_marker_layer:cell_mismatch
- - - - -
Structure Marker Layer
- - - - -
M1      (1;0)
M2      (1;0)
M4      (1;0)
M5      (1;0)

test2.rs:22:layout_integrity_by_marker_layer:cell_mismatch
- - - - -
Structure Marker Layer
- - - - -
M2      (1;0)
M5      (1;0)

test2.rs:28:layout_integrity_by_marker_layer:cell_mismatch
- - - - -
Structure Marker Layer
- - - - -
M1      (1;0)
M4      (1;0)

```

每个单元状态的细节报告在 run_details/lidb.log 中 (节选):

```

Processing layout cell C
Database: test.lidb
  All Layers: Hierarchically equivalent cells found
  Layers (3;0): Hierarchically equivalent cells found
  Layers (4;0) (2;0): Hierarchically equivalent cells found
  Layers (2;0): Hierarchically equivalent cells found
Marker Layer Check:
  Result      = No Data on Marker Layer

Processing layout cell B
Database: test.lidb
  All Layers: No hierarchically equivalent cells found
  Layers (3;0): No hierarchically equivalent cells found
  Layers (4;0) (2;0): Hierarchically equivalent cells found
  Layers (2;0): Hierarchically equivalent cells found
Marker Layer Check:
  Result      = No Data on Marker Layer

Processing layout cell M6
...

```

```

Marker Layer Check:
  Result      = Matched
  Marker Layer = (1;0)
  Matched Cell = M6
  Database     = test.lidb

Processing layout cell M5
...
Marker Layer Check:
  Result      = Matched
  Marker Layer = (1;0)
  Matched Cell = M5
  Database     = test.lidb

Processing layout cell TOP
Marker Layer Check:
  Result      = No Data on Marker Layer

```

C.2.7 标记层检查

Marker Layer Check

`layout_integrity_by_marker_layer()` 不按名称检查单元，而是检查在指定标记层上有数据的任意单元。它在输入 LIDB 中查找同样在标记层上有数据的任意单元的任一组匹配校验和。

```

{
  @ "Layout Integrity Check";
  layout_integrity_by_marker_layer(
    marker_layer = { 8,0 }
  );
}

```

本例以层 8 为标记层检查单元。`run_details/lidb.log` 中匹配与不匹配单元示例如下：

```

-----
                        Layout Integrity Processing Log
                        11/08/2010 09:45:33
-----

Processing layout cell VIA
Marker Layer Check:
  Result      = No Match Found
  Marker Layer = (8;0)

Processing layout cell INV
Marker Layer Check:
  Result      = Matched
  Marker Layer = (8;0)
  Matched Cell = INV
  Database     = /home/user/orig.lidb

```

从 `cell.LAYOUT_ERRORS` 可见，标记层检查的不匹配按通过/失败报告，不报告单个层或单元引用的不匹配：

```
Layout Integrity Check
  layout_integrity_by_marker_layer:cell_mismatch ... 3 violations found.
...

runset8.rs:17:layout_integrity_by_marker_layer:cell_mismatch
- - - - -
Structure Marker Layer
- - - - -
AD4FUL      (8;0)
VIA          (8;0)
TGATE        (8;0)
```

若只想检查被标记单元中的特定层，使用 `check_layers` 参数：

```
{
  @ "Layout Integrity Check";
  layout_integrity_by_marker_layer(
    marker_layer = { 8,0 },
    check_layers = { {==6}, {==8}, {==10} }
  );
}
```

上例中，若单元在层 6、8、10 上均匹配，则视为完整匹配。

C.2.8 混合 Milkyway 与 GDSII 流程

Mixed Milkyway and GDSII Flow

对于混合 Milkyway 与 GDSII 流程，`layout_integrity_options()` 指定用于检查 Milkyway 单元的 LIDB。要对替换用的 GDSII 或 OASIS 文件执行版图完整性检查，须在 `milkyway_merge_library_options()` 中为每个替换库指定 LIDB。

```
library(
  library_name = "EX_ADDER_3",
  format = MILKYWAY,
  cell = "AD4FUL"
);

milkyway_merge_library_options(
  libraries = {
    {
      library_name = "EX_ADDER_3",
      replacement_libraries = {
        {
          file = "EX_ADDER_3.GDS",
          format = GDSII,
          layout_integrity = {
            { db_name = "gds.lidb" }
          }
        }
      }
    }
  }
}
```

```
    }
  }
);

layout_integrity_options(
  databases = {
    { db_name = "milkyway.lidb" }
  }
);

{
  @ "Layout Integrity Check";
  layout_integrity_by_cell(
    cells = {"*"}
  );
}
```

IC Validator 运行后，run_details/lidb.log 显示 Milkyway 单元对照 milkyway.lidb 检查，而 GDSII 单元对照 gds.lidb 检查：

```
-----
                        Layout Integrity Processing Log
                        11/08/2010 09:56:43
-----

Processing layout cell AD4FUL
Cell check:
  Result      = Matched
  Matched Cell = AD4FUL
  Database     = /home/user/milkyway.lidb

Processing layout cell VIA
Cell check:
  Result      = Matched
  Matched Cell = VIA
  Database     = /home/user/gds.lidb
```

C.3 icv_lidb 实用程序

The icv_lidb Utility

icv_lidb 实用程序管理 LIDB。可用于：

- 从输入版图创建 LIDB；
- 合并多个 LIDB；
- 使用现有 LIDB 快速验证输入版图。

C.3.1 命令行选项

Command-Line Options

icv_lidb 的命令行语法支持三种简单使用模型。

- 从输入版图生成 LIDB:

```
icv_lidb -i input_layout -o output_lidb [-c top_cell]
```

- 使用现有 LIDB 验证输入版图:

```
icv_lidb -i input_layout -validate golden_lidb [-c top_cell]
```

- 合并多个 LIDB:

```
icv_lidb -merge lidb1 lidb2... -o output_lidb
```

表 C.1 说明该实用程序的命令行选项。

表 C.1: icv_lidb 实用程序命令行选项

语法	说明
-c <i>cell</i>	输入版图的顶层单元。省略时处理库中全部单元。
-elpc <i>num</i> -error_limit_per_check <i>num</i>	设置每类检查的错误上限。默认值为 100。注意: 设为 <code>ERROR_LIMIT_MAX</code> 时存储直至工具定义最大上限的全部错误。也可给出具体数字提高上限; 但对大量错误可能降低性能并增加磁盘占用, 不推荐。设为 0 抑制存储全部错误。
-help	显示命令行选项列表并退出。
-i <i>input_layout ...</i>	创建 LIDB 或对照现有 LIDB 验证版图时的输入版图名。创建 LIDB 时可指定多个文件, 将全部版图的校验和纳入输出 LIDB。各格式示例: - GDSII: <code>icv_lidb -i /path/to/file.gds</code> - OASIS: <code>icv_lidb -i /path/to/file.oas</code> - Milkyway: <code>icv_lidb -i /library/path/LIBRARY_NAME</code> - OpenAccess: <code>icv_lidb -i /path/to/lib.defs/LIBRARY_NAME</code>
-lf <i>filename</i>	选择写入版图完整性数据库的层与对象类型(多边形、边、文本)子集。例如, 仅当层文件包含包含 <code>layer1</code> 的 <code>assign_edge()</code> 时, 层 1 上的边才写入数据库。层文件存入 LIDB, 使对照该数据库的任何验证运行都限制在层文件中的层与对象类型。层文件不得含 <code>options</code> 或 <code>runset</code> 函数。若文件为空或不含 <code>assign</code> 语句, 则忽略该选项, 行为如同未指定。
-merge <i>lidb ...lidb</i>	将指定的输入 LIDB 合并到输出数据库。
-o <i>output_lidb</i>	生成的输出版图完整性数据库(LIDB)。若文件已存在则覆盖。
-oa_layer_map <i>layer_mapping_file</i>	OpenAccess 数据库的可选层映射文件。映射文件中不存在的层上的数据不读取。见 <code>openaccess_options()</code> 的 <code>layer_mapping_file</code> 参数。
-oa_view <i>view_name</i>	OpenAccess 数据库顶层单元的可选视图。

语法	说明
-of <i>options_file</i>	可选文件，含 IC Validator runset OPTIONS 节。可含任意 OPTIONS 节函数，包括 <code>gds_options()</code> 、 <code>milkyway_options()</code> 、 <code>oasis_options()</code> 。
-V	显示实用程序版本并退出。
-validate <i>golden_lidb</i>	对照指定的黄金 LIDB 对输入版图执行版图完整性检查。结果在 <code>cell.LAYOUT_ERRORS</code> 文件中。

C.3.2 退出状态

Exit Status

`icv_lidb` 的退出状态见表 C.2。

表 C.2: `icv_lidb` 退出状态值

退出状态值	定义
0	成功运行。若验证输入版图，则未发现差异。
1	成功运行且发现差异。
负数	发生错误。

C.3.3 输出文件

Output Files

`icv_lidb` 的输出文件见表 C.3。

表 C.3: `icv_lidb` 输出文件

文件名	说明
<code>lidb_details/icv_lidb.sum</code>	摘要文件，含运行信息；屏幕全部输出写入该文件。含：所用命令行选项；输入输出文件信息；运行时间与内存；成功或失败指示。
<code>lidb_details/icv_lidb.log</code>	版图验证或数据库生成运行中所处理全部单元的日志。验证运行中可能指出设计单元是否匹配黄金 LIDB 单元，以及匹配的是哪个黄金 LIDB 单元。
<code>cell.LAYOUT_ERRORS</code>	版图验证运行中发现的全部版图完整性错误。若未给出顶层单元，文件名为 <code>cell.LAYOUT_ERRORS</code> 。

C.4 `icv_lidb_report` 实用程序

The `icv_lidb_report` Utility

`icv_lidb_report` 实用程序报告 LIDB 内容。对数据库中每一组校验和，显示黄金版图路径、用户 ID 与时间戳，并列有条目的单元。输出发送到屏幕，以及当前目录中的 `icv_lidb_report.sum` 摘要文件。

C.4.1 命令行选项

Command-Line Options

命令行语法为：

```
icv_lidb_report -i input_lidb
```

表 C.4 说明命令行选项。

表 C.4: icv_lidb_report 命令行选项

语法	说明
-help	显示命令行选项列表并退出。
-i <i>input_lidb</i>	要报告的输入版图完整性数据库。
-V	显示实用程序版本并退出。

C.4.2 输出文件

Output Files

输出文件见表 C.5。

表 C.5: icv_lidb_report 输出文件

文件名	说明
icv_lidb_report.sum	摘要文件，含运行信息；屏幕全部输出写入该文件。含：所用命令行选项；输入输出文件信息。

附录 D

IC Validator PXL Debugger

IC Validator PXL Debugger

IC Validator PXL Debugger 协助你构造 `runset` 或 `runset` 的一部分，使你能够查看正在编写的每条设计规则或器件配置函数中的中间阶段。

PXL Debugger 包括以下各节：

- 运行 PXL Debugger（Running the PXL Debugger）
- PXL Debugger 命令（PXL Debugger Commands）

D.1 运行 PXL Debugger

Running the PXL Debugger

要运行 PXL Debugger，在 IC Validator 命令中使用 `-debug` 命令行选项：

```
% icv -debug runset.rs [-debug_source file]
```

`-debug_source` 命令行选项允许以批处理模式运行 PXL Debugger。指定先前调试运行中用 `record` 命令写出的文件。PXL Debugger 的输出显示在屏幕上。

当 `runset` 执行开始时：

- 屏幕消息指示 PXL Debugger 已启动；
- 运行在第一个断点处暂停。显示可执行行之前五与之后四行；
- 显示 PXL Debugger 提示符（ICVD>>）。

例如：

```
Starting ICV Debugger
13 |         t = t + dttoi((x * y**2) / z);
14 |     }
15 |     t = dttoi(t / x);
16 |     note (" f2(3) = "+ t);
17 | }
--> 18 | xx:integer;
    19 | f2(3);
    20 | note("end. ");
```

```
ICVD>>
```

在提示符处输入希望 PXL Debugger 执行的操作。见下文「PXL Debugger 命令」。

注意

PXL Debugger 设计为顺序执行 PXL 代码。若启用了分布式处理选项，将生成警告并忽略这些选项。
PXL Debugger 在 runset 与用户函数中单步执行代码，但不在远程（remote）函数中单步执行。

D.2 PXL Debugger 命令

PXL Debugger Commands

IC Validator PXL Debugger 的命令见表 D.1。^a

表 D.1: PXL Debugger 命令

命令	定义
h[elp]	打印 PXL Debugger 命令列表。
s[tep]	步入函数调用。见 step 命令说明。
n[ext]	一次性执行完函数。见 next 命令说明。
q[uit]	退出 PXL Debugger 与 IC Validator 运行。见 quit 命令说明。
c[ont]	继续直至下一断点或运行结束。见 cont 命令说明。
fin[ish]	执行直至当前函数返回。见 finish 命令说明。
b[reak] b[reak] [file:]line b[reak] function	设置断点。默认为当前文件: 行。见 break 命令说明。
info b[reak] [number]	显示断点状态。默认为全部断点。见 info break 命令说明。
dis[able] [number]	禁用断点。默认为禁用全部断点。见 disable 命令说明。
en[able] [number]	启用断点。默认为启用全部断点。见 enable 命令说明。
d[ele]te number	删除指定断点。见 delete 命令说明。
l[ist] l[ist] [file:]line l[ist] function	列出源码。默认为当前文件与行。见 list 命令说明。
where	显示函数调用栈。见 where 命令说明。
pwd	打印 IC Validator PXL Debugger 的当前工作目录。例如: ICVD>> pwd Working directory /remote/data/ICV
p[rint] expression	打印表达式的值。见 print 命令说明。
record [file]	保存当前调试会话的命令。默认为 ICVpid.pdi 。见 record 命令说明。

命令	定义
<code>source file</code>	从文件执行命令。见 <code>source</code> 命令说明。

^a 命令可按方括号所示缩写。

D.2.1 break

break

`break` 命令设置断点。有四种方式：

1. `break`（无选项）：在当前文件的当前行设置断点。

```

4 |
5 | f2 : function (
6 |     x : integer
7 | ) returning ans : integer
8 | {
--> 9 |     ans = x;
10 |     note("0. ans = " + ans);
11 | }
12 |
13 | xx: integer = 0;
ICVD>> b
Breakpoint #1, f2() at test.rs:9
ICVD>>
```

2. `break line`：在当前文件的指定行号设置断点。

```

ICVD>> b 13
Breakpoint #1,      f2() at test.rs:13
ICVD>>
```

3. `break file:line`：在指定文件的指定行号设置断点。

```

ICVD>> b test.rs:15
Breakpoint #2, _MAIN_() at          test.rs:15
ICVD>>
```

4. `break function`：在指定函数内第一条可执行行设置断点。

```

8 | {
9 |     ans = x;
10 |     note("0. ans = " + ans);
11 | }
12 |
--> 13 | xx: integer = 0;
14 | xx = f2(3);
15 | note("1. f2(3)= " + xx);
ICVD>> b f2
Breakpoint #1,      f2() at test.rs:9
ICVD>>
```

此方法可在重载函数上设置断点。下例中 f2 可返回 integer 或 void。指定 2 选择选项 2（返回 integer）：

```
ICVD>> b f2
[0] cancel
[1] all
[2] f2() returning integer, at test.rs:9
[3] f2() returning void, at test.rs:16
> 2
```

指定 1 选择选项 1（返回 integer 与 void）：

```
Breakpoint #1, f2() at test.rs:9
ICVD>> b f2
[0] cancel
[1] all
[2] f2() returning integer, at test.rs:9
[3] f2() returning void, at test.rs:16
> 1
Breakpoint #2, f2() at test.rs:9
Breakpoint #3, f2() at test.rs:16
ICVD>>
```

D.2.2 cont

cont

cont 命令执行 runset 直至完成，或在指定断点处停止。

下例中 runset 完成调试运行：

```
1 | #include <math.rh>
2 | #include <diagnostics.rh>
3 |
--> 4 | f: list of integer = {1, 1};
5 |
6 | for (i in 2 thru 19) {
7 |     f.push_back(f[i-2] + f[i-1]);
8 | }
ICVD>> c

Completed ICV Debugger Run
ICVD>>
```

下例中执行停在循环内的断点：

```
ICVD>> b 13
Breakpoint #1,      f2() at test.rs:13
ICVD>> c
8 | {
9 |     t : integer = 0;
10 |    y : integer = 10;
11 |    z : integer = 100;
12 |    for ( i=1 to x) {
```

```

B --> 13 |          t = t + dtol((x * y**2) / z);
        14 |      }
        15 |      t = dtol(t / x);
        16 |      note (" f2(3) = "+ t);
        17 | }
Breakpoint #1,      f2() at test.rs:13
ICVD>> c
...
Breakpoint #1,      f2() at test.rs:13
ICVD>>

```

D.2.3 delete

delete

`delete` 命令删除指定断点。格式为：

```
delete number
```

```

ICVD>> info b
Breakpoint #1,      _MAIN_() at      test.rs:7      Enabled
Breakpoint #2,      _MAIN_() at      test.rs:16      Enabled
ICVD>> d 1
ICVD>> info b
Breakpoint #2,      _MAIN_() at      test.rs:16      Enabled
ICVD>>

```

D.2.4 disable

disable

`disable` 命令禁用指定断点。有两种方式：

1. `disable`（无断点号）：禁用全部断点。

```

ICVD>> info b
Breakpoint #1,      _MAIN_() at      test.rs:7      Enabled
Breakpoint #2,      _MAIN_() at      test.rs:18      Enabled
ICVD>> dis
ICVD>> info b
Breakpoint #1,      _MAIN_() at      test.rs:7      Disabled
Breakpoint #2,      _MAIN_() at      test.rs:18      Disabled

```

2. `disable number`：仅禁用该断点。

```

ICVD>> dis 1
ICVD>> info b
Breakpoint #1,      _MAIN_() at      test.rs:13      Disabled
Breakpoint #2,      _MAIN_() at      test.rs:16      Enabled
ICVD>>

```

D.2.5 enable

enable

`enable` 命令启用先前禁用的断点。有两种方式：

1. `enable`（无断点号）：启用全部断点。

```
ICVD>> info b
Breakpoint #1,      _MAIN_() at      test.rs:7      Disabled
Breakpoint #2,      _MAIN_() at      test.rs:18     Disabled
ICVD>> en
ICVD>> info b
Breakpoint #1,      _MAIN_() at      test.rs:7      Enabled
Breakpoint #2,      _MAIN_() at      test.rs:18     Enabled
ICVD>>
```

2. `enable number`：仅启用该断点。

```
ICVD>> info b
Breakpoint #1,      _MAIN_() at      test.rs:7      Disabled
ICVD>> en 1
ICVD>> info b
Breakpoint #1,      _MAIN_() at      test.rs:7      Enabled
ICVD>>
```

D.2.6 finish

finish

`finish` 命令执行到所在函数结束。但若遇到断点仍会停止。

下例中运行继续直至当前函数完成：

```

8 | {
9 |     t : integer = 0;
10 |    y : integer = 10;
11 |    z : integer = 100;
12 |    for ( i=1 to x) {
--> 13 |        t = t + dtoi((x * y**2) / z);
14 |    }
15 |    t = dtoi(t / x);
16 |    note (" f2(3) = "+ t);
17 | }
ICVD>> fin
test.rs:16 f2(3) = 3
15 |    t = dtoi(t / x);
16 |    note (" f2(3) = "+ t);
17 | }
18 | xx:integer;
19 | f2(3);
--> 20 | note("end. ");
```

下例中运行继续直至遇到断点：

```

ICVD>> fin
test.rs:11 fibonacci(16) = 1597
...
B --> 11 |      note("fibonacci(" + i + ") = " + f[i]);
...
Breakpoint #2, _MAIN_() at      test.rs:11
ICVD>> fin
test.rs:11 fibonacci(17) = 2584
...
Breakpoint #2, _MAIN_() at      test.rs:11
ICVD>>

```

D.2.7 info break

info break

`info break` 命令列出本 PXL Debugger 会话中设置的断点信息。有两种方式：

1. `info break`（无断点号）：列出全部断点。

```

ICVD>> info b
Breakpoint #1,      _MAIN_() at      test.rs:7      Enabled
Breakpoint #2,      _MAIN_() at      test.rs:16     Enabled

```

2. `info break number`：仅列出该断点信息。

```

ICVD>> info b 2
Breakpoint #2, _MAIN_() at      test.rs:16      Enabled
ICVD>>

```

D.2.8 list

list

`list` 命令列出源码行。有四种方式：

1. `list`（无选项）：从上次打印的行起列出 10 行。

```

1 | #include <math.rh>
2 | #include <diagnostics.rh>
3 |
--> 4 | f: list of integer = {1, 1};
5 |
6 | for (i in 2 thru 19) {
7 |     f.push_back(f[i-2] + f[i-1]);
8 | }
ICVD>> list
9 |
10 | /* comment 1
...
17 | for (i in 0 thru 19) {

```

```

18 |      note("fibonacci(" + i + ") = " + f[i]);
ICVD>>

```

2. **list line:** 从当前文件打印指定行号之前五行与之后四行。

3. **list file:line:** 从指定文件打印指定行号之前五行与之后四行。

```

ICVD>>  list test.rs:4
1 | #include <math.rh>
2 | #include <diagnostics.rh>
3 |
--> 4 | f: list of integer = {1, 1};
5 |
6 | for (i in 2 thru 19) {
7 |     f.push_back(f[i-2] + f[i-1]);
8 | }
ICVD>>

```

4. **list function:** 打印指定函数内第一条可执行行之五行与之后四行。

```

ICVD>>  list f2
4 |
5 | f2 : function (
6 |     x : integer
7 | ) returning void
8 | {
9 |     t : integer = 0;
...
ICVD>>

```

D.2.9 next

next

next 命令将 PXL Debugger 提示符移到 **runset** 中下一可执行行。对函数调用执行 **next** 时，会完成该函数执行，再移到 **runset** 中下一可执行行。

在 **runset** 可执行行上给出 **next**:

```

13 |      t = t + dtol((x * y**2) / z);
...
--> 18 | xx:integer;
19 | f2(3);
20 | note("end. ");
ICVD>> n
...
--> 19 | f2(3);
20 | note("end. ");
ICVD>>

```

在函数调用上给出 **next**:

```

--> 13 | xx = f2(3);
14 | note("1. f2(3)= " + xx);

```



```
ICVD>> n
...
--> 14 | note("1. f2(3)= " + xx);
ICVD>>
```

D.2.10 print

print

print 命令打印变量与简单表达式的值。格式为：

```
print expression
```

打印变量值：

```
ICVD>> p ans
ans = 3
```

打印循环变量：

```
ICVD>> p i
i = 3
```

打印表达式：

```
ICVD>> p xx
xx = 0
ICVD>> p (xx+7)*20
(xx+7)*20 = 140
ICVD>>
```

D.2.11 quit

quit

quit 命令结束调试会话与 IC Validator 运行。

```
ICVD>> q
IC Validator Run: Time= 0:00:04 User=0.06 System=0.01
Overall icv_engine time = 0:00:04
IC Validator is done.
```

D.2.12 record

record

record 命令将当前调试会话的命令保存到 ASCII 文件。有两种方式：

1. record（未指定文件名）：命令保存到 *ICVpid.pdi*，其中 *pid* 为 UNIX 进程 ID。

```
ICVD>> record
Saving Commands to 'ICV22347.pdi'
ICVD>>
```

2. **record file**: 可指定带或不带路径的文件名。

```
ICVD>> record myfile.cmd
Saving Commands to 'myfile.cmd'
ICVD>>
```

ASCII 文件 **verbatim** 存储除 **break** 与 **list** 外的全部命令；**break** 与 **list** 以解析后的文件路径存储。例如会话后记录文件含：

```
b f2
list f2
b b.rs:6
q
```

D.2.13 source

source

source 命令执行指定文件中的 PXL Debugger 命令。命令完成后，控制返回用户，除非源文件中的 **quit** 使调试器结束。格式为：

```
source file
```

例如：

```
ICVD>> source ICVDebug9398.pdi
ICVD>> n
...
ICVD>> b 7
Breakpoint #1, _MAIN_() at      test.rs:7
ICVD>> b 16
Breakpoint #2, _MAIN_() at      test.rs:16
ICVD>> c
...
Breakpoint #1, _MAIN_() at      test.rs:7
End of commands from commandfile
ICVD>>
```

D.2.14 step

step

step 命令允许步入函数调用，执行函数内每一行。光标移到函数内第一条可执行行。

下例中步入 **f2**，第一条可执行行为 9：

```
--> 19 | f2(3);
      20 | note("end. ");
ICVD>> s
      4 |
      5 | f2 : function (
...
--> 9 |      t : integer = 0;
```

```
...  
ICVD>>
```

D.2.15 where

where

`where` 命令打印 `runset` 中某语句的函数调用栈。下例显示已有两次函数调用：

```
ICVD>> where  
#0      | sum()      at test.rs:27  
#1      | _MAIN_()   at test.rs:32
```

下例显示已有三次函数调用：

```
ICVD>> where  
#0      | add()      at test.rs:22  
#1      | sum()      at test.rs:27  
#2      | _MAIN_()   at test.rs:32  
ICVD>>
```

附录 E

PYDB Perl API

PYDB Perl API

本附录介绍如何使用 IC Validator 工具提供的 Perl 应用程序编程接口（API）访问 IC Validator 错误数据库（PYDB）。

PYDB Perl API 使你能直接访问 IC Validator 工具产生的错误数据。例如，你可以：

- 为特定违例生成错误信息；
- 统计违例数量以供跟踪。

数据库与 Perl API 包括以下各节：

- PYDB Perl API 概述（PYDB Perl API Overview）
- 访问 PYDB（Accessing the PYDB）
- 常见任务示例（Examples of Common Tasks）
- PYDB 模式（PYDB Schema）

E.1 PYDB Perl API 概述

PYDB Perl API Overview

使用 PYDB Perl API，你可以访问 IC Validator 工具发现的违例信息，并按需处理数据。IC Validator 仅将含错误数据的违例与单元存入 PYDB。LAYOUT_ERRORS 文件通常显示 PYDB 中可用的大部分信息。

IC Validator 产生的错误数据以表的形式存储在 SQLite 数据库中；该数据库即为 PYDB。表的结构及其相互关系称为数据库模式（schema）。模式信息见「PYDB 模式」。

要正确查询复杂数据库，必须了解数据库模式。可用 SQLite 命令访问数据库并返回信息；这些命令可能较复杂，取决于所需返回的信息。

为便于在无需学习数据库模式的情况下访问错误数据，所提供的 PYDB Perl API 可连接并检索 PYDB。与使用 SQLite 命令相比，它使与错误数据库的交互容易得多。其内置功能包括：

- 连接到 PYDB 服务器；
- 提交查询并确保多个数据库服务器不会相互竞争；
- 提供高级函数以检索 PYDB 特定信息，而无需编写复杂的 SQLite 查询；
- 处理结果。

PYDB Perl API 位于 ICV_INSTALL_DIRECTORY/contrib/pydb/。子目录见表 E.1。

表 E.1: pydb 目录的子目录

子目录	说明
bin/	含示例脚本。这些脚本展示如何使用 PYDB 模块。
lib/	含公共库模块。每个文件有简短说明，后接该模块提供的全部类方法的详细说明。

E.2 访问 PYDB

Accessing the PYDB

PYDB Perl API 提供高级接口，使你能处理错误数据，并在需要时对错误数据库运行直接 SQLite 查询。

注意

PYDB 模式与服务器通信协议可能随版本变化。因此，访问 PYDB 时必须使用生成该 PYDB 的同一 IC Validator 版本所附带的 API。

使用 IC Validator 提供的 PYDB Perl API 访问 PYDB，请按下列步骤：

1. 加载 **PYDB** 库。
在任何数据库命令之前，在 Perl 脚本中包含下列代码。该代码从 IC Validator 安装导入所需模块：
- ```
push @INC, $ENV{"ICV_INSTALL_DIRECTORY"}."/contrib/pydb/lib";
require "PYDB.pm";
```
2. 连接数据库。  
在 Perl 脚本中包含下列代码以连接数据库：
- ```
$dbh = PYDB::Connect($db_path,$db_name);
if (! $dbh) {
    print "ERROR: Error connecting to database: $PYDB::errstr\n";
    exit;
}
```
- 若连接成功，返回数据库句柄。
3. 执行数据库查询。
使用 PYDB Perl API 可访问错误信息并按需处理数据。仅含错误数据的违例与单元存入数据库。LAYOUT_ERRORS 文件通常显示 PYDB 中可用的大部分信息。示例见「常见任务示例」。
4. 断开数据库连接。
在 Perl 脚本中包含下列代码以断开连接：
- ```
$dbh->Disconnect();
```

## E.3 常见任务示例

### Examples of Common Tasks

下列示例展示一些常见任务。

### E.3.1 列出数据库中的违例

#### List violations in the database

```
All violations stored in the PYDB
foreach $vio ($dbh->Violations()) {
 print "Violation: \"".$vio->Comment()."\n";
}

Find a specific violation
$vio = $dbh->GetViolationByComment("M1.S1.1: Metal 1 space");
if ($vio) {
 print "Found M1 violation, with ".$vio->ErrorCount().
 " error(s).\n";
}

Find violations that match a regular expression
foreach $vio ($dbh->GetViolationsByCommentRegex(qr/M1\..*/)) {
 print "M1 rule: \"".$vio->Comment()."\n";
}
```

### E.3.2 列出数据库中的单元

#### List cells in the database

```
All cells stored in the PYDB
foreach $cell ($dbh->Cells()) {
 print "Cell: \"".$cell->CellName()."\n";
}

Find a specific cell
$cell = $dbh->GetCellByName("TOP");
if ($cell) {
 print "Found top cell.\n";
}

Find cells that match a regular expression
foreach $cell ($dbh->GetCellsByNameRegex(qr/STDCELL.*/)) {
 print "Found cell: \"".$cell->CellName()."\n";
}
```

### E.3.3 访问特定违例中的错误

#### Access errors in a specific violation

```
$vio = $dbh->GetViolationByComment("M1.S1.1: Metal 1 space");
if ($vio) {
```

```

foreach $command ($vio->Commands()) {
 print "Command ".$command->Comment().
 " produced ".$command->NumErrorsUnlimited().
 " errors, details are available for ".
 $command->NumErrors()."\\n";

 foreach $cell ($command->Cells()) {
 # Errors are stored by command/cell, the error
 # iterator can be used to access them
 $iter = $dbh->ListErrors($command, $cell);
 if (! $iter) {
 print "ERROR: $PYDB::errstr\\n";
 $dbh->Disconnect();
 exit;
 }

 while ($error = $iter->NextError()) {
 # Bounding box of error
 @bbox = $error->GetBBox();
 # Error polygons
 foreach $polygon ($error->Polygons()) {
 # List of points making up the polygon
 @points = $polygon->Points();
 }
 # Access to specific data fields by column name
 $distance = $error->Get("Distance");
 # Certain types of errors might have child errors
 foreach $child_error ($error->Children()) {
 # ...
 }
 }
 }
}
}

```

### E.3.4 执行直接 SQLite 查询

#### Perform a direct SQLite query

Perl API 提供与多数 Perl 安装自带的 DBI 模块类似的接口。

```

Prepare the query
$sth = $dbh->prepare("SELECT CellID, CellName FROM pyCellNames");
if (! $sth) {
 print "ERROR: $PYDB::errstr\\n";
 $dbh->Disconnect();
 exit;
}

Execute the query

```

```
if (! $sth->execute()) {
 print "ERROR: $PYDB::errstr\n";
 $dbh->Disconnect();
 exit;
}

Retrieve the data
while ($ref = $sth->fetchrow_hashref()) {
 print "Cell name=".$ref->{'CellName'}.
 ", id=".$ref->{'CellID'}."\n";
}
```

## E.4 PYDB 模式

### PYDB Schema

在 PYDB 模式中，错误存储在错误表中。

- 每个表由可引用若干相关列的数据行组成，类似电子表格。
- 每个表至少需要一个唯一键以标识各条记录。某些表引用多个键。

每种命令类型至少有一个表；含有子错误的命令还有额外表。错误分类信息也存储在 PYDB 的若干表中。  
表 E.2 展示 PYDB 中一些常用表。

表 E.2: PYDB 组织结构

| 表                         | 存储的信息                                                                                           |
|---------------------------|-------------------------------------------------------------------------------------------------|
| pyCellNames               | 含单元信息。每个命令可从不同单元写入错误。                                                                           |
| pyCommandCells            | 列出某命令含错误的单元。                                                                                    |
| pyCommands                | 对产生错误的每个 runset 命令含一条记录。多个命令（例如 <code>internal1()</code> 与 <code>external2()</code> ）可写入同一违例注释。 |
| pyErrors_<command type>   | 含各条错误数据。特定命令类型的错误表动态创建，并含该命令类型特有的数据列。                                                           |
| pyErrorTableFormat        | 定义错误表的数据列。                                                                                      |
| pyErrorTables             | 定义与某命令类型关联的错误表层次。                                                                               |
| pyPolygons_<command type> | 含各条错误形状。                                                                                        |
| pyRunInfo                 | 含一般信息。                                                                                          |
| pyViolations              | 定义 PYDB 中存储错误的违例规则注释。                                                                           |



# 附录 F

## 第三方许可

### *Third-Party Licenses*

本附录提供 IC Validator 工具所用第三方软件的许可信息。

- 许可概述 (Licensing Overview)
- OSS 软件包声明 (OSS Package Notices)
  - ANTLR
  - Boost
  - cx\_Freeze
  - Flatbuffers
  - libdp
  - libxml2
  - MariaDB Connector
  - MariaDB
  - MCPP Public Domain Code
  - NumPy\*
  - patchELF
  - pyparsing\*
  - Python 3.6 License
  - scikit-learn
  - SciPy\*
  - Shroud-1.0
  - SQLite
  - TclLib
  - Tcl/Tk (注: tbcload 代码是 Tcl 的一部分)
  - zlib
  - Zstandard
- 标准 OSS 许可文本 (Standard OSS License Text)

- Apache-2.0
- Artistic-1.0-Perl
- GPL-1.0
- GPL-3.0

## F.1 许可概述

### Licensing Overview

本文档包含与 Synopsys® IC Validator 产品（下称“SOFTWARE”）所含自由与开源软件（“OSS”）相关的许可信息。适用的 OSS 许可条款约束 Synopsys® 对 SOFTWARE 的分发以及您对 SOFTWARE 的使用。Synopsys® 以及 SOFTWARE 的第三方作者、许可方与分发方，对因任何使用与分发 SOFTWARE 而产生的全部保证与全部责任予以免责。若 OSS 在与 Synopsys® 的协议下提供，且该协议不同于适用的 OSS 许可，则那些条款仅由 Synopsys® 单独提供。

Synopsys® 已在下文转载 OSS 软件包中出现的版权与其他许可声明。尽管 Synopsys® 力求为每个 OSS 软件包提供完整准确的版权与许可信息，但 Synopsys® 不声明或保证下列信息完整、正确或无误。鼓励 SOFTWARE 接收方：(a) 核查所识别的 OSS 软件包以确认本文所提供许可信息的准确性；(b) 将本文档中发现的任何不准确或错误通知 Synopsys®，以便 Synopsys® 相应更新本文档。

某些 OSS 许可（例如 GNU General Public Licenses、GNU Library/Lesser General Public Licenses、Affero General Public Licenses、Mozilla Public Licenses、Common Development and Distribution Licenses、Common Public License 以及 Eclipse Public License）要求：与所分发 OSS 二进制相对应的源代码，须按同一 OSS 许可的条款向接收方或其他请求者提供。

希望收到此类对应源代码副本的接收方或请求者，应向 Synopsys® 邮寄请求至：

Synopsys  
Attn: Open Source Requests  
690 E. Middlefield Road  
Mountain View, CA 94043

提交的全部 OSS 请求中请提供下列信息：

- 您请求源代码的 OSS 软件包；
- 所请求 OSS 软件包随其分发的 Synopsys® 产品（以及任何可用的版本信息）；
- Synopsys® 可就该请求联系您的电子邮箱（如有）；以及
- 请求源代码的投递邮寄地址。

已在 OSS 软件包名称后添加星号（\*），以表示这些组件可能包含在使用 ZeBu 开发的软件或硬件产品中。这些组件的 OSS 许可信息也转载于随 SOFTWARE 提供的 IC Validator 可再分发 OSS 声明文本文件中。

#### 注意

下列各 OSS 软件包与标准许可小节中的法律许可原文保留英文；中文仅译章节说明与概述。

## F.2 OSS 软件包声明

### OSS Package Notices

#### F.2.1 ANTLR

##### ANTLR

以下为 ANTLR 的许可声明原文。

```
ANTLR 2 License
[The BSD License]
Copyright (c) 2012 Terence Parr and Sam Harwell
All rights reserved.
```

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of the author nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

#### F.2.2 Boost

##### Boost

以下为 Boost 的许可声明原文。

```
Boost Software License - Version 1.0 - August 17th, 2003
Permission is hereby granted, free of charge, to any person or organization obtaining
a copy of the software and accompanying documentation covered by this license (the
“Software”) to use, reproduce, display, distribute, execute, and transmit the Software,
and to prepare derivative works of the Software, and to permit third-parties to whom the
Software is furnished to do so, all subject to the following:
The copyright notices in the Software and this entire statement, including the above
license grant, this restriction and the following disclaimer, must be included in all copies
of the Software, in whole or in part, and all derivative works of the Software, unless such
copies or derivative works are solely in the form of machine-executable object code
```

generated by a source language processor.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT. IN NO EVENT SHALL THE COPYRIGHT HOLDERS OR ANYONE

DISTRIBUTING THE SOFTWARE BE LIABLE FOR ANY DAMAGES OR OTHER LIABILITY, WHETHER IN CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

## F.2.3 cx\_Freeze

### cx\_Freeze

以下为 cx\_Freeze 的许可声明原文。

Project homepage/download site: <http://cx-freeze.readthedocs.io/>; [https://pypi.python.org/pypi/cx\\_Freeze/4.3.4](https://pypi.python.org/pypi/cx_Freeze/4.3.4)

Project licensing notices: <http://cx-freeze.readthedocs.io/en/latest/license.html>

Copyright © 2007-2017, Anthony Tuininga.

Copyright © 2001-2006, Computronix (Canada) Ltd., Edmonton, Alberta, Canada.

All rights reserved.

NOTE: this license is derived from the Python Software Foundation License which can be found at <http://www.python.org/psf/license>

License for cx\_Freeze 6.0b1

1. This LICENSE AGREEMENT is between the copyright holders and the Individual or Organization ("Licensee") accessing and otherwise using cx\_Freeze software in source or binary form and its associated documentation.
2. Subject to the terms and conditions of this License Agreement, the copyright holders hereby grant Licensee a nonexclusive, royalty-free, world-wide license to reproduce, analyze, test, perform and/or display publicly, prepare derivative works, distribute, and otherwise use cx\_Freeze alone or in any derivative version, provided, however, that this License Agreement and this notice of copyright are retained in cx\_Freeze alone or in any derivative version prepared by Licensee.
3. In the event Licensee prepares a derivative work that is based on or incorporates cx\_Freeze or any part thereof, and wants to make the derivative work available to others as provided herein, then Licensee hereby agrees to include in any such work a brief summary of the changes made to cx\_Freeze.
4. The copyright holders are making cx\_Freeze available to Licensee on an "AS IS" basis. THE COPYRIGHT HOLDERS MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED. BY WAY OF EXAMPLE, BUT NOT LIMITATION, THE COPYRIGHT HOLDERS MAKE NO AND DISCLAIM ANY REPRESENTATION OR WARRANTY OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE OR THAT THE USE OF CX\_FREEZE WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.

5. THE COPYRIGHT HOLDERS SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF CX\_FREEZE FOR ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS A RESULT OF MODIFYING, DISTRIBUTING, OR OTHERWISE USING CX\_FREEZE, OR ANY DERIVATIVE THEREOF, EVEN IF ADVISED OF THE POSSIBILITY THEREOF.

6. This License Agreement will automatically terminate upon a material breach of its terms and conditions.

7. Nothing in this License Agreement shall be deemed to create any relationship of agency, partnership, or joint venture between the copyright holders and Licensee. This License Agreement does not grant permission to use copyright holder's trademarks or trade name in a trademark sense to endorse or promote products or services of Licensee, or any third party.

8. By copying, installing or otherwise using cx\_Freeze, Licensee agrees to be bound by the terms and conditions of this License Agreement. Computronix® is a registered trademark of Computronix (Canada) Ltd.

## F.2.4 Flatbuffers

### Flatbuffers

以下为 Flatbuffers 的许可声明原文。

Project homepage/download site: <http://google.github.io/flatbuffers/>; <https://github.com/google/flatbuffers>

Project licensing notices: <https://github.com/google/flatbuffers/blob/master/LICENSE.txt>

LICENSE.txt:

See Apache-2.0 in the Standard OSS License Text on page 542."

## F.2.5 libdp

### libdp

以下为 libdp 的许可声明原文。

1. The "Software", below, refers to the Tcl-DP system, developed by the Tcl-DP group (in either source-code, object-code or executable-code form), and related documentation, and a "work based on the Software" means a work based on either the Software, on part of the Software, or on any derivative work of the Software under copyright law: that is, a work containing all or a portion of the Tcl-DP system, either verbatim or with modifications. Each licensee is addressed as "you" or "Licensee."
2. Cornell University as the parent organization of the Tcl-DP group holds copyrights in the Software. The copyright holder reserves all rights except those expressly granted to licensees, and U.S. Government license rights.
3. Permission is hereby granted to use, copy, modify, and to redistribute to others. If you distribute a copy or copies of the Software, or you modify a copy or copies of the

Software or any portion of it, thus forming a work based on the Software, and make and/or distribute copies of such work, you must meet the following conditions:

1. If you make a copy of the Software (modified or verbatim) it must include the copyright notice and this license.
2. You must cause the modified Software to carry prominent notices stating that you changed specified portions of the Software.

4. LICENSEE AGREES THAT THE EXPORT OF GOODS AND/OR TECHNICAL DATA FROM THE UNITED STATES MAY REQUIRE SOME FORM OF EXPORT CONTROL LICENSE FROM THE U.S. GOVERNMENT AND THAT FAILURE TO OBTAIN SUCH EXPORT CONTROL LICENSE MAY RESULT IN CRIMINAL LIABILITY UNDER U.S. LAWS.
5. Portions of the Software resulted from work developed under a U.S. Government Contract and are subject to the following license: the Government is granted for itself and others acting on its behalf a paid-up, nonexclusive, irrevocable worldwide license in this computer software to reproduce, prepare derivative works, and perform publicly and display publicly.
6. Disclaimer of warranty: Licensor provides the software on an ``as is'' basis. Licensor does not warrant, guarantee, or make any representations regarding the use or results of the software with respect to its correctness, accuracy, reliability or performance. The entire risk of the use and performance of the software is assumed by licensee. ALL WARRANTIES INCLUDING, WITHOUT LIMITATION, ANY WARRANTY OF FITNESS FOR A PARTICULAR PURPOSE OR MERCHANTABILITY ARE HEREBY EXCLUDED.
7. Lack of maintenance or support services: Licensee understands and agrees that licensor is under no obligation to provide maintenance, support or update services, notices of latent defects, or correction of defects for the software.
8. Limitation of liability, indemnification: Even if advised of the possibility of damages, under no circumstances shall licensor be liable to licensee or any third party for damages of any character, including, without limitation, direct, indirect, incidental, consequential or special damages, loss of profits, loss of use, loss of goodwill, computer failure or malfunction. Licensee agrees to indemnify and hold harmless licensor for any and all liability licensor may incur as a result of licensee's use of the software.

## F.2.6 libxml2

### libxml2

以下为 libxml2 的许可声明原文。

Libxml2 is the XML C parser and toolkit developed for the Gnome project (but usable outside of the Gnome platform), it is free software available under the MIT License. The MIT license:

The MIT License (MIT)

Copyright (c) <year> <copyright holders>

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

## F.2.7 MariaDB Connector

### MariaDB Connector

以下为 MariaDB Connector 的许可声明原文。

Synopsys modifications: None  
 MariaDB Connector is licensed under the GNU Lesser General Public License, version 2.1, (LGPL-2.1). Software licensed under the LGPL-2.1 is free software that comes with ABSOLUTELY NO WARRANTY. You may modify, redistribute, or otherwise use the GPL software under the terms of the LGPL-2.1 (<http://www.gnu.org/licenses/old-licenses/lgpl-2.1.html>).  
 GNU LESSER GENERAL PUBLIC LICENSE  
 Version 2.1, February 1999

Copyright (C) 1991, 1999 Free Software Foundation, Inc. 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

[This is the first released version of the Lesser GPL. It also counts as the successor of the GNU Library Public License, version 2, hence the version number 2.1.]

#### TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License Agreement applies to any software library or other program which contains a notice placed by the copyright holder or other authorized party saying it may be distributed under the terms of this Lesser General Public License (also called "this License"). Each licensee is addressed as "you".

A "library" means a collection of software functions and/or data prepared so as to be conveniently linked with application programs (which use some of those functions and data) to form executables.

The "Library", below, refers to any such software library or work which has been distributed under these terms. A "work based on the Library" means either the Library or any derivative work under copyright law: that is to say, a work containing the Library or a portion of it, either verbatim or with modifications and/or translated straightforwardly into another language. (Hereinafter, translation is included without limitation in the term "modification".)

"Source code" for a work means the preferred form of the work for making modifications to it. For a library, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the library.

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running a program using the Library is not restricted, and output from such a program is covered only if its contents constitute a work based on the Library (independent of the use of the Library in a tool for writing it). Whether that is true depends on what the Library does and what the program that uses the Library does.

1. You may copy and distribute verbatim copies of the Library's complete source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and distribute a copy of this License along with the Library.  
 You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Library or any portion of it, thus forming a work based on the Library, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:
- The modified work must itself be a software library.
  - You must cause the files modified to carry prominent notices stating that you changed the files and the date of any change.
  - You must cause the whole of the work to be licensed at no charge to all third parties under the terms of this License.
  - If a facility in the modified Library refers to a function or a table of data to be supplied by an application program that uses the facility, other than as an argument passed when the facility is invoked, then you must make a good faith effort to ensure that, in the event an application does not supply such function or table, the facility still operates, and performs whatever part of its purpose remains meaningful.

(For example, a function in a library to compute square roots has a purpose that is entirely well-defined independent of the application. Therefore, Subsection 2d requires that any application-supplied function or table used by this function must be optional: if the application does not supply it, the square root function must still compute square roots.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Library, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Library, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Library.

In addition, mere aggregation of another work not based on the Library with the Library (or with a work based on the Library) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may opt to apply the terms of the ordinary GNU General Public License instead of this License to a given copy of the Library. To do this, you must alter all the notices that refer to this License, so that they refer to the ordinary GNU General Public License, version 2, instead of to this License. (If a newer version than version 2 of the ordinary GNU General Public License has appeared, then you can specify that version instead if you wish.) Do not make any other change in these notices.

Once this change is made in a given copy, it is irreversible for that copy, so the ordinary GNU General Public License applies to all subsequent copies and derivative works made from that copy.

This option is useful when you wish to copy part of the code of the Library into a program that is not a library.

4. You may copy and distribute the Library (or a portion or derivative of it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange.

If distribution of object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place satisfies the requirement to distribute the source code, even though third parties are not compelled to copy the source along with the object code.

5. A program that contains no derivative of any portion of the Library, but is designed to work with the Library by being compiled or linked with it, is called a "work that uses the Library". Such a work, in isolation, is not a derivative work of the Library, and therefore



falls outside the scope of this License.

However, linking a "work that uses the Library" with the Library creates an executable that is a derivative of the Library (because it contains portions of the Library), rather than a "work that uses the library". The executable is therefore covered by this License. Section 6 states terms for distribution of such executables.

When a "work that uses the Library" uses material from a header file that is part of the Library, the object code for the work may be a derivative work of the Library even though the source code is not. Whether this is true is especially significant if the work can be linked without the Library, or if the work is itself a library. The threshold for this to be true is not precisely defined by law.

If such an object file uses only numerical parameters, data structure layouts and accessors, and small macros and small inline functions (ten lines or less in length), then the use of the object file is unrestricted, regardless of whether it is legally a derivative work. (Executables containing this object code plus portions of the Library will still fall under Section 6.)

Otherwise, if the work is a derivative of the Library, you may distribute the object code for the work under the terms of Section 6. Any executables containing that work also fall under Section 6, whether or not they are linked directly with the Library itself.

6. As an exception to the Sections above, you may also combine or link a "work that uses the Library" with the Library to produce a work containing portions of the Library, and distribute that work under terms of your choice, provided that the terms permit

modification of the work for the customer's own use and reverse engineering for debugging such modifications.

You must give prominent notice with each copy of the work that the Library is used in it and that the Library and its use are covered by this License. You must supply a copy of this License. If the work during execution displays copyright notices, you must include the copyright notice for the Library among them, as well as a reference directing the user to the copy of this License. Also, you must do one of these things:

- Accompany the work with the complete corresponding machine-readable source code for the Library including whatever changes were used in the work (which must be distributed under Sections 1 and 2 above); and, if the work is an executable linked with the Library, with the complete machine-readable "work that uses the Library", as object code and/or source code, so that the user can modify the Library and then relink to produce a modified executable containing the modified Library. (It is understood that the user who changes the contents of definitions files in the Library will not necessarily be able to recompile the application to use the modified definitions.)
- Use a suitable shared library mechanism for linking with the Library. A suitable mechanism is one that (1) uses at run time a copy of the library already present on the user's computer system, rather than copying library functions into the executable, and (2) will operate properly with a modified version of the library, if the user installs one, as long as the modified version is interface-compatible with the version that the work was made with.
- Accompany the work with a written offer, valid for at least three years, to give the same user the materials specified in Subsection 6a, above, for a charge no more than the cost of performing this distribution.
- If distribution of the work is made by offering access to copy from a designated place, offer equivalent access to copy the above specified materials from the same place.
- Verify that the user has already received a copy of these materials or that you have already sent this user a copy.

For an executable, the required form of the "work that uses the Library" must include any data and utility programs needed for reproducing the executable from it. However, as a special exception, the materials to be distributed need not include anything that is normally distributed (in either source or binary form) with the major components

(compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

It may happen that this requirement contradicts the license restrictions of other proprietary libraries that do not normally accompany the operating system. Such a contradiction means you cannot use both them and the Library together in an executable that you distribute.

7. You may place library facilities that are a work based on the Library side-by-side in a single library together with other library facilities not covered by this License, and distribute such a combined library, provided that the separate distribution of the work based on the Library and of the other library facilities is otherwise permitted, and provided that you do these two things:
  - Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities. This must be distributed under the terms of the Sections above.
  - Give prominent notice with the combined library of the fact that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.
8. You may not copy, modify, sublicense, link with, or distribute the Library except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, link with, or distribute the Library is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.
9. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Library or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Library (or any work based on the Library), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Library or works based on it.
10. Each time you redistribute the Library (or any work based on the Library), the recipient automatically receives a license from the original licensor to copy, distribute, link with or modify the Library subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties with this License.
11. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Library at all. For example, if a patent license would not permit royalty-free redistribution of the Library by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Library.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply, and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system which is implemented by public license practices. Many people have made generous

contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

12. If the distribution and/or use of the Library is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Library under this license may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.
13. The Free Software Foundation may publish revised and/or new versions of the Lesser General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. Each version is given a distinguishing version number. If the Library specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Library does not specify a license version number, you may choose any version ever published by the Free Software Foundation.
14. If you wish to incorporate parts of the Library into other free programs whose distribution conditions are incompatible with these, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

#### NO WARRANTY

- BECAUSE THE LIBRARY IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE LIBRARY, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE LIBRARY "AS IS" WITHOUT

WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE LIBRARY IS WITH YOU. SHOULD THE LIBRARY PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

- IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE LIBRARY AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE LIBRARY (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE LIBRARY TO OPERATE WITH ANY OTHER SOFTWARE), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

## F.2.8 MariaDB

### MariaDB

以下为 MariaDB 的许可声明原文。

Synopsys modifications: None

MariaDB is licensed under the GNU General Public License, version 2, (GPL-2.0).

Software licensed under the GPL-2.0 is free software that comes with ABSOLUTELY NO WARRANTY. You may modify, redistribute, or otherwise use the GPL software under the terms of the GPL-2.0 (<http://www.gnu.org/licenses/old-licenses/gpl-2.0.html>).''

GNU GENERAL PUBLIC LICENSE

Version 2.1, June 1991

Copyright (C) 1989, 1991 Free Software Foundation, Inc. 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

#### TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License applies to any program or other work which contains a notice placed by the copyright holder saying it may be distributed under the terms of this General Public License. The "Program", below, refers to any such program or work, and a "work based on the Program" means either the Program or any derivative work under copyright law: that is to say, a work containing the Program or a portion of it, either verbatim or with modifications and/or translated into another language. (Hereinafter, translation is included without limitation in the term "modification".) Each licensee is addressed as "you".

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running the Program is not restricted, and the output from the Program is covered only if its contents constitute a work based on the Program (independent of having been made by running the Program). Whether that is true depends on what the Program does.

1. You may copy and distribute verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and give any other recipients of the Program a copy of this License along with the Program. You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.
2. You may modify your copy or copies of the Program or any portion of it, thus forming a work based on the Program, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:
  - You must cause the modified files to carry prominent notices stating that you changed the files and the date of any change.
  - You must cause any work that you distribute or publish, that in whole or in part contains or is derived from the Program or any part thereof, to be licensed as a whole at no charge to all third parties under the terms of this License.
  - If the modified program normally reads commands interactively when run, you must cause it, when started running for such interactive use in the most ordinary way, to print or display an announcement including an appropriate copyright notice and a notice that there is no warranty (or else, saying that you provide a warranty) and that users may redistribute the program under these conditions, and telling the user how to view a copy of this License. (Exception: if the Program itself is interactive but does not normally print such an announcement, your work based on the Program is not required to print an announcement.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Program, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the

Program, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Program.

In addition, mere aggregation of another work not based on the Program with the Program (or with a work based on the Program) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may copy and distribute the Program (or a work based on it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you also do one of the following:
  - Accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,
  - Accompany it with a written offer, valid for at least three years, to give any third party, for a charge no more than your cost of physically performing source distribution, a complete machine-readable copy of the corresponding source code, to be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,
  - Accompany it with the information you received as to the offer to distribute corresponding source code. (This alternative is allowed only for noncommercial distribution and only if you received the program in object code or executable form with such an offer, in accord with Subsection b above.)

The source code for a work means the preferred form of the work for making modifications to it. For an executable work, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the executable. However, as a special exception, the source code distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

If distribution of executable or object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place counts as distribution of the source code, even though third parties are not compelled to copy the source along with the object code.
4. You may not copy, modify, sublicense, or distribute the Program except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense or distribute the Program is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.
5. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Program or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Program (or any work based on the Program), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Program or works based on it.
6. Each time you redistribute the Program (or any work based on the Program), the recipient automatically receives a license from the original licensor to copy, distribute or modify the Program subject to these terms and conditions. You may not impose any

further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties to this License.

7. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Program at all. For example, if a patent license would not permit royalty-free redistribution of the Program by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Program.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system, which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

8. If the distribution and/or use of the Program is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Program under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

9. The Free Software Foundation may publish revised and/or new versions of the General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. Each version is given a distinguishing version number. If the Program specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of this License, you may choose any version ever published by the Free Software Foundation.
10. If you wish to incorporate parts of the Program into other free programs whose distribution conditions are different, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

#### NO WARRANTY

- BECAUSE THE PROGRAM IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM

PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

- IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

## F.2.9 MCPP Public Domain Code

### MCPP Public Domain Code

以下为 MCPP Public Domain Code 的许可声明原文。

Copyright (c) 1998, 2002-2007 Kiyoshi Matsui <kmatsui@t3.rim.or.jp>  
All rights reserved.

This software including the files in this directory is provided under the following license. Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR 'AS IS' AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

## F.2.10 NumPy\*

### NumPy\*

以下为 NumPy\* 的许可声明原文。

Project homepage/download site: <http://www.numpy.org/>; <https://github.com/numpy/numpy>  
Project licensing notices: <https://numpy.org/doc/stable/license.html>  
LICENSE.txt

Copyright (c) 2005-2017, NumPy Developers. All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- \* Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- \* Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- \* Neither the name of the NumPy Developers nor the names of any contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

The NumPy repository and source distributions bundle several libraries that are compatibly licensed. We list these here.

Name: Numpydoc Files: doc/sphinxext/numpydoc/\* License: 2-clause BSD For details, see doc/sphinxext/LICENSE.txt

Name: scipy-sphinx-theme Files: doc/scipy-sphinx-theme/\* License: 3-clause BSD, PSF and Apache 2.0 For details, see doc/sphinxext/LICENSE.txt

Name: lapack-lite Files: numpy/linalg/lapack\_lite/\* License: 3-clause BSD For details, see numpy/linalg/lapack\_lite/LICENSE.txt

Name: tempita Files: tools/npv\_tempita/\* License: BSD derived For details, see tools/npv\_tempita/license.txt

Name: dragon4 Files: numpy/core/src/multiarray/dragon4.c License: One of a kind For license text, see numpy/core/src/multiarray/dragon4.cd/doc/sphinxext/LICENSE.txt:

-----

The files - numpydoc.py - docscrape.py - docscrape\_sphinx.py - phantom\_import.py have the following license: Copyright (C) 2008 Stefan van der Walt <stefan@mentat.za.net>, Pauli Virtanen <pav@iki.fi>

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:



1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR ``AS IS'' AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

-----  
The files - compiler\_unparse.py - comment\_eater.py - traitsdoc.py have the following license:

This software is OSI Certified Open Source Software. OSI Certified is a certification mark of the Open Source Initiative. Copyright (c) 2006, Enthought, Inc. All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

\* Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer. \* Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution. \* Neither the name of Enthought, Inc. nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

The file - plot\_directive.py originates from Matplotlib (<http://matplotlib.sf.net/>) which has the following license: Copyright (c) 2002-2008 John D. Hunter; All Rights Reserved.

1. This LICENSE AGREEMENT is between John D. Hunter ("JDH"), and the Individual or Organization ("Licensee") accessing and otherwise using matplotlib software in source or binary form and its associated documentation.
2. Subject to the terms and conditions of this License Agreement, JDH hereby grants Licensee a nonexclusive, royalty-free, world-wide license to reproduce, analyze, test, perform and/or display publicly, prepare derivative works, distribute, and otherwise use matplotlib 0.98.3 alone or in any derivative version, provided, however, that JDH's License Agreement and JDH's notice of copyright, i.e., "Copyright (c) 2002-2008 John D. Hunter; All Rights Reserved" are retained in matplotlib 0.98.3 alone or in any derivative version prepared by Licensee.
3. In the event Licensee prepares a derivative work that is based on or incorporates matplotlib 0.98.3 or any part thereof, and wants to make the derivative work available to others as provided herein, then Licensee hereby agrees to include in any such work a brief summary of the changes made to matplotlib 0.98.3.
4. JDH is making matplotlib 0.98.3 available to Licensee on an "AS IS" basis. JDH MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED. BY WAY OF EXAMPLE, BUT NOT LIMITATION, JDH MAKES NO AND DISCLAIMS ANY REPRESENTATION OR WARRANTY OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE OR THAT THE USE OF MATPLOTLIB 0.98.3 WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.
5. JDH SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF MATPLOTLIB 0.98.3 FOR ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS A RESULT OF MODIFYING, DISTRIBUTING, OR OTHERWISE USING MATPLOTLIB 0.98.3, OR ANY DERIVATIVE THEREOF, EVEN IF ADVISED OF THE POSSIBILITY THEREOF.
6. This License Agreement will automatically terminate upon a material breach of its terms and conditions.
7. Nothing in this License Agreement shall be deemed to create any relationship of agency, partnership, or joint venture between JDH and Licensee. This License Agreement does not grant permission to use JDH trademarks or trade name in a trademark sense to endorse or promote products or services of Licensee, or any third party.
8. By copying, installing or otherwise using matplotlib 0.98.3, Licensee agrees to be bound by the terms and conditions of this License Agreement.

numpy/linalg/lapack\_lite/LICENSE.txt:

Copyright (c) 1992-2013 The University of Tennessee and The University of Tennessee Research Foundation. All rights reserved. Copyright (c) 2000-2013 The University of California Berkeley. All rights reserved. Copyright (c) 2006-2013 The University of Colorado Denver. All rights reserved.

\$COPYRIGHT\$

Additional copyrights may follow

\$HEADER\$

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer listed in this license in the documentation and/or other materials provided with the distribution.
- Neither the name of the copyright holders nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

The copyright holders provide no reassurances that the source code provided does not infringe any patent, copyright, or any other intellectual property rights of third parties. The copyright holders disclaim any liability to any recipient for claims brought against recipient by any third party for infringement of that parties intellectual property rights.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

tools/npv\_tempita/license.txt:

License

-----

Copyright (c) 2008 Ian Bicking and Contributors

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the

following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

numpy/core/src/multiarray/dragon4.c:

```
/*
 * Copyright (c) 2014 Ryan Juckett
 * http://www.ryanjuckett.com/
 *
 * This software is provided 'as-is', without any express or implied
 * warranty. In no event will the authors be held liable for any damages
 * arising from the use of this software.
 *
 * Permission is granted to anyone to use this software for any purpose,
 * including commercial applications, and to alter it and redistribute it
 * freely, subject to the following restrictions:
 *
 * 1. The origin of this software must not be misrepresented; you must
 * not
 * claim that you wrote the original software. If you use this
 * software
 * in a product, an acknowledgment in the product documentation would
 * be
 * appreciated but is not required.
 *
 * 2. Altered source versions must be plainly marked as such, and must
 * not be
 *
 * misrepresented as being the original software.
 *
 * 3. This notice may not be removed or altered from any source
 * distribution.
 */
```

## F.2.11 patchELF

### patchELF

以下为 patchELF 的许可声明原文。

Project homepage/download site: <https://nixos.org/patchelf.html>  
 Project licensing notices:  
 COPYING:

See GPL-3.0 in the Standard OSS License Text on page 542."

## F.2.12 pyparsing\*

### pyparsing\*

以下为 pyparsing\* 的许可声明原文。

```
Project homepage/download site: https://pypi.python.org/pypi/pyparsing
Project licensing notices:
module pyparsing.py
#
Copyright (c) 2003-2016 Paul T. McGuire
#
Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:
#
The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.
#
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
#
```

## F.2.13 Python 3.6 License

### Python 3.6 License

以下为 Python 3.6 License 的许可声明原文。

PYTHON SOFTWARE FOUNDATION LICENSE VERSION 2

-----

1. This LICENSE AGREEMENT is between the Python Software Foundation ("PSF"), and the Individual or Organization ("Licensee") accessing and otherwise using this software ("Python") in source or binary form and its associated documentation.
2. Subject to the terms and conditions of this License Agreement, PSF hereby grants Licensee a nonexclusive, royalty-free, world-wide license to reproduce, analyze, test, perform and/or display publicly, prepare derivative works, distribute, and otherwise use Python alone or in any derivative version, provided, however, that PSF's License Agreement and PSF's notice of copyright, i.e., "Copyright (c) 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014 2015, 2016 Python Software Foundation; All Rights Reserved" are retained in Python alone or in any derivative version prepared by

Licensee.

3. In the event Licensee prepares a derivative work that is based on or incorporates Python or any part thereof, and wants to make the derivative work available to others as provided herein, then Licensee hereby agrees to include in any such work a brief summary of the changes made to Python.
4. PSF is making Python available to Licensee on an "AS IS" basis. PSF MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED. BY WAY OF EXAMPLE, BUT NOT LIMITATION, PSF MAKES NO AND DISCLAIMS ANY REPRESENTATION OR WARRANTY OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE OR THAT THE USE OF PYTHON WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.
5. PSF SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF PYTHON FOR ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS A RESULT OF MODIFYING, DISTRIBUTING, OR OTHERWISE USING PYTHON, OR ANY DERIVATIVE THEREOF, EVEN IF ADVISED OF THE POSSIBILITY THEREOF.
6. This License Agreement will automatically terminate upon a material breach of its terms and conditions.
7. Nothing in this License Agreement shall be deemed to create any relationship of agency, partnership, or joint venture between PSF and Licensee. This License Agreement does not grant permission to use PSF trademarks or trade name in a trademark sense to endorse or promote products or services of Licensee, or any third party.
8. By copying, installing or otherwise using Python, Licensee agrees to be bound by the terms and conditions of this License Agreement.

## F.2.14 scikit-learn

### scikit-learn

以下为 scikit-learn 的许可声明原文。

Project homepage/download site: <http://scikit-learn.org/>  
 Project licensing notices:  
 New BSD License

Copyright (c) 2007-2018 The scikit-learn developers. All rights reserved.  
 Redistribution and use in source and binary forms, with or without  
 modification, are permitted provided that the following conditions are  
 met:

- a. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- b. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- c. Neither the name of the Scikit-learn Developers nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS

IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

## F.2.15 SciPy\*

### SciPy\*

以下为 SciPy\* 的许可声明原文。

Project homepage/download site: <https://www.SciPy.org>; <https://github.com/SciPy/SciPy>  
 Project licensing notices: <https://github.com/SciPy/SciPy/blob/master/LICENSE.txt>  
 Copyright (c) 2001, 2002 Enthought, Inc. All rights reserved.  
 Copyright (c) 2003-2017 SciPy Developers. All rights reserved.  
 Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- a. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- b. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- c. Neither the name of Enthought nor the names of the SciPy Developers may be used to endorse or promote products derived from this software without specific prior written permission. THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDERS OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

SciPy bundles a number of libraries that are compatibly licensed. We list these here.

Name: Numpydoc

Files: doc/sphinxext/numpydoc/\*

License: 2-clause BSD

For details, see doc/sphinxext/LICENSE.txt

Name: scipy-sphinx-theme

Files: doc/scipy-sphinx-theme/\*

License: 3-clause BSD, PSF and Apache 2.0

For details, see doc/sphinxext/LICENSE.txt

Name: Six

Files: scipy/\_lib/six.py

```

License: MIT
 For details, see the header inside scipy/_lib/six.py
Name: Decorator
Files: scipy/_lib/decorator.py
License: 2-clause BSD
 For details, see the header inside scipy/_lib/decorator.py
Name: ID
Files: scipy/linalg/src/id_dist/*
License: 3-clause BSD
 For details, see scipy/linalg/src/id_dist/doc/doc.tex
Name: L-BFGS-B
Files: scipy/optimize/lbfgsb/*
License: BSD license
 For details, see scipy/optimize/lbfgsb/README
Name: SuperLU
Files: scipy/sparse/linalg/dsolve/SuperLU/*
License: 3-clause BSD
 For details, see scipy/sparse/linalg/dsolve/SuperLU/License.txt
Name: ARPACK
Files: scipy/sparse/linalg/eigen/arpack/ARPACK/*
License: 3-clause BSD
 For details, see scipy/sparse/linalg/eigen/arpack/ARPACK/COPYING

Name: Qhull
Files: scipy/spatial/qhull/*
License: Qhull license (BSD-like)
 For details, see scipy/spatial/qhull/COPYING.txt
Name: Cephes
Files: scipy/special/cephes/*
License: 3-clause BSD
 Distributed under 3-clause BSD license with permission from the author,
 see https://lists.debian.org/debian-legal/2004/12/msg00295.html
 Cephes Math Library Release 2.8: June, 2000
 Copyright 1984, 1995, 2000 by Stephen L. Moshier
 This software is derived from the Cephes Math Library and is
 incorporated herein by permission of the author.
 All rights reserved.
 Redistribution and use in source and binary forms, with or without
 modification, are permitted provided that the following conditions are
 met:
 * Redistributions of source code must retain the above copyright notice,
 this list of conditions and the following disclaimer.
 * Redistributions in binary form must reproduce the above copyright
 notice, this list of conditions and the following disclaimer in the
 documentation and/or other materials provided with the distribution.
 * Neither the name of the <organization> nor the names of its
 contributors may be used to endorse or promote products derived from
 this software without specific prior written permission.
 THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
 "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
 LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
 PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL <COPYRIGHT HOLDER>
 BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
 CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF
 SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS
 INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN

```



CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Name: Faddeeva

Files: scipy/special/Faddeeva.\*

License: MIT

Copyright (c) 2012 Massachusetts Institute of Technology

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM,

DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

doc/sphinxext/LICENSE.txt:

-----  
-----

The files - numpydoc.py - docscrape.py - docscrape\_sphinx.py - phantom\_import.py have the following license:

Copyright (C) 2008 Stefan van der Walt <stefan@mentat.za.net>, Pauli Virtanen <pav@iki.fi>

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR ``AS IS'' AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

-----  
-----

The files - compiler\_unparse.py - comment\_eater.py - traitsdoc.py have the following license:

This software is OSI Certified Open Source Software. OSI Certified is a certification mark of the Open Source Initiative.

Copyright (c) 2006, Enthought, Inc. All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

\* Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer. \* Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution. \* Neither the name of Enthought, Inc. nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED

TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

-----  
-----

The file - plot\_directive.py originates from Matplotlib

(<http://matplotlib.sf.net/>) which has the following license:

Copyright (c) 2002-2008 John D. Hunter; All Rights Reserved.

1. This LICENSE AGREEMENT is between John D. Hunter ("JDH"), and the Individual or Organization ("Licensee") accessing and otherwise using matplotlib software in source or binary form and its associated documentation.
2. Subject to the terms and conditions of this License Agreement, JDH hereby grants Licensee a nonexclusive, royalty-free, world-wide license to reproduce, analyze, test, perform and/or display publicly, prepare derivative works, distribute, and otherwise use matplotlib 0.98.3 alone or in any derivative version, provided, however, that JDH's License Agreement and JDH's notice of copyright, i.e., "Copyright (c) 2002-2008 John D. Hunter; All Rights Reserved" are retained in matplotlib 0.98.3 alone or in any derivative version prepared by Licensee.
3. In the event Licensee prepares a derivative work that is based on or incorporates matplotlib 0.98.3 or any part thereof, and wants to make the derivative work available to others as provided herein, then Licensee hereby agrees to include in any such work a brief summary of the changes made to matplotlib 0.98.3.
4. JDH is making matplotlib 0.98.3 available to Licensee on an "AS IS" basis. JDH MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED. BY WAY OF EXAMPLE, BUT NOT LIMITATION, JDH MAKES NO AND DISCLAIMS ANY REPRESENTATION OR WARRANTY OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE OR THAT THE USE OF MATPLOTLIB 0.98.3 WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.
5. JDH SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF MATPLOTLIB 0.98.3 FOR ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS A RESULT OF MODIFYING, DISTRIBUTING, OR OTHERWISE USING MATPLOTLIB 0.98.3, OR ANY DERIVATIVE THEREOF, EVEN IF ADVISED OF THE POSSIBILITY

THEREOF.

6. This License Agreement will automatically terminate upon a material breach of its terms and conditions.

7. Nothing in this License Agreement shall be deemed to create any relationship of agency, partnership, or joint venture between JDH and Licensee. This License Agreement does not grant permission to use JDH trademarks or trade name in a trademark sense to endorse or promote products or services of Licensee, or any third party.

8. By copying, installing or otherwise using matplotlib 0.98.3, Licensee agrees to be bound by the terms and conditions of this License Agreement.

numpy/linalg/lapack\_lite/LICENSE.txt:

Copyright (c) 1992-2013 The University of Tennessee and The University of Tennessee Research Foundation. All rights reserved. Copyright (c) 2000-2013 The University of California Berkeley. All rights reserved.

Copyright (c) 2006-2013 The University of Colorado Denver. All rights reserved.

\$COPYRIGHT\$

Additional copyrights may follow

\$HEADER\$

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer listed in this license in the documentation and/or other materials provided with the distribution.
- Neither the name of the copyright holders nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

The copyright holders provide no reassurances that the source code provided does not infringe any patent, copyright, or any other intellectual property rights of third parties. The copyright holders disclaim any liability to any recipient for claims brought against recipient by any third party for infringement of that parties intellectual property rights.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

tools/npy\_tempita/license.txt:

License

-----

Copyright (c) 2008 Ian Bicking and Contributors

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the

```

"Software")), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit
persons to whom the Software is furnished to do so, subject to the
following conditions:
The above copyright notice and this permission notice shall be included
in all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,

TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
scipy/_lib/six.py
"""Utilities for writing code that runs on Python 2 and 3"""
Copyright (c) 2010-2012 Benjamin Peterson
#
Permission is hereby granted, free of charge, to any person obtaining a
copy of
this software and associated documentation files (the "Software"), to
deal in
the Software without restriction, including without limitation the
rights to
use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of
the Software, and to permit persons to whom the Software is furnished
to do so,
subject to the following conditions:
#
The above copyright notice and this permission notice shall be included
in all
copies or substantial portions of the Software.
#
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS
FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR
COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
WHETHER
IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
scipy/_lib/decorator.py
LICENSE
#
Copyright (c) 2005-2015, Michele Simionato
All rights reserved.
Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are
met:
Redistributions of source code must retain the above copyright

```

```

notice, this list of conditions and the following disclaimer.
Redistributions in bytecode form must reproduce the above copyright
notice, this list of conditions and the following disclaimer in
the documentation and/or other materials provided with the
distribution.
THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDERS OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT,
INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING,

BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS
OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND
ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR
TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE
USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH
DAMAGE.
scipy/linalg/src/id_dist/doc/doc.tex
The present document and all of the software
in the accompanying distribution (which is contained in the directory
{\tt id_dist} and its subdirectories, or in the file
{\tt id_dist.tar.gz}), is
\bigskip
Copyright \copyright\ 2014 by P.-G. Martinsson, V. Rokhlin,
Y. Shkolnisky, and M. Tygert.
\bigskip
All rights reserved.
\bigskip
Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are
met:
\begin{enumerate}
\item Redistributions of source code must retain the above copyright
notice, this list of conditions, and the following disclaimer.
\item Redistributions in binary form must reproduce the above copyright
notice, this list of conditions, and the following disclaimer in the
documentation and/or other materials provided with the distribution.
\item None of the names of the copyright holders may be used to endorse
or promote products derived from this software without specific prior
written permission.
\end{enumerate}
\bigskip
THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS ``AS IS'' AND ANY
EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNERS BE
LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO,
PROCUREMENT

OF
SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR
BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY,
WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE
OR

```

```

OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
scipy/optimize/lbfgsb/README
License of L-BFGS-B (Fortran code)
=====

The version included here (in lbfgsb.f) is 3.0 (released April 25, 2011).
It was written by Ciyou Zhu, Richard Byrd, and Jorge Nocedal
<nocedal@ece.nwu.edu>. It carries the following condition for use:
"""

This software is freely available, but we expect that all publications
describing work using this software, or all commercial products using it,
quote at least one of the references given below. This software is
released under the BSD License.

References
* R. H. Byrd, P. Lu and J. Nocedal. A Limited Memory Algorithm for Bound
Constrained Optimization, (1995), SIAM Journal on Scientific and
Statistical Computing, 16, 5, pp. 1190-1208.
* C. Zhu, R. H. Byrd and J. Nocedal. L-BFGS-B: Algorithm 778: L-BFGS-B,
FORTRAN routines for large scale bound constrained optimization (1997),
ACM Transactions on Mathematical Software, 23, 4, pp. 550 - 560.
* J.L. Morales and J. Nocedal. L-BFGS-B: Remark on Algorithm 778:
L-BFGS-B, FORTRAN routines for large scale bound constrained optimization
(2011), ACM Transactions on Mathematical Software, 38, 1.
"""

License for the Python wrapper
=====
Copyright (c) 2004 David M. Cooke <cookedm@physics.mcmaster.ca>
Permission is hereby granted, free of charge, to any person obtaining a
copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to permit
persons to whom the Software is furnished to do so, subject to the
following conditions:
The above copyright notice and this permission notice shall be included
in all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN
NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM,
DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR
OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE
USE OR OTHER DEALINGS IN THE SOFTWARE.
scipy/sparse/linalg/dsolve/SuperLU/License.txt
Copyright (c) 2003, The Regents of the University of California, through
Lawrence Berkeley National Laboratory (subject to receipt of any required
approvals from U.S. Dept. of Energy)
All rights reserved.
Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are
met:
(1) Redistributions of source code must retain the above copyright
notice, this list of conditions and the following disclaimer.
(2) Redistributions in binary form must reproduce the above copyright
notice, this list of conditions and the following disclaimer in the

```

documentation and/or other materials provided with the distribution.  
 (3) Neither the name of Lawrence Berkeley National Laboratory, U.S. Dept. of Energy nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS  
 "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT

LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

scipy/sparse/linalg/eigen/arpack/ARPACK/COPYING

BSD Software License

Pertains to ARPACK and P\_ARPACK

Copyright (c) 1996-2008 Rice University.

Developed by D.C. Sorensen, R.B. Lehoucq, C. Yang, and K. Maschhoff.

All rights reserved.

Arpack has been renamed to arpack-ng.

Copyright (c) 2001-2011 - Scilab Enterprises

Updated by Allan Cornet, Sylvestre Ledru.

Copyright (c) 2010 - Jordi Gutiérrez Hermoso (Octave patch)

Copyright (c) 2007 - Sébastien Fabbro (gentoo patch)

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer listed in this license in the documentation and/or other materials provided with the distribution.
- Neither the name of the copyright holders nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS

"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT

LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

scipy/spatial/qhull/COPYING.txt

Qhull, Copyright (c) 1993-2015

C.B. Barber

Arlington, MA

and

The National Science and Technology Research Center for Computation and Visualization of Geometric Structures  
(The Geometry Center)  
University of Minnesota  
email: qhull@qhull.org

This software includes Qhull from C.B. Barber and The Geometry Center. Qhull is copyrighted as noted above. Qhull is free software and may be obtained via http from [www.qhull.org](http://www.qhull.org). It may be freely copied, modified, and redistributed under the following conditions:

1. All copyright notices must remain intact in all files.
2. A copy of this text file must be distributed along with any copies of Qhull that you redistribute; this includes copies that you have modified, or copies of programs or other software products that include Qhull.
3. If you modify Qhull, you must include a notice giving the name of the person performing the modification, the date of modification, and the reason for such modification.
4. When distributing modified versions of Qhull, or other software products that include Qhull, you must provide notice that the original source code may be obtained as noted above.
5. There is no warranty or other guarantee of fitness for Qhull, it is provided solely "as is". Bug reports or fixes may be sent to [qhull\\_bug@qhull.org](mailto:qhull_bug@qhull.org); the authors may or may not act on them as they desire.

## F.2.16 Shroud-1.0

### Shroud-1.0

以下为 Shroud-1.0 的许可声明原文。

```
Project homepage: http://www.cpan.org/authors/id/C/CR/CRAIC/shroud-1.0
Project license:
#!/usr/bin/perl
shroud
Copyright 2000 Robert Jones, Craic Computing, All rights reserved.
This program is free software; you can redistribute it and/or modify it
under the same terms as Perl itself.
The software is supplied as is, with absolutely no warranty.
Perl5 is Copyright (C) 1993-2005, by Larry Wall and others.
```

It is free software; you can redistribute it and/or modify it under the terms of either:

- a) the GNU General Public License as published by the Free Software Foundation; either external linkversion 1, or (at your option) any later versionexternal link, or
- b) the "Artistic License".

For those of you that choose to use the GNU General Public License, my interpretation of the GNU General Public License is that no Perl script falls under the terms of the GPL unless you explicitly put said script under the terms of the GPL yourself.



Furthermore, any object code linked with perl does not automatically fall under the terms of the GPL, provided such object code only adds definitions of subroutines and variables, and does not otherwise impair

the resulting interpreter from executing any standard Perl script. I consider linking in C subroutines in this manner to be the moral equivalent of defining subroutines in the Perl language itself. You may sell such an object file as proprietary provided that you provide or offer to provide the Perl source, as specified by the GNU General Public License. (This is merely an alternate way of specifying input to the program.) You may also sell a binary produced by the dumping of a running Perl script that belongs to you, provided that you provide or offer to provide the Perl source as specified by the GPL. (The fact that a Perl interpreter and your code are in the same binary file is, in this case, a form of mere aggregation.)

This is my interpretation of the GPL. If you still have concerns or difficulties understanding my intent, feel free to contact me. Of course, the Artistic License spells all this out for your protection, so you may prefer to use that.

-- Larry Wall

See Artistic-1.0-Perl in the Standard OSS License Text on page 542."See GPL-1.0 in the Standard OSS License Text on page 542."

## F.2.17 SQLite

### SQLite

以下为 SQLite 的许可声明原文。

Project homepage/download site: <https://www.sqlite.org/index.html>  
Project licensing notices: <https://www.sqlite.org/copyright.html>  
SQLite Is Public Domain

All of the code and documentation in SQLite has been dedicated to the public domain by the authors. All code authors, and representatives of the companies they work for, have signed affidavits dedicating their contributions to the public domain and originals of those signed affidavits are stored in a firesafe at the main offices of Hwaci. Anyone is free to copy, modify, publish, use, compile, sell, or distribute the original SQLite code, either in source code form or as a compiled binary, for any purpose, commercial or non-commercial, and by any means.

The previous paragraph applies to the deliverable code and documentation in SQLite - those parts of the SQLite library that you actually bundle and ship with a larger application. Some scripts used as part of the build process (for example the "configure" scripts generated by autoconf) might fall under other open-source licenses. Nothing from these build scripts ever reaches the final deliverable SQLite library, however, and so the licenses associated with those scripts should not be a factor in assessing your rights to copy and use the SQLite library.

All of the deliverable code in SQLite has been written from scratch. No code has been taken from other projects or from the open internet. Every line of code can be traced back to its original author, and all of those authors have public domain dedications on file. So the SQLite code base is clean and is uncontaminated with licensed code from other projects.

#### Open-Source, not Open-Contribution

SQLite is open-source, meaning that you can make as many copies of it as you want and do whatever you want with those copies, without limitation. But SQLite is not open-contribution. In order to keep SQLite in the public domain and ensure that the code does not become contaminated with proprietary or licensed content, the project does not accept patches from unknown persons.

All of the code in SQLite is original, having been written specifically for use by SQLite. No code has been copied from unknown sources on the internet.

#### Warranty of Title

SQLite is in the public domain and does not require a license. Even so, some organizations want legal proof of their right to use SQLite. Circumstances where this occurs include the following:

Your company desires indemnity against claims of copyright infringement.

You are using SQLite in a jurisdiction that does not recognize the public domain.

You are using SQLite in a jurisdiction that does not recognize the right of an author to dedicate their work to the public domain.

You want to hold a tangible legal document as evidence that you have the legal right to use and distribute SQLite.

Your legal department tells you that you have to purchase a license.

If any of the above circumstances apply to you, Hwaci, the company that employs all the developers of SQLite, will sell you a Warranty of Title for SQLite. A Warranty of Title is a legal document that asserts that the claimed authors of SQLite are the true authors, and that the authors have the legal right to dedicate the SQLite to the public domain, and that Hwaci will vigorously defend against challenges to those claims. All proceeds from the sale of SQLite Warranties of Title are used to fund continuing improvement and support of SQLite.

#### Contributed Code

In order to keep SQLite completely free and unencumbered by copyright, the project does not accept patches. If you would like to make a suggested change, and include a patch as a proof-of-concept, that would be great. However please do not be offended if we rewrite your patch from scratch.

## F.2.18 TclLib

### TclLib

以下为 TclLib 的许可声明原文。

This software is copyrighted by Ajuba Solutions and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files. The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE. THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARS. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

## F.2.19 Tcl/Tk

### Tcl/Tk

以下为 Tcl/Tk 的许可声明原文。

The following terms apply to the all versions of the core Tcl/Tk releases, the Tcl/Tk browser plug-in version 2.0, Tcllib, and TclBlend and Jacl version 1.0. Please note that the TclPro tools are under a different license agreement. This agreement is part of the standard Tcl/Tk distribution as the file named "license.terms".

#### Tcl/Tk License Terms

This software is copyrighted by the Regents of the University of California, Sun Microsystems, Inc., Scriptics Corporation, Ajuba Solutions, and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the

licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARS. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

## F.2.20 zlib

### zlib

以下为 zlib 的许可声明原文。

```
License
/* zlib.h -- interface of the 'zlib' general purpose compression library version 1.2.8, April
28th, 2013
Copyright (C) 1995-2013 Jean-loup Gailly and Mark Adler

This software is provided 'as-is', without any express or implied warranty. In no event will
the authors be held liable for any damages arising from the use of this software.
Permission is granted to anyone to use this software for any purpose, including
commercial applications, and to alter it and redistribute it freely, subject to the following
restrictions:
1. The origin of this software must not be misrepresented; you must not claim that you
 wrote the original software. If you use this software in a product, an acknowledgement
 in the product documentation would be appreciated but is not required.
2. Altered source versions must be plainly marked as such, and must not be
 misrepresented as being the original software.
3. This notice may not be removed or altered from any source distribution.
Jean-loup Gailly (jloup@gzip.org) Mark Adler (madler@alumni.caltech.edu)
*/
```

## F.2.21 Zstandard

### Zstandard

以下为 Zstandard 的许可声明原文。

```
Project homepage: https://github.com/facebook/zstd
Project license: https://github.com/facebook/zstd/blob/master/LICENSE
BSD License

For Zstandard software

Copyright (c) 2016-present, Facebook, Inc. All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are
met:

* Redistributions of source code must retain the above copyright notice,
this list of conditions and the following disclaimer.

* Redistributions in binary form must reproduce the above copyright
notice, this list of conditions and the following disclaimer in the
documentation and/or other materials provided with the distribution.

* Neither the name Facebook nor the names of its contributors may be
used to endorse or promote products derived from this software without
specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
```

## F.3 标准 OSS 许可文本

### Standard OSS License Text

本节转载若干标准开源许可的完整法律文本（英文原文）。

#### F.3.1 Apache-2.0

##### Apache-2.0

以下为 Apache-2.0 许可原文。

```
Project homepage/download site: http://google.github.io/flatbuffers/; https://github.com/
google/flatbuffers
Project licensing notices: https://github.com/google/flatbuffers/blob/master/LICENSE.txt
LICENSE.txt
Apache LicenseVersion 2.0, January 2004 http://www.apache.org/licenses/

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and
distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the
copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other
```

entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell,

sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

(a) You must give any other recipients of the Work or Derivative Works a copy of this License; and

(b) You must cause any modified files to carry prominent notices stating that You changed the files; and

(c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and

(d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and

distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such

Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS\

APPENDIX: How to apply the Apache License to your work.

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "[ ]" replaced with your own identifying information. (Don't include the brackets!) The



text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright 2014 Google Inc.

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

## F.3.2 Artistic-1.0-Perl

### Artistic-1.0-Perl

以下为 Artistic-1.0-Perl 许可原文。

The "Artistic License"

Preamble

The intent of this document is to state the conditions under which a Package may be copied, such that the Copyright Holder maintains some semblance of artistic control over the development of the package, while giving the users of the package the right to use and distribute the Package in a more-or-less customary fashion, plus the right to make reasonable modifications.

Definitions:

"Package" refers to the collection of files distributed by the Copyright Holder, and derivatives of that collection of files created through textual modification.

"Standard Version" refers to such a Package if it has not been modified, or has been modified in accordance with the wishes of the Copyright Holder as specified below.

"Copyright Holder" is whoever is named in the copyright or copyrights for the package.

"You" is you, if you're thinking about copying or distributing this Package.

"Reasonable copying fee" is whatever you can justify on the basis of media cost, duplication charges, time of people involved, and so on. (You will not be required to justify it to the Copyright Holder, but only to the computing community at large as a market that must bear the fee.)

"Freely Available" means that no fee is charged for the item itself, though there may be fees involved in handling the item. It also means that recipients of the item may redistribute it under the same conditions they received it.

- You may make and give away verbatim copies of the source form of the Standard Version of this Package without restriction, provided that you duplicate all of the original copyright notices and associated disclaimers.
- You may apply bug fixes, portability fixes and other modifications derived from the Public Domain or from the Copyright Holder. A Package modified in such a way shall still be considered the Standard Version.
- You may otherwise modify your copy of this Package in any way, provided that you insert a prominent notice in each changed file stating how and when you changed that file, and provided that you do at least ONE of the following:
  - place your modifications in the Public Domain or otherwise make them Freely Available, such as by posting said modifications to Usenet or an equivalent medium, or placing the modifications on a major archive site such as uunet.uu.net, or by allowing the Copyright Holder to include your modifications in the Standard Version of the Package.
  - use the modified Package only within your corporation or organization.
  - rename any non-standard executables so the names do not conflict with standard executables, which must also be provided, and provide a separate manual page for each non-standard executable that clearly documents how it differs from the Standard Version.
  - make other distribution arrangements with the Copyright Holder.
- You may distribute the programs of this Package in object code or executable form, provided that you do at least ONE of the following:
  - distribute a Standard Version of the executables and library files, together with instructions (in the manual page or equivalent) on where to get the Standard Version.
  - accompany the distribution with the machine-readable source of the Package with your modifications.
  - give non-standard executables non-standard names, and clearly document the differences in manual pages (or equivalent), together with instructions on where to get the Standard Version.
  - make other distribution arrangements with the Copyright Holder.
- You may charge a reasonable copying fee for any distribution of this Package. You may charge any fee you choose for support of this Package. You may not charge a fee for this Package itself. However, you may distribute this Package in aggregate with other (possibly

commercial) programs as part of a larger (possibly commercial) software distribution provided that you do not advertise this Package as a product of your own. You may embed this Package's interpreter within an executable of yours (by linking); this shall be construed as a mere form of aggregation, provided that the complete Standard Version of the interpreter is so embedded.

- The scripts and library files supplied as input to or produced as output from the programs of this Package do not automatically fall under the copyright of this Package, but belong to whoever generated them, and may be sold commercially, and may be aggregated with this Package. If such scripts or library files are aggregated with this Package via the so-called "undump" or "unexec" methods of producing a binary executable image, then distribution of such an image shall neither be construed as a distribution of this Package nor shall it fall under the restrictions of Paragraphs 3 and 4, provided that you do not represent such an executable image as a Standard Version of this Package.
- subroutines (or comparably compiled subroutines in other languages) supplied by you and linked into this Package in order to emulate subroutines and variables of the language defined by this Package shall not be considered part of this Package, but are the equivalent

of input as in Paragraph 6, provided these subroutines do not change the language in any way that would cause it to fail the regression tests for the language.

- Aggregation of this Package with a commercial distribution is always permitted provided that the use of this Package is embedded; that is, when no overt attempt is made to make this Package's interfaces visible to the end user of the commercial distribution. Such use shall not be construed as a distribution of this Package.
- The name of the Copyright Holder may not be used to endorse or promote products derived from this software without specific prior written permission.
- THIS PACKAGE IS PROVIDED "AS IS" AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

The End

### F.3.3 GPL-1.0

#### GPL-1.0

以下为 GPL-1.0 许可原文。

GNU GENERAL PUBLIC LICENSE Version 1, February 1989

Copyright (C) 1989 Free Software Foundation, Inc. 59 Temple Place, Suite 330, Boston, MA 02111-1307, USA Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

## Preamble

The license agreements of most software companies try to keep users at the mercy of those companies. By contrast, our General Public License is intended to guarantee your freedom to share and change free software--to make sure the software is free for all its users. The General Public License applies to the Free Software Foundation's software and to any other program whose authors commit to using it. You can use it for your programs, too.

When we speak of free software, we are referring to freedom, not price. Specifically, the General Public License is designed to make sure that you have the freedom to give away or sell copies of free software, that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs; and that you know you can do these things.

To protect your rights, we need to make restrictions that forbid anyone to deny you these rights or to ask you to surrender the rights. These restrictions translate to certain responsibilities for you if you distribute copies of the software, or if you modify it.

For example, if you distribute copies of a such a program, whether gratis or for a fee, you must give the recipients all the rights that you have. You must make sure that they, too, receive or can get the source code. And you must tell them their rights.

We protect your rights with two steps: (1) copyright the software, and (2) offer you this license which gives you legal permission to copy, distribute and/or modify the software.

Also, for each author's protection and ours, we want to make certain that everyone understands that there is no warranty for this free software. If the software is modified by someone else and passed on, we want its recipients to know that what they have is not the original, so that any problems introduced by others will not reflect on the original authors' reputations.

The precise terms and conditions for copying, distribution and modification follow. GNU GENERAL PUBLIC LICENSE TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License Agreement applies to any program or other work which contains a notice placed by the copyright holder saying it may be distributed under the terms of this General Public License. The "Program", below, refers to any such program or work, and a "work based on the Program" means either the Program or any work containing the Program or a portion of it, either verbatim or with modifications. Each licensee is addressed as "you".

- You may copy and distribute verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this General Public License and to the absence of any warranty; and

give any other recipients of the Program a copy of this General Public License along with the Program. You may charge a fee for the physical act of transferring a copy.

- You may modify your copy or copies of the Program or any portion of it, and copy and distribute such modifications under the terms of Paragraph 1 above, provided that you also do the following:
  - cause the modified files to carry prominent notices stating that you changed the files and the date of any change; and
  - cause the whole of any work that you distribute or publish, that in whole or in part contains the Program or any part thereof, either with or without modifications, to be licensed at no charge to all third parties under the terms of this General Public License (except that you may choose to grant warranty protection to some or all third parties, at your option).
  - If the modified program normally reads commands interactively when run, you must cause it, when started running for such interactive use in the simplest and most usual way, to print or display an announcement including an appropriate copyright notice and a notice that there is no warranty (or else, saying that you provide a warranty) and that users may redistribute the program under these conditions, and telling the user how to view a copy of this General Public License.
  - You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

Mere aggregation of another independent work with the Program (or its derivative) on a volume of a storage or distribution medium does not bring the other work under the scope of these terms. 3. You may copy and distribute the Program (or a portion or derivative of it, under Paragraph 2) in object code or executable form under the terms of Paragraphs 1 and 2 above provided that you also do one of the following:

- accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Paragraphs 1 and 2 above; or,
- accompany it with a written offer, valid for at least three years, to give any third party free (except for a nominal charge for the cost of distribution) a complete machine-readable copy of the corresponding source code, to be distributed under the terms of Paragraphs 1 and 2 above; or,
- accompany it with the information you received as to where the corresponding source code may be obtained. (This alternative is

allowed only for noncommercial distribution and only if you received

the program in object code or executable form alone.)

Source code for a work means the preferred form of the work for making modifications to it. For an executable file, complete source code means all the source code for all modules it contains; but, as a special exception, it need not include source code for modules which are standard libraries that accompany the operating system on which the executable file runs, or for standard header files or definitions files that accompany that operating system.

4. You may not copy, modify, sublicense, distribute or transfer the Program except as expressly provided under this General Public License. Any attempt otherwise to copy, modify, sublicense, distribute or transfer the Program is void, and will automatically terminate your rights to use the Program under this License. However, parties who have received copies, or rights to use copies, from you under this General Public License will not have their licenses terminated so long as such parties remain in full compliance.

5. By copying, distributing or modifying the Program (or any work based on the Program) you indicate your acceptance of this license to do so, and all its terms and conditions.

6. Each time you redistribute the Program (or any work based on the Program), the recipient automatically receives a license from the original licensor to copy, distribute or modify the Program subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. 7. The Free Software Foundation may publish revised and/or new versions of the General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies a version number of the license which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the license, you may choose any version ever published by the Free Software Foundation.

8. If you wish to incorporate parts of the Program into other free programs whose distribution conditions are different, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of

preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

9. BECAUSE THE PROGRAM IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND,

EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

10. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS Appendix: How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to humanity, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

Copyright (C) 19yy

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 1, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307, USA.

Also add information on how to contact you by electronic and paper mail.

If the program is interactive, make it output a short notice like this when it starts in an interactive mode:

Gnomovision version 69, Copyright (C) 19xx name of author Gnomovision comes with ABSOLUTELY NO WARRANTY; for details type `show w'. This is free software, and you are welcome to redistribute it under certain conditions; type `show c' for details.

The hypothetical commands `show w' and `show c' should show the appropriate parts of the General Public License. Of course, the commands

you use may be called something other than `show w' and `show c'; they could even be mouse-clicks or menu items--whatever suits your program.

You should also get your employer (if you work as a programmer) or your school, if any, to sign a "copyright disclaimer" for the program, if necessary. Here a sample; alter the names:

Yoyodyne, Inc., hereby disclaims all copyright interest in the program `Gnomovision' (a program to direct compilers to make passes at assemblers) written by James Hacker.

, 1 April 1989 Ty Coon, President of Vice

That's all there is to it!

## F.3.4 GPL-3.0

### GPL-3.0

以下为 GPL-3.0 许可原文。

GNU GENERAL PUBLIC LICENSE Version 3, 29 June 2007

Copyright (C) 2007 Free Software Foundation, Inc. <<http://fsf.org/>>  
Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The GNU General Public License is a free, copyleft license for software and other kinds of works.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a program--to make sure it remains free software for all its users. We, the Free Software Foundation, use the

GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that



you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could

make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

#### TERMS AND CONDITIONS

##### 0. Definitions.

"This License" refers to version 3 of the GNU General Public License.

"Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

"The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" may be individuals or organizations.

To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the earlier work or a work "based on" the earlier work.

A "covered work" means either the unmodified Program or a work based on the Program.

To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

#### 1. Source Code.

The "source code" for a work means the preferred form of the work for making modifications to it. "Object code" means any non-source form of a work.

A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work.

## 2. Basic Permissions.

All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its

content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

## 3. Protecting Users' Legal Rights From Anti-Circumvention Law.

No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

## 4. Conveying Verbatim Copies.

You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

#### 5. Conveying Modified Source Versions.

You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

- a) The work must carry prominent notices stating that you modified it, and giving a relevant date.
- b) The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".
- c) You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.
- d) If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

#### 6. Conveying Non-Source Forms.

You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

- a) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange.
- b) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you

offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge.

c) Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b.

d) Convey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the object code. If the place to copy the object code is a network server, the Corresponding Source may be on a different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements.

e) Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A "User Product" is either (1) a "consumer product", which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, "normally used" refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

#### 7. Additional Terms.

"Additional permissions" are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal

in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

a) Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or

b) Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or

c) Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or

d) Limiting the use for publicity purposes of names of licensors or authors of the material; or

e) Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or

f) Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered "further restrictions" within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

## 8. Termination.

You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

#### 9. Acceptance Not Required for Having Copies.

You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

#### 10. Automatic Licensing of Downstream Recipients.

Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

#### 11. Patents.

A "contributor" is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.



Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy,

free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

12. No Surrender of Others' Freedom.

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence

you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

#### 13. Use with the GNU Affero General Public License.

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

#### 14. Revised Versions of this License.

The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

#### 15. Disclaimer of Warranty.

THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO,

THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

#### 16. Limitation of Liability.

IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

#### 17. Interpretation of Sections 15 and 16.

If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

#### END OF TERMS AND CONDITIONS

#### How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively state the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

<one line to give the program's name and a brief idea of what it does.>  
Copyright (C) <year> <name of author>

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <http://www.gnu.org/licenses/>.

Also add information on how to contact you by electronic and paper mail.

If the program does terminal interaction, make it output a short notice like this when it starts in an interactive mode:

```
<program> Copyright (C) <year> <name of author> This program comes with
ABSOLUTELY NO WARRANTY; for details type `show w'. This is free software,
and you are welcome to redistribute it under certain conditions; type
`show c' for details.
```

The hypothetical commands ``show w'` and ``show c'` should show the appropriate parts of the General Public License. Of course, your program's commands might be different; for a GUI interface, you would use an "about box".

You should also get your employer (if you work as a programmer) or school, if any, to sign a "copyright disclaimer" for the program, if necessary. For more information on this, and how to apply and follow the GNU GPL, see <http://www.gnu.org/licenses/>.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read <http://www.gnu.org/philosophy/why-not-lgpl.html>.